



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



Gruppo RubberDuck

email: GroupRubberDuck@gmail.com

Specifica Tecnica

Stato	Approvato
Versione	1.0.0
Autori	Aldo Bettega Davide Lorenzon Ana Maria Draghici Filippo Guerra Felician Mario Necsulescu Davide Testolin
Verificatori	Aldo Bettega Davide Lorenzon Filippo Guerra Felician Mario Necsulescu
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin BlueWind srl

Vers.	Data	Autore	Verificatore	Descrizione
1.0.0	2026-05-22	Aldo Bettega	Aldo Bettega	Approvazione
0.13.0	2026-05-19	Aldo Bettega	Davide Lorenzon	Scrittura sezione MVVM
0.12.0	2026-05-16	Aldo Bettega	Davide Lorenzon	Rivisitazione complessiva dei diagrammi e creazione dei diagrammi di dominio, asset _G , device e generazione report
0.11.0	2026-05-14	Felician Mario Necsulescu	Aldo Bettega	Riviste classi di importazione _G ed esportazione _G : <u>Sezione 5.4</u> e <u>Sezione 5.5</u>
0.10.0	2026-05-12	Felician Mario Necsulescu	Aldo Bettega	Riviste classi di sessione e valutazione _G : <u>Sezione 5.6</u> e <u>Sezione 5.7.1</u>
0.9.0	2026-05-06	Ana Maria Draghici	Aldo Bettega	Scomposizione sezioni a partire dalla <u>Sezione 5.2</u> : Import, ValutazioneRequisiti, Dashboard _G , Export e Frontend
0.8.0	2026-05-03	Ana Maria Draghici	Aldo Bettega	Scomposizione sezioni a partire dalla <u>Sezione 5.2</u> : Device, Asset _G e Session
0.7.3	2026-04-22	Davide Testolin	Ana Maria Draghici	Migliorata sezione <u>Sezione 3.5</u>
0.7.2	2026-04-20	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern comportamentali e architeturali
0.7.1	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern: creazionali e strutturali <u>Sezione 4.2</u> , <u>Sezione 4.3</u>

Vers.	Data	Autore	Verificatore	Descrizione
0.7.0	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei principi di design <u>Sezione 4.1</u>
0.6.4	2026-04-19	Davide Lorenzon	Aldo Bettega	Revisione architeturale della sezione relativa alla modifica degli asset _G
0.6.3	2026-04-19	Davide Lorenzon	Aldo Bettega	Bozza delle classi relative alla generazione del report di conformità _G
0.6.2	2026-04-19	Davide Lorenzon	Aldo Bettega	Bozza delle classi relative a import ed export tramite file di dispositivo _G e modello
0.6.1	2026-04-17	Filippo Guerra	Aldo Bettega	Bozza della classe valutazione _G <u>Sezione 5.6</u>
0.6.0	2026-04-17	Filippo Guerra	Davide Lorenzon	Stesura vista_dati
0.5.0	2026-04-16	Ana Maria Draghici	Davide Lorenzon	Stesura vista_dati <u>Sezione 3.5</u>
0.4.0	2026-04-16	Aldo Bettega	Davide Lorenzon	Stesura diagramma dominio e <u>Sezione 5.2</u>
0.3.0	2026-04-15	Aldo Bettega	Davide Lorenzon	Stesura della <u>Sezione 3.4</u>
0.2.0	2026-04-10	Aldo Bettega	Davide Lorenzon	Stesura della <u>Sezione 3.1</u> e <u>Sezione 3.2</u>
0.1.2	2026-04-09	Davide Lorenzon	Filippo Guerra	Bozza iniziale della <u>Sezione 3.3</u> Architettura di deployment.
0.1.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Bozza iniziale della <u>Sezione 2</u> Tecnologie.

Vers.	Data	Autore	Verificatore	Descrizione
0.1.0	2026-04-08	Aldo Bettega	Felician Mario Necsulescu	Stesura introduzio- ne
0.0.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Creazione del docu- mento

Indice

1) Introduzione	1
1.1) Scopo del documento	1
1.2) Scopo del prodotto	1
1.3) Glossario	1
1.4) Riferimenti	2
1.4.1) Riferimenti normativi	2
1.4.2) Riferimenti informativi	2
2) Tecnologie	3
2.1) Backend	3
2.2) Frontend	6
2.3) Persistenza dei dati	7
3) Architettura di Sistema	8
3.1) Premessa: approccio di lavoro usato	8
3.2) Architettura Logica	8
3.2.1) Stile architetturale	8
3.2.2) Motivazioni della scelta architetturale	9
3.2.3) Limiti dell'architettura	9
3.2.4) Diagramma dei package	10
3.3) Architettura di Deployment	11
3.3.1) Motivazioni della scelta	11
3.3.2) Realizzazione Pratica e Topologia	11
3.4) Diagrammi di sequenza	12
3.4.1) UC05 - Importazione _G dispositivo _G	13
3.4.2) UC26, UC27 - Valutazione _G di un nodo _G e transizione di stato	15
3.5) Schema dati	17
3.5.1) Scelte Architettureali e Formalismo	17
3.5.2) Schema Dati Device	17
3.5.3) Schema ComplianceStandard	19
3.5.4) Gestione degli Esiti e Storicità	22
4) Design Pattern	23
4.1) Principi di Design	23
4.1.1) Dependency Injection	23
4.2) Design Pattern Creazionali	23
4.2.1) Factory	23
4.3) Design Pattern strutturali	24
4.3.1) Adapter	24
4.4) Design Pattern Comportamentali	24
4.4.1) Template Method	24
4.5) Pattern MVVM nel Frontend	25
4.5.1) Model	25

4.5.2)	ViewModel	25
4.5.3)	View	26
5)	Diagrammi delle classi	27
5.1)	Dominio	27
5.1.1)	Device	28
5.1.2)	AssetG	29
5.1.3)	AssetAnagraphic	29
5.1.4)	AssetProperties	30
5.1.5)	AssetType	31
5.1.6)	AssetEvidence	31
5.1.7)	ComplianceStandard	32
5.1.8)	Requirement	33
5.1.9)	DecisionTree	34
5.1.10)	Node	34
5.1.11)	DecisionNode	35
5.1.12)	LeafNode	36
5.1.13)	StandardVerdict	36
5.1.14)	EvaluationEngine	37
5.1.15)	EvaluationState	38
5.1.16)	RequirementEvaluationResult	38
5.1.17)	AssetEvaluationResult	39
5.1.18)	DeviceEvaluationResult	40
5.1.19)	NodeDetail	41
5.1.20)	RequirementEvaluationDetail	42
5.1.21)	AssetEvaluationDetail	43
5.1.22)	DeviceEvaluationDetail	43
5.1.23)	EvaluationSession	44
5.1.24)	SessionHandler	45
5.1.25)	ExportedFile	45
5.1.26)	DeviceSummary	46
5.2)	Device	47
5.2.1)	CreateDevice	47
5.2.1.1)	FlaskWriteDeviceController	47
5.2.1.2)	CreateDeviceUseCase	48
5.2.1.3)	CreateDeviceCommand	48
5.2.1.4)	CreateDeviceService	49
5.2.1.5)	RegisterDevicePort	49
5.2.1.6)	MongoDeviceAdapter	50
5.2.2)	DeleteDevice	51
5.2.2.1)	DeleteDeviceUseCase	51
5.2.2.2)	DeleteDeviceCommand	52

5.2.2.3)	DeleteDeviceService	52
5.2.2.4)	DeleteDevicePort	53
5.2.3)	UpdateDevice	53
5.2.3.1)	UpdateDeviceUseCase	54
5.2.3.2)	UpdateDeviceCommand	54
5.2.3.3)	UpdateDeviceService	55
5.2.3.4)	SaveDevicePort	55
5.2.3.5)	FindDevicePort	56
5.2.4)	GetDeviceDetail	56
5.2.4.1)	FlaskQueryDeviceController	57
5.2.4.2)	GetDeviceDetailUseCase	57
5.2.4.3)	GetDeviceDetailService	58
5.2.4.4)	GetDeviceDetailCommand	58
5.2.4.5)	GetComplianceStandardUseCase	59
5.2.4.6)	GetComplianceStandardService	59
5.2.4.7)	GetComplianceStandardCommand	60
5.2.4.8)	FindStandardPort	60
5.2.4.9)	MongoStandardAdapter	61
5.2.5)	GetDeviceList	61
5.2.5.1)	GetDeviceListUseCase	62
5.2.5.2)	GetDeviceListService	62
5.2.5.3)	DeviceSummary	63
5.2.5.4)	FindAllDevicesPort	63
5.2.6)	GetDeviceEvaluationDetail	64
5.2.6.1)	FlaskDeviceEvaluationDetailController	64
5.2.6.2)	GetDeviceEvaluationDetailCommand	65
5.2.6.3)	GetDeviceEvaluationDetailUseCase	65
5.2.6.4)	GetDeviceEvaluationDetailService	66
5.2.6.5)	DTO	66
5.2.6.5.1)	DeviceEvaluationDTO	67
5.2.6.5.2)	AssetEvaluationSummaryDTO	67
5.3)	AssetG	68
5.3.1)	CreateAsset	68
5.3.1.1)	FlaskWriteAssetController	69
5.3.1.2)	CreateAssetUseCase	69
5.3.1.3)	CreateAssetCommand	70
5.3.1.4)	CreateAssetService	71
5.3.1.5)	InMemoryEvaluationSessionCache	71
5.3.1.6)	SaveEvaluationSessionPort	72
5.3.1.7)	GetEvaluationSessionPort	72
5.3.2)	UpdateAsset	73

5.3.2.1)	UpdateAssetUseCase	74
5.3.2.2)	UpdateAssetCommand	74
5.3.2.3)	UpdateAssetService	75
5.3.3)	DeleteAsset	76
5.3.3.1)	DeleteAssetCommand	77
5.3.3.2)	DeleteAssetUseCase	77
5.3.3.3)	DeleteAssetService	78
5.3.4)	GetAssetAnagraphic	79
5.3.4.1)	FlaskQueryAssetController	80
5.3.4.2)	GetAssetAnagraphicCommand	80
5.3.4.3)	GetAssetAnagraphicUseCase	81
5.3.4.4)	GetAssetAnagraphicService	81
5.3.5)	GetAssetEvaluationDetail	82
5.3.5.1)	FlaskAssetEvaluationDetailController	82
5.3.5.2)	GetAssetEvaluationDetailCommand	83
5.3.5.3)	GetAssetEvaluationDetailUseCase	83
5.3.5.4)	GetAssetEvaluationDetailService	84
5.3.5.5)	DTO	84
5.3.5.5.1)	AssetEvaluationDTO	85
5.3.5.5.2)	RequirementEvaluationSummaryDTO	85
5.3.6)	GetRequirement	86
5.3.6.1)	FlaskRequirementEvaluationDetailController	86
5.3.6.2)	GetRequirementEvaluationDetailUseCase	87
5.3.6.3)	GetRequirementEvaluationDetailCommand	88
5.3.6.4)	GetRequirementEvaluationDetailService	88
5.3.6.5)	DTO	89
5.3.6.5.1)	RequirementEvaluationDTO	89
5.3.6.5.2)	DependencySummaryDTO	90
5.3.6.5.3)	DecisionTreeDTO	90
5.3.6.5.4)	LeafNodeDTO	91
5.3.6.5.5)	DecisionNodeDTO	91
5.3.6.5.6)	NodeBaseDTO	92
5.4)	Import	92
5.4.1)	ImportDevice	92
5.4.1.1)	UploadFileController	93
5.4.1.2)	FlaskImportDeviceController	93
5.4.1.3)	ImportDeviceUseCase	94
5.4.1.4)	ImportDeviceCommand	94
5.4.1.5)	AllowedDeviceFileExtension	95
5.4.1.6)	ImportDeviceService	95
5.4.1.7)	FileDeviceImporterPort	96

5.4.1.8)	FileDeviceImporter	96
5.4.1.9)	XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter	97
5.4.1.10)	FileDeviceImporterFactoryPort	98
5.4.1.11)	ConcreteFileDeviceImporterFactory	98
5.5)	Export	99
5.5.1)	ExportDevice	99
5.5.1.1)	FlaskExportDeviceController	100
5.5.1.2)	ExportDeviceUseCase	100
5.5.1.3)	ExportDeviceCommand	101
5.5.1.4)	ExportDeviceService	101
5.5.1.5)	FileDeviceExporterFactoryPort	102
5.5.1.6)	FileDeviceExporterPort	102
5.5.1.7)	FileDeviceExporter	103
5.5.1.8)	ConcreteFileDeviceExporterFactory	103
5.5.1.9)	CSVFileDeviceExporter, JSONFileDeviceExporter, XMLFileDeviceExporter	104
5.5.2)	ExportReport	105
5.5.2.1)	FlaskExportReportController	105
5.5.2.2)	GenerateReportUseCase	106
5.5.2.3)	GenerateReportCommand	107
5.5.2.4)	ReportFormat	107
5.5.2.5)	GenerateReportService	107
5.5.2.6)	ReportGeneratorPort	108
5.5.2.7)	PdfReportGenerator	109
5.6)	Session	110
5.6.1)	OpenEvaluationSession	110
5.6.1.1)	EvaluationSessionController	111
5.6.1.2)	OpenEvaluationSessionCommand	111
5.6.1.3)	OpenEvaluationSessionUseCase	112
5.6.1.4)	OpenEvaluationSessionService	112
5.6.1.5)	SessionCoordinator	113
5.6.1.6)	EvaluationSessionExistPort	114
5.6.1.7)	CreateEvaluationSessionPort	114
5.6.2)	CommitEvaluationSession	115
5.6.2.1)	CommitEvaluationSessionUseCase	116
5.6.2.2)	CommitEvaluationSessionService	116
5.6.2.3)	CommitEvaluationSessionCommand	117
5.6.3)	CloseEvaluationSession	118
5.6.3.1)	CloseEvaluationSessionUseCase	118
5.6.3.2)	CloseEvaluationSessionService	119

5.6.3.3)	CloseEvaluationSessionCommand	119
5.6.3.4)	DeleteSessionPort	120
5.7)	Area navigazione	121
5.7.1)	EvaluateDecisionNode	121
5.7.1.1)	EvaluateDecisionNodeController	122
5.7.1.2)	EvaluateDecisionNodeUseCase	122
5.7.1.3)	EvaluateDecisionNodeCommand	123
5.7.1.4)	EvaluateDecisionNodeService	123
5.7.2)	InsertJustification	124
5.7.2.1)	FlaskInsertJustificationController	125
5.7.2.2)	InsertJustificationUseCase	125
5.7.2.3)	InsertJustificationCommand	126
5.7.2.4)	InsertJustificationService	126
5.8)	Architettura Frontend: Widget Semplici	127
5.8.1)	Shared	128
5.8.1.1)	Validation Rule	128
5.8.1.2)	Field Definition	129
5.8.1.3)	Use Form Model	129
5.8.1.4)	Definizioni di Dominio (Constants)	130
5.8.2)	Component	131
5.8.2.1)	AsyncButton	131
5.8.2.2)	BaseModal	133
5.8.2.3)	FormField	134
5.8.2.4)	Toast	134
5.8.2.5)	FileDropZone	135
5.8.2.6)	Device Form	136
5.8.2.7)	Asset _G Form	137
5.8.3)	Widget	138
5.8.3.1)	AssetCreate	138
5.8.3.1.1)	AssetCreateWidget	138
5.8.3.2)	AssetDelete	140
5.8.3.2.1)	AssetDeleteWidget	141
5.8.3.3)	AssetEditWidget	142
5.8.3.3.1)	AssetEditWidget	142
5.8.3.4)	DeviceCreate	143
5.8.3.4.1)	DeviceCreateWidget	144
5.8.3.5)	DeviceEdit	145
5.8.3.5.1)	DeviceEditWidget	146
5.8.3.6)	DeviceExportWidget	147
5.8.3.7)	DeviceImport	148
5.8.3.7.1)	DeviceImportWidget	149

5.8.3.8)	SessionClose	150
5.8.3.8.1)	SessionCloseWidget	151
5.8.3.9)	SessionCommitAndClose	152
5.8.3.9.1)	SessionCommitAndCloseWidget	152
5.8.3.10)	SessionCommit	153
5.8.3.10.1)	SessionCommitWidget	154
5.8.3.11)	SessionOpen	155
5.8.3.11.1)	SessionOpenWidget	155
5.9)	Architettura Frontend: Modulo Decision Tree _G	156
5.9.1)	Logica di valutazione _G	157
5.9.1.1)	TreeStructure	158
5.9.1.2)	Node	159
5.9.1.3)	DecisionNode	159
5.9.1.4)	LeafNode	160
5.9.1.5)	NodeDTO	161
5.9.1.6)	NodeType	162
5.9.1.7)	LeafNodeValue	162
5.9.1.8)	EvaluationEngine	163
5.9.2)	APIClient	163
5.9.3)	EvaluationApiClient	163
5.9.4)	Layout	165
5.9.4.1)	LayoutNodeDTO	165
5.9.4.2)	EdgeDTO	166
5.9.4.3)	HierarchyNode	167
5.9.4.4)	LayoutConfig	168
5.9.4.5)	LayoutResult	168
5.9.4.6)	D3LayoutEngine	169
5.9.5)	Store	170
5.9.5.1)	DecisionTreeStore	170
5.9.6)	Componenti	172
5.9.6.1)	RequirementEvaluationWidget	172
5.9.6.2)	TreeSidebar	174
5.9.6.3)	DecisionTreeWidget	175
5.9.6.4)	TreeCanvas	176
5.9.6.5)	EvaluationBadge	178
5.9.6.6)	JustificationForm	179
5.9.6.7)	Point	180
5.9.6.8)	StateConfig	180
5.9.6.9)	UiDecisionNode	181
5.9.6.10)	UiLeafNode	182
5.9.6.11)	ViewBox	182

6) Tracciamento	184
6.1) Tracciamento dei Requisiti	184
6.1.1) Requisiti Obbligatori	184
6.1.2) Requisiti Desiderabili	190
6.1.3) Requisiti Opzionali	191
7) Gestione degli Errori	201
7.0.1) Il Flusso di Traduzione degli Errori (Exception Mapping)	201

Lista delle tabelle

Tabella 1	Backend	3
Tabella 2	Frontend	6
Tabella 3	Schema dati: Root — Device	18
Tabella 4	Schema dati: Sub-documento — assets[N]	19
Tabella 5	Schema dati: Sub-documento — evaluations[N]	19
Tabella 6	Schema dati: Root — ComplianceStandard	21
Tabella 7	Schema dati: Sub-documento — requirements[N]	21
Tabella 8	Schema dati: Sub-documento — DecisionTree	21
Tabella 9	Schema dati: Sub-documento — DecisionNode	22
Tabella 10	Schema dati: Sub-documento — LeafNode	22

Lista delle immagini

Figura 1	Diagramma dei package	10
Figura 2	Schema logico: Documento Device	18
Figura 3	Schema logico: ComplianceStandard	20
Figura 4		27
Figura 5	Modello di Dominio	27
Figura 6	Device	28
Figura 7	Asset _G	29
Figura 8	AssetAnagraphic	29
Figura 9	AssetProperties	30
Figura 10	AssetType	31
Figura 11	AssetEvidence	31
Figura 12	ComplianceStandard	32
Figura 13	Requirement	33
Figura 14	DecisionTree	34
Figura 15	Node	34
Figura 16	DecisionNode	35
Figura 17	LeafNode	36
Figura 18	StandardVerdict	36
Figura 19	EvaluationEngine	37
Figura 20	EvaluationState	38
Figura 21	RequirementEvaluationResult	38
Figura 22	AssetEvaluationResult	39
Figura 23	DeviceEvaluationResult	40
Figura 24	NodeDetail	41
Figura 25	RequirementEvaluationDetail	42
Figura 26	AssetEvaluationDetail	43
Figura 27	DeviceEvaluationDetail	43
Figura 28	EvaluationSession	44
Figura 29	SessionHandler	45
Figura 30	ExportedFile	45
Figura 31	DeviceSummary	46
Figura 32	Caso d'uso _G CreateDevice	47
Figura 33	FlaskWriteDeviceController	47
Figura 34	CreateDeviceUseCase	48
Figura 35	CreateDeviceCommand	48
Figura 36	CreateDeviceService	49
Figura 37	RegisterDevicePort	49
Figura 38	MongoDeviceAdapter	50
Figura 39	Caso d'uso _G DeleteDevice	51
Figura 40	DeleteDeviceUseCase	51

Figura 41	DeleteDeviceCommand	52
Figura 42	DeleteDeviceService	52
Figura 43	DeleteDevicePort	53
Figura 44	Caso d'uso _G UpdateDevice	53
Figura 45	UpdateDeviceUseCase	54
Figura 46	UpdateDeviceCommand	54
Figura 47	UpdateDeviceService	55
Figura 48	SaveDevicePort	55
Figura 49	FindDevicePort	56
Figura 50	GetDeviceDetail	56
Figura 51	FlaskQueryDeviceController	57
Figura 52	GetDeviceDetailUseCase	57
Figura 53	GetDeviceDetailService	58
Figura 54	GetDeviceDetailCommand	58
Figura 55	GetComplianceStandardUseCase	59
Figura 56	GetComplianceStandardService	59
Figura 57	GetComplianceStandardCommand	60
Figura 58	FindStandardPort	60
Figura 59	MongoStandardAdapter	61
Figura 60	Caso d'uso _G GetDeviceList	61
Figura 61	GetDeviceListUseCase	62
Figura 62	GetDeviceListService	62
Figura 63	DeviceSummary	63
Figura 64	FindAllDevicesPort	63
Figura 65	GetDeviceEvaluationDetail	64
Figura 66	FlaskDeviceEvaluationDetailController	64
Figura 67	GetDeviceEvaluationDetailCommand	65
Figura 68	GetDeviceEvaluationDetailUseCase	65
Figura 69	GetDeviceEvaluationDetailService	66
Figura 70	DeviceEvaluationDTO	67
Figura 71	Caso d'uso _G CreateAsset	68
Figura 72	FlaskWriteAssetController	69
Figura 73	CreateAssetUseCase	69
Figura 74	CreateAssetCommand	70
Figura 75	CreateAssetService	71
Figura 76	InMemoryEvaluationSessionCache	71
Figura 77	SaveEvaluationSessionPort	72
Figura 78	GetEvaluationSessionPort	72
Figura 79	Caso d'uso _G UpdateAsset	73
Figura 80	UpdateAssetUseCase	74
Figura 81	UpdateAssetCommand	74

Figura 82	UpdateAssetService	75
Figura 83	Caso d'uso _G DeleteAsset	76
Figura 84	DeleteAssetCommand	77
Figura 85	DeleteAssetUseCase	77
Figura 86	DeleteAssetService	78
Figura 87	Caso d'uso _G GetAssetAnagraphic	79
Figura 88	FlaskQueryAssetController	80
Figura 89	GetAssetAnagraphicCommand	80
Figura 90	GetAssetAnagraphicUseCase	81
Figura 91	GetAssetAnagraphicService	81
Figura 92	GetAssetEvaluationDetail	82
Figura 93	GetAssetEvaluationDetail	82
Figura 94	GetAssetEvaluationDetailCommand	83
Figura 95	GetAssetEvaluationDetailUseCase	83
Figura 96	GetAssetEvaluationDetailService	84
Figura 97	AssetEvaluationDTO	85
Figura 98	Caso d'uso _G GetRequirement	86
Figura 99	FlaskRequirementEvaluationDetailController	86
Figura 100	GetRequirementEvaluationDetailUseCase	87
Figura 101	GetRequirementEvaluationDetailCommand	88
Figura 102	GetRequirementEvaluationDetailService	88
Figura 103	RequirementEvaluationDTO	89
Figura 104	Caso d'uso _G ImportDevice	92
Figura 105	UploadFileController	93
Figura 106	FlaskImportDeviceController	93
Figura 107	ImportDeviceUseCase	94
Figura 108	ImportDeviceCommand	94
Figura 109	AllowedDeviceFileExtension	95
Figura 110	ImportDeviceService	95
Figura 111	FileDeviceImporterPort	96
Figura 112	FileDeviceImporter	96
Figura 113	XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter	97
Figura 114	FileDeviceImporterFactoryPort	98
Figura 115	ConcreteFileDeviceImporterFactory	98
Figura 116	Caso d'uso _G ExportDevice	99
Figura 117	FlaskExportDeviceController	100
Figura 118	ExportDeviceUseCase	100
Figura 119	ExportDeviceCommand	101
Figura 120	ExportDeviceService	101
Figura 121	FileDeviceExporterFactoryPort	102

Figura 122	FileDeviceExporterPort	102
Figura 123	FileDeviceExporter	103
Figura 124	ConcreteFileDeviceExporterFactory	103
Figura 125	CSVFileDeviceExporter, JSONFileDeviceExporter, XMLFileDeviceExporter	104
Figura 126	ExportReport	105
Figura 127	ExportReportController	105
Figura 128	GenerateReportUseCase	106
Figura 129	GenerateReportCommand	107
Figura 130	GenerateReportService	107
Figura 131	ReportGeneratorPort	108
Figura 132	PdfReportGenerator	109
Figura 133	Caso d'uso _G OpenEvaluationSession	110
Figura 134	EvaluationSessionController	111
Figura 135	OpenEvaluationSessionCommand	111
Figura 136	OpenEvaluationSessionUseCase	112
Figura 137	OpenEvaluationSessionService	112
Figura 138	SessionCoordinator	113
Figura 139	EvaluationSessionExistPort	114
Figura 140	CreateEvaluationSessionPort	114
Figura 141	Caso d'uso _G CommitEvaluationSession	115
Figura 142	CommitEvaluationSessionUseCase	116
Figura 143	CommitEvaluationSessionService	116
Figura 144	CommitEvaluationSessionCommand	117
Figura 145	Caso d'uso _G CloseEvaluationSession	118
Figura 146	CloseEvaluationSessionUseCase	118
Figura 147	CloseEvaluationSessionService	119
Figura 148	CloseEvaluationSessionCommand	119
Figura 149	DeleteSessionPort	120
Figura 150	Caso d'uso _G EvaluateDecisionNode	121
Figura 151	EvaluateDecisionNodeController	122
Figura 152	EvaluateDecisionNodeUseCase	122
Figura 153	EvaluateDecisionNodeCommand	123
Figura 154	EvaluateDecisionNodeService	123
Figura 155	Caso d'uso _G InsertJustification	124
Figura 156	FlaskInsertJustificationController	125
Figura 157	InsertJustificationUseCase	125
Figura 158	InsertJustificationCommand	126
Figura 159	InsertJustificationService	126
Figura 160	Shared - ValidationRule	128
Figura 161	Shared - FieldDefinition	129

Figura 162	Shared - UseFormModel	129
Figura 163	Shared - Form Fields Configurations	130
Figura 164	Componente - AsyncButton	131
Figura 165	Componente - BaseModal	133
Figura 166	Componente - FormField	134
Figura 167	Componente - Toast	134
Figura 168	Componente - FileDropZone	135
Figura 169	Component - DeviceForm	136
Figura 170	Component - AssetForm	137
Figura 171	AssetCreate	138
Figura 172	Widget - AssetCreateWidget	138
Figura 173	AssetDelete	140
Figura 174	Widget - AssetDeleteWidget	141
Figura 175	AssetEdit	142
Figura 176	Widget - AssetEditWidget	142
Figura 177	DeviceCreate	143
Figura 178	Widget - DeviceCreateWidget	144
Figura 179	DeviceEdit	145
Figura 180	Widget - DeviceEditWidget	146
Figura 181	Widget - DeviceExportWidget	147
Figura 182	DeviceImport	148
Figura 183	Widget - DeviceImportWidget	149
Figura 184	SessionClose	150
Figura 185	Widget - SessionCloseWidget	151
Figura 186	SessionCommitAndClose	152
Figura 187	Widget - SessionCommitAndCloseWidget	152
Figura 188	SessionCommit	153
Figura 189	Widget - SessionCommitWidget	154
Figura 190	SessionOpen	155
Figura 191	Widget - SessionOpenWidget	155
Figura 192	Logica di valutazione _G	157
Figura 193	TreeStructure	158
Figura 194	Node	159
Figura 195	DecisionNode	159
Figura 196	LeafNode	160
Figura 197	NodeDTO	161
Figura 198	NodeType	162
Figura 199	Leaf Node Value	162
Figura 200	EvaluationEngine	163
Figura 201	Dettaglio dell'EvaluationApiClient	163
Figura 202	LayoutEngineComplessivo	165

Figura 203	LayoutNodeDTO	165
Figura 204	EdgeDTO	166
Figura 205	HierarchyNode	167
Figura 206	LayoutConfig	168
Figura 207	LayoutResult	168
Figura 208	D3LayoutEngine	169
Figura 209	DecisionTreeComplessivo	170
Figura 210	DecisionTreeStore	170
Figura 211	RequirementEvaluationWidget	172
Figura 212	RequirementEvaluationWidget Dettaglio	172
Figura 213	TreeSideBar	174
Figura 214	DecisionTreeWidget	175
Figura 215	TreeCanvas	176
Figura 216	EvaluationBadge	178
Figura 217	JustificationForm	179
Figura 218	Point	180
Figura 219	StateConfig	180
Figura 220	UiDecisionNode	181
Figura 221	UiLeafNode	182
Figura 222	ViewBox	182
Figura 223	Diagramma dei requisiti obbligatori soddisfatti	190
Figura 224	Diagramma dei requisiti desiderabili soddisfatti	191
Figura 225	Diagramma dei requisiti opzionali soddisfatti	200

1) Introduzione

1.1) Scopo del documento

Il presente documento intende fornire una descrizione approfondita dell'architettura del prodotto software, delineando con chiarezza le componenti che lo costituiscono, le loro responsabilità e le interazioni reciproche all'interno del sistema. La Specifica Tecnica funge da riferimento principale per la fase di progettazione e codifica, garantendo coerenza con i requisiti analizzati e consolidando la maturità architetturale del prodotto.

Nello specifico, questo documento si propone di:

- **Definire l'architettura logica e i design pattern:** descrivere l'organizzazione dei moduli e le logiche di interazione, evidenziando come i pattern adottati garantiscano un codice modulare e testabile.
- **Motivare lo stack tecnologico:** giustificare la scelta delle tecnologie utilizzate in funzione della scalabilità del sistema e della qualità complessiva del prodotto finale.
- **Delineare l'architettura di deployment:** illustrare la distribuzione delle componenti negli ambienti di esecuzione e le strategie di rilascio adottate.
- **Fornire un framework per l'evoluzione del software :** stabilire le linee guida necessarie per facilitare la comprensione del sistema, agevolando futuri interventi di estensione_G e manutenzione_G.

1.2) Scopo del prodotto

Il prodotto si prefigge di automatizzare e digitalizzare il processo di verifica_G della conformità_G alla normativa EN 18031_G, sostituendo le attuali procedure manuali, spesso onerose e soggette a errore umano, con una soluzione software interattiva.

Il sistema è progettato per guidare l'utente attraverso l'esecuzione di alberi decisionali (Decision Tree_G) complessi, permettendo la valutazione_G sistematica dei requisiti di sicurezza per dispositivi IoT_G e la generazione di esiti certi (Pass_G, Fail_G, N.A.). L'integrazione di funzionalità per l'importazione_G di dati strutturati e una dashboard_G di monitoraggio in tempo reale mira a garantire la massima tracciabilità del processo, ottimizzando i tempi operativi della proponente e assicurando un elevato standard di affidabilità e manutenibilità dei risultati ottenuti.

1.3) Glossario

Per garantire precisione terminologica senza appesantire la lettura, in questo documento i termini tecnici presenti nel Glossario sono segnalati con un pedice "G", ad esempio:

termine_G: indica che il termine è definito nel Glossario e può essere consultato per chiarimenti.

Questo metodo consente di mantenere il testo chiaro e tecnicamente corretto, permettendo al lettore di riferirsi al Glossario solo quando necessario, senza interrompere il flusso della lettura.

1.4) Riferimenti

1.4.1) Riferimenti normativi

- [Capitolato d'appalto C1 - Automated EN18031 Compliance Verification di BlueWind Srl](#)
Ultima consultazione: 8 aprile 2026;
- [Regolamento del progetto](#)
Ultima consultazione: 8 aprile 2026;
- [Standard ISO/IEC/IEEE 12207:2017](#)
Ultima consultazione: 10 gennaio 2026;
- [Standard ISO/IEC 9126](#)
Ultima consultazione: 10 gennaio 2026;

1.4.2) Riferimenti informativi

- [Glossario del gruppo](#)
Ultima consultazione: 8 aprile 2026;
- [Software Engineering, Ian Sommerville](#)
Ultima consultazione: 10 gennaio 2026;
- [Documentazione del gruppo GroupRubberDuck](#)
Ultima consultazione: 8 aprile 2026;

2) Tecnologie

In questa sezione vengono descritte le tecnologie utilizzate nello sviluppo del progetto **Automated EN18031 Compliance Verification_G**.

Per ogni tecnologia è riportata la versione di riferimento e il relativo ruolo all'interno del progetto

2.1) Backend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
Python	3.12.X	Linguaggio interpretato scelto per la rapidità di sviluppo, l'alta leggibilità e il vasto ecosistema.
Framework Principale		
Flask	3.1.3	Micro-framework web utilizzato come infrastruttura principale di backend. Consente una gestione flessibile e leggera del routing per esporre le interfacce API. Buona documentazione e possibilità di confronto tecnico con la proponente.
Dipendenze principali		
Waitress	3.0.2	Server WSGI leggero. Gestisce la concorrenza delle richieste in modo affidabile.
Fpdf2	2.8.7	Libreria per la generazione dinamica di documenti e reportistica in formato PDF _G direttamente lato server.
Pymongo	4.17.0	Driver ufficiale per l'integrazione con MongoDB.
Pydantic	2.13.3	Utilizzato per la validazione _G rigida e type-safe dei dati in ingresso e delle variabili d'ambiente di sistema.
Python-dotenv	1.2.2	Gestione semplificata delle configurazioni e delle variabili d'ambiente.

Nome	Versione	Descrizione
Jinja2	3.1.2	Template engine HTML con un'ottima integrazione con Flask.
Werkzeug	3.1.8	Server di sviluppo locale.
Watchdog	2.3.X	Permette l'hot reloading durante lo sviluppo locale, passando le modifiche al server di sviluppo senza necessità di riavvio e preservando lo stato dell'applicazione.
Dynamic Testing		
Pytest	8.4.2	Framework di testing adottato per la sua sintassi concisa. Utilizzato per automatizzare i «Sanity Tests» e validare l'integrazione di sistema alla radice del progetto.
Pytest-cov	7.1.0	Estensione _G di Pytest per la misurazione della Code Coverage.
Pytest-html	4.2.0	Plugin per generare report visivi dei test in formato HTML self-contained, utili per la documentazione delle run di sistema.
Pytest-json _G -report	1.5.0	Estensione _G per esportare i risultati dei test in formato JSON _G , facilitando l'integrazione con pipeline CI/CD o tool di analisi custom.
Pytest-mock	3.15.1	Wrapper per la libreria “unittest.mock” nativa di Python. Semplifica la creazione di mock, stub e spy durante i test unitari.
Mongomock	4.1.2	Libreria per effettuare il mocking del database MongoDB. Permette di eseguire test di integrazione _G in memoria senza la necessità di sollevare un'istanza reale del database.
Static Testing		
Ruff _G	0.15.12	Linter e formatter estremamente veloce (scritto in Rust). Impone e garantisce standard di

Nome	Versione	Descrizione
		codice puliti e uniformi senza rallentare lo sviluppo.
Mypy _G	1.20.2	Analizzatore statico basato sul Type Hinting. Previene i bug e gli errori di tipo a tempo di sviluppo prima ancora dell'esecuzione
Radon	6.0.0	Strumento di analisi statica del codice utilizzato per calcolare metriche del software come la complessità ciclomatica e l'indice di manutenibilità.
Sviluppo		
Poetry _G	2.3.4	Gestore moderno delle dipendenze e degli ambienti virtuali (.venv). Garantisce build riproducibili tra i vari sviluppatori tramite il file di blocco (poetry _G .lock).

Tabella 1: Backend

2.2) Frontend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
JavaScript	ES2020	Linguaggio client-side universale.
Framework Principale		
Vue.js	3.5.32	Framework progressivo con una curva di apprendimento morbida e un'ottima implementazione della reattività. L'adozione della Composition API permette di scrivere logica modulare e riutilizzabile.
Dipendenze principali		
pinia	3.0.4	Gestore dello stato ufficiale e leggero per Vue.
tailwindcss		Framework CSS utility-first per un'agile stilizzazione dell'interfaccia.
D3	7.9.0	Libreria usata per i calcoli grafici.
Dynamic Testing		
vitest	4.1.X	Framework di unit testing veloce e nativo per Vite. Utilizzato per validare la logica isolata dei componenti.
vue-test-utils	2.4.X	Libreria ufficiale affiancata a Vitest per montare e simulare le interazioni dell'utente sulle singole isole Vue durante i test.
JSDOM	29.1.X	Permette a Vitest di simulare il DOM di un browser direttamente all'interno del terminale Node.js, rendendo i test leggeri e veloci.
Vitest Coverage V8	4.1.X	Motore per la misurazione della Code Coverage. Fornisce metriche precise su linee, branch e funzioni testate.

Nome	Versione	Descrizione
Sviluppo		
vite	8.0.X	Tool di build per minificare e raggruppare i file JS/CSS corrispondenti ai componenti Vue, ottimizzandoli per la produzione.

Tabella 2: Frontend

2.3) Persistenza dei dati

MongoDB è un database NoSQL orientato ai documenti. A differenza dei classici database relazionali (che usano tabelle e colonne rigide), archivia le informazioni in documenti flessibili molto simili al formato JSON_G.

La versione utilizzata è la 7.XX

Permette di aggiungere o rimuovere campi dai dati senza riscrivere l'intera organizzazione delle informazioni del database.

Nel progetto **Automated EN18031 Compliance Verification_G** viene utilizzato come sistema per la persistenza dei dati, in particolare per la rappresentazione dei dispositivi sottoposti alle valutazioni e per la rappresentazione del modello di standard tramite configurazioni esterne.

Il suo utilizzo permette di rappresentare facilmente strutture dati annidate, tramite una sintassi JSON_G.

Evita la gestione di join complessi tipici di un database relazionale, offre una maggiore sicurezza rispetto alla gestione manuale di file di rappresentazione interni.

È facilmente integrabile nel sistema backend

3) Architettura di Sistema

3.1) Premessa: approccio di lavoro usato

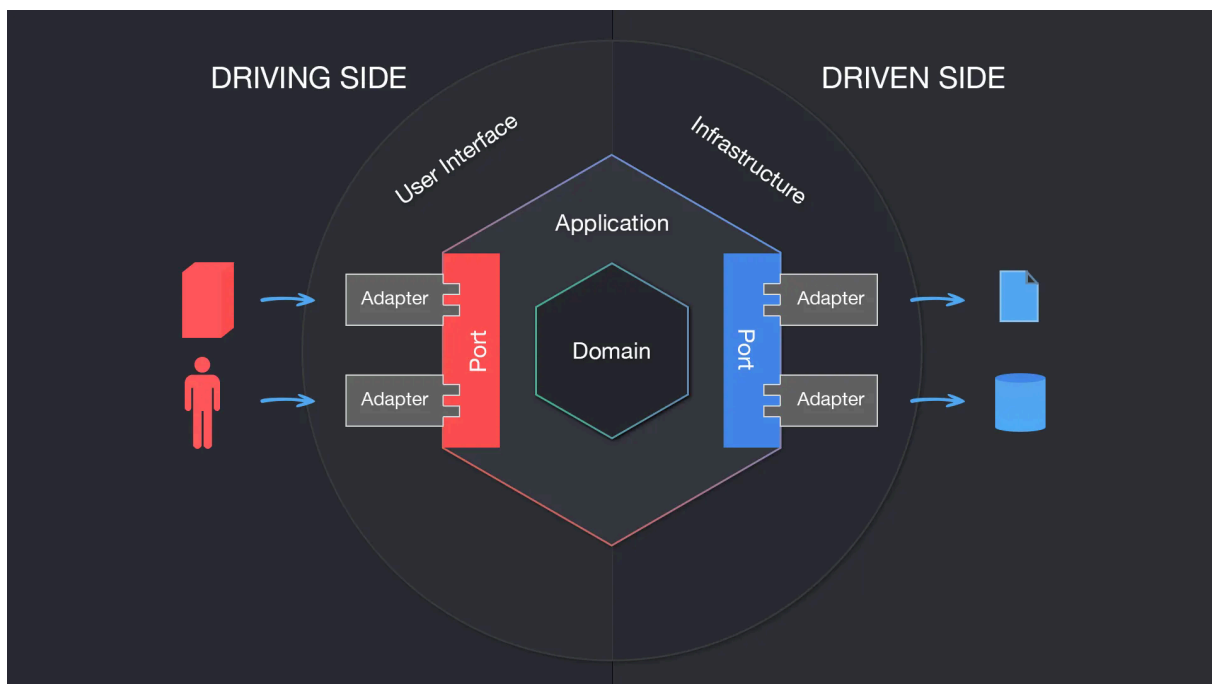
L'obiettivo della fase di progettazione è definire un'architettura software in grado di soddisfare pienamente i requisiti di sistema, operando secondo criteri di sostenibilità e ottimizzazione delle risorse (design-to-cost).

La metodologia adottata per la definizione dell'architettura è di tipo Top-Down. Il sistema viene inizialmente concepito nella sua interezza come un'unica entità astratta e, attraverso un processo di esplorazione funzionale, viene progressivamente decomposto in sotto-sistemi logici più piccoli e gestibili.

3.2) Architettura Logica

3.2.1) Stile architetturale

L'architettura adottata per realizzare il prodotto è l'architettura esagonale. Questo tipo di architettura ha come obiettivo primario isolare la logica di business dal resto. Questo approccio garantisce un'elevata testabilità del sistema e rende il nucleo del software completamente agnostico rispetto ai dettagli implementativi (framework, database o interfacce). Il sistema è strutturato in livelli concentrici, organizzati come segue:



- **Domain (Core):** Rappresenta il nucleo centrale dell'applicazione. Contiene la logica di business pura ed è rigorosamente privo di dipendenze esterne.
- **Services:** Definiscono i casi d'uso dell'applicazione (Application Services) e fungono da orchestratori. Implementano le Inbound Ports per gestire le richieste in ingresso, coordinano gli oggetti del Domain affinché eseguano la logica di business pura

e utilizzano le Outbound Ports per delegare all'esterno operazioni infrastrutturali (come il salvataggio su database).

- **Ports:** Costituiscono il punto di connessione tra il nucleo e il mondo esterno, permettendo una comunicazione strutturata senza creare accoppiamento. Si suddividono in Inbound Ports (definiscono i casi d'uso accessibili dall'esterno) e Outbound Ports (permettono al nucleo di definire interfacce per interagire con i servizi esterni).
- **Adapters:** Rappresentano lo strato più esterno e fungono da traduttori tra le tecnologie specifiche e il nucleo. Si suddividono in Driving/Input Adapters (adattatori in entrata, che guidano l'applicazione ricevendo input e invocando le Inbound Ports) e Driven/Output Adapters (adattatori in uscita, che vengono guidati dall'applicazione per interagire con l'infrastruttura esterna tramite le Outbound Ports).

3.2.2) Motivazioni della scelta architetturale

Le motivazioni per le quali questa architettura è stata scelta sono le seguenti:

- **Testabilità:** l'assenza di dipendenze esterne nel livello di Dominio permette di scrivere Unit Test in puro Python in modo semplice e veloce. È possibile testare la logica di business in modo isolato, iniettando mock o stub per simulare le interfacce, senza la necessità di avviare il server Flask o connettersi al database MongoDB.
- **Divisione del nucleo:** il nucleo dell'applicazione è totalmente disaccoppiato dall'infrastruttura. Essendo ignaro del livello di presentazione (l'interfaccia utente in HTML, CSS e Vue.js) e del livello di persistenza (il database NoSQL MongoDB), il sistema garantisce un'alta tolleranza ai cambiamenti. È possibile sostituire o aggiornare le tecnologie esterne senza dover alterare la logica di business.
- **Sviluppo parallelo:** la rigorosa definizione dei contratti di comunicazione (le Porte) nelle fasi iniziali di progettazione permette al team di procedere in parallelo. Frontend, backend e integrazione con il database possono essere sviluppati simultaneamente da membri diversi del gruppo, riducendo i colli di bottiglia.
- **Inversione delle dipendenze:** l'architettura applica rigorosamente questo principio poiché è la parte interna dell'esagono a dettare i contratti (le interfacce) a cui i servizi esterni devono sottostare, e non viceversa. In questo modo, il nucleo dell'applicazione mantiene il controllo assoluto del flusso ed è protetto dai potenziali effetti collaterali (side effects) derivanti dall'infrastruttura.

3.2.3) Limiti dell'architettura

Gli aspetti negativi di questa scelta sono:

- **Ripida curva di apprendimento:** l'architettura esagonale richiede una profonda comprensione dei principi SOLID, in particolare la Dependency Injection e usare questo pattern per la prima volta richiede profondo studio. Questo rischio sarà mitigato da una precisa fase di progettazione che semplificherà la codifica.
- **Elevato overhead iniziale:** La ferrea separazione dei livelli impone la stesura di un'abbondante quantità di codice infrastrutturale (boilerplate). Risulta necessario

definire contratti astratti (Porte), implementazioni concrete (Adattatori) e orchestratori (Servizi), allungando i tempi di sviluppo nelle prime fasi del progetto. Il gruppo ha tuttavia accettato questo costo iniziale, ritenendolo un investimento necessario a fronte del drastico abbattimento dei futuri costi di manutenzione_G e della massima testabilità garantita al nucleo applicativo.

3.2.4) Diagramma dei package

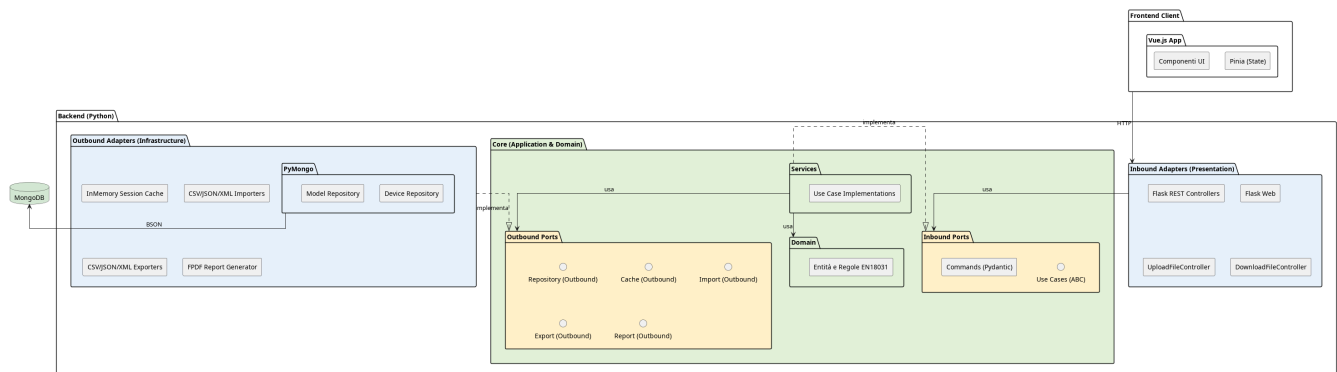


Figura 1: Diagramma dei package

Il diagramma illustra l'organizzazione logica del sistema Automated EN18031 Compliance Verification_G, fondata sull'architettura esagonale. Il sistema è strutturalmente ripartito in un livello di presentazione esterno (Frontend Client), sviluppato tramite il framework Vue.js, e un nucleo applicativo (Backend), implementato in Python.

Per garantire una rigorosa separazione delle responsabilità e il pieno rispetto del principio di Inversione delle Dipendenze, il Backend si articola nei seguenti livelli tipici dell'architettura esagonale:

1. **Adapters:** Costituisce il confine tecnologico del sistema, responsabile della comunicazione con l'esterno. Si divide in:
 - **Inbound Adapters** (Driving Adapters): Agiscono da punto di ingresso per il sistema. Intercettano gli input esterni (es. le richieste HTTP/REST inviate dal Frontend), ne effettuano il parsing e traducono tali direttive in invocazioni dirette verso le Inbound Ports, senza mai accedere alla logica di business.
 - **Outbound Adapters** (Driven Adapters): Rappresentano le tecnologie concrete di uscita (es. il driver MongoDB o le librerie di generazione PDF_G). Il loro compito è implementare materialmente le interfacce definite nel livello Outbound Ports, traducendo i comandi astratti in operazioni specifiche per l'infrastruttura.
2. **Core** (Nucleo Applicativo): È il cuore del software, totalmente agnostico e privo di dipendenze verso framework web o database. Al suo interno è stratificato in:
 - **Ports:** Funge da barriera di isolamento. Definisce le interfacce in puro Python, disaccoppiando la tecnologia dall'applicazione:
 - **Inbound Ports** (Primary Ports): Definiscono le interfacce (i contratti) relative ai Casi d'Uso (Use Cases) che il sistema offre. Specificano «cosa» il sistema è

in grado di fare, permettendo agli Inbound Adapters di invocare le operazioni senza dover conoscere l'implementazione sottostante.

- **Outbound Ports** (Secondary Ports): Definiscono i contratti astratti di cui il dominio ha bisogno per comunicare con l'esterno (ad esempio, le interfacce del Repository Pattern per l'accesso ai dati). Queste porte vengono usate dal livello Services e implementate dagli Outbound Adapters.
- **Services**: Costituiscono il livello di orchestrazione. Implementano concretamente i Casi d'Uso definiti nelle Inbound Ports, coordinano il flusso di esecuzione richiamando le entità di dominio, e si avvalgono delle Outbound Ports per la persistenza dei dati.
- **Domain**: Incapsula le regole di business pure e la logica della norma EN18031. Contiene la modellazione formale delle entità e la validazione strutturale dei dati (supportata da Pydantic), operando in totale e completo isolamento.

3.3) Architettura di Deployment

Il sistema adotta un'architettura a Monolite Modulare.

L'intera applicazione è concepita, sviluppata e rilasciata come un'unica unità eseguibile.

3.3.1) Motivazioni della scelta

Questa scelta è coerente con lo sviluppo di una web app locale.

In questo contesto un'architettura a monolite modulare, rispetto all'architettura a microservizi, offre i seguenti vantaggi:

- **Semplicità di Deployment**: tutte le funzionalità sono contenute nella singola unità operativa;
- **Latenze ridotte**: lo scambio di informazioni avviene completamente in locale.

Non si è optato per un'architettura monolitica tradizionale in cui spesso il codice è fortemente accoppiato.

La rigorosa divisione in moduli logici isolati permette di coniugare la semplicità di rilascio dell'applicazione come singola unità con alcuni dei vantaggi organizzativi tipici delle architetture a microservizi:

- **Separazione delle responsabilità** tra i vari ambiti del dominio.
- **Semplicità di sviluppo e manutenibilità** a lungo termine.
- **Facilità di testing**, permettendo di collaudare i singoli moduli in totale isolamento.

3.3.2) Realizzazione Pratica e Topologia

Per la distribuzione viene usato Docker_G come tool di containerizzazione e Docker_G Compose come tool di orchestrazione.

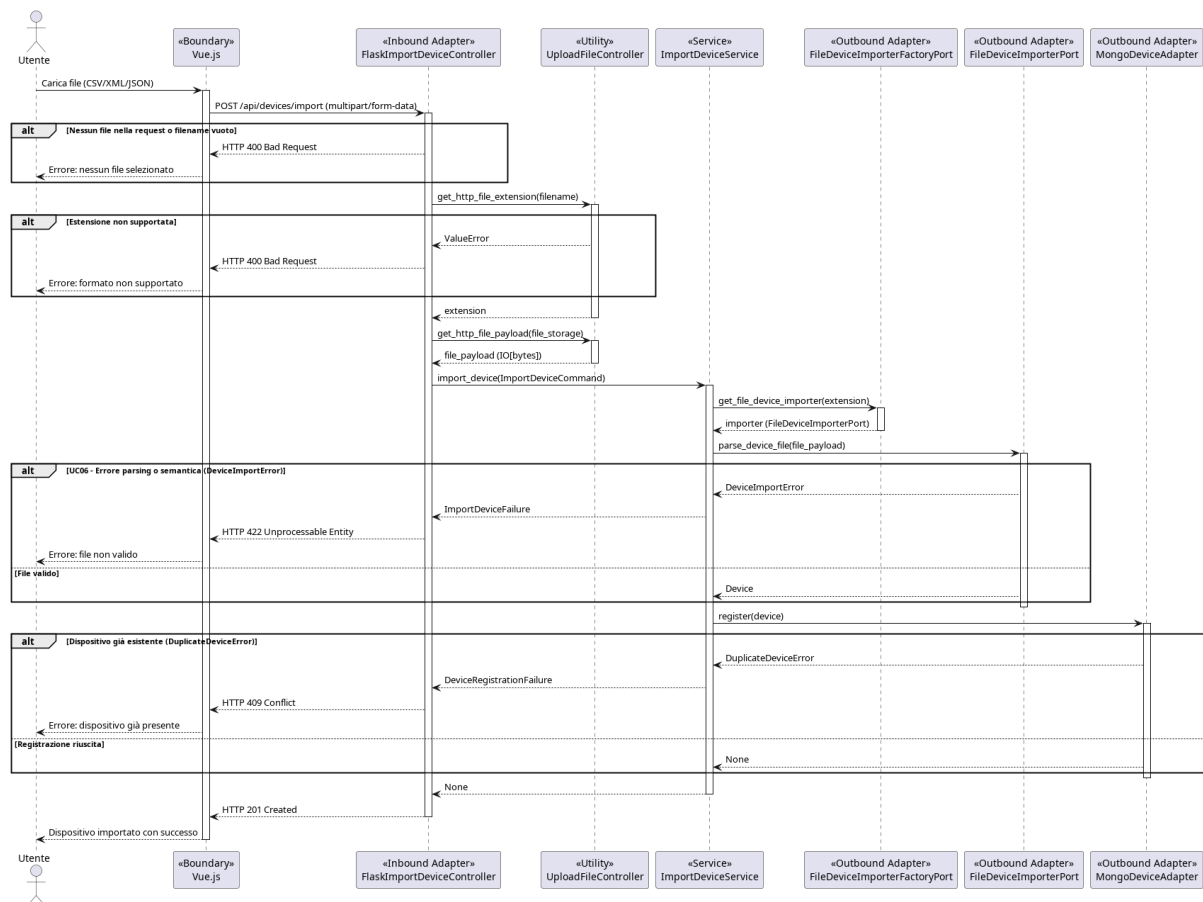
1. Nodi di Esecuzione:

- **Browser dell'utente**, l'interfaccia_G utente è realizzata tramite browser;
 - **Server Backend**, contiene la core logic dell'applicazione e risponde alle richieste dell'utente;
 - **Database MongoDB**, sistema di permanenza;
2. **Packaging e isolamento**. L'infrastruttura è orchestrata tramite Docker_G Compose e prevede i seguenti container:
- **Web-app**
Contiene l'ambiente Python, il framework Flask, gli asset_G statici del frontend e altre dipendenze¹
 - **MongoDB**
Basato sull'immagine ufficiale di MongoDB. Per prevenire la perdita dei dati questo container è agganciato a un Docker_G Volume locale.
3. **Comunicazioni e protocolli di rete**
- I nodi comunicano attraverso i seguenti canali e protocolli:
- **Tra Browser e Backend:**
La comunicazione avviene in chiaro tramite protocollo HTTP/REST
 - **Tra Backend e Database:** Flask comunica con MongoDB utilizzando il protocollo binario proprietario.
- La porta del database non è esposta verso il browser né verso l'esterno, garantendo l'integrità dei dati.
4. **Gestione del Traffico Web**
- Tra il browser dell'utente e l'applicazione Python è interposto un server **WSGI** .
- Questo strato intermedio garantisce stabilità, gestisce la concorrenza delle richieste e le instrada in modo sicuro verso la logica applicativa.

3.4) Diagrammi di sequenza

La seguente sezione illustra il comportamento dinamico del sistema tramite diagrammi di sequenza, focalizzandosi sui casi d'uso di maggiore interesse. Questi modelli descrivono l'ordine cronologico dei messaggi scambiati tra gli attori esterni, i componenti infrastrutturali e il nucleo applicativo.

¹Vedi sezione [Tecnologie - Backend](#)

3.4.1) UC05 - Importazione_G dispositivo_G

Il caso d'uso_G consente all'utente di caricare un file (CSV_G, JSON_G o XML_G) contenente i dati di un dispositivo_G e di registrarlo nel sistema. Il flusso coinvolge tre componenti principali: il controller HTTP, il servizio applicativo e il repository.

Flusso Principale

1. Ricezione della richiesta HTTP:

il FlaskImportDeviceController riceve una richiesta POST /devices/import. Prima di delegare al servizio, verifica_G la presenza del file nella richiesta e l'estensione_G dello stesso. Se il file è assente o l'estensione_G non è supportata, restituisce immediatamente HTTP 400 senza propagare la richiesta al layer applicativo.

2. Costruzione del Command:

Se la validazione_G HTTP supera il controllo, il controller costruisce un ImportDeviceCommand contenente il percorso del file e il formato dichiarato, e lo passa all'ImportDeviceService.

3. Parsing del file:

il servizio delega il parsing al FileDeviceImporterPort tramite il metodo parse_device_file. Il port è implementato dall'adapter corretto in base al formato del file (CSV_G, JSON_G, XML_G). Il parser legge il file e, se il formato è valido e i dati sono semanticamente corretti, restituisce direttamente un oggetto Device del dominio.

4. Registrazione del dispositivo_G:

il Device restituito dal parser viene passato al DeviceRepositoryPort tramite il metodo register. Il repository persiste il dispositivo_G nel database.

5. Risposta di successo:

Al termine del flusso, il controller restituisce HTTP 201 Created.

Flussi alternativi

- **[File non valido o formato non supportato]**

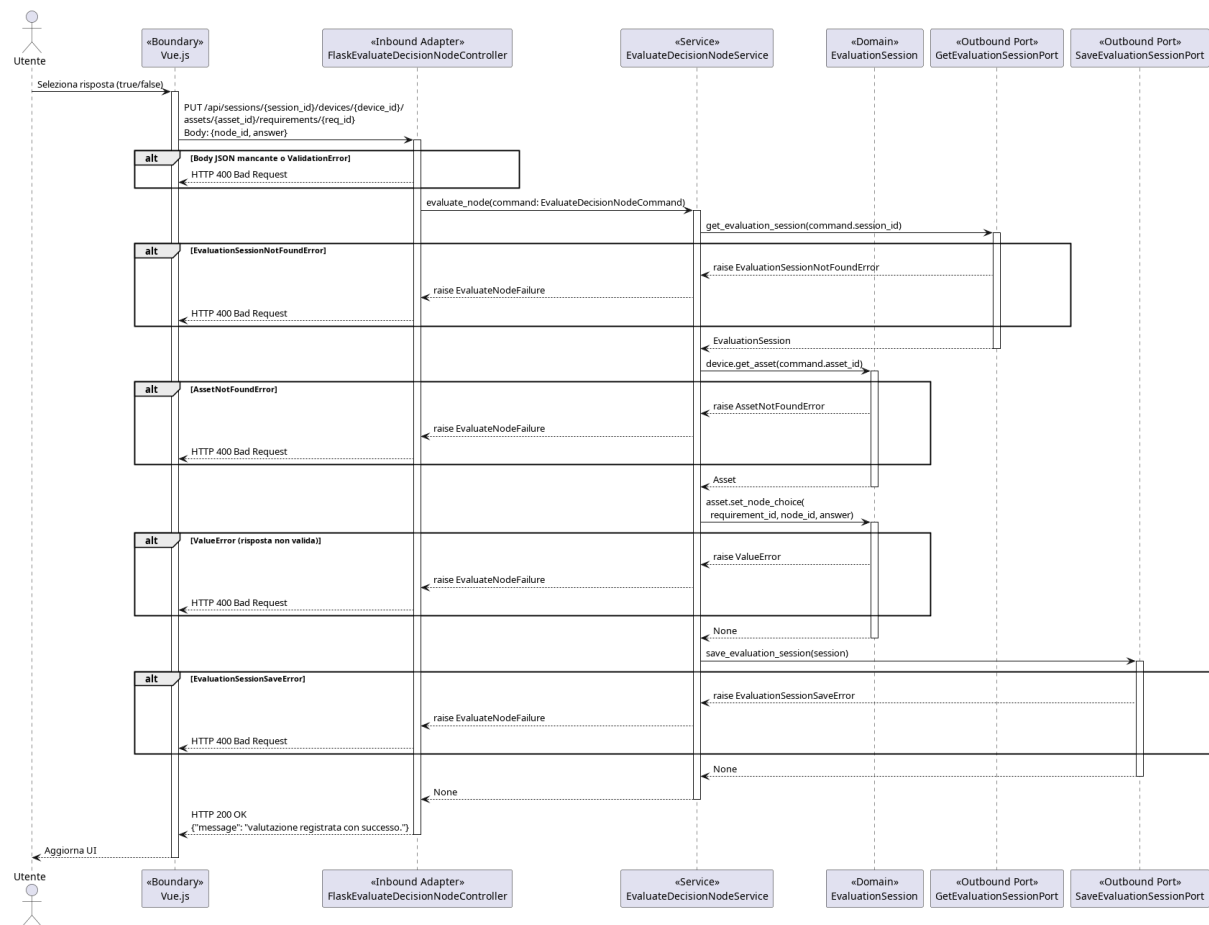
Se il file è assente nella richiesta o l'estensione_G non rientra tra quelle accettate (csv_G, json_G, xml_G), il FlaskImportDeviceController restituisce HTTP 400 Bad Request. Il servizio non viene mai invocato.

- **[Errore di parsing o dati non validi]**

Se il file non è leggibile, ha una struttura malformata, o contiene dati semanticamente non validi (es. campo obbligatorio mancante, tipo non riconosciuto), il FileDeviceImporterPort solleva una DeviceImportError. Il servizio cattura l'eccezione e la traduce in un ImportErrorFailure, che il controller mappa in HTTP 422 Unprocessable Entity.

- **[Dispositivo_G già registrato]**

Se un dispositivo_G con lo stesso identificatore è già presente nel repository, register() solleva una DuplicateDeviceError. Il servizio la traduce in un DeviceRegistrationFailure, che il controller mappa in HTTP 409 Conflict.

3.4.2) UC26, UC27 - Valutazione_G di un nodo_G e transizione di stato

Il caso d'uso_G consente all'utente di registrare la risposta a un nodo_G decisionale nell'albero di valutazione_G di un requisito_G. Il flusso coinvolge il controller HTTP, il servizio applicativo, due port distinte per la sessione e gli oggetti di dominio EvaluationSession e Asset_G.

Flusso principale:

1. Ricezione della richiesta HTTP:

Il FlaskEvaluateDecisionNodeController riceve una richiesta PUT sull'endpoint `/api/sessions/{session_id}/devices/{device_id}/assets/{asset_id}/requirements/{req_id}`. Prima di procedere, verifica_G la presenza del body JSON_G. Se assente, restituisce immediatamente HTTP 400 senza invocare il service.

2. Costruzione del Command:

Il controller istanzia un EvaluateDecisionNodeCommand (modello Pydantic) con i parametri di path e i campi node_id e answer estratti dal body. Se la validazione_G Pydantic fallisce per campi mancanti o tipo non valido, restituisce HTTP 400 senza propagare la richiesta al layer applicativo.

3. Recupero della sessione:

L'EvaluateDecisionNodeService riceve il command e delega il recupero della ses-

sione alla *GetEvaluationSessionPort* tramite *get_evaluation_session(session_id)*. La port restituisce l'*EvaluationSession* completa, incluso il device e i suoi *asset_G*.

4. Navigazione al dominio:

Il service naviga l'aggregato di dominio per recuperare l'*asset_G* su cui operare tramite *session.device.get_asset(asset_id)*, ottenendo l'oggetto *Asset_G* corrispondente.

5. Registrazione della scelta:

Il service invoca *asset_G.set_node_choice(requirement_id, node_id, answer)* sull'oggetto *Asset_G*. Questa chiamata registra in memoria la risposta dell'utente per il nodo_G specificato all'interno del requisito_G.

6. Persistenza della sessione:

Il service passa la *EvaluationSession* aggiornata alla *SaveEvaluationSessionPort* tramite *save_evaluation_session(session)*. La responsabilità di persistere l'intera sessione è delegata alla port: il service non conosce il meccanismo di storage.

7. Risposta di successo:

Il service restituisce *None*. Il controller risponde con HTTP 200 OK e il messaggio {«message»: «valutazione_G registrata con successo.»}.

Flussi alternativi:

- **[Body JSON_G mancante o non valido]**

Se il body della richiesta è assente o *node_id/answer* non rispettano i tipi attesi da *EvaluateDecisionNodeCommand*, il controller restituisce HTTP 400 Bad Request. Il service non viene mai invocato.

- **[Sessione non trovata]**

Se la *GetEvaluationSessionPort* non trova una sessione con l'identificatore fornito, solleva *EvaluationSessionNotFoundError*. Il service cattura l'eccezione e la traduce in *EvaluateNodeFailure*. Il controller restituisce HTTP 400 Bad Request.

- **[Asset_G non trovato]**

Se il device nella sessione non contiene un *asset_G* con l'identificatore fornito, il dominio solleva *AssetNotFoundError*. Il service la traduce in *EvaluateNodeFailure*. Il controller restituisce HTTP 400 Bad Request.

- **[Risposta non valida per il nodo_G]**

Se *set_node_choice* riceve un valore non ammesso per il nodo_G specificato, il dominio solleva *ValueError*. Il service la traduce in *EvaluateNodeFailure*. Il controller restituisce HTTP 400 Bad Request.

- **[Errore di salvataggio]**

Se la *SaveEvaluationSessionPort* non riesce a persistere la sessione, solleva *EvaluationSessionSaveError*. Il service la traduce in *EvaluateNodeFailure*. Il controller restituisce HTTP 400 Bad Request.

3.5) Schema dati

In questa sezione viene illustrata l'organizzazione logica dei dati all'interno del database MongoDB.

3.5.1) Scelte Architettureali e Formalismo

A differenza dei sistemi relazionali basati su tabelle, MongoDB è un database **NoSQL orientato ai documenti** con struttura *schema-flexible*. Questa scelta architetturale è ideale per gestire strutture dati gerarchiche e dinamiche (come gli alberi decisionali), evitando il sovraccarico computazionale derivante da query di JOIN complesse.

Per la modellazione visiva non si utilizzano diagrammi Entity-Relationship, in quanto non idonei ai database documentali.

Si adotta invece il formalismo di Hackolade (consultabile al link: https://hackolade.com/schemas/Yelp_Challenge_dataset_documentation.html), che descrive chiaramente la gerarchia dei campi, i tipi di dato e le nidificazioni.

Nota sulla validazione_G dei dati: Benché i documenti vengano illustrati con tipi logici (incluso il tipo `enum` per chiarezza espositiva), la validazione_G strutturale di base sarà garantita dall'adapter predisposto alle comunicazioni col database.

Le due *collection* principali del database sono:

- **Collection «Compliance Standards»:** Catalogo di sola lettura dei template. Contiene le definizioni degli standard, le versioni e la logica degli alberi decisionali.
- **Collection «Devices»:** Registro dinamico dei device censiti. Memorizza lo stato delle valutazioni e il riferimento puntuale al modello normativo_G applicato.

3.5.2) Schema Dati Device

Il documento *Device* è l'entità centrale del sistema. La sua struttura gerarchica rispecchia la scomposizione fisica e funzionale dell'oggetto analizzato in tre blocchi logici:

1. **Header Identificativo (Root):** Informazioni anagrafiche, tecniche e puntatore al modello normativo_G.
2. **Lista Asset_G (Nodi Figli):** Array di sub-documenti che modellano le singole componenti o funzionalità del dispositivo_G.
3. **Registro Valutazioni (Nodi Terminali):** Dizionario delle risposte fornite per i requisiti applicabili a ciascun asset_G.

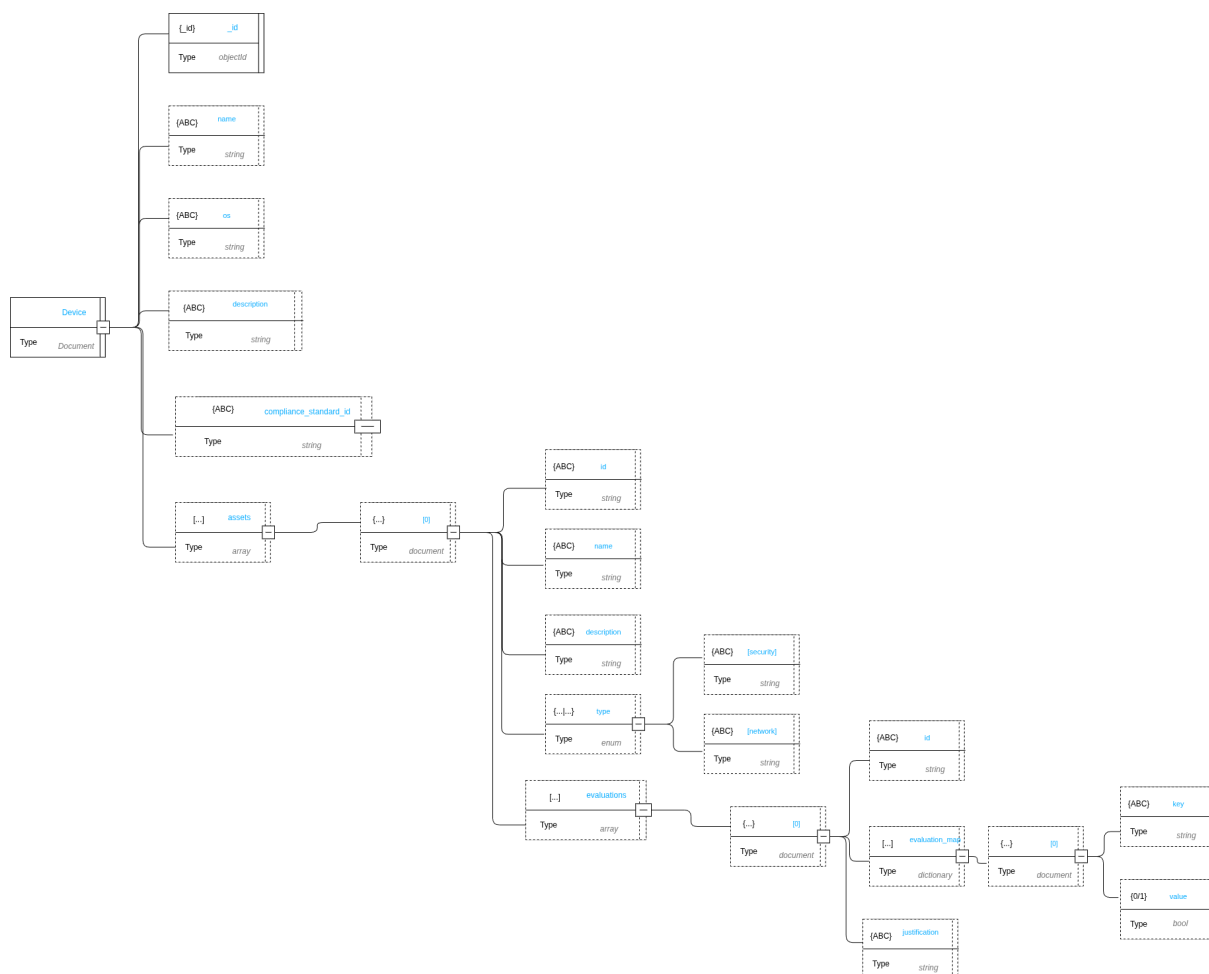


Figura 2: Schema logico: Documento Device

Root --- Device

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco generato dal database.
<code>name</code>	string	Nome assegnato dall'utente al dispositivo _G . Valore obbligatorio.
<code>os</code>	string	Sistema operativo o firmware _G installato.
<code>description</code>	string	Testo libero descrittivo del dispositivo _G .
<code>compliance_standard_id</code>	string	Id del modello di riferimento
<code>assets</code>	array	Elenco degli asset _G associati al dispositivo _G . Può essere inizializzato come array vuoto <code>[]</code> .

Tabella 3: Schema dati: Root — Device

Sub-documento --- assets[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	ID applicativo per l'identificazione univoca dell'asset _G nel dispositivo _G .
<code>name</code>	string	Nome descrittivo dell'asset _G .
<code>description</code>	string	Testo libero descrittivo dell'asset _G .
<code>type</code>	enum	Categoria logica dell'asset _G . Valori ammessi: <code>security</code> , <code>network</code> .
<code>evaluations</code>	array	Elenco delle valutazioni fornite per questo asset _G . Può essere vuoto <code>[]</code> .

Tabella 4: Schema dati: Sub-documento — assets[N]

Sub-documento --- evaluations[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Riferimento al codice del requisito _G presente nel modello.
<code>evaluation_map</code>	dictionary	Mappa chiave-valore delle risposte. La chiave (string) corrisponde al codice del nodo _G decisionale, il valore <code>bool</code> rappresenta la risposta associata.
<code>justification</code>	string	Testo descrittivo della motivazione. Obbligatorio a livello applicativo in caso di esito <code>FAIL_G</code> o <code>NA</code> .

Tabella 5: Schema dati: Sub-documento — evaluations[N]

3.5.3) Schema ComplianceStandard

La collection «Modelli» contiene la definizione strutturale degli standard normativi. A differenza del Dispositivo_G, orientato allo stato, il Modello descrive la logica di valutazione_G. Si suddivide in:

- **Metadati:** Versionamento e identificazione.
- **Albero dei Requisiti:** Struttura che definisce il testo della norma e la logica decisionale per guidare l'utente alla valutazione_G finale.

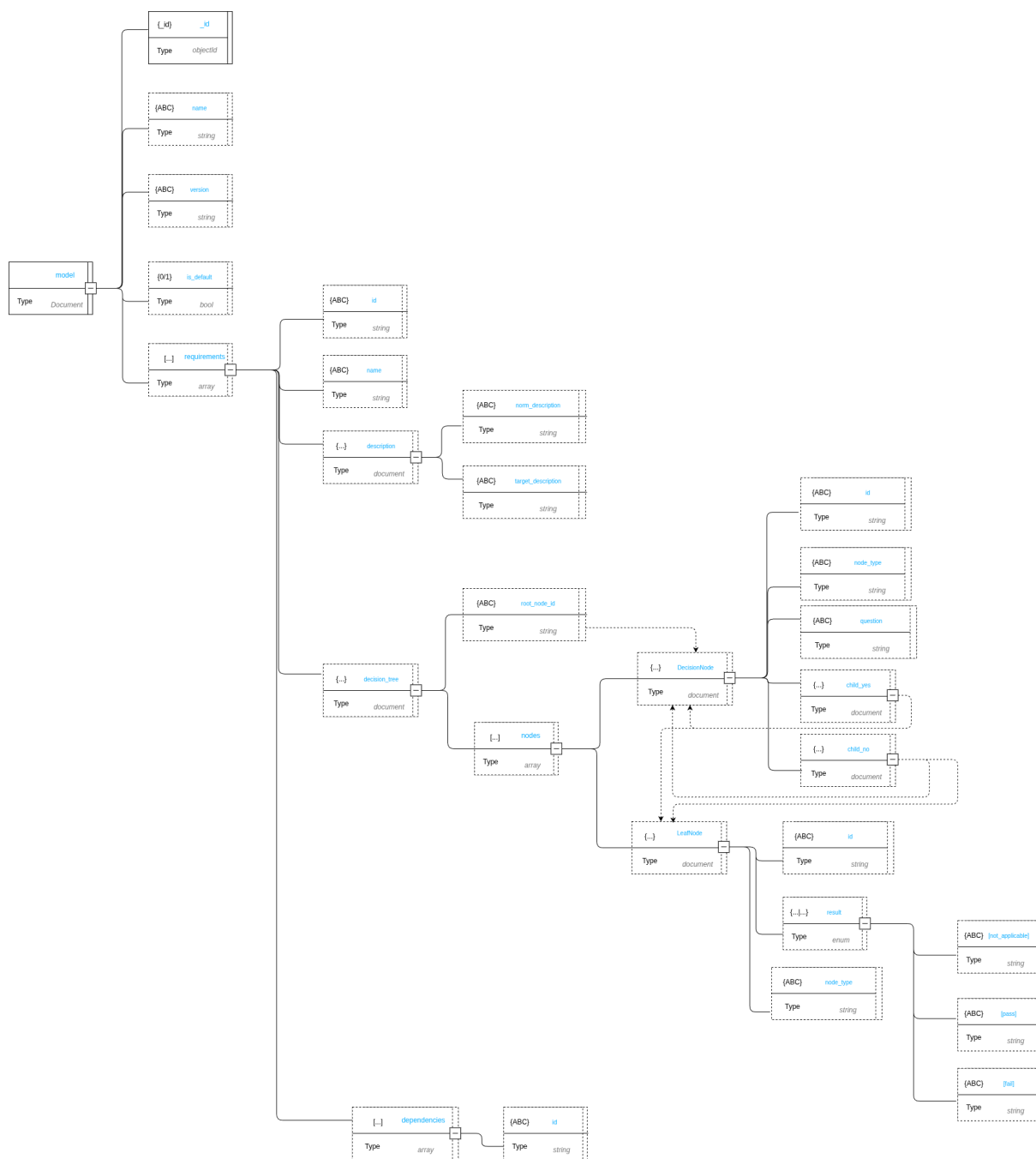


Figura 3: Schema logico: ComplianceStandard

Root --- ComplianceStandard

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco del modello.
<code>name</code>	string	Nome dello standard normativo _G .
<code>version</code>	string	Identificativo della versione del modello.
<code>is_default</code>	bool	Flag che indica se il modello è quello predefinito per nuove valutazioni.
<code>requirements</code>	array	Elenco dei requisiti che compongono la normativa.

Tabella 6: Schema dati: Root — ComplianceStandard

Sub-documento --- requirements[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Identificativo testuale univoco del requisito _G .
<code>name</code>	string	Titolo sintetico del requisito _G .
<code>description</code>	document	Testo descrittivi del contesto normativo (<code>norm_description</code> , <code>target_description</code>).
<code>decision_tree</code>	document	Albero decisionale per questo requisito _G .
<code>dependencies</code>	array	Elenco di codici di requisiti propedeutici da soddisfare prima della valutazione _G .

Tabella 7: Schema dati: Sub-documento — requirements[N]

Sub-documento --- DecisionNode

Campo	Tipo BSON	Descrizione
<code>root_node</code>	string	Codice identificativo del nodo _G logico iniziale.
<code>nodes</code>	array	Lista dei nodi che compongono il decision tree _G . Il <code>node_type</code> funge da discriminante tra nodi di decisione e nodi foglia. Valori ammessi da <code>node_type</code> : <code>decision_node</code> <code>leaf_node</code> .

Tabella 8: Schema dati: Sub-documento — DecisionTree

Sub-documento --- DecisionNode

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Codice identificativo del nodo _G logico.
<code>node_type</code>	enum	<code>decision_node</code> , identifica un nodo _G di decisione.
<code>question</code>	string	Testo della domanda posta all'utente.
<code>child_yes</code>	document	Riferimento al nodo _G successivo sul ramo associato alla risposta affermativa.
<code>child_no</code>	document	Riferimento al nodo _G successivo sul ramo associato alla risposta negativa.

Tabella 9: Schema dati: Sub-documento — DecisionNode

Sub-documento --- LeafNode

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Codice identificativo del nodo _G foglia.
<code>node_type</code>	enum	<code>leaf_node</code> , identifica un nodo _G foglia.
<code>result</code>	enum	Stato terminale del percorso logico. Valori ammessi: <code>PASS_G</code> , <code>FAIL_G</code> , <code>NA</code> .

Tabella 10: Schema dati: Sub-documento — LeafNode

3.5.4) Gestione degli Esiti e Storicità

La separazione netta tra Modello e Dispositivo_G ottimizza la memorizzazione e la gestione dei dati portando i seguenti vantaggi:

- **Zero Ridondanza:** Il documento Dispositivo_G salva unicamente il dizionario delle risposte fornite, mentre il documento Modello conserva esclusivamente la struttura statica dell'albero. Questo evita di duplicare l'intera gerarchia normativa per ogni dispositivo_G.
- **Disaccoppiamento:** Viene lasciato alla logica applicativa il compito di «fondere» i dati, compilando a runtime il template del Modello con le risposte storicizzate nel Dispositivo_G.
- **Versionamento:** La collection Modelli conserva tutte le versioni rilasciate di uno standard. Poiché il Dispositivo_G punta esplicitamente a una singola e specifica versione, lo storico delle certificazioni passate rimane inalterato e coerente anche a fronte di futuri aggiornamenti normativi.

4) Design Pattern

4.1) Principi di Design

4.1.1) Dependency Injection

Descrizione del pattern

La dependency injection prevede che le dipendenze necessarie a una classe o a un modulo non vengano create internamente, ma fornite dall'esterno.

Generalmente questo avviene alla costruzione, tramite un setter o come parametro di un metodo.

Motivazioni dell'utilizzo del pattern

La dependency injection permette di realizzare la Dependency Inversion e garantisce che le classi di dominio non dipendano mai da servizi secondari, framework o tecnologie esterne.

Utilizzo del pattern nel progetto

All'interno del progetto viene usata la Constructor Injection. Il factory di Flask (`create_app()`) funge da **Composition Root**: è l'unico punto centralizzato del sistema a conoscere le implementazioni concrete delle porte e ad occuparsi di iniettarle nei service applicativi.

4.2) Design Pattern Creazionali

4.2.1) Factory

Descrizione del pattern

Il pattern Factory è un pattern creazionale che astrae il processo di creazione degli oggetti.

Delega a un componente specifico la responsabilità di creare oggetti complessi, restituendoli attraverso un'interfaccia comune.

Motivazioni dell'utilizzo del pattern

In un'architettura esagonale, i layer di dominio devono dipendere da interfacce (Porte) e mai da implementazioni concrete.

Se un service dovesse istanziare direttamente le proprie dipendenze, finirebbe per accoppiarsi a librerie o tecnologie esterne.

Il Factory risolve questo problema incapsulando la conoscenza di «come» e «quale» oggetto concreto creare.

Utilizzo del pattern nel progetto

Nel progetto il pattern è applicato a due livelli distinti:

1. **Application Factory:** A livello di framework, la funzione `create_app()` di Flask agisce come macro-factory, assemblando l'applicazione e le sue dipendenze principali all'avvio.

2. **Domain/Service Factory:** A livello applicativo, vengono utilizzate factory specifiche per costruire a runtime le implementazioni corrette di alcune interfacce che richiedono una selezione basata sulle azioni dell'utente (creare il FileExporter corrispondente al formato richiesto dall'utente), restituendo ai service oggetti pronti all'uso che rispettano l'interfaccia_G attesa.

4.3) Design Pattern strutturali

4.3.1) Adapter

Descrizione del pattern

L'Adapter è un pattern strutturale che traduce l'interfaccia_G di un componente esterno nell'interfaccia_G che il sistema si aspetta.

Permette a due componenti incompatibili di collaborare senza che nessuno dei due debba cambiare la propria interfaccia_G.

Motivazioni dell'utilizzo del pattern

Il dominio definisce le proprie interfacce in termini di concetti di business.

Tramite l'uso sistematico di porte e adapter è possibile ridurre l'accoppiamento permettendo l'aggiornamento parallelo dei sistemi e lasciando all'adapter il compito di tradurre le nuove interfacce

Utilizzo del pattern nel progetto

Al fine di implementare correttamente l'architettura esagonale nel sistema backend, il pattern adapter è stato usato in modo sistematico per gestire le comunicazioni verso servizi esterni.

Gli utilizzi principali sono nella comunicazione col database e con i servizi di interpretazione e generazione di file

4.4) Design Pattern Comportamentali

4.4.1) Template Method

Descrizione del pattern

Il Template Method è un pattern comportamentale che definisce lo scheletro_G di un algoritmo in una classe astratta, delegando alle sottoclassi l'implementazione dei passi specifici.

La sequenza complessiva è fissa; ciò che cambia è il comportamento dei singoli passi.

Motivazioni dell'utilizzo del pattern

Tutti i formati di export condividono la stessa sequenza operativa: preparare la struttura, scrivere i dati, finalizzare l'output.

Senza Template Method questa sequenza dovrebbe essere replicata in ogni exporter concreto, introducendo duplicazione e rischio di inconsistenze.

Il Template Method consente di scrivere la sequenza una sola volta nella classe astratta, lasciando alle sottoclassi solo i dettagli specifici del formato.

Utilizzo del pattern nel progetto

Viene utilizzato durante l'esportazione_G di informazioni sotto formato di file in diversi formati.

4.5) Pattern MVVM nel Frontend

Il frontend adotta il pattern Model-View-ViewModel tramite la Composition API di Vue 3. La struttura del file .vue riflette naturalmente questa separazione: il blocco `<script setup>` realizza il ViewModel, il blocco `<template>` dichiara il binding tra ViewModel e View tramite il meccanismo di data binding del framework. Il ViewModel non istanzia né invoca direttamente le View — le conosce solo come destinatari di dati e sorgenti di eventi.

Il pattern MVVM si applica ai widget che gestiscono stato complesso. I widget che incapsulano una singola azione senza stato interno (eliminazione, esportazione_G, gestione sessione) seguono un pattern riconducibile al Command, descritto nella sezione dedicata ai widget semplici.

4.5.1) Model

Il Model rappresenta i dati e le regole che li governano, indipendentemente da qualsiasi elemento visivo.

Per i widget basati su form, il Model è composto dal composabile `useFormModel` e dalle definizioni di dominio (`assetFormFields`, `deviceFormFields`). `useFormModel` gestisce lo stato reattivo dei campi, la validazione_G client-side e l'integrazione degli errori provenienti dal server. Le definizioni di dominio specificano campi, valori iniziali e regole di validazione_G come funzioni pure. La logica di business del Model non dipende dal framework — utilizza la reattività di Vue esclusivamente come meccanismo di notifica.

Per il modulo di valutazione_G dei requisiti, il Model è composto da classi di dominio pure (`DecisionNode`, `LeafNode`, `TreeStructure`, `EvaluationEngine`), prive di qualsiasi dipendenza_G da Vue, e dallo store Pinia (`DecisionTreeStore`) che mantiene lo stato reattivo della valutazione_G. Lo store incapsula lo stato dei dati e le operazioni di dominio (calcolo del percorso attivo, gestione delle risposte) ed è acceduto esclusivamente dal ViewModel.

4.5.2) ViewModel

Il ViewModel orchestra il collegamento tra Model e View: espone lo stato reattivo, gestisce le azioni dell'utente e coordina la comunicazione con il backend.

Per i widget basati su form, il ViewModel è il widget smart che inizializza `useFormModel` con le definizioni di dominio, lo collega alla View tramite props binding, e orchestra la chiamata HTTP di creazione o modifica. La chiamata HTTP è contenuta

direttamente nel widget anziché in un API client dedicato: trattandosi di una singola operazione (POST o PUT), l'introduzione di una classe separata avrebbe aggiunto complessità senza beneficio. Il widget resta comunque testabile in isolamento poiché l'URL dell'endpoint non è hardcoded ma iniettato dal livello di integrazione tramite props.

Per il modulo di valutazione_G dei requisiti, il ViewModel è il RequirementEvaluationWidget, unico componente che accede allo store. Calcola le proprietà derivate necessarie ai componenti figli, le passa come props e traduce gli eventi emessi dai figli in azioni sullo store. La comunicazione con il backend è delegata a un EvaluationApiClient dedicato, scelta motivata dalla presenza di operazioni multiple (salvataggio risposte, salvataggio giustificazione, recupero stato, recupero dettaglio) che avrebbero appesantito il widget se gestite direttamente al suo interno.

4.5.3) View

La View è responsabile esclusivamente del rendering. Ricevono dati tramite props ed emettono eventi senza contenere logica applicativa né accedere direttamente allo stato dell'applicazione. Il binding reattivo tra ViewModel e View è gestito dal framework Vue tramite props e data binding dichiarativo nel template.

5) Diagrammi delle classi

5.1) Dominio

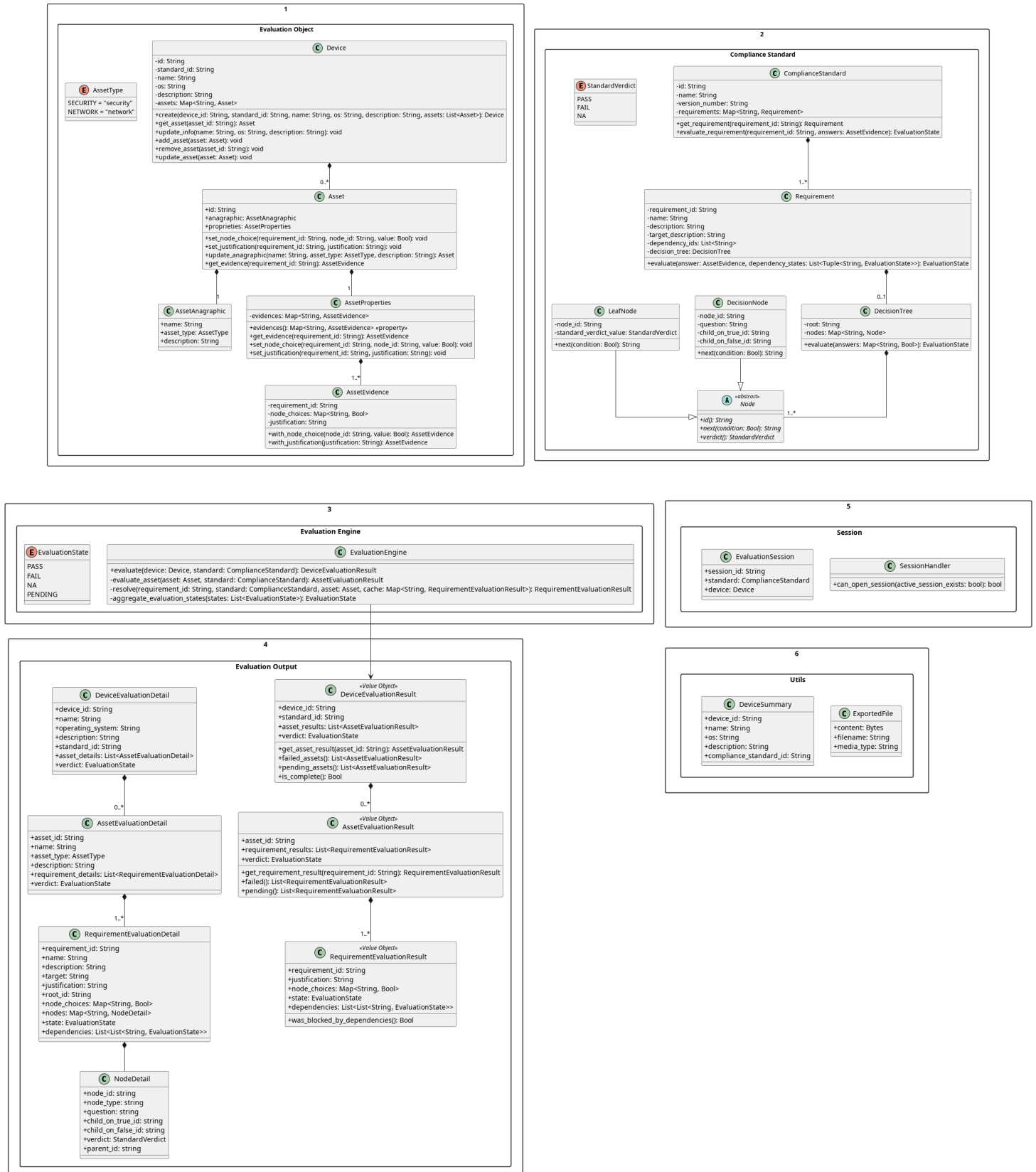


Figura 5: Modello di Dominio

5.1.1) Device

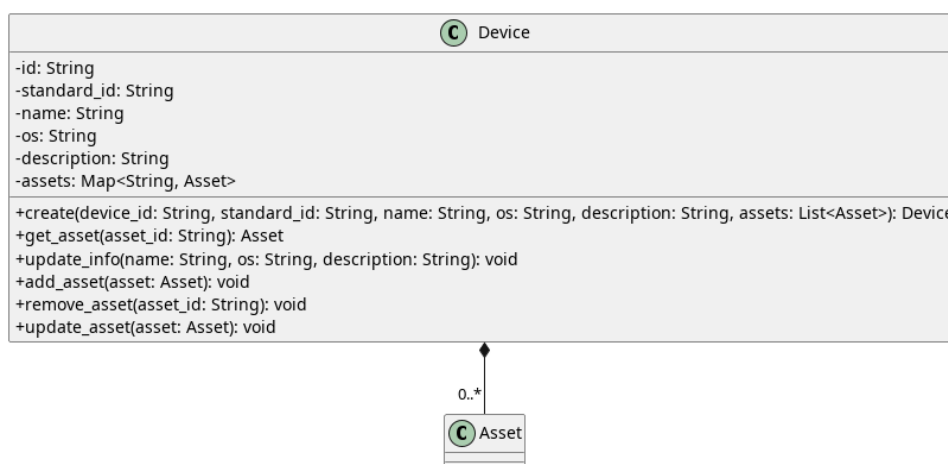


Figura 6: Device

Descrizione

Device è l'entità centrale del dominio che rappresenta il Dispositivo_G oggetto della valutazione_G di conformità. È associata alla classe *Asset_G* con una relazione di composizione avente cardinalità 0..*.

Attributi

- - `id: String` — identificativo univoco del Dispositivo_G.
- - `standard_id: String` — identificativo dello standard di conformità_G associato al dispositivo_G.
- - `name: String` — nome del Dispositivo_G.
- - `os: String` — sistema operativo del Dispositivo_G.
- - `description: String` — descrizione testuale del Dispositivo_G.
- - `assets: Map<String, AssetG>` — mappa degli asset_G associati al Dispositivo_G, indicizzati per il loro identificativo.

Metodi

- + `create(device_id: String, standard_id: String, name: String, os: String, description: String, assets: List<AssetG>): Device` — metodo per la creazione e istanziazione di un nuovo oggetto Dispositivo_G.
- + `get_asset(asset_id: String): AssetG` — recupera l'Asset_G corrispondente all'identificativo fornito.
- + `update_info(name: String, os: String, description: String): void` — aggiorna le informazioni anagrafiche del Dispositivo_G (nome, sistema operativo e descrizione).
- + `add_asset(assetG: AssetG): void` — aggiunge un nuovo Asset_G alla mappa del Dispositivo_G.
- + `remove_asset(asset_id: String): void` — rimuove l'Asset_G identificato da `asset_id` dalla mappa del Dispositivo_G.

- + update_asset(asset_G: Asset_G): void — aggiorna un Asset_G esistente

5.1.2) Asset_G

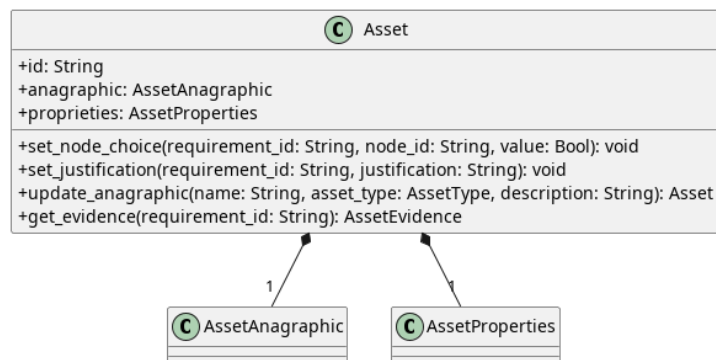


Figura 7: Asset_G

Descrizione

Asset_G è l'entità di dominio che rappresenta un asset_G oggetto di valutazione_G di conformità_G all'interno di una sessione.

Attributi

- - asset_id: String — identificativo univoco dell'Asset_G.
- - asset_anagraphic: AssetAnagraphic — oggetto che incapsula i dati anagrafici dell'asset_G.
- - asset_proprieties: AssetProperties — oggetto che incapsula le proprietà dell'asset_G.

Metodi

- + set_node_choice(requirement_id: String, node_id: String, value: Bool): void — imposta o aggiorna la scelta (risposta) effettuata per un determinato nodo_G decisionale relativo a un requisito_G.
- + set_justification(requirement_id: String, value: Bool): void — imposta la giustificazione per un determinato requisito_G.
- + update_anagraphic(name: String, type: AssetType, description: String): void — aggiorna le informazioni anagrafiche dell'asset_G (nome, tipologia e descrizione), delegando l'aggiornamento all'istanza interna di *AssetAnagraphic*.

5.1.3) AssetAnagraphic

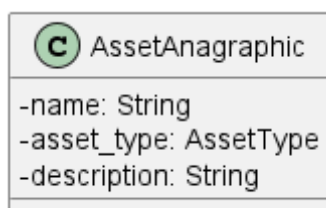


Figura 8: AssetAnagraphic

Descrizione

AssetAnagraphic è l'entità delegata all'incapsulamento delle informazioni anagrafiche e dei metadati di base di un generico *asset_G*.

Attributi

- - name: String — stringa di testo che rappresenta il nome identificativo dell'*asset_G*.
- - type: AssetType — attributo che definisce la tipologia o la categoria di appartenenza dell'*asset_G*.
- - description: String — stringa di testo destinata a contenere una descrizione estesa, note o dettagli aggiuntivi riguardanti le caratteristiche fisiche o logiche dell'*asset_G*.

Metodi

AssetAnagraphic definisce dei semplici getter, qui omessi per poca rilevanza.

5.1.4) AssetProperties

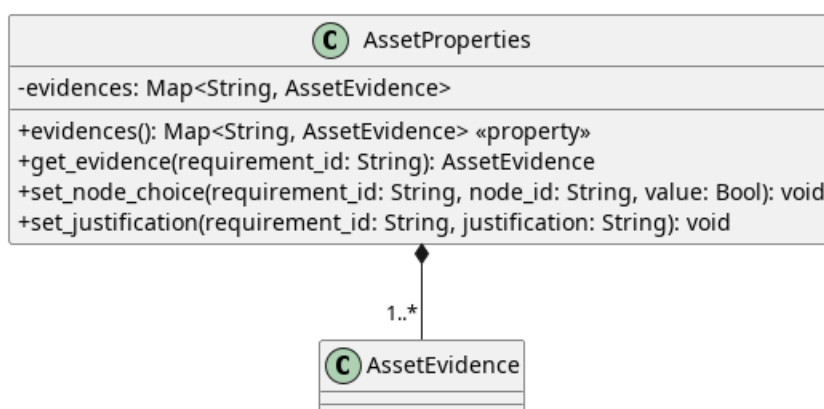


Figura 9: AssetProperties

Descrizione

AssetProperties è l'entità delegata alla gestione dello stato valutativo e delle proprietà specifiche di un *asset_G*. Presenta una relazione di composizione con la classe *AssetEvidence* con cardinalità `1..*`, gestendone il ciclo di vita all'interno di una lista.

Attributi

- + asset_evidence_list: List<AssetEvidence> — struttura dati che incapsula e gestisce l'elenco delle evidenze (scelte e giustificazioni) associate all'*asset_G*.

Metodi

- + set_node_choice(requirement_id: String, node_id: String, value: Bool): void — imposta o aggiorna la scelta (risposta) effettuata per un determinato *nodog* decisionale relativo a un *requisitog*.

- + `set_justification(requirement_id: String, value: Bool): void` — imposta o aggiorna la giustificazione testuale per un determinato nodo_G di un requisito_G.
- + `get_evidence(requirement_id: String): AssetEvidence | void` — recupera l'oggetto *AssetEvidence* associato a un determinato identificativo di requisito_G. Restituisce l'evidenza_G se presente, altrimenti void (nessun valore/null).

5.1.5) AssetType

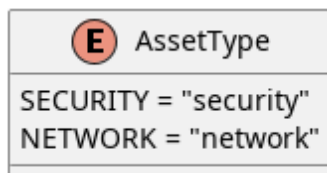


Figura 10: AssetType

Descrizione

AssetType è l'enumerazione che definisce le categorie o le tipologie logiche di appartenenza di un determinato Asset_G all'interno del dominio valutativo.

Attributi

- - `SECURITY: String` — valore enumerato che identifica un asset_G di tipo sicurezza («security»).
- - `NETWORK: String` — valore enumerato che identifica un asset_G di tipo rete («network»).

Metodi

AssetType non definisce metodi.

5.1.6) AssetEvidence

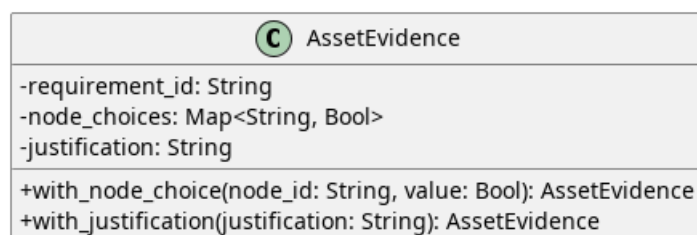


Figura 11: AssetEvidence

Descrizione

AssetEvidence è un Value Object immutabile che incapsula in modo sicuro le evidenze di valutazione_G associate a un singolo requisito_G, comprendendo le risposte fornite ai nodi decisionali e l'eventuale giustificazione testuale. Essendo immutabile, ogni modifica genera una nuova istanza.

Attributi

- - `requirement_id`: `String` — identificativo univoco del requisito_G a cui le evidenze fanno riferimento.
- - `node_choices`: `Map<String, Bool>` — mappa immutabile che associa l'identificativo di un nodo_G decisionale alla risposta booleana fornita.
- - `justification`: `String` — stringa di testo contenente la giustificazione opzionale per la valutazione_G del requisito_G.

Metodi

- + `with_node_choice(node_id: String, value: Bool): AssetEvidence` — crea e restituisce una nuova istanza dell'oggetto contenente la mappa delle risposte aggiornata con il nuovo valore per il nodo_G specificato.
- + `with_justification(justification: String): AssetEvidence` — crea e restituisce una nuova istanza dell'oggetto aggiornata con il testo della giustificazione fornito.

5.1.7) ComplianceStandard

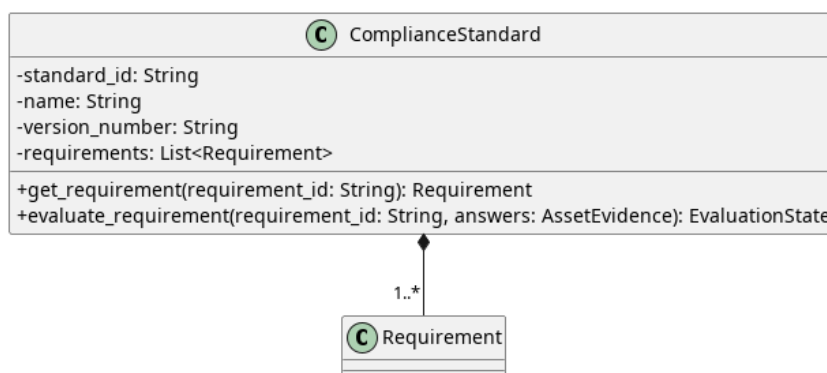


Figura 12: ComplianceStandard

Descrizione

ComplianceStandard è l'entità radice che rappresenta una norma o standard di conformità_G. È composta da una collezione di requisiti validati univocamente. La classe è progettata per essere immutabile garantendo l'integrità dei dati tramite incapsulamento.

Attributi

- - `standard_id`: `String` — identificativo univoco dello standard.
- - `name`: `String` — nome descrittivo dello standard.
- - `version_number`: `String` — versione specifica dello standard.
- - `requirements`: `List<Requirement>` — tupla immutabile contenente tutti i requisiti associati allo standard.

Metodi

- + `get_requirement(requirement_id: String): Requirement` — recupera uno specifico requisito_G tramite il suo identificativo, sollevando un'eccezione se non presente.
- + `evaluate_requirement(requirement_id: String, answers: AssetEvidence): EvaluationState` — delega al requisito_G specificato la valutazione_G del suo stato basandosi sulle evidenze fornite.

5.1.8) Requirement

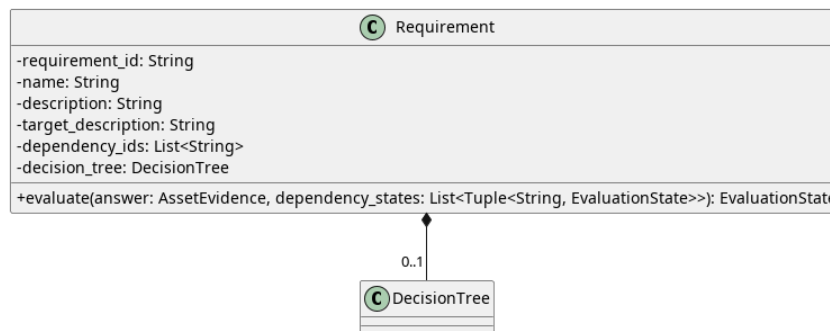


Figura 13: Requirement

Descrizione

Requirement è un'entità immutabile che definisce un singolo requisito_G di conformità_G, includendo le sue dipendenze e l'albero decisionale per valutarlo. Applica regole di business stringenti, come il fallimento automatico se il verdetto è «Non Applicabile» (NA) e manca la giustificazione testuale.

Attributi

- - `requirement_id: String` — identificativo univoco del requisito_G.
- - `name: String` — nome del requisito_G.
- - `description: String` — descrizione completa del requisito_G.
- - `target_description: String` — descrizione dell'obiettivo del requisito_G.
- - `decision_tree: DecisionTree` — albero decisionale contenente la logica per la valutazione_G.
- - `dependency_ids: List<String>` — tupla contenente gli ID di altri requisiti da cui questo dipende.

Metodi

- + `evaluate(answer: AssetEvidence, dependency_states: List): EvaluationState` — calcola lo stato di valutazione_G controllando prima lo stato delle dipendenze e poi interrogando l'albero decisionale.
- - `_check_dependencies(dependencies: List): EvaluationState | None` — metodo privato che verifica_G le dipendenze: se una fallisce, blocca la valutazione_G in FAIL_G o PENDING.

5.1.9) DecisionTree

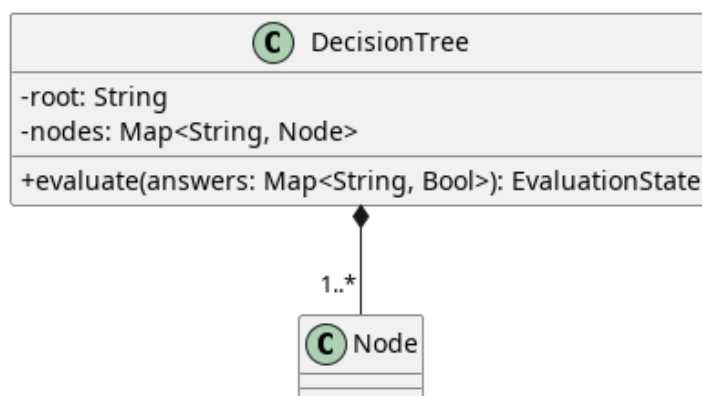


Figura 14: DecisionTree

Descrizione

DecisionTree è il componente che incapsula la logica strutturata di valutazione_G navigando un grafo di nodi. In fase di inizializzazione convalida l'assenza di nodi duplicati e l'esistenza della radice. Durante la valutazione_G, traccia i nodi visitati per prevenire cicli infiniti.

Attributi

- - `root_id: String` — identificativo del nodo_G iniziale (radice) da cui parte la valutazione_G.
- - `nodes: Map<String, Node>` — mappa immutabile che associa gli ID dei nodi alle rispettive istanze.

Metodi

- + `evaluate(answers: Map<String, Bool>): EvaluationState` — naviga l'albero partendo dalla radice e utilizzando le risposte fornite, restituendo lo stato finale della valutazione_G.

5.1.10) Node

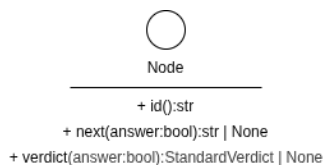


Figura 15: Node

Descrizione

Node è un'interfaccia_G che definisce il contratto base per tutti gli elementi che compongono un albero decisionale.

Metodi

- - `id(): String` — proprietà astratta che restituisce l'identificativo del nodo_G.

- - `verdict(): StandardVerdict | None` — proprietà astratta che restituisce il verdetto del nodo_G, se applicabile.
- + `next(condition: Bool | None): String | None` — metodo astratto che determina l'ID del nodo_G successivo basandosi sulla condizione fornita.

5.1.11) DecisionNode

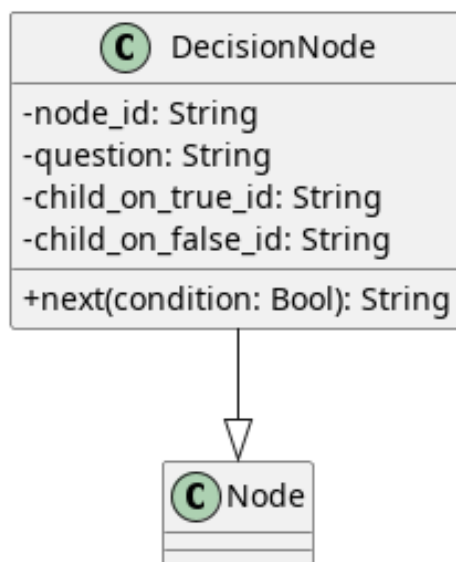


Figura 16: DecisionNode

Descrizione

DecisionNode è un oggetto immutabile che implementa *Node*. Rappresenta uno snodo dell'albero che pone una domanda e biforca il percorso di valutazione_G.

Attributi

- - `node_id: String` — identificativo del nodo_G.
- - `question: String` — testo della domanda posta all'utente.
- - `child_on_true_id: String` — ID del nodo_G successivo se la risposta è affermativa.
- - `child_on_false_id: String` — ID del nodo_G successivo se la risposta è negativa.

Metodi

- + `id(): str` — restituisce l'id del nodo_G.
- + `next(condition: Bool | None): String | None` — restituisce l'ID del nodo_G figlio appropriato a seconda del valore booleano della risposta.
- + `verdict(): None` — restituisce costantemente `None` poiché uno snodo non produce un verdetto finale.

5.1.12) LeafNode

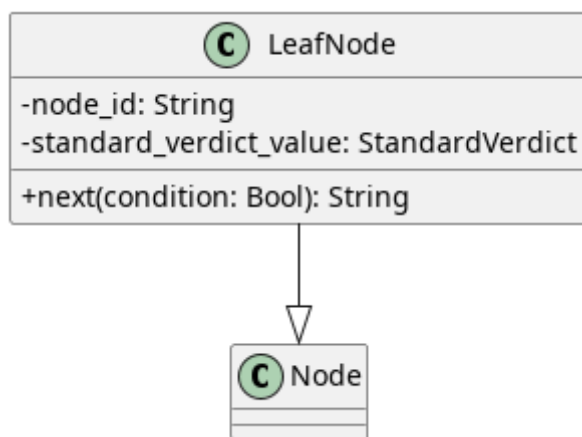


Figura 17: LeafNode

Descrizione

LeafNode è un oggetto immutabile che implementa *Node* e rappresenta la terminazione di un percorso decisionale (foglia), contenendo il risultato finale della valutazione_G.

Attributi

- - `node_id: String` — identificativo del nodo_G foglia.
- - `verdict_value: StandardVerdict` — il verdetto definitivo di conformità_G associato alla foglia.

Metodi

- + `id(): str` — restituisce l'id del nodo_G.
- + `next(condition: Bool | None): None` — restituisce costantemente `None` poiché una foglia non possiede nodi successivi.
- + `verdict(): StandardVerdict` — restituisce il valore del verdetto di conformità_G.

5.1.13) StandardVerdict

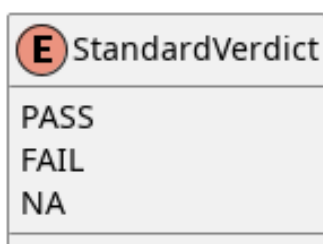


Figura 18: StandardVerdict

Descrizione

StandardVerdict è un'enumerazione di stringhe che definisce formalmente i possibili verdeti terminali generati da un albero decisionale.

Attributi

- - PASS_G: String — indica che il requisito_G è soddisfatto («pass_G»).
- - FAIL_G: String — indica che il requisito_G non è soddisfatto («fail_G»).
- - NA: String — indica che il requisito_G non è applicabile al contesto («not_applicable»).

Metodi

StandardVerdict non definisce metodi.

5.1.14) EvaluationEngine

 EvaluationEngine
<pre> +evaluate(device: Device, standard: ComplianceStandard): DeviceEvaluationResult -evaluate_asset(asset: Asset, standard: ComplianceStandard): AssetEvaluationResult -resolve(requirement_id: String, standard: ComplianceStandard, asset: Asset, cache: Map<String, RequirementEvaluationResult>): RequirementEvaluationResult -aggregate_evaluation_states(states: List<EvaluationState>): EvaluationState </pre>

Figura 19: EvaluationEngine

Descrizione

EvaluationEngine è il componente del dominio responsabile di orchestrare il processo di valutazione_G di un intero Device rispetto a un ComplianceStandard fornito. Il motore elabora iterativamente gli asset_G, risolvendo in modo ricorsivo le dipendenze tra i requisiti e applicando tecniche di memoizzazione per ottimizzare le performance e prevenire valutazioni ridondanti.

Attributi

La classe non definisce attributi di stato interni, agendo come puro gestore della logica di business.

Metodi

- + evaluate(device: Device, standard: ComplianceStandard): DeviceEvaluationResult — valuta il dispositivo_G calcolando e aggregando i risultati di tutti i suoi asset_G, restituendo infine l'esito globale.
- - _evaluate_asset(asset_G: Asset_G, standard: ComplianceStandard): AssetEvaluationResult — metodo privato che valuta un singolo asset_G contro tutti i requisiti dello standard, avvalendosi di una cache per mantenere i risultati e supportare la memoizzazione.
- - _resolve(requirement_id: String, standard: ComplianceStandard, asset_G: Asset_G, cache: Map): RequirementEvaluationResult — risolve la valutazione_G di uno specifico requisito_G verificando prima ricorsivamente le sue dipendenze. Se l'asset_G non presenta evidenze per il requisito_G, imposta forzatamente lo stato su PENDING. Successivamente, salva il risultato nella cache.
- - _aggregate_evaluation_states(states: List<EvaluationState>): EvaluationState — analizza una serie di stati e ne calcola il verdetto aggregato:

restituisce $FAIL_G$ se rileva almeno un fallimento, $PENDING$ se vi sono valutazioni incomplete, altrimenti restituisce $PASS_G$.

5.1.15) EvaluationState

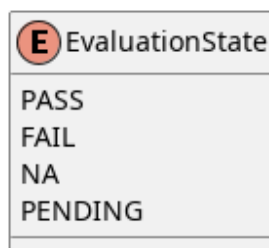


Figura 20: EvaluationState

Descrizione

EvaluationState è un'enumerazione di stringhe (*StrEnum*) che definisce formalmente i possibili stati di avanzamento o di conclusione di una valutazione_G.

Attributi

- - $PASS_G$: *String* — indica che la valutazione_G è stata superata con successo («pass_G»).
- - $FAIL_G$: *String* — indica che la valutazione_G ha dato esito negativo («fail_G»).
- - NA : *String* — indica che il requisito_G non è applicabile al contesto valutato («not_applicable»).
- - $PENDING$: *String* — indica che la valutazione_G è attualmente in sospeso_G per mancanza di evidenze o risposte complete («pending»).

Metodi

- + `from_verdict(verdict: StandardVerdict): EvaluationState` — metodo di classe che converte un *StandardVerdict* nel corrispondente *EvaluationState*, sollevando un *ValueError* qualora non esista una mappatura definita.

5.1.16) RequirementEvaluationResult

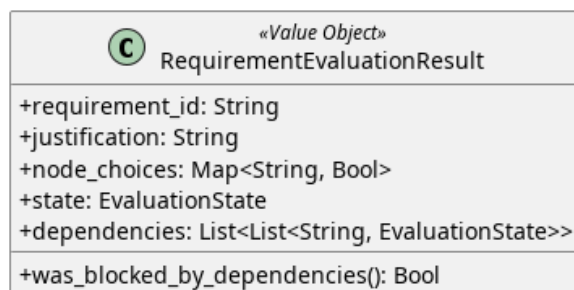


Figura 21: RequirementEvaluationResult

Descrizione

RequirementEvaluationResult è un Value Object immutabile che incapsula l'esito finale della valutazione_G di un singolo requisito_G.

Attributi

- + requirement_id: String — identificativo del requisito_G valutato.
- + justification: String — giustificazione fornita per la valutazione_G.
- + node_choices: MappingProxyType<String, Bool> — mappa immutabile delle risposte fornite ai nodi decisionali.
- + state: EvaluationState — stato finale calcolato per il requisito_G.
- + dependencies: List — tupla contenente l'ID e lo stato delle dipendenze del requisito_G.

Metodi

- + was_blocked_by_dependencies(): Bool — verifica_G se l'esito è stato bloccato a causa di una o più dipendenze che non hanno raggiunto lo stato di PASS_G.

5.1.17) AssetEvaluationResult

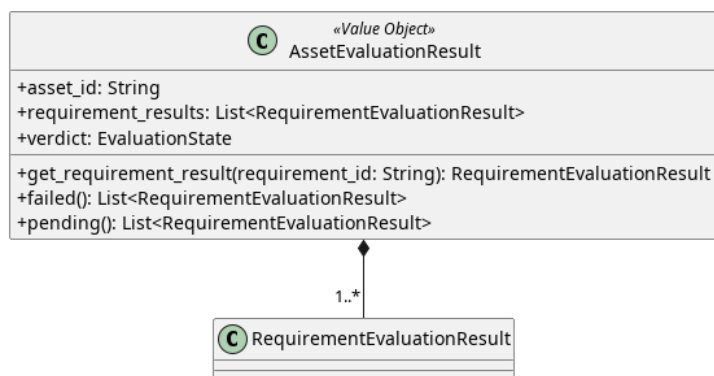


Figura 22: AssetEvaluationResult

Descrizione

AssetEvaluationResult è un Value Object immutabile che raggruppa tutti i risultati dei requisiti calcolati per un singolo asset_G, determinandone il verdetto complessivo.

Attributi

- + asset_id: String — identificativo dell'asset_G valutato.
- + requirement_results: List<RequirementEvaluationResult> — tupla contenente i risultati di tutti i requisiti valutati per questo asset_G.
- + verdict: EvaluationState — stato di conformità_G globale dell'asset_G.

Metodi

- + get_requirement_result(requirement_id: String): RequirementEvaluationResult | None — cerca e restituisce il risultato di uno specifico requisito_G, se presente.

- + failed(): List<RequirementEvaluationResult> — filtra e restituisce esclusivamente i requisiti che hanno prodotto uno stato di FAIL_G.
- + pending(): List<RequirementEvaluationResult> — filtra e restituisce esclusivamente i requisiti con valutazione_G ancora in corso_G o incompleta (PENDING).

5.1.18) DeviceEvaluationResult

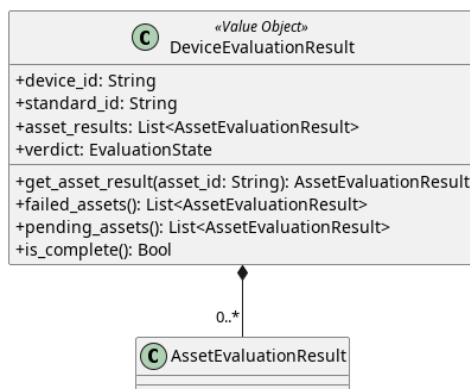


Figura 23: DeviceEvaluationResult

Descrizione

DeviceEvaluationResult rappresenta l'esito globale e immutabile della valutazione_G di un intero dispositivo_G rispetto a uno standard. Raggruppa i risultati di tutti i suoi asset_G.

Attributi

- + device_id: String — identificativo del dispositivo_G.
- + standard_id: String — identificativo dello standard di riferimento.
- + asset_results: List<AssetEvaluationResult> — tupla con i risultati aggregati per ogni asset_G.
- + verdict: EvaluationState — esito finale della valutazione_G complessiva del dispositivo_G.

Metodi

- + get_asset_result(asset_id: String): AssetEvaluationResult | None — recupera il risultato di un asset_G specifico all'interno del dispositivo_G.
- + failed_assets(): List<AssetEvaluationResult> — restituisce gli asset_G che non hanno superato la valutazione_G.
- + pending_assets(): List<AssetEvaluationResult> — restituisce gli asset_G la cui valutazione_G è ancora in sospeso_G.
- + is_complete(): Bool — restituisce True se l'intera valutazione_G del dispositivo_G è conclusa (non ci sono stati PENDING).

5.1.19) NodeDetail

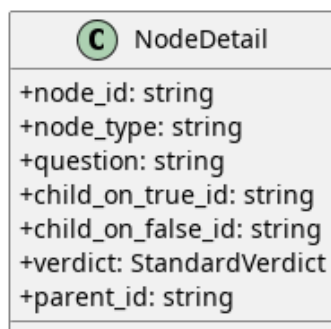


Figura 24: NodeDetail

Descrizione

NodeDetail è un oggetto di dominio, immutabile, utilizzato per esporre i dettagli strutturali e il contenuto informativo di un nodo_G dell'albero decisionale.

Attributi

- + `node_id`: `String` — identificativo del nodo_G.
- + `node_type`: `String` — indica la natura del nodo_G («decision» o «leaf»).
- + `question`: `String` | `None` — testo della domanda, presente solo nei nodi decisionali.
- + `child_on_true_id`: `String` | `None` — riferimento al nodo_G figlio per risposta affermativa.
- + `child_on_false_id`: `String` | `None` — riferimento al nodo_G figlio per risposta negativa.
- + `verdict`: `StandardVerdict` | `None` — esito associato al nodo_G, valorizzato solo per i nodi foglia.
- + `parent_id`: `String` | `None` — identificativo del nodo_G genitore, utile per navigare l'albero a ritroso.

Metodi

NodeDetail non definisce metodi.

5.1.20) RequirementEvaluationDetail

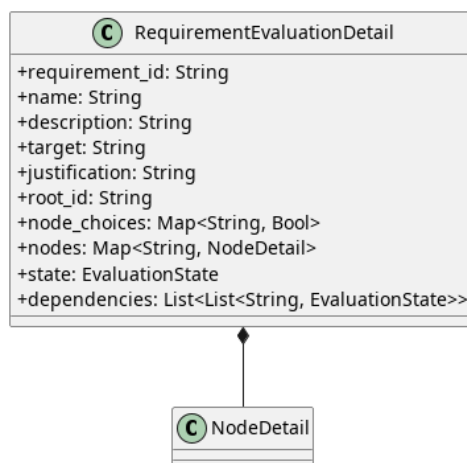


Figura 25: RequirementEvaluationDetail

Descrizione

RequirementEvaluationDetail è un oggetto immutabile che consolida tutte le informazioni di un requisito_G (anagrafica_G, albero, nodi) e il relativo stato di valutazione_G nel contesto di un asset_G. È concepito per arricchire il risultato grezzo con i test completi.

Attributi

- + `requirement_id: String` — identificativo univoco.
- + `name: String` — nome del requisito_G.
- + `description: String` — testo descrittivo del requisito_G.
- + `target: String` — obiettivo prefissato.
- + `justification: String` — motivazione associata alla risposta.
- + `root_id: String` — identificativo del nodo_G radice dell'albero.
- + `node_choices: MappingProxyType<String, Bool>` — mappa delle risposte effettuate.
- + `nodes: Map<String, NodeDetail>` — dizionario di tutti i nodi appartenenti all'albero di questo requisito_G.
- + `state: EvaluationState` — esito attuale.
- + `dependencies: List` — lista delle dipendenze correlate con il relativo stato.

Metodi

RequirementEvaluationDetail non definisce metodi.

5.1.21) AssetEvaluationDetail

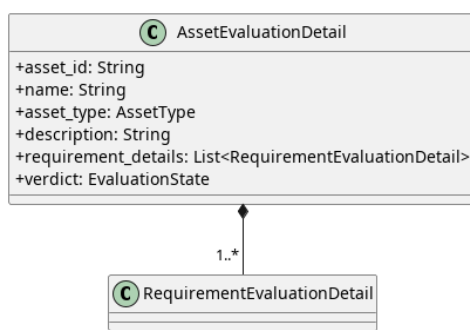


Figura 26: AssetEvaluationDetail

Descrizione

AssetEvaluationDetail è un oggetto immutabile che aggrega tutti i dettagli descrittivi e di valutazione_G dei requisiti di uno specifico asset_G.

Attributi

- + `asset_id: String` — identificativo dell'asset_G.
- + `name: String` — nome dell'asset_G.
- + `asset_type: AssetType` — categoria di appartenenza.
- + `description: String` — descrizione aggiuntiva.
- + `requirement_details: List<RequirementEvaluationDetail>` — insieme arricchito di dettagli per ogni requisito_G.
- + `verdict: EvaluationState` — stato globale dell'asset_G.

Metodi

AssetEvaluationDetail non definisce metodi.

5.1.22) DeviceEvaluationDetail

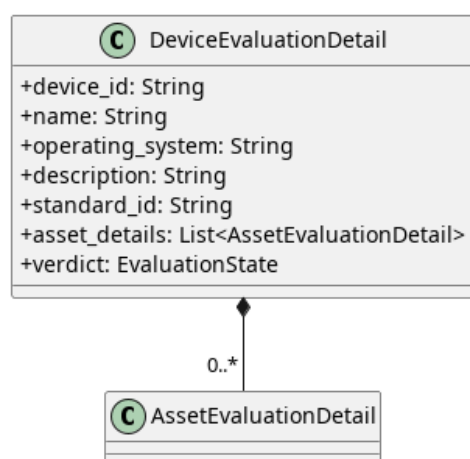


Figura 27: DeviceEvaluationDetail

Descrizione

DeviceEvaluationDetail è il livello radice della struttura di dettaglio: consolida e struttura in modo gerarchico tutte le informazioni testuali e di valutazione_G per l'intero dispositivo_G.

Attributi

- + device_id: String — identificativo del dispositivo_G.
- + name: String — nome del dispositivo_G.
- + operating_system: String — sistema operativo in uso.
- + description: String — breve descrizione del dispositivo_G.
- + standard_id: String — identificativo dello standard applicato.
- + asset_details: List<AssetEvaluationDetail> — dettaglio di ogni asset_G.
- + verdict: EvaluationState — stato complessivo.

Metodi

DeviceEvaluationDetail non definisce metodi.

5.1.23) EvaluationSession

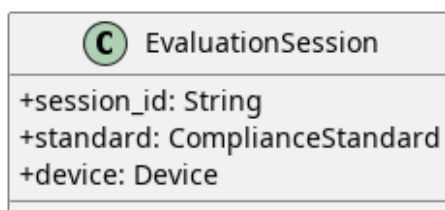


Figura 28: EvaluationSession

Descrizione

EvaluationSession è la classe che modella il contesto centrale di un'operazione di valutazione_G. Raggruppa e associa un identificativo univoco di sessione a uno specifico dispositivo_G e allo standard di conformità_G di riferimento scelto per l'analisi.

Attributi

- + session_id: String — identificativo univoco della sessione di valutazione_G.
- + standard: ComplianceStandard — istanza dello standard di conformità_G applicato per la valutazione_G corrente.
- + device: Device — istanza del dispositivo_G che viene sottoposto a valutazione_G.

Metodi

EvaluationSession non definisce metodi.

5.1.24) SessionHandler

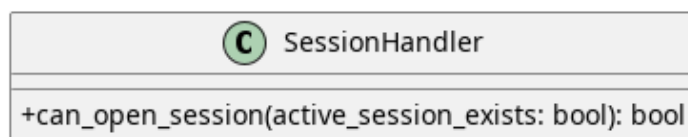


Figura 29: SessionHandler

Descrizione

SessionHandler è il componente di dominio che incapsula le regole di business fondamentali relative all'apertura di una nuova sessione. Valuta la fattibilità dell'operazione controllando le precondizioni del sistema.

Tale contesto viene passato tramite parametro di funzione. Sebbene al momento la logica sia molto semplice, questa struttura risulterà utile per estensioni future.

Attributi

La classe non definisce attributi di stato interni, agendo come puro gestore di logica.

Metodi

- `+ can_open_session(active_session_exists: Bool): Bool` — verifica_G se è possibile avviare una nuova sessione, restituendo `False` nel caso in cui ne esista già una attiva (impedendo sovrapposizioni), altrimenti restituisce `True`.

5.1.25) ExportedFile

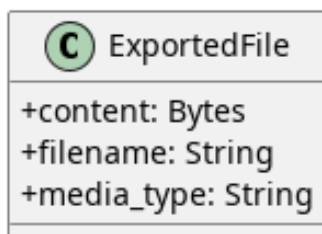


Figura 30: ExportedFile

Descrizione

ExportedFile è un oggetto immutabile utilizzato per incapsulare in un'unica struttura il contenuto binario e i metadati di un file pronto per l'esportazione_G o il download.

Attributi

- `+ content: IO<bytes>` — stream binario contenente i dati grezzi del file da esportare.
- `+ filename: String` — nome del file, comprensivo della relativa estensione_G.
- `+ media_type: String` — tipo di media o formato MIME (ad esempio, «application/pdf_G» o «application/json_G») associato al file.

Metodi

ExportedFile non definisce metodi.

5.1.26) DeviceSummary

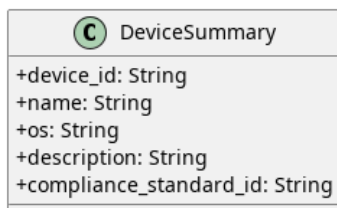


Figura 31: DeviceSummary

Descrizione

DeviceSummary è un oggetto immutabile utilizzato per incapsulare e trasportare una vista sintetica e leggera delle informazioni anagrafiche di un dispositivo_G. È progettato per fornire i dati essenziali senza esporre l'intera struttura complessa o gli asset_G associati, risultando ideale per le operazioni di visualizzazione o per la generazione di elenchi.

Attributi

- `+ device_id: String` — identificativo univoco del dispositivo_G.
- `+ name: String` — nome assegnato al dispositivo_G.
- `+ os: String` — sistema operativo in uso sul dispositivo_G.
- `+ description: String` — breve descrizione testuale del dispositivo_G.
- `+ compliance_standard_id: String` — identificativo dello standard di conformità_G a cui il dispositivo_G fa riferimento.

Metodi

DeviceSummary non definisce metodi.

5.2) Device

5.2.1) CreateDevice



Figura 32: Caso d'uso_G CreateDevice

Il diagramma illustra l'architettura del modulo di creazione dei Dispositivi secondo i principi dell'architettura esagonale.

- Per la definizione di *Device*, vedere la **Sezione 5.1.1**.

5.2.1.1) FlaskWriteDeviceController



Figura 33: FlaskWriteDeviceController

Descrizione

FlaskWriteDeviceController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione dei Dispositivi e le inoltra al livello applicativo.

Attributi

- - `create_device_use_case`: `CreateDeviceUseCase` — inbound port usata per la creazione di un device.
- - `update_device_use_case`: `UpdateDeviceUseCase` — inbound port usata per la modifica di un device.
- - `delete_device_use_case`: `DeleteDeviceUseCase` — inbound port usata per l'eliminazione di un device.

Metodi

- + `create_device(req: Request): Response` — riceve la richiesta HTTP di creazione di un nuovo `DispositivoG`, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `update_device(req: Request): Response` — riceve la richiesta HTTP di modifica di un `DispositivoG` esistente, estrae i dati aggiornati e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `delete_device(req: Request): Response` — riceve la richiesta HTTP di eliminazione di un `DispositivoG`, estrae l'identificativo dalla richiesta e lo inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.2.1.2) CreateDeviceUseCase

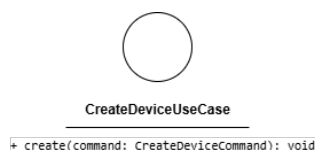


Figura 34: CreateDeviceUseCase

Descrizione

CreateDeviceUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per la creazione di un nuovo `DispositivoG`. Viene implementata da *CreateDeviceService* e utilizzata da *FlaskWriteDeviceController*.

Attributi

CreateDeviceUseCase non definisce attributi.

Metodi

- + `create(command: CreateDeviceCommand): void` — firma del metodo delegato all'esecuzione della logica di creazione a partire dai dati contenuti nel comando.

5.2.1.3) CreateDeviceCommand

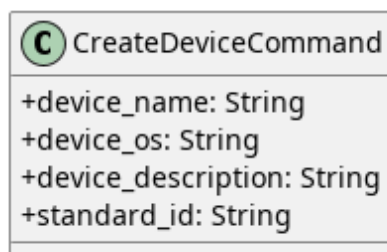


Figura 35: CreateDeviceCommand

Descrizione

CreateDeviceCommand è il Command Object che veicola i dati necessari alla creazione di un `DispositivoG` dal controller al service. L'uso di questo pattern separa la

struttura dei dati in ingresso dall'entità di dominio, rendendo esplicita l'intenzione dell'operazione.

Attributi

- + device_name: String — nome del nuovo Dispositivo_G.
- + device_os: String — sistema operativo del Dispositivo_G.
- + device_description: String — descrizione testuale del Dispositivo_G.
- + standard_id: String — identificativo dello standard di conformità_G associato.

Metodi

CreateDeviceCommand non definisce metodi.

5.2.1.4) CreateDeviceService



Figura 36: CreateDeviceService

Descrizione

CreateDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di creazione di un Dispositivo_G. Implementa l'interfaccia_G *CreateDeviceUseCase*, riceve il comando in ingresso, costruisce l'entità di dominio e ne coordina la persistenza tramite *RegisterDevicePort*.

Attributi

- - register_device_port: RegisterDevicePort — porta outbound utilizzata per la persistenza del nuovo Dispositivo_G.

Metodi

- + create(command: CreateDeviceCommand): void — concretizza il contratto definito da *CreateDeviceUseCase*. Mappa i dati del comando nell'entità *Device* e ne richiede la registrazione tramite *RegisterDevicePort*.

5.2.1.5) RegisterDevicePort

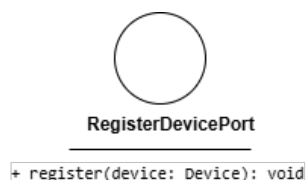


Figura 37: RegisterDevicePort

Descrizione

RegisterDevicePort è l'interfaccia_G (Outbound Port) che definisce il contratto per la registrazione di un nuovo Dispositivo_G nel sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *CreateDeviceService*.

Attributi

RegisterDevicePort non definisce attributi.

Metodi

- + `register(device: Device): void` — firma del metodo che esegue l'inserimento fisico del Dispositivo_G nel sistema di persistenza.

5.2.1.6) MongoDeviceAdapter



Figura 38: MongoDeviceAdapter

Descrizione

MongoDeviceAdapter è la classe dell'Outbound Adapter che implementa le porte out-bound del sistema, traducendo le operazioni del dominio in interazioni concrete con MongoDB tramite `pymongo.Collection`.

Attributi

- - `collection: pymongo.Collection` — riferimento alla collezione MongoDB su cui vengono eseguite le operazioni di persistenza.

Metodi

- + `save(device: Device): void` — salva le modifiche a un Dispositivo_G esistente nella collezione.
- + `register(device: Device): void` — inserisce un nuovo Dispositivo_G nella collezione.
- + `delete(device_id: String): void` — rimuove il Dispositivo_G identificato da `device_id` dalla collezione.
- + `find_by_id(device_id: String): Device` — recupera il Dispositivo_G corrispondente all'identificativo fornito.

- + find_all(): List<DeviceSummary> — recupera la lista sintetica di tutti i Dispositivi presenti nella collezione.

5.2.2) DeleteDevice

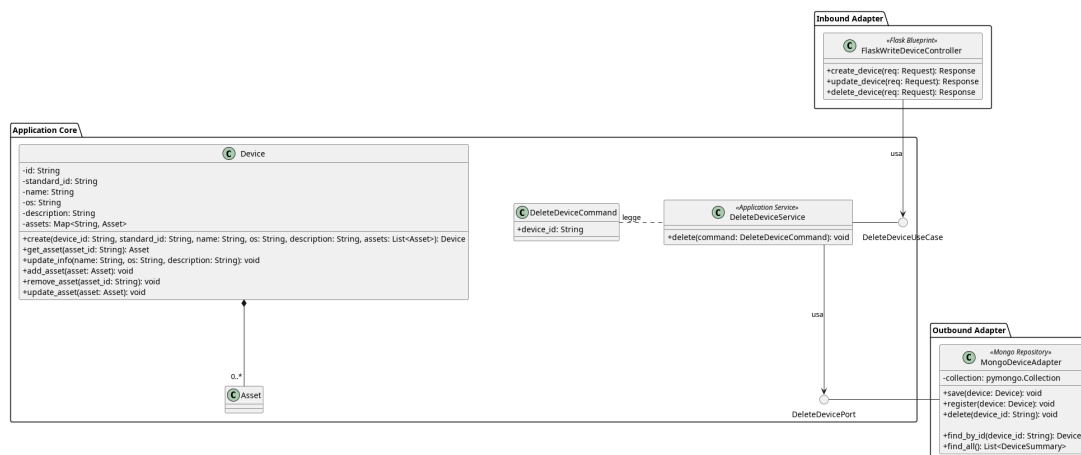


Figura 39: Caso d'uso DeleteDevice

Il diagramma illustra l'architettura del modulo dedicato all'eliminazione di un Dispositivo_G.

- Per la definizione di *FlaskWriteDeviceController*, vedere la [Sezione 5.2.1.1](#).
- Per la definizione di *Device*, vedere la [Sezione 5.1.1](#).
- Per la definizione di *MongoDeviceAdapter*, vedere la [Sezione 5.2.1.6](#).

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso_G.

5.2.2.1) DeleteDeviceUseCase

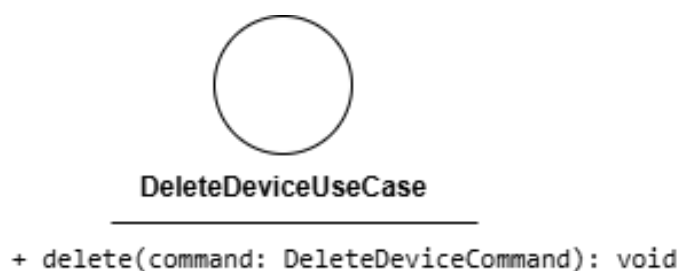


Figura 40: DeleteDeviceUseCase

Descrizione

DeleteDeviceUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per l'eliminazione di un Dispositivo_G. Viene implementata da *DeleteDeviceService* e utilizzata da *FlaskWriteDeviceController*.

Attributi

DeleteDeviceUseCase non definisce attributi.

Metodi

- + delete(device_id: String): void — firma del metodo delegato all'esecuzione della logica di eliminazione a partire dall'identificativo del Dispositivo_G.

5.2.2.2) DeleteDeviceCommand

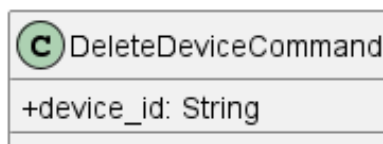


Figura 41: DeleteDeviceCommand

Descrizione

DeleteDeviceCommand è il Command Object utilizzato per trasportare i dati necessari all'eliminazione di un Dispositivo_G.

Attributi

- + device_id: String — identificativo univoco del Dispositivo_G da eliminare.

Metodi

DeleteDeviceCommand non definisce metodi propri.

5.2.2.3) DeleteDeviceService

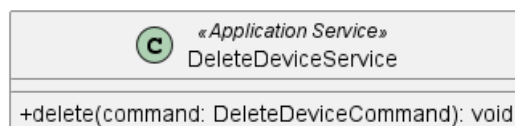


Figura 42: DeleteDeviceService

Descrizione

DeleteDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di eliminazione di un Dispositivo_G. Implementa l'interfaccia *DeleteDeviceUseCase* e riceve un *DeleteDeviceCommand* contenente l'identificativo del Dispositivo_G, coordinandone la rimozione dal sistema.

Attributi

- - delete_device_port: DeleteDevicePort — porta outbound utilizzata per eliminare il device

Metodi

- + delete(command: DeleteDeviceCommand): void — riceve il Command Object contenente l'identificativo univoco del Dispositivo_G e ne coordina la rimozione tramite *DeleteDevicePort*.

5.2.2.4) DeleteDevicePort

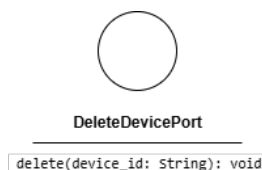


Figura 43: DeleteDevicePort

Descrizione

DeleteDevicePort è l'interfaccia_G (Outbound Port) che definisce il contratto per la rimozione fisica di un Dispositivo_G dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *DeleteDeviceService*.

Attributi

DeleteDevicePort non definisce attributi.

Metodi

- + `delete(device_id: String): void` — firma del metodo che esegue la rimozione fisica del Dispositivo_G identificato da `device_id` dal sistema di persistenza.

5.2.3) UpdateDevice

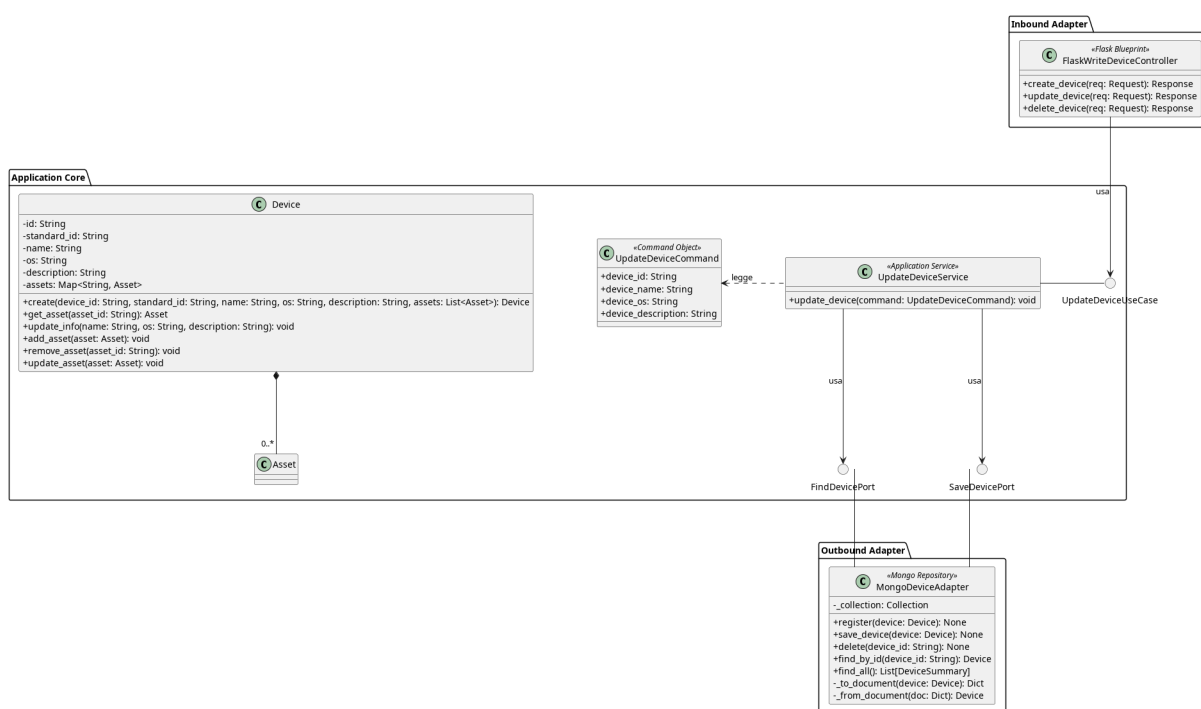


Figura 44: Caso d'uso_G UpdateDevice

Il diagramma illustra l'architettura del modulo dedicato alla modifica e al salvataggio dello stato di un Dispositivo_G esistente.

- Per la definizione di *FlaskWriteDeviceController*, vedere la **Sezione 5.2.1.1**.
- Per la definizione di *Device*, vedere la **Sezione 5.1.1**.

- Per la definizione di *MongoDeviceAdapter*, vedere la **Sezione 5.2.1.6**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.3.1) UpdateDeviceUseCase

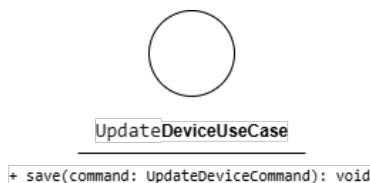


Figura 45: UpdateDeviceUseCase

Descrizione

UpdateDeviceUseCase è l'interfacciag (Inbound Port) che definisce il contratto per la modifica di un Dispositivo_G esistente. Viene implementata da *UpdateDeviceService* e utilizzata da *FlaskWriteDeviceController*.

Attributi

UpdateDeviceUseCase non definisce attributi.

Metodi

- `update_device(command: UpdateDeviceCommand)` — firma del metodo delegato all'esecuzione della logica di aggiornamento a partire dai dati contenuti nel comando.

5.2.3.2) UpdateDeviceCommand



Figura 46: UpdateDeviceCommand

Descrizione

UpdateDeviceCommand è il Command Object che veicola i dati necessari alla modifica di un Dispositivo_G dal controller al service. Analogamente a *CreateDeviceCommand*, separa la struttura dei dati in ingresso dall'entità di dominio.

Attributi

- `+ device_id: String` — identificativo univoco del Dispositivo_G da aggiornare.
- `+ device_name: String` — nome aggiornato del Dispositivo_G.

- + device_os: String — sistema operativo aggiornato.
- + device_description: String — descrizione testuale aggiornata.

Metodi

UpdateDeviceCommand non definisce metodi.

5.2.3.3) UpdateDeviceService

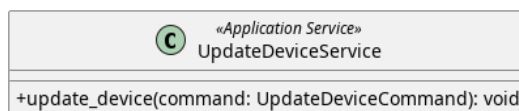


Figura 47: UpdateDeviceService

Descrizione

UpdateDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di aggiornamento di un *DispositivoG* esistente. Implementa l'interfaccia *UpdateDeviceUseCase*, riceve il comando in ingresso e ne coordina la persistenza tramite *SaveDevicePort*.

Attributi

- - find_device_port: FindDevicePort — utilizza la porta di outbound per prelevare il device
- - save_device_port: SaveDevicePort — utilizza la porta di outbound per salvare le modifiche

Metodi

- + update_device(command: UpdateDeviceCommand): void — concretizza il contratto definito da *SaveDeviceUseCase*. Mappa i dati del comando nell'entità *Device* e ne richiede l'aggiornamento tramite *SaveDevicePort*.

5.2.3.4) SaveDevicePort

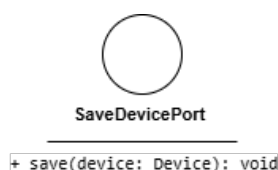


Figura 48: SaveDevicePort

Descrizione

SaveDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per l'aggiornamento fisico di un *DispositivoG* nel sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *UpdateDeviceService*.

Attributi

UpdateDevicePort non definisce attributi.

Metodi

- + `save(device: Device): void` — firma del metodo che esegue l'aggiornamento fisico del Dispositivo_G nel sistema di persistenza.

5.2.3.5) FindDevicePort

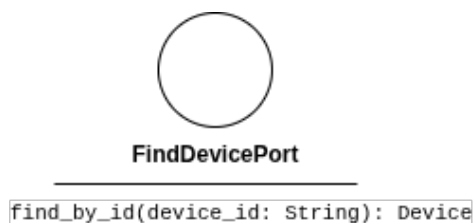


Figura 49: FindDevicePort

Descrizione

FindDevicePort è l'interfaccia_G (Outbound Port) che definisce il contratto per prelevare un Device tramite id. Viene implementata da *MongoDeviceAdapter* e utilizzata da *UpdateDeviceService*.

Attributi

FindDevicePort non definisce attributi.

Metodi

- + `find_by_id(device_id: String): Device` — firma del metodo che esegue l'aggiornamento fisico del Dispositivo_G nel sistema di persistenza.

5.2.4) GetDeviceDetail

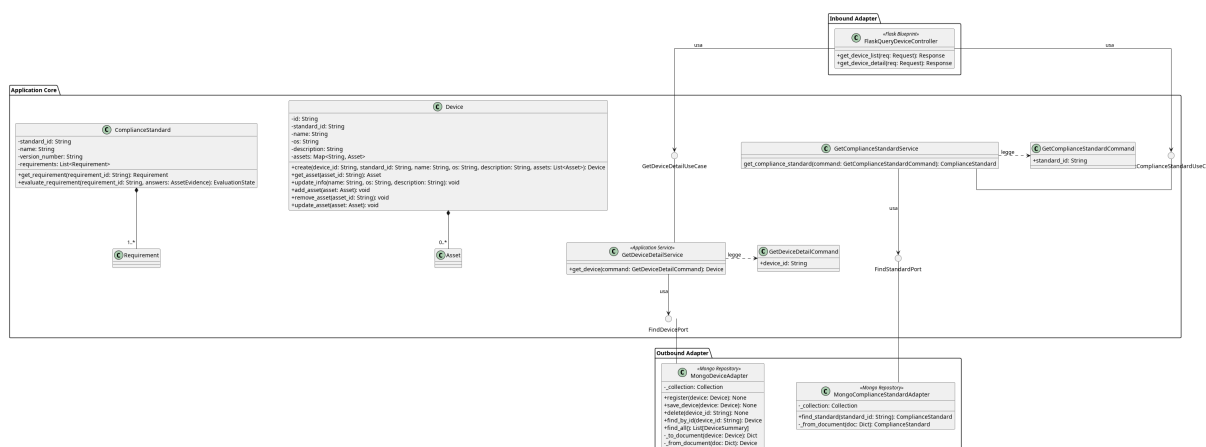


Figura 50: GetDeviceDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero del dettaglio di un Dispositivo_G.

- Per la definizione di *Device*, vedere la **Sezione 5.1.1**.

- Per la definizione di *MongoDeviceAdapter*, vedere la **Sezione 5.2.1.6**.
- Per la definizione di *FindDevicePort*, vedere la **Sezione 5.2.3.5**.
- Per la definizione di *ComplianceStandard*, vedere la **Sezione 5.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso_G.

5.2.4.1) FlaskQueryDeviceController



Figura 51: FlaskQueryDeviceController

Descrizione

FlaskQueryDeviceController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di lettura relative ai Dispositivi e le inoltra al livello applicativo.

Attributi

- - `get_device_list_use_case`: `GetDeviceListUseCase` — inbound port usata per prendere la lista dei dispositivi
- - `get_device_detail_use_case`: `GetDeviceDetailUseCase` — inbound port usata per prendere il dettaglio di un dispositivo_G
- - `get_compliance_standard_use_case`: `GetComplianceStandardUseCase` — inbound port usata per prendere lo Standard

Metodi

- + `get_device_list(req: Request): Response` — riceve la richiesta HTTP di recupero della lista dei Dispositivi e restituisce una risposta HTTP con l'elenco sintetico.
- + `get_device_detail(req: Request): Response` — riceve la richiesta HTTP di recupero del dettaglio di un Dispositivo_G specifico e restituisce una risposta HTTP con i dati completi, per recuperare questi dati utilizza le porte *GetDeviceDetailUseCase* per le informazioni del dispositivo_G e *GetComplianceStandardUseCase* per recuperare lo standard associato.

5.2.4.2) GetDeviceDetailUseCase

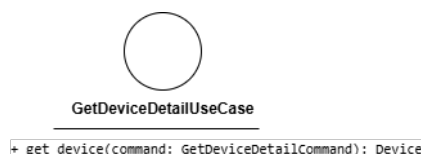


Figura 52: GetDeviceDetailUseCase

Descrizione

GetDeviceDetailUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per il recupero del dettaglio di un Dispositivo_G. Viene implementata da *GetDeviceDetailService* e utilizzata da *FlaskQueryDeviceController*.

Attributi

GetDeviceDetailUseCase non definisce attributi.

Metodi

- + `get_device(command: GetDeviceDetailCommand): Device` — firma del metodo delegato al recupero del Dispositivo_G corrispondente al Command fornito.

5.2.4.3) GetDeviceDetailService

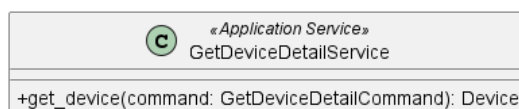


Figura 53: GetDeviceDetailService

Descrizione

GetDeviceDetailService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di un Dispositivo_G. Implementa l'interfaccia_G *GetDeviceDetailUseCase* e coordina il recupero dei dati tramite la porta outbound *FindDevicePort*.

Attributi

- `find_device_port: FindDevicePort` — outbound port usata per prelevare un dispositivo_G.

Metodi

- + `get_device(command: GetDeviceDetailCommand): Device` — concretizza il contratto definito da *GetDeviceDetailUseCase*. Recupera il Dispositivo_G corrispondente all'identificativo contenuto nel Command tramite *FindDevicePort*.

5.2.4.4) GetDeviceDetailCommand

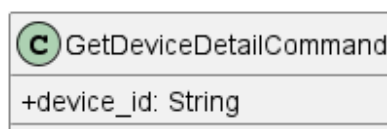


Figura 54: GetDeviceDetailCommand

Descrizione

GetDeviceDetailCommand è utilizzato per trasportare i dati necessari al recupero del dettaglio di un *DispositivoG*. Incapsula i parametri di input del metodo esposto da *GetDeviceDetailUseCase*.

Attributi

- + `device_id`: String — identificativo univoco del *DispositivoG* di cui recuperare il dettaglio.

Metodi

GetDeviceDetailCommand non definisce metodi propri.

5.2.4.5) GetComplianceStandardUseCase

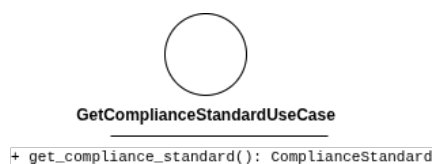


Figura 55: GetComplianceStandardUseCase

Descrizione

GetComplianceStandardUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per il recupero dello standard. Viene implementata da *GetComplianceStandardService* e utilizzata da *FlaskQueryDeviceController*.

Attributi

GetComplianceStandardUseCase non definisce attributi.

Metodi

- + `get_compliance_standard(command: GetComplianceStandardCommand): ComplianceStandard` — firma del metodo delegato al recupero dello Standard corrispondente al Command fornito.

5.2.4.6) GetComplianceStandardService

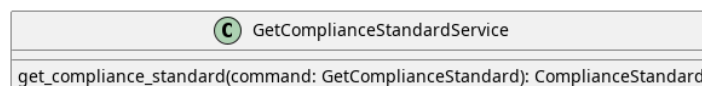


Figura 56: GetComplianceStandardService

Descrizione

GetComplianceStandardService è il service applicativo appartenente all'Application Core responsabile del recupero di un Compliance Standard. Implementa l'interfaccia_G *GetComplianceStandardUseCase* e coordina il recupero dei dati tramite la porta out-bound *FindStandardPort*.

Attributi

- `find_standard_port`: `FindStandardPort` — outbound port usata per prelevare lo Standard.

Metodi

- + `get_compliance_standard(command: GetComplianceStandardCommand): ComplianceStandard` — concretizza il contratto definito da *GetComplianceStandardUseCase*. Recupera il Compliance Standard corrispondente all'identificativo contenuto nel Command tramite *FindStandardPort*.

5.2.4.7) GetComplianceStandardCommand

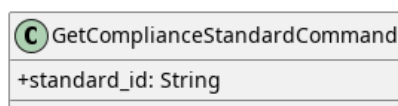


Figura 57: GetComplianceStandardCommand

Descrizione

GetComplianceStandardCommand è utilizzato per trasportare i dati necessari al recupero del dettaglio di un Compliance Standard. Incapsula i parametri di input del metodo esposto da *GetComplianceStandardUseCase*.

Attributi

- + `standard_id`: `String` — identificativo univoco dello Standard di cui recuperare il dettaglio.

Metodi

GetComplianceStandardCommand non definisce metodi propri.

5.2.4.8) FindStandardPort

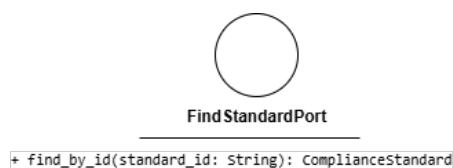


Figura 58: FindStandardPort

Descrizione

FindStandardPort è l'interfaccia_G (Outbound Port) che definisce il contratto per il recupero di uno standard di conformità_G dal sistema di persistenza. Viene implementata da *MongoStandardAdapter* e utilizzata da *OpenEvaluationSessionService*.

Attributi

FindStandardPort non definisce attributi.

Metodi

- + find_by_id(standard_id: String): ComplianceStandard — firma del metodo che recupera lo standard di conformità_G corrispondente all'identificativo fornito.

5.2.4.9) MongoStandardAdapter



Figura 59: MongoStandardAdapter

Descrizione

MongoStandardAdapter è la classe dell'Outbound Adapter annotata come *Mongo Repository* che implementa *FindStandardPort*, traducendo le operazioni di recupero degli standard di conformità_G in interazioni concrete con MongoDB.

Attributi

MongoStandardAdapter non definisce attributi propri nel diagramma.

Metodi

- + save(standard: ComplianceStandard): void — persiste uno standard di conformità_G nel database.
- + find_by_id(standard_id: String): ComplianceStandard — recupera lo standard di conformità_G corrispondente all'identificativo fornito.

5.2.5) GetDeviceList

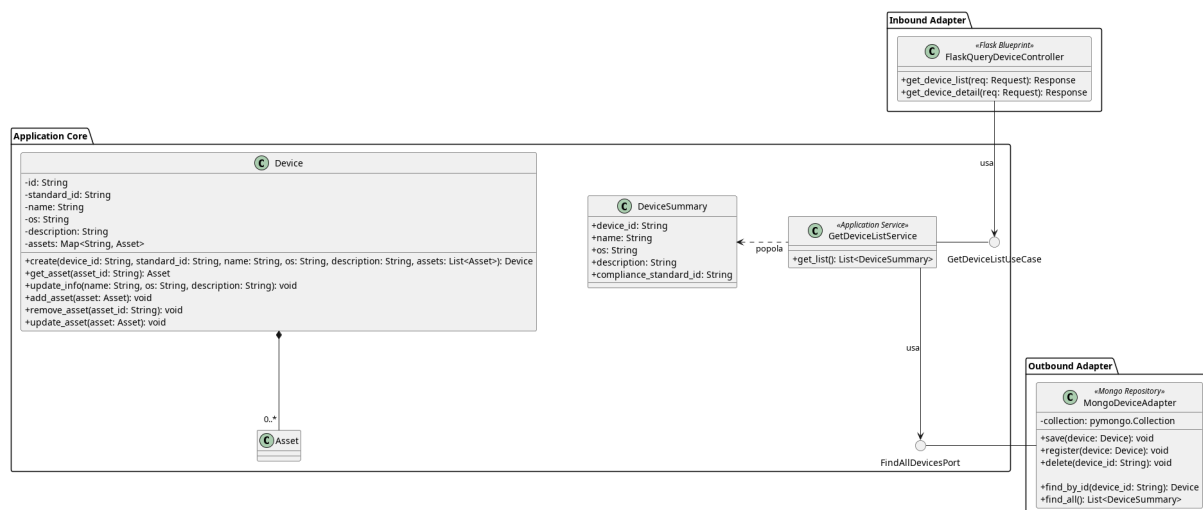


Figura 60: Caso d'uso_G GetDeviceList

Il diagramma illustra l'architettura del modulo dedicato al recupero della lista sintetica dei Dispositivi.

- Per la definizione di *FlaskQueryDeviceController*, vedere la **Sezione 5.2.4.1**.
- Per la definizione di *Device*, vedere la **Sezione 5.1.1**.
- Per la definizione di *MongoDeviceAdapter*, vedere la **Sezione 5.2.1.6**.
- Per la definizione di *DeviceSummary*, vedere la **Sezione 5.1.26**

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.5.1) GetDeviceListUseCase

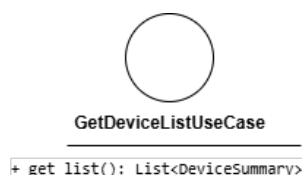


Figura 61: GetDeviceListUseCase

Descrizione

GetDeviceListUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per il recupero della lista sintetica dei Dispositivi. Viene implementata da *GetDeviceListService* e utilizzata da *FlaskQueryDeviceController*.

Attributi

GetDeviceListUseCase non definisce attributi.

Metodi

- + get_list(): List<DeviceSummary> — firma del metodo delegato al recupero della lista sintetica di tutti i Dispositivi presenti nel sistema.

5.2.5.2) GetDeviceListService



Figura 62: GetDeviceListService

Descrizione

GetDeviceListService è il service applicativo appartenente all'Application Core responsabile del recupero della lista sintetica dei Dispositivi. Implementa l'interfaccia_G *GetDeviceListUseCase* e coordina il recupero tramite la porta outbound *FindAllDevicesPort*.

Attributi

- - `find_port: FindAllDevicesPort` — porta outbound utilizzata per il recupero della lista dei Dispositivi dal sistema di persistenza.

Metodi

- + `get_list(): List<DeviceSummary>` — concretizza il contratto definito da *GetDeviceListUseCase*. Recupera la lista sintetica di tutti i Dispositivi tramite *FindAllDevicesPort*.

5.2.5.3) DeviceSummary

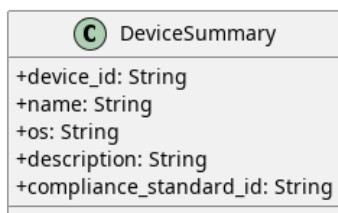


Figura 63: DeviceSummary

Descrizione

DeviceSummary è il Data Transfer Object che veicola la rappresentazione sintetica di un *Dispositivo_G* verso il livello di presentazione. Espone esclusivamente le informazioni necessarie alla visualizzazione in lista, evitando di esporre l'intera entità di dominio.

Attributi

- + `device_id: String` — identificativo univoco del dispositivo_G
- + `name: String` — nome del dispositivo_G
- + `os: String` — sistema operativo del dispositivo_G
- + `description: String` — descrizione del dispositivo_G
- + `compliance_standard_id: String` — identificativo univoco dello Standard

Metodi

DeviceSummary non definisce metodi.

5.2.5.4) FindAllDevicesPort

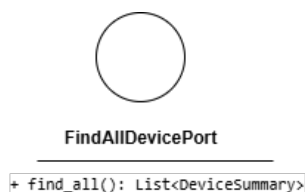


Figura 64: FindAllDevicesPort

Descrizione

FindAllDevicesPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero della lista sintetica di tutti i Dispositivi dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *GetDeviceListService*.

Attributi

FindAllDevicesPort non definisce attributi.

Metodi

- `+ find_all(): List<DeviceSummary>` — firma del metodo che recupera la lista sintetica di tutti i Dispositivi presenti nel sistema di persistenza.

5.2.6) GetDeviceEvaluationDetail

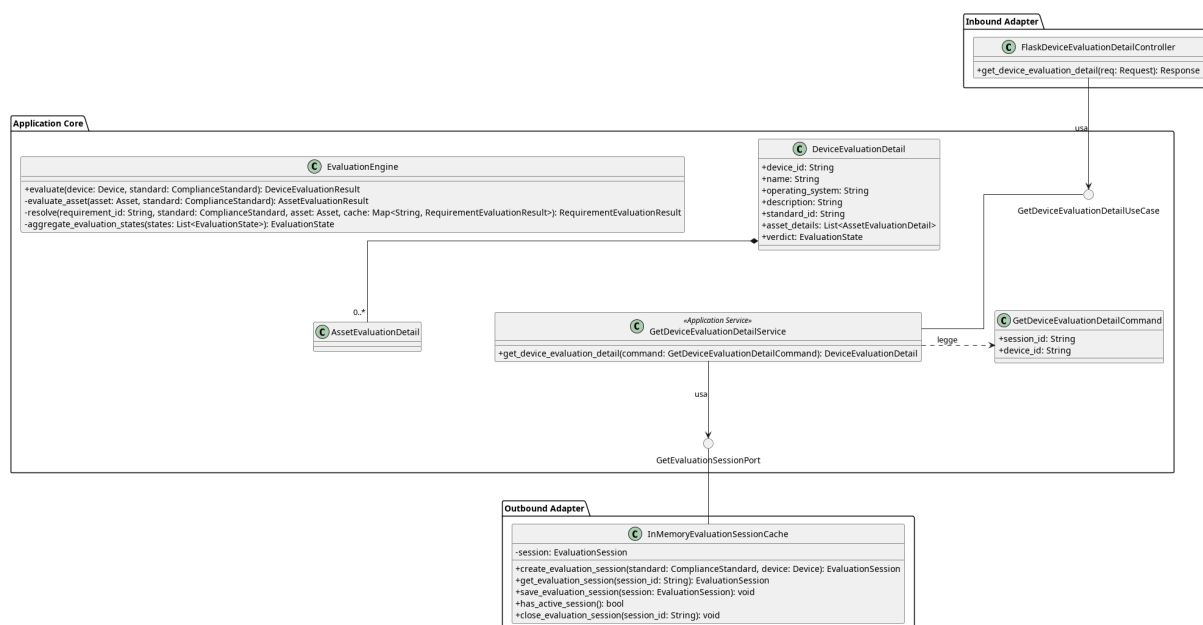


Figura 65: GetDeviceEvaluationDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero della dashboard_G di un Dispositivo_G, che aggrega le informazioni della sessione di valutazione_G attiva in una vista sintetica.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la **Sezione 5.3.1.7**.
- Per la definizione di *EvaluationEngine*, vedere la **Sezione 5.1.14**
- per la definizione di *DeviceEvaluationDetail*, vedere la di **Sezione 5.1.22**

5.2.6.1) FlaskDeviceEvaluationDetailController

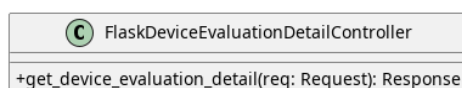


Figura 66: FlaskDeviceEvaluationDetailController

Descrizione

FlaskDeviceEvaluationDetailController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di lettura di un device per la dashboard_G.

Attributi

- get_device_evaluation_detail_use_case: *GetDeviceEvaluationDetailUseCase*
— inbound port usata per prelevare un *DeviceEvaluationDetail*.

Metodi

- + get_device_evaluation_detail(req: Request): Response — riceve la richiesta HTTP di recupero di un *DeviceEvaluationDetail* per la dashboard_G.

5.2.6.2) GetDeviceEvaluationDetailCommand

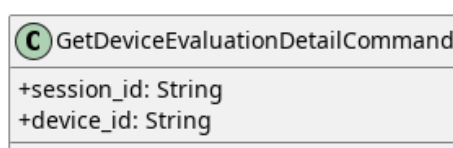


Figura 67: GetDeviceEvaluationDetailCommand

Descrizione

GetDeviceEvaluationDetailCommand è il Command Object utilizzato per trasportare i dati necessari al recupero della dashboard_G di un Dispositivo_G. Incapsula i parametri di input del metodo esposto da *GetDeviceEvaluationDetailUseCase*.

Attributi

- + session_id: String — identificativo univoco della sessione
- + device_id: String — identificativo univoco del Dispositivo_G di cui recuperare le informazioni.

Metodi

GetDeviceEvaluationDetailCommand non definisce metodi propri.

5.2.6.3) GetDeviceEvaluationDetailUseCase



Figura 68: GetDeviceEvaluationDetailUseCase

Descrizione

GetDeviceEvaluationDetailUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per il recupero della dashboard_G di un Dispositivo_G. Viene implementata da *GetDeviceEvaluationDetailService* e utilizzata da *FlaskQueryDashboardController*.

Attributi

GetDeviceEvaluationDetailUseCase non definisce attributi.

Metodi

- + `get_device_evaluation_detail(command: GetDeviceEvaluationDetailCommand): DeviceEvaluationDetail` — firma del metodo delegato al recupero e all'aggregazione delle informazioni della sessione di valutazione_G.

5.2.6.4) GetDeviceEvaluationDetailService

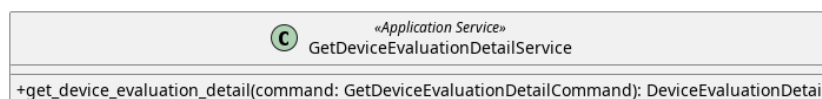


Figura 69: GetDeviceEvaluationDetailService

Descrizione

GetDeviceEvaluationDetailService è il service applicativo appartenente all'Application Core responsabile del recupero della dashboard_G di un Dispositivo_G. Implementa l'interfaccia_G *GetDeviceEvaluationDetailUseCase*, recupera la sessione attiva tramite *GetEvaluationSessionPort* e costruisce l' *EvaluationDetailCommand* con le informazioni aggregate.

Attributi

- - `get_evaluation_session_port: GetEvaluationSessionPort` — outbound port per prelevare la sessione di valutazione_G.

Metodi

- + `get_device_evaluation_detail(command: GetDeviceEvaluationDetailCommand): DeviceEvaluationDetail` —

concretizza il contratto definito da *GetDeviceEvaluationDetailUseCase*. Recupera la sessione attiva, aggrega le informazioni del Dispositivo_G e dei suoi Asset_G e restituisce la rappresentazione *DeviceEvaluationDetail*.

5.2.6.5) DTO

Qui vengono elencati i DTO usati dal controller per gestire ed esporre i dettagli della valutazione_G di un dispositivo_G.

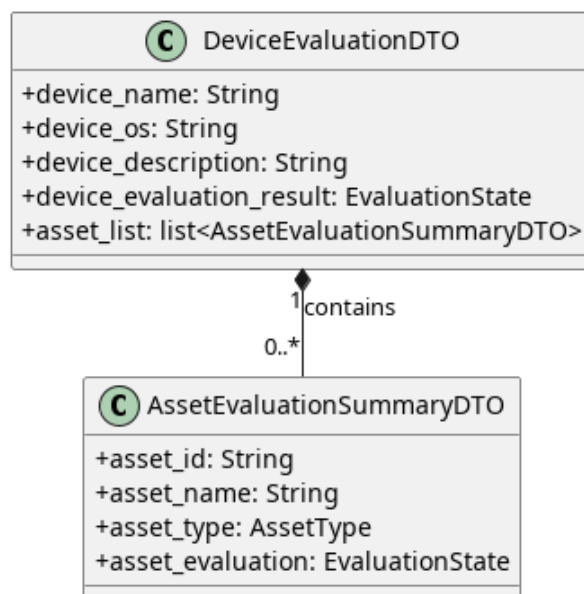


Figura 70: DeviceEvaluationDTO

5.2.6.5.1) DeviceEvaluationDTO

Descrizione

DeviceEvaluationDTO è il Data Transfer Object principale utilizzato per consolidare e trasportare le informazioni anagrafiche e l'esito complessivo della valutazione_G di un intero dispositivo_G. Funge da aggregatore principale per la vista dashboard_G, includendo al suo interno una lista sintetica dello stato di tutti gli asset_G ad esso associati.

Attributi

- + `device_name`: String — nome assegnato al dispositivo_G.
- + `device_os`: String — sistema operativo in uso sul dispositivo_G.
- + `device_description`: String — breve descrizione testuale del dispositivo_G.
- + `device_evaluation_result`: EvaluationState — stato globale e finale della valutazione_G di conformità_G per l'intero dispositivo_G.
- + `asset_list`: List<AssetEvaluationSummaryDTO> — tupla contenente i DTO di riepilogo per ciascun asset_G analizzato all'interno del dispositivo_G.

Metodi

DeviceEvaluationDTO non definisce metodi.

5.2.6.5.2) AssetEvaluationSummaryDTO

Descrizione

AssetEvaluationSummaryDTO è un Data Transfer Object leggero e di supporto, istanziato esclusivamente per popolare la lista degli asset_G all'interno di un *DeviceEvaluationDTO*. Fornisce una vista sintetica contenente le informazioni

essenziali e il verdetto di un singolo asset_G, risultando ottimale per le visualizzazioni a elenco o per le tabelle riassuntive nella dashboard_G.

Attributi

- + asset_id: String — identificativo univoco dell'asset_G valutato.
- + asset_name: String — nome dell'asset_G.
- + asset_type: AssetType — categoria o tipologia a cui appartiene l'asset_G.
- + asset_evaluation: EvaluationState — stato corrente e finale della valutazione_G specifica per questo asset_G.

Metodi

AssetEvaluationSummaryDTO non definisce metodi.

5.3) Asset_G

5.3.1) CreateAsset

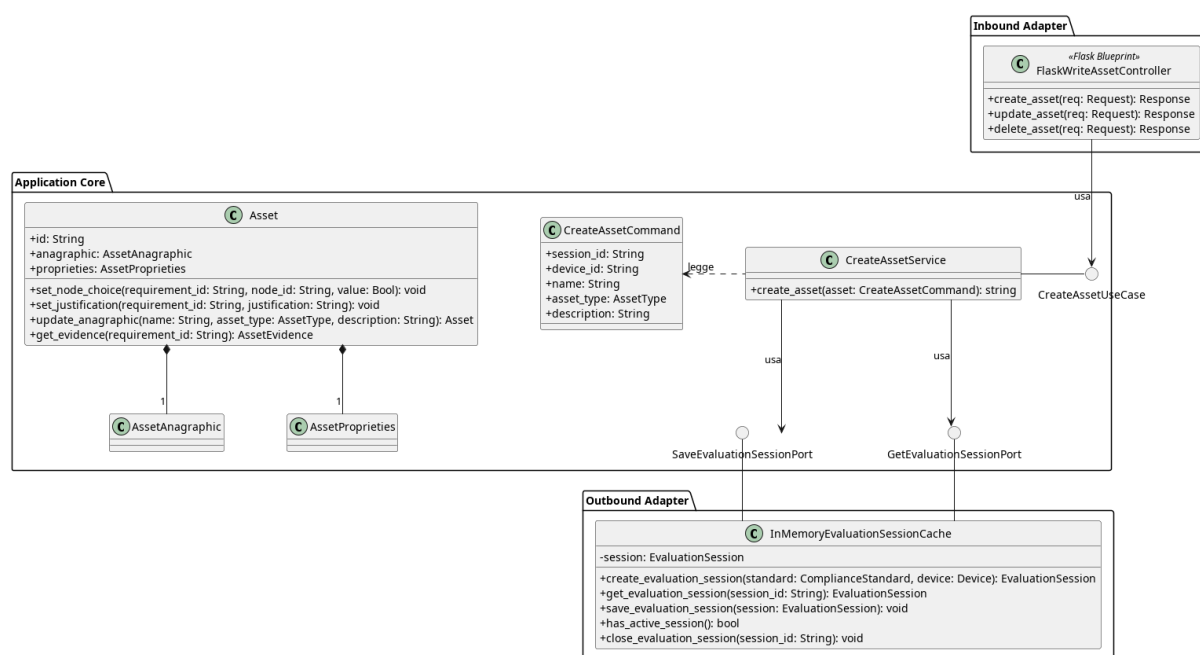


Figura 71: Caso d'uso_G CreateAsset

Il diagramma illustra l'architettura del modulo di creazione di un Asset_G all'interno di una sessione di valutazione_G attiva.

- per la definizione di Asset_G, vedere la di **Sezione 5.1.2**

5.3.1.1) FlaskWriteAssetController



Figura 72: FlaskWriteAssetController

Descrizione

FlaskWriteAssetController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione (scrittura) degli Asset_G e le inoltra al livello applicativo.

Attributi

- - `create_asset_use_case`: `CreateAssetUseCase` — inbound port usata per la creazione di un asset_G
- - `delete_asset_use_case`: `DeleteAssetUseCase` — inbound port usata per la rimozione di un asset_G
- - `update_asset_use_case`: `UpdateAssetUseCase` — inbound port usata per l'aggiornamento di un asset_G

Metodi

- + `create_asset(req: Request): Response` — riceve la richiesta HTTP di creazione di un nuovo Asset_G, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `update_asset(req: Request): Response` — riceve la richiesta HTTP di aggiornamento di un Asset_G esistente, estrae i dati modificati e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `delete_asset(req: Request): Response` — riceve la richiesta HTTP di eliminazione di un Asset_G, estrae l'identificativo dalla richiesta e lo inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.3.1.2) CreateAssetUseCase

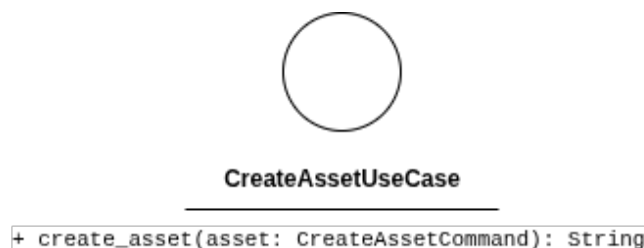


Figura 73: CreateAssetUseCase

Descrizione

CreateAssetUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per la creazione di un nuovo Asset_G all'interno della sessione di valutazione_G. Viene implementata da *CreateAssetService* e utilizzata da *FlaskWriteAssetController*.

Attributi

CreateAssetUseCase non definisce attributi.

Metodi

- + `create_asset(assetG: CreateAssetCommand): String` — firma del metodo delegato all'esecuzione della logica di creazione a partire dai dati contenuti nel comando.

5.3.1.3) CreateAssetCommand

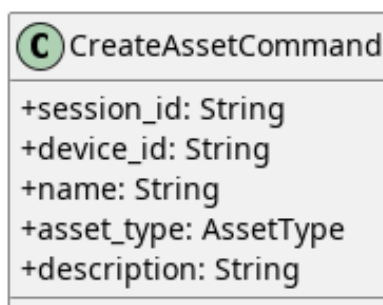


Figura 74: CreateAssetCommand

Descrizione

CreateAssetCommand è il Command Object che veicola i dati necessari alla creazione di un Asset_G dal controller al service. Separa la struttura dei dati in ingresso dall'entità di dominio, rendendo esplicita l'intenzione dell'operazione.

Attributi

- + `session_id: String` — identificativo della sessione di valutazione_G attiva a cui l'Asset_G viene associato.
- + `device_id: String` — identificativo del dispositivo_G che lo contiene.
- + `name: String` — nome del nuovo Asset_G.
- + `type: AssetType` — tipo dell'Asset_G.
- + `description: String` — descrizione testuale dell'Asset_G.

Metodi

CreateAssetCommand non definisce metodi.

5.3.1.4) CreateAssetService

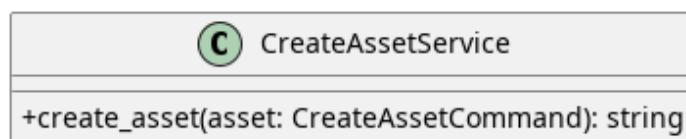


Figura 75: CreateAssetService

Descrizione

CreateAssetService è il service applicativo appartenente all'Application Core responsabile della logica di creazione di un *AssetG*. Implementa l'interfaccia *CreateAssetUseCase*, recupera la sessione attiva tramite *GetEvaluationSession*, aggiunge il nuovo *AssetG* e persiste la sessione aggiornata tramite *SaveEvaluationSession*.

Attributi

- - `save_evaluation_session_port`: *SaveEvaluationSessionPort* — outbound port usata per il salvataggio delle modifiche nella sessione
- - `get_evaluation_session_port`: *GetEvaluationSessionPort* — outbound port usata per prelevare la sessione di valutazione

Metodi

- + `create_asset(assetG: CreateAssetCommand): string` — concretizza il contratto definito da *CreateAssetUseCase*. Recupera la sessione attiva, vi aggiunge il nuovo *AssetG* e ne persiste lo stato aggiornato. Ritorna l'id dell'*assetG* creato.

5.3.1.5) InMemoryEvaluationSessionCache

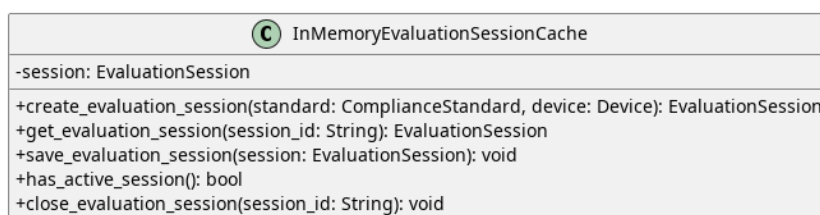


Figura 76: InMemoryEvaluationSessionCache

Descrizione

InMemoryEvaluationSessionCache è l'Outbound Adapter che funge da Session Cache. Poiché il sistema prevede un utilizzo mono-utente, la classe gestisce in memoria una singola sessione di valutazione *G* attiva per volta.

Attributi

- - `session`: *EvaluationSession* — contiene l'oggetto *EvaluationSession*

Metodi

- + `create_evaluation_session(standard: ComplianceStandard, device: Device): EvaluationSession` — crea e registra una nuova sessione di valutazione_G.
- + `get_evaluation_session(session_id: String): EvaluationSession` — recupera la sessione corrispondente all'identificativo fornito.
- + `save_evaluation_session(session: EvaluationSession): void` — aggiorna la sessione in memoria.
- + `has_active_session(): bool` — verifica_G se esiste una sessione attiva.
- + `close_evaluation_session(session_id: String): void` — chiude la sessione identificata da `session_id`.

5.3.1.6) SaveEvaluationSessionPort



Figura 77: SaveEvaluationSessionPort

Descrizione

SaveEvaluationSessionPort è l'interfaccia_G (Outbound Port) che definisce il contratto per il salvataggio della sessione di valutazione_G nel sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da tutti i service del modulo che modificano lo stato della sessione, tra cui *CreateAssetService*, *SaveAssetService* e *DeleteAssetService*.

Attributi

SaveEvaluationSessionPort non definisce attributi.

Metodi

- + `save_evaluation_session(session: EvaluationSession): void` — firma del metodo che persiste la sessione aggiornata nel sistema in memoria.

5.3.1.7) GetEvaluationSessionPort



Figura 78: GetEvaluationSessionPort

Descrizione

GetEvaluationSessionPort è l'interfaccia_G (Outbound Port) che definisce il contratto per il recupero della sessione di valutazione_G dal sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da tutti i service del

modulo che necessitano di accedere alla sessione corrente, tra cui *CreateAssetService*, *SaveAssetService*, *DeleteAssetService* e *GetAssetAnagraphicService*.

Attributi

GetEvaluationSessionPort non definisce attributi.

Metodi

- + `get_evaluation_session(session_id: String): EvaluationSession` — firma del metodo che recupera la sessione di valutazione_G attiva corrispondente all'identificativo fornito.

5.3.2) UpdateAsset

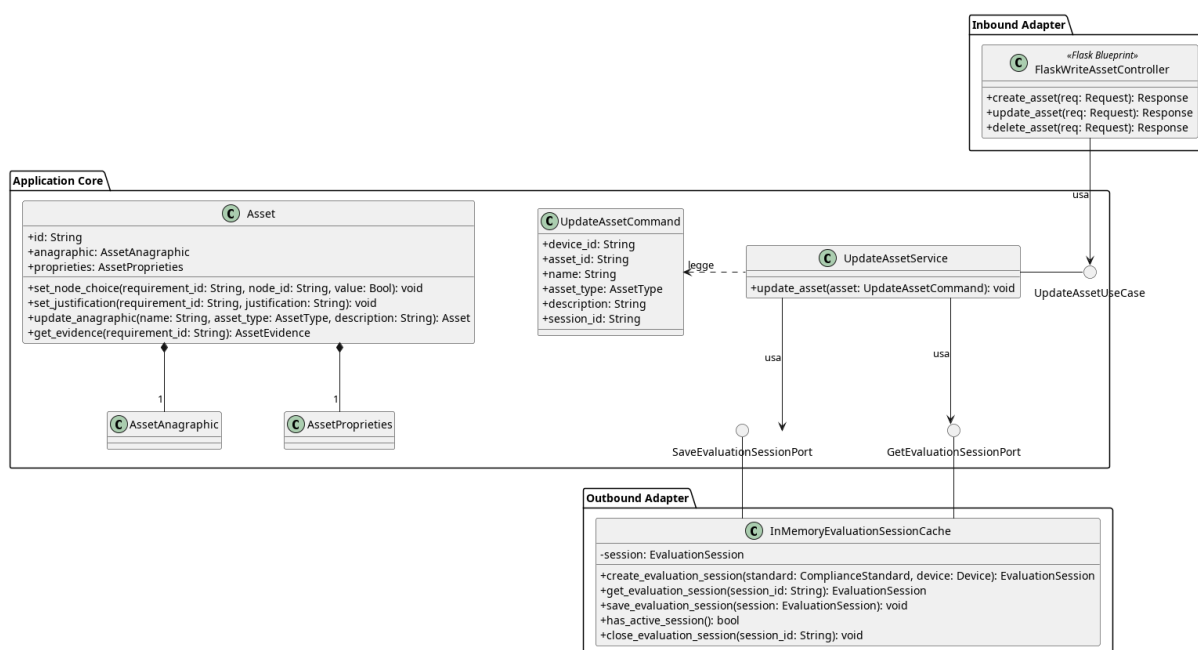


Figura 79: Caso d'uso_G UpdateAsset

Il diagramma illustra l'architettura del modulo di modifica di un Asset_G esistente all'interno di una sessione di valutazione_G attiva.

- Per la definizione di *FlaskWriteAssetController*, vedere la **Sezione 5.3.1.1**.
- Per la definizione di *Asset_G*, vedere la **Sezione 5.1.2**.
- Per la definizione di *SaveEvaluationSession*, vedere la **Sezione 5.3.1.6**.
- Per la definizione di *GetEvaluationSession*, vedere la **Sezione 5.3.1.7**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la **Sezione 5.3.1.5**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso_G.

5.3.2.1) UpdateAssetUseCase



Figura 80: UpdateAssetUseCase

Descrizione

UpdateAssetUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per la modifica di un *Asset_G* esistente all'interno della sessione di valutazione_G. Viene implementata da *UpdateAssetService* e utilizzata da *FlaskWriteAssetController*.

Attributi

UpdateAssetUseCase non definisce attributi.

Metodi

- + `update_asset(assetG: UpdateAssetCommand): void` — firma del metodo delegato all'esecuzione della logica di aggiornamento a partire dai dati contenuti nel comando.

5.3.2.2) UpdateAssetCommand

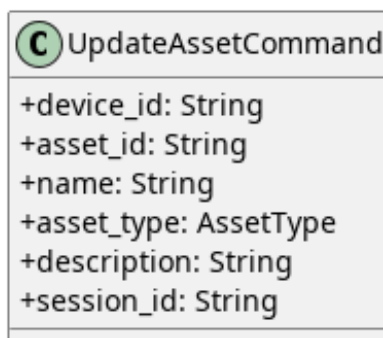


Figura 81: UpdateAssetCommand

Descrizione

UpdateAssetCommand è il Command Object che veicola i dati necessari alla modifica di un *Asset_G* dal controller al service. Separa la struttura dei dati in ingresso dall'entità di dominio.

Attributi

- + `device_id: String` — identificativo del device a cui appartiene.
- + `asset_id: String` — identificativo univoco dell'*Asset_G* da modificare.
- + `name: String` — nome aggiornato dell'*Asset_G*.
- + `asset_type: AssetType` — tipo aggiornato dell'*Asset_G*.
- + `description: String` — descrizione testuale aggiornata dell'*Asset_G*.

- + session_id: String — identificativo della sessione di valutazione_G attiva.

Metodi

UpdateAssetCommand non definisce metodi.

5.3.2.3) UpdateAssetService

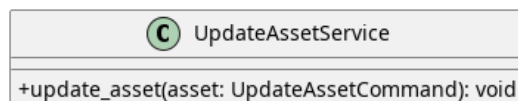


Figura 82: UpdateAssetService

Descrizione

UpdateAssetService è il service applicativo appartenente all'Application Core responsabile della logica di aggiornamento di un Asset_G esistente. Implementa l'interfaccia_G *UpdateAssetUseCase*, recupera la sessione attiva tramite *GetEvaluationSession*, aggiorna l'Asset_G corrispondente e persiste la sessione modificata tramite *SaveEvaluationSession*.

Attributi

- - save_evaluation_session_port: SaveEvaluationSessionPort — outbound port usata per il salvataggio delle modifiche nella sessione
- - get_evaluation_session_port: GetEvaluationSessionPort — outbound port usata per prelevare la sessione di valutazione_G

Metodi

- + update_asset(asset_G: UpdateAssetCommand): void — concretizza il contratto definito da *UpdateAssetUseCase*. Recupera la sessione attiva, individua l'Asset_G da aggiornare tramite asset_id e ne persiste lo stato modificato.

5.3.3) DeleteAsset

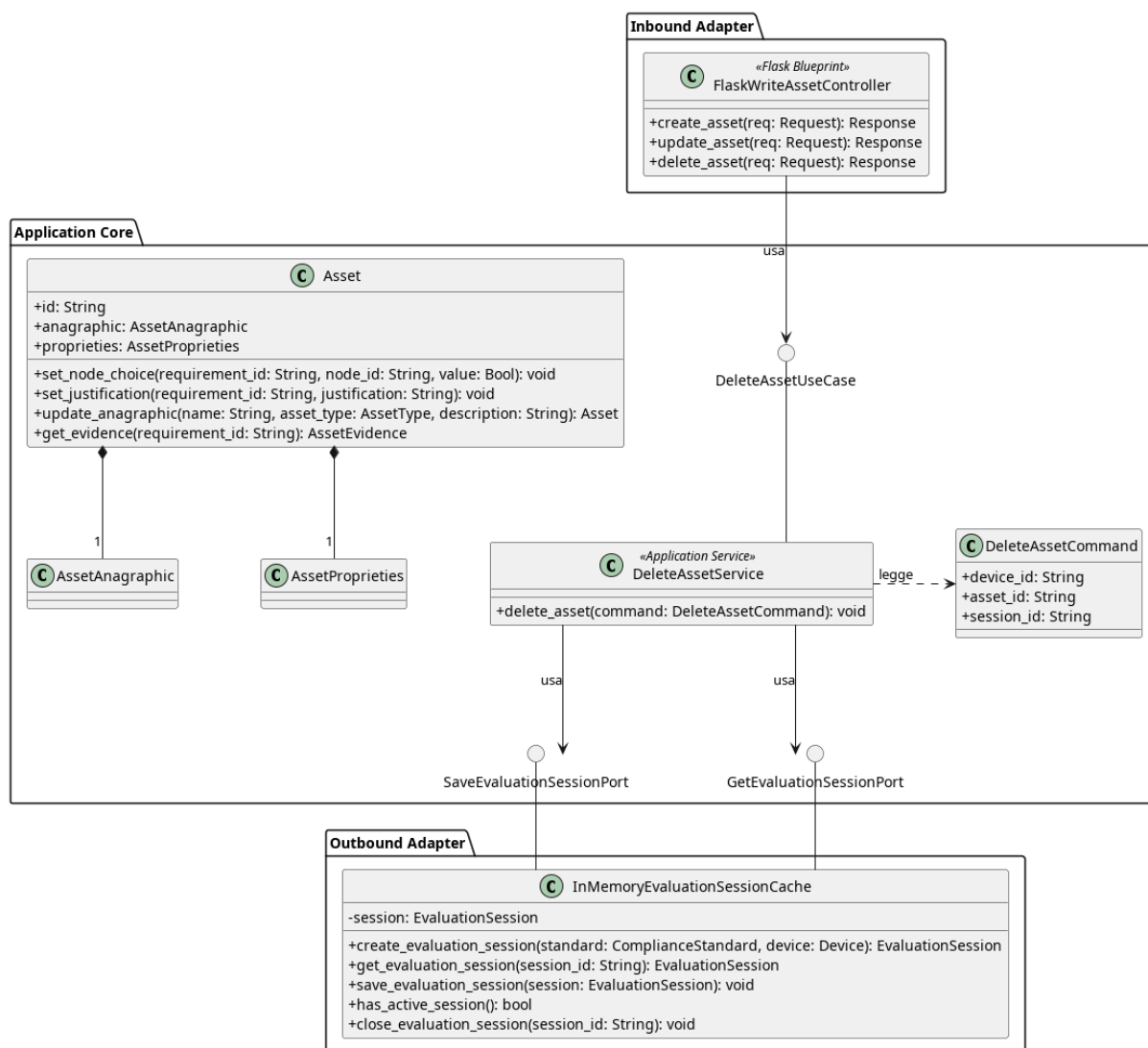


Figura 83: Caso d'uso_G DeleteAsset

Il diagramma illustra l'architettura del modulo di eliminazione di un Asset_G esistente all'interno di una sessione di valutazione_G attiva.

- Per la definizione di *FlaskWriteAssetController*, vedere la **Sezione 5.3.1.1.**
- Per la definizione di *Asset_G*, vedere la **Sezione 5.1.2.**
- Per la definizione di *SaveEvaluationSession*, vedere la **Sezione 5.3.1.6.**
- Per la definizione di *GetEvaluationSession*, vedere la **Sezione 5.3.1.7.**
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la **Sezione 5.3.1.5.**

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.3.1) DeleteAssetCommand



Figura 84: DeleteAssetCommand

Descrizione

DeleteAssetCommand è il Command Object utilizzato per trasportare i dati necessari all'eliminazione di un `AssetG` dalla sessione di valutazione_G. Incapsula i parametri di input del metodo esposto da *DeleteAssetUseCase*.

Attributi

- `+ device_id: String` — identificativo univoco del dispositivo_G che contiene l'`assetG`.
- `+ asset_id: String` — identificativo univoco dell'`AssetG` da eliminare.
- `+ session_id: String` — identificativo univoco della sessione.

Metodi

DeleteAssetCommand non definisce metodi propri.

5.3.3.2) DeleteAssetUseCase



Figura 85: DeleteAssetUseCase

Descrizione

DeleteAssetUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per l'eliminazione di un `AssetG` dalla sessione di valutazione_G. Viene implementata da *DeleteAssetService* e utilizzata da *FlaskWriteAssetController*.

Attributi

DeleteAssetUseCase non definisce attributi.

Metodi

- + delete_asset(command: DeleteAssetCommand): void — firma del metodo delegato all'esecuzione della logica di eliminazione a partire dai dati contenuti nel Command.

5.3.3.3) DeleteAssetService

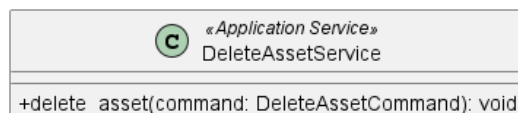


Figura 86: DeleteAssetService

Descrizione

DeleteAssetService è il service applicativo appartenente all'Application Core responsabile della logica di eliminazione di un Asset_G. Implementa l'interfaccia_G *DeleteAssetUseCase*, riceve un *DeleteAssetCommand* contenente gli identificativi necessari, recupera la sessione tramite *GetEvaluationSession*, rimuove l'Asset_G corrispondente e persiste la sessione aggiornata tramite *SaveEvaluationSession*.

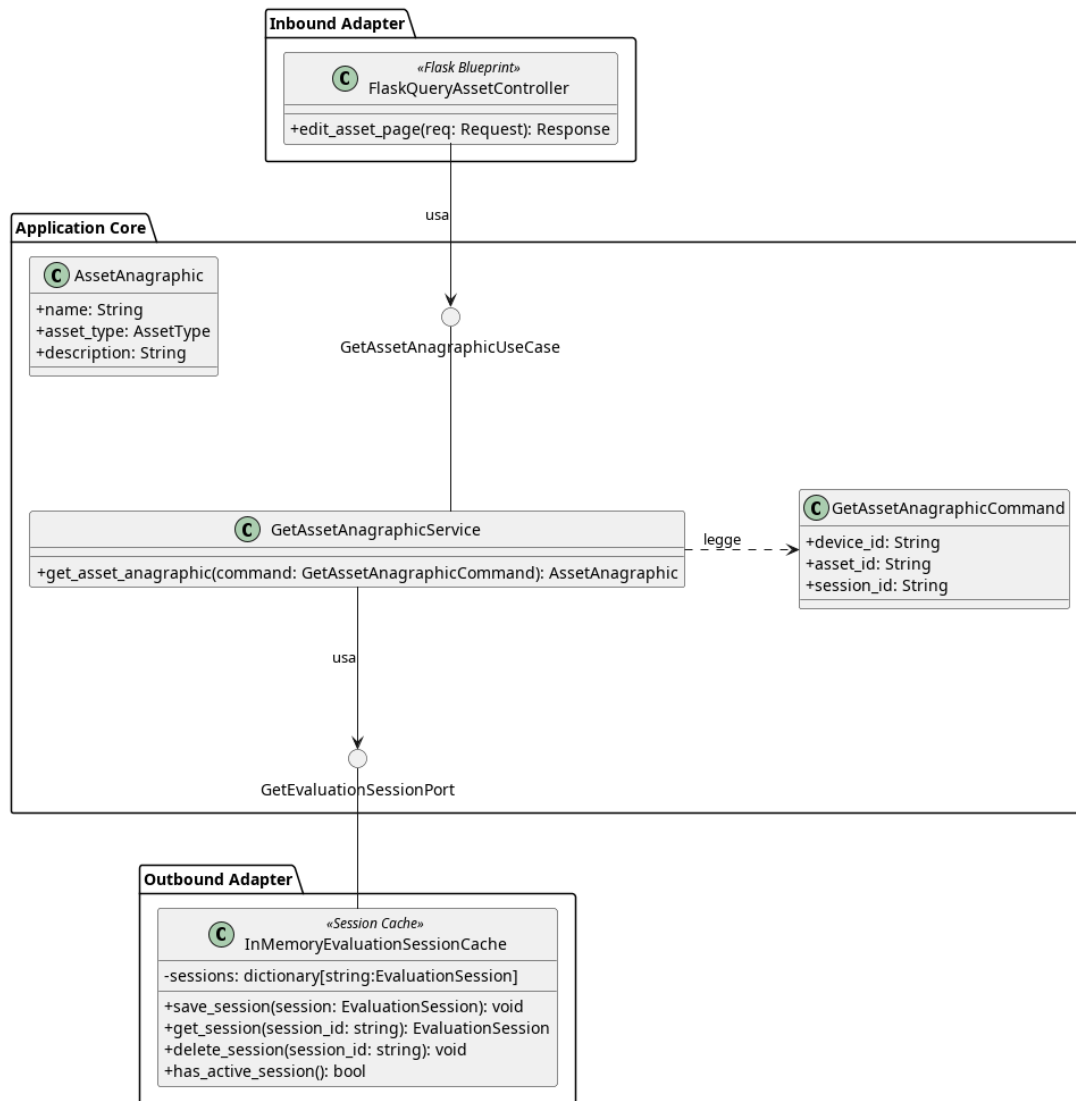
Attributi

- - save_evaluation_session_port: SaveEvaluationSessionPort — outbound port usata per il salvataggio delle modifiche nella sessione.
- - get_evaluation_session_port: GetEvaluationSessionPort — outbound port usata per prelevare la sessione di valutazione_G.

Metodi

- + delete_asset(command: DeleteAssetCommand): void — concretizza il contratto definito da *DeleteAssetUseCase*. Recupera la sessione attiva, individua e rimuove l'Asset_G corrispondente e ne persiste lo stato aggiornato.

5.3.4) GetAssetAnagraphic

Figura 87: Caso d'uso_G GetAssetAnagraphic

Il diagramma illustra l'architettura del modulo dedicato al recupero del dettaglio di un `AssetG` all'interno di una sessione di valutazione_G attiva.

- Per la definizione di `InMemoryEvaluationSessionCache`, vedere la **Sezione 5.3.1.5**.
- Per la definizione di `GetEvaluationSessionPort`, vedere la **Sezione 5.3.1.7**.
- Per la definizione di `AssetAnagraphic`, vedere la **Sezione 5.1.3**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso_G.

5.3.4.1) FlaskQueryAssetController



Figura 88: FlaskQueryAssetController

Descrizione

FlaskQueryAssetController è un adapter di input (controller) basato su Flask. Ha la responsabilità di intercettare le richieste web in lettura relative agli Asset_G, interagire con i casi d'uso applicativi e restituire le viste HTML (template) popolate con i dati di dominio mappati tramite Data Transfer Object (DTO).

Attributi

- - `_get_asset_anagraphic_use_case`: `GetAssetAnagraphicUseCase` — istanza del caso d'uso_G iniettata a runtime, necessaria per recuperare le informazioni anagrafiche di un asset_G specifico.

Metodi

- + `edit_asset_page(session_id: String, device_id: String, asset_id: String): Response` — delega al caso d'uso_G il recupero dell'anagrafica_G dell'asset_G, mappa le informazioni di dominio in un `AssetAnagraphicDTO` e restituisce il template HTML precompilato per la visualizzazione/modifica.

5.3.4.2) GetAssetAnagraphicCommand

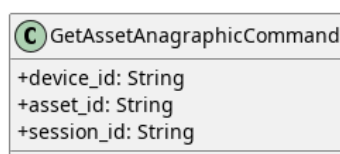


Figura 89: GetAssetAnagraphicCommand

Descrizione

GetAssetAnagraphicCommand è il Command Object utilizzato per trasportare i dati necessari al recupero del dettaglio di un Asset_G. Incapsula i parametri di input del metodo esposto da *GetAssetAnagraphicUseCase*.

Attributi

- + `device_id: String` — identificativo univoco del device che contiene l'asset_G.
- + `asset_id: String` — identificativo univoco dell'Asset_G di cui recuperare il dettaglio.
- + `session_id: String` — identificativo univoco della sessione di valutazione_G corrente.

Metodi

GetAssetAnagraphicCommand non definisce metodi propri.

5.3.4.3) GetAssetAnagraphicUseCase

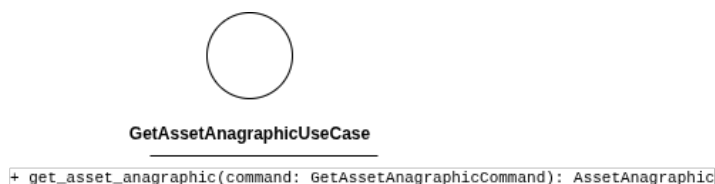


Figura 90: GetAssetAnagraphicUseCase

Descrizione

GetAssetAnagraphicUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per il recupero del dettaglio di un Asset_G. Viene implementata da *GetAssetAnagraphicService*.

Attributi

GetAssetAnagraphicUseCase non definisce attributi.

Metodi

- + `get_asset_anagraphic(command: GetAssetAnagraphicCommand): AssetAnagraphic` — firma del metodo delegato al recupero dell'anagrafica_G dell'asset_G ricercato.

5.3.4.4) GetAssetAnagraphicService

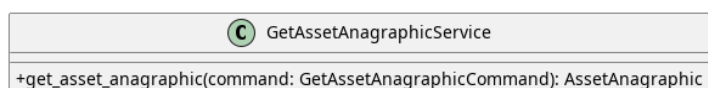


Figura 91: GetAssetAnagraphicService

Descrizione

GetAssetAnagraphicService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di un Asset_G. Implementa l'interfaccia_G *GetAssetAnagraphicUseCase*, recupera la sessione attiva tramite *GetEvaluationSession* e ritorna l'*AssetAnagraphic* tramite *AssetG*.

Attributi

- - `get_evaluation_session_port: GetEvaluationSessionPort` — outbound port usata per prelevare la sessione di valutazione_G.

Metodi

- + `get_asset_anagraphic(command: GetAssetAnagraphicCommand): AssetAnagraphic` — concretizza il contratto definito da

GetAssetAnagraphicUseCase. Recupera la sessione attiva, individua l'Asset_G richiesto e ne estrae l'*AssetAnagraphic*.

5.3.5) GetAssetEvaluationDetail

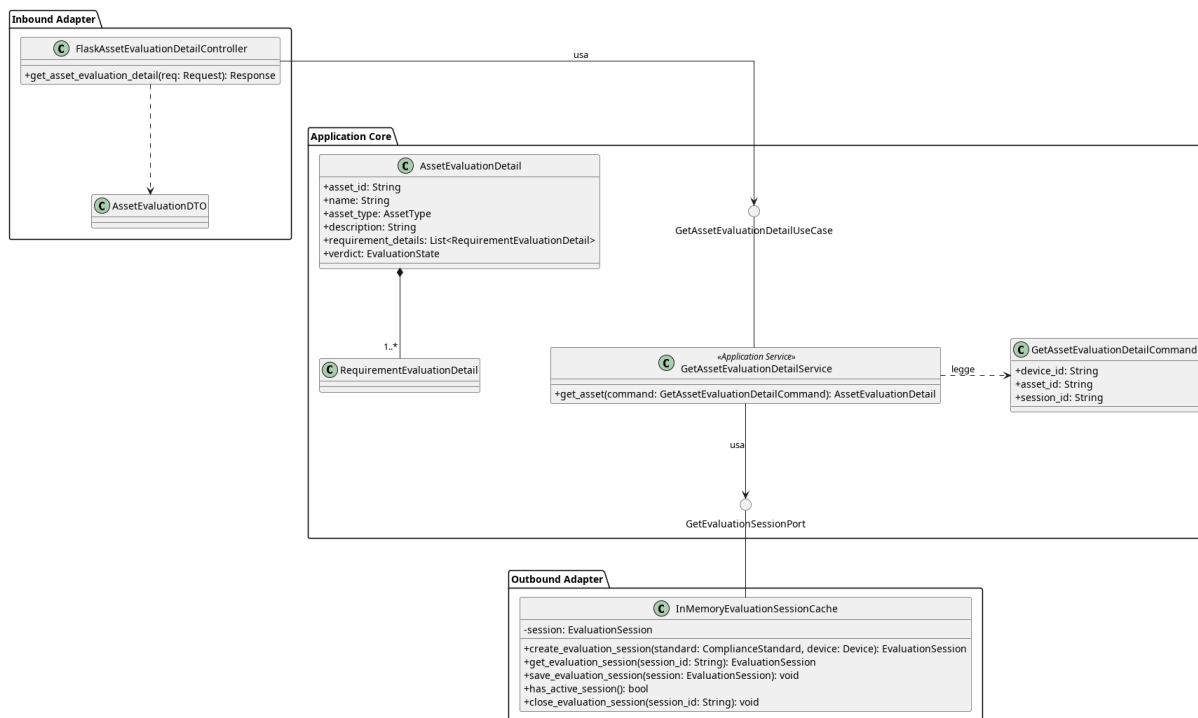


Figura 92: GetAssetEvaluationDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero di un *AssetEvaluationDetail* contenente informazioni anagrafiche e stato di valutazione_G del dispositivo_G.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la **Sezione 5.3.1.7**.
- Per la definizione di *AssetEvaluationDetail*, vedere la **Sezione 5.1.21**.

5.3.5.1) FlaskAssetEvaluationDetailController

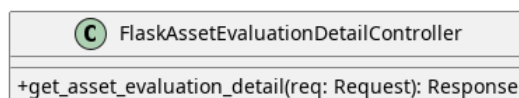


Figura 93: GetAssetEvaluationDetail

Descrizione

FlaskAssetEvaluationDetailController è un adapter di input (controller) che ha la responsabilità di intercettare le richieste per la visualizzazione del dettaglio di valutazione_G di un asset_G, interrogare il core applicativo (attraverso le porte di inbound) e presentare i risultati o gli eventuali errori di dominio alla vista.

Attributi

- - `_get_asset_ev_detail_use_case`: `GetAssetEvaluationDetailUseCase` — istanza del caso d'uso_G necessaria per recuperare i dettagli della valutazione_G dell'asset_G richiesto.

Metodi

- + `get_asset_evaluation_detail(session_id: String, device_id: String, asset_id: String)`: `Response` — preleva tramite la porta di inbound l'oggetto di dominio *AssetEvaluationDetail*.

5.3.5.2) GetAssetEvaluationDetailCommand

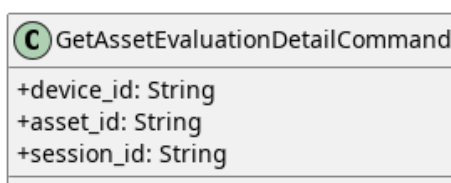


Figura 94: GetAssetEvaluationDetailCommand

Descrizione

GetAssetEvaluationDetailCommand è il Command Object utilizzato per trasportare i dati necessari al recupero di un *AssetEvaluationDetail*. Incapsula i parametri di input del metodo esposto da *GetAssetEvaluationDetailUseCase*.

Attributi

- + `device_id`: `String` — identificativo univoco del Dispositivo_G di cui recuperare le informazioni.
- + `asset_id`: `String` — identificativo univoco dell'Asset_G.
- + `session_id`: `String` — identificativo univoco della sessione.

Metodi

GetAssetEvaluationDetailCommand non definisce metodi propri.

5.3.5.3) GetAssetEvaluationDetailUseCase



Figura 95: GetAssetEvaluationDetailUseCase

Descrizione

GetAssetEvaluationDetailUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per il recupero di un *AssetEvaluationDetail*. Viene implementata da *GetAssetEvaluationDetailService* e utilizzata da *FlaskAssetEvaluationDetailController*.

Attributi

GetAssetEvaluationDetailUseCase non definisce attributi.

Metodi

- + `get_asset(command: GetAssetEvaluationDetailCommand): AssetEvaluationDetail` — firma del metodo delegato al recupero e all'aggregazione delle informazioni della sessione di valutazione_G.

5.3.5.4) GetAssetEvaluationDetailService



Figura 96: GetAssetEvaluationDetailService

Descrizione

Concretizza il contratto definito da *GetAssetEvaluationDetailUseCase*. Recupera la sessione di valutazione_G specificata, esegue la valutazione_G del dispositivo_G tramite l'engine e isola i risultati dell'Asset_G richiesto. Infine, aggrega i dati anagrafici di tale Asset_G con i risultati della valutazione_G dei suoi requisiti, restituendo la rappresentazione *AssetEvaluationDetail*.

Attributi

- - `get_evaluation_session_port: GetEvaluationSessionPort` — outbound port usata per recuperare la sessione.

Metodi

- + `get_asset(command: GetAssetEvaluationDetailCommand): AssetEvaluationDetail` —

concretizza il contratto definito da *GetAssetEvaluationDetailUseCase*. Recupera la sessione attiva, aggrega le informazioni del Dispositivo_G e dei suoi Asset_G e restituisce la rappresentazione *AssetEvaluationDetail*.

5.3.5.5) DTO

Qui vengono elencati i DTO usati dal controller per gestire ed esporre i dettagli della valutazione_G di un asset_G verso l'interfaccia_G frontend o le API.

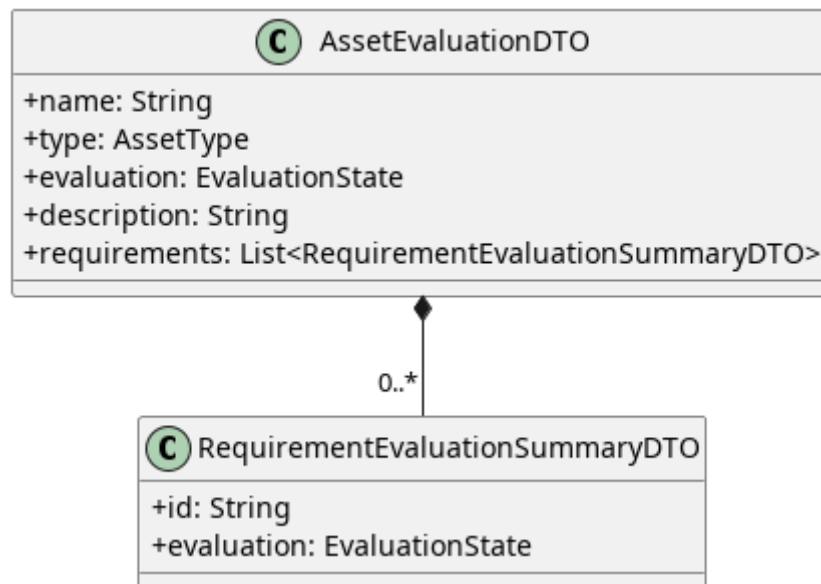


Figura 97: AssetEvaluationDTO

5.3.5.5.1) AssetEvaluationDTO

Descrizione

AssetEvaluationDTO è il Data Transfer Object principale utilizzato per esporre all'interfaccia_G utente i dettagli anagrafici e l'esito complessivo della valutazione_G di uno specifico asset_G. Agisce come aggregatore, includendo al suo interno una lista sintetica dello stato di tutti i requisiti ad esso associati.

Attributi

- + name: String — nome dell'asset_G valutato.
- + type: AssetType — categoria o tipologia a cui appartiene l'asset_G.
- + evaluation: EvaluationState — stato globale e finale della valutazione_G di conformità_G per l'intero asset_G.
- + description: String — descrizione testuale aggiuntiva dell'asset_G.
- + requirements: List<RequirementEvaluationSummaryDTO> — tupla contenente i DTO di riepilogo per ciascun requisito_G associato all'asset_G.

Metodi

AssetEvaluationDTO non definisce metodi.

5.3.5.5.2) RequirementEvaluationSummaryDTO

Descrizione

RequirementEvaluationSummaryDTO è un Data Transfer Object estremamente leggero e di supporto, istanziato esclusivamente per popolare la lista dei requisiti all'interno di un *AssetEvaluationDTO*. Fornisce una vista sintetica limitata all'identificativo e al verdetto, ideale per le visualizzazioni a elenco o in forma di tabella.

Attributi

- + id: String — identificativo univoco del requisito_G valutato.
- + evaluation: EvaluationState — stato corrente della valutazione_G specifica per questo requisito_G.

Metodi

RequirementEvaluationSummaryDTO non definisce metodi.

5.3.6) GetRequirement

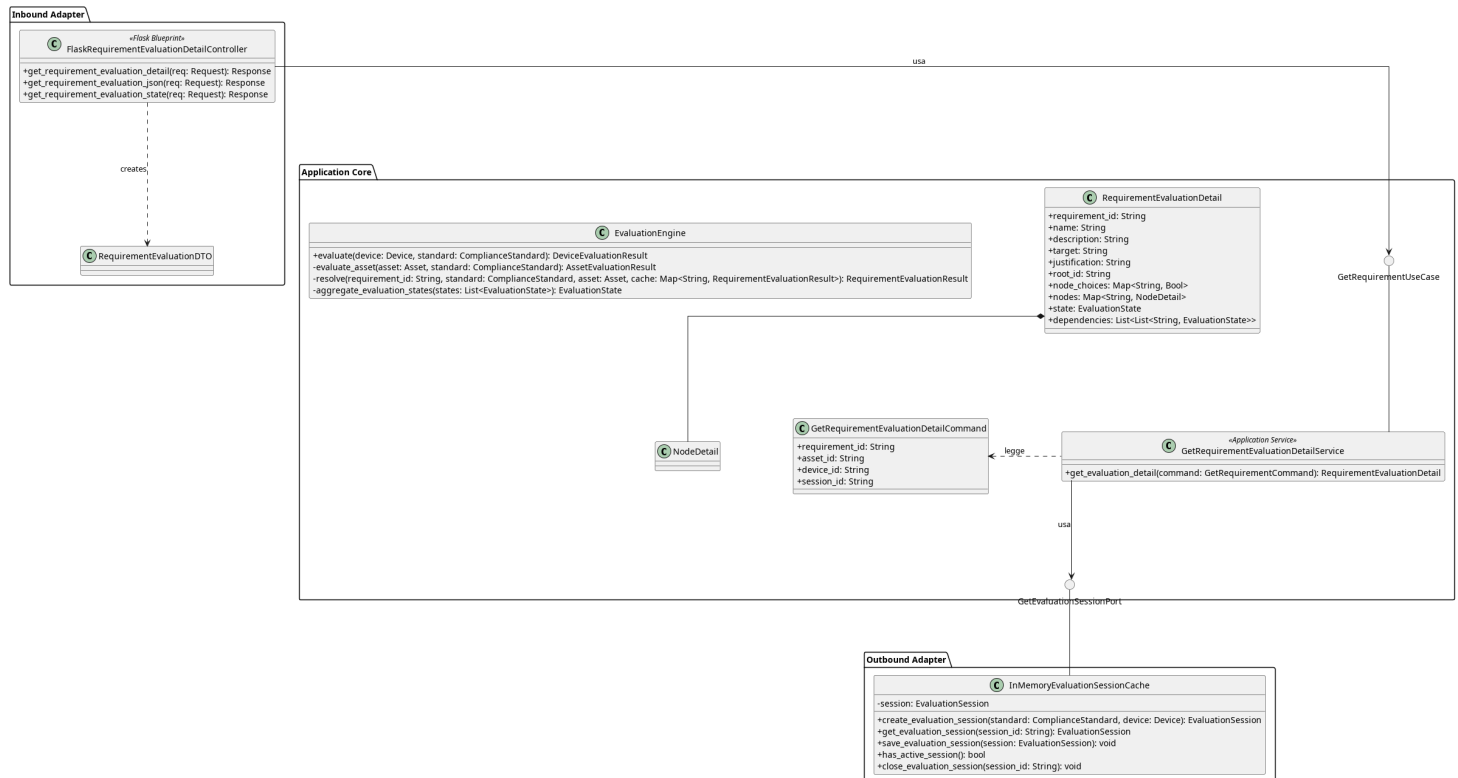


Figura 98: Caso d'uso_G GetRequirement

Il diagramma illustra l'architettura del modulo dedicato al recupero di un requisito_G di conformità_G con il relativo albero decisionale e le dipendenze associate.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la [Sezione 5.3.1.5](#).
- Per la definizione di *GetEvaluationSessionPort*, vedere la [Sezione 5.3.1.7](#).
- Per la definizione di *EvaluationEngine*, vedere la [Sezione 5.1.14](#).
- Per la definizione di *RequirementEvaluationDetail*, vedere la [Sezione 5.1.20](#).

5.3.6.1) FlaskRequirementEvaluationDetailController

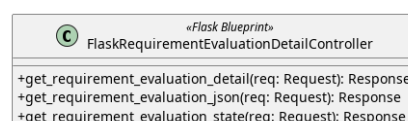


Figura 99: FlaskRequirementEvaluationDetailController

Descrizione

FlaskRequirementEvaluationDetailController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di recupero di un requisito_G di conformità e le inoltra al livello applicativo.

Attributi

- - `get_requirement_ev_detail_use_case:`
GetRequirementEvaluationDetailUseCase — inbound port usata per recuperare l'oggetto *RequirementEvaluationDetail*

Metodi

- + `get_requirement_evaluation_detail(req: Request): Response` — endpoint GET che restituisce l'intera pagina HTML con il dettaglio completo del requisito_G valutato.
- + `get_requirement_evaluation_json(req: Request): Response` — endpoint GET che restituisce il dettaglio completo della valutazione_G del requisito_G in formato JSON_G.
- + `get_requirement_evaluation_state(req: Request): Response` — endpoint GET leggero che restituisce esclusivamente lo stato attuale di valutazione_G in formato JSON_G.

5.3.6.2) GetRequirementEvaluationDetailUseCase



Figura 100: GetRequirementEvaluationDetailUseCase

Descrizione

GetRequirementEvaluationDetailUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per il recupero di un requisito_G di conformità_G. Viene implementata da *GetRequirementEvaluationService* e utilizzata da *FlaskRequirementEvaluationDetailController*.

Attributi

GetRequirementEvaluationDetailUseCase non definisce attributi.

Metodi

- + `get_evaluation_detail(command: GetRequirementEvaluationDetailCommand): RequirementEvaluationDetail` — firma del metodo delegato al recupero del requisito_G corrispondente ai parametri incapsulati nel comando fornito in input.

5.3.6.3) GetRequirementEvaluationDetailCommand

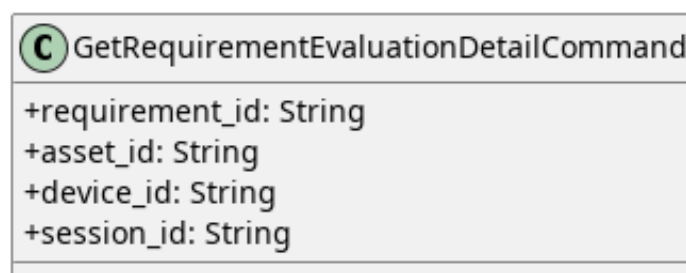


Figura 101: GetRequirementEvaluationDetailCommand

Descrizione

GetRequirementEvaluationDetailCommand è l'oggetto che veicola i parametri necessari al recupero di un requisito_G dal controller al service.

Attributi

- + `requirement_id: String` — identificativo del requisito_G da recuperare.
- + `asset_id: String` — identificativo dell'asset_G oggetto di valutazione_G.
- + `device_id: String` — identificativo del dispositivo_G contenente l'asset_G.
- + `session_id: String` — identificativo della sessione di valutazione_G attiva.

Metodi

GetRequirementEvaluationDetailCommand non definisce metodi.

5.3.6.4) GetRequirementEvaluationDetailService

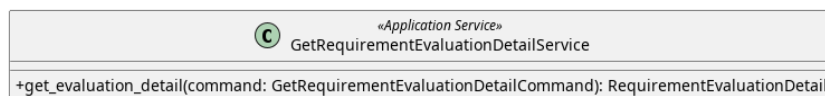


Figura 102: GetRequirementEvaluationDetailService

Descrizione

GetRequirementEvaluationDetailService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di valutazione_G di un singolo requisito_G. Implementa l'interfaccia_G

GetRequirementEvaluationDetailUseCase, legge i parametri dal *GetRequirementEvaluationDetailCommand*, recupera la sessione attiva tramite *GetEvaluationSessionPort*, esegue la valutazione_G tramite l'*EvaluationEngine* e costruisce un *RequirementEvaluationDetail*.

Attributi

- `get_evaluation_session_port: GetEvaluationSessionPort`
- `evaluation_engine: EvaluationEngine`

Metodi

- + get_evaluation_detail(command: GetRequirementEvaluationDetailCommand): RequirementEvaluationDetail — concretizza il contratto definito da *GetRequirementEvaluationDetailUseCase*. Recupera la sessione attiva, individua il requisito_G richiesto utilizzando i parametri incapsulati nel comando e ne costruisce la relativa rappresentazione sotto forma di *RequirementEvaluationDetail*.

5.3.6.5) DTO

Qui vengono elencati i DTO usati dal controller per gestire ed esporre i dettagli della valutazione_G di un requisito_G.

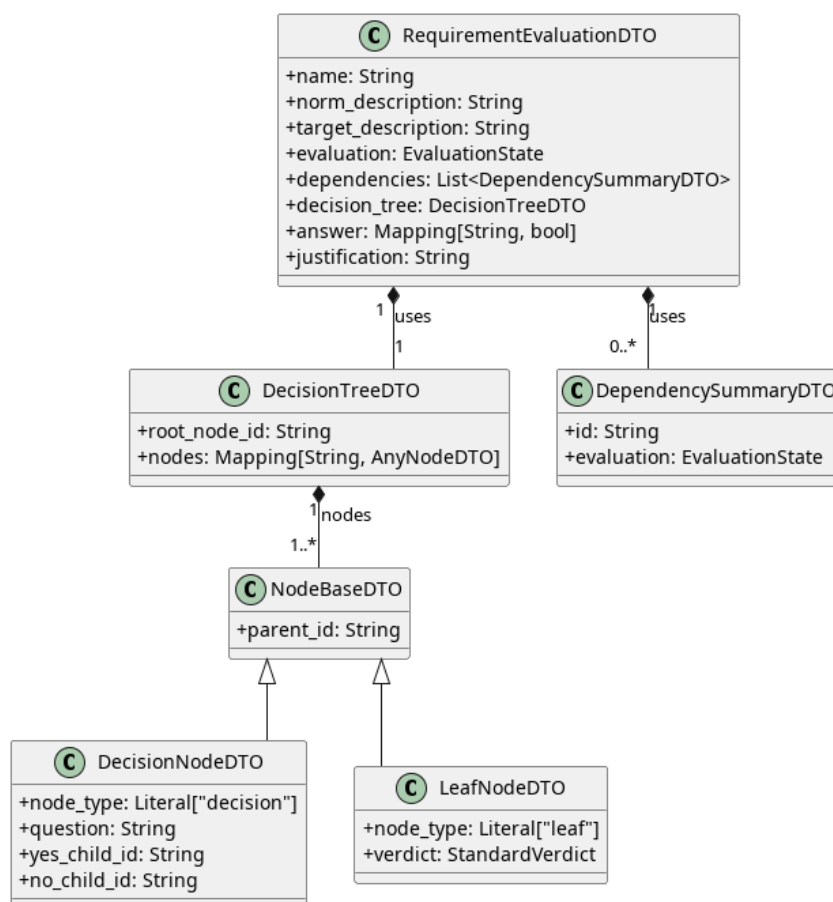


Figura 103: RequirementEvaluationDTO

5.3.6.5.1) RequirementEvaluationDTO

Descrizione

RequirementEvaluationDTO è il Data Transfer Object principale che consolida tutte le informazioni necessarie per la visualizzazione del dettaglio di un singolo requisito_G valutato. Raccoglie i dati anagrafici, l'esito della valutazione_G, lo stato delle dipendenze, la struttura dell'albero decisionale e le risposte fornite dall'utente.

Attributi

- + name: String — nome del requisito_G.

- + norm_description: String — descrizione normativa estesa del requisito_G.
- + target_description: String — descrizione dell'obiettivo del requisito_G.
- + evaluation: EvaluationState — stato corrente e complessivo della valutazione_G.
- + dependencies: List<DependencySummaryDTO> — collezione di DTO che riepilogano lo stato dei requisiti da cui questo dipende.
- + decision_tree: DecisionTreeDTO — DTO che mappa la struttura dell'albero decisionale associato al requisito_G.
- + answer: Map<String, Bool> — mappa che associa gli ID dei nodi decisionali alle risposte fornite.
- + justification: String | None — eventuale giustificazione testuale fornita durante la valutazione_G.

Metodi

RequirementEvaluationDTO non definisce metodi.

5.3.6.5.2) DependencySummaryDTO

Descrizione

DependencySummaryDTO è un Data Transfer Object leggero utilizzato per esporre esclusivamente l'identificativo e lo stato di valutazione_G di un requisito_G che funge da dipendenza_G.

Attributi

- + id: String — identificativo del requisito_G dipendente.
- + evaluation: EvaluationState — stato attuale della valutazione_G della dipendenza_G.

Metodi

DependencySummaryDTO non definisce metodi.

5.3.6.5.3) DecisionTreeDTO

Descrizione

DecisionTreeDTO è il DTO responsabile di incapsulare l'intera struttura dell'albero decisionale. Espone il riferimento al nodo_G di partenza e l'insieme di tutti i nodi che compongono l'albero, utilizzando il polimorfismo per rappresentare sia i nodi decisionali che quelli terminali.

Attributi

- + root_node_id: String — identificativo del nodo_G radice da cui inizia la navigazione dell'albero.

- + nodes: Map<String, NodeDTO> — dizionario che associa gli identificativi univoci dei nodi ai rispettivi oggetti DTO (che possono essere istanze di *DecisionNodeDTO* o *LeafNodeDTO*).

Metodi

DecisionTreeDTO non definisce metodi.

5.3.6.5.4) LeafNodeDTO

Descrizione

LeafNodeDTO estende *NodeBaseDTO* e rappresenta un nodo_G terminale (foglia) dell'albero decisionale. Definisce l'esito finale della valutazione_G per quel particolare percorso.

Attributi

- + parent_id: String | None — identificativo del nodo_G genitore (ereditato).
- + node_type: String — costante valorizzata a «leaf», utilizzata come discriminatore dal frontend e dai validatori per distinguere il tipo di nodo_G.
- + verdict: StandardVerdict — verdetto di conformità_G finale associato alla foglia.

Metodi

LeafNodeDTO non definisce metodi.

5.3.6.5.5) DecisionNodeDTO

Descrizione

DecisionNodeDTO estende *NodeBaseDTO* e rappresenta uno snodo intermedio dell'albero decisionale. Contiene la domanda da porre all'utente e i riferimenti per la navigazione in base alla risposta data.

Attributi

- + parent_id: String | None — identificativo del nodo_G genitore (ereditato).
- + node_type: String — costante valorizzata a «decision», utilizzata come discriminatore strutturale.
- + question: String — testo della domanda posta dal nodo_G decisionale.
- + yes_child_id: String | None — identificativo del nodo_G figlio da visitare nel caso la risposta sia affermativa.
- + no_child_id: String | None — identificativo del nodo_G figlio da visitare nel caso la risposta sia negativa.

Metodi

DecisionNodeDTO non definisce metodi.

5.3.6.5.6) NodeBaseDTO

Descrizione

NodeBaseDTO è la classe base per i DTO che rappresentano i nodi dell'albero decisionale. Fattorizza le proprietà condivise in modo trasparente da tutti i nodi (sia foglie che decisionali).

Attributi

- + `parent_id`: String | None — identificativo del nodo_G genitore, necessario per permettere la navigazione a ritroso dell'albero da parte dell'interfaccia_G utente.

Metodi

NodeBaseDTO non definisce metodi.

5.4) Import

5.4.1) ImportDevice

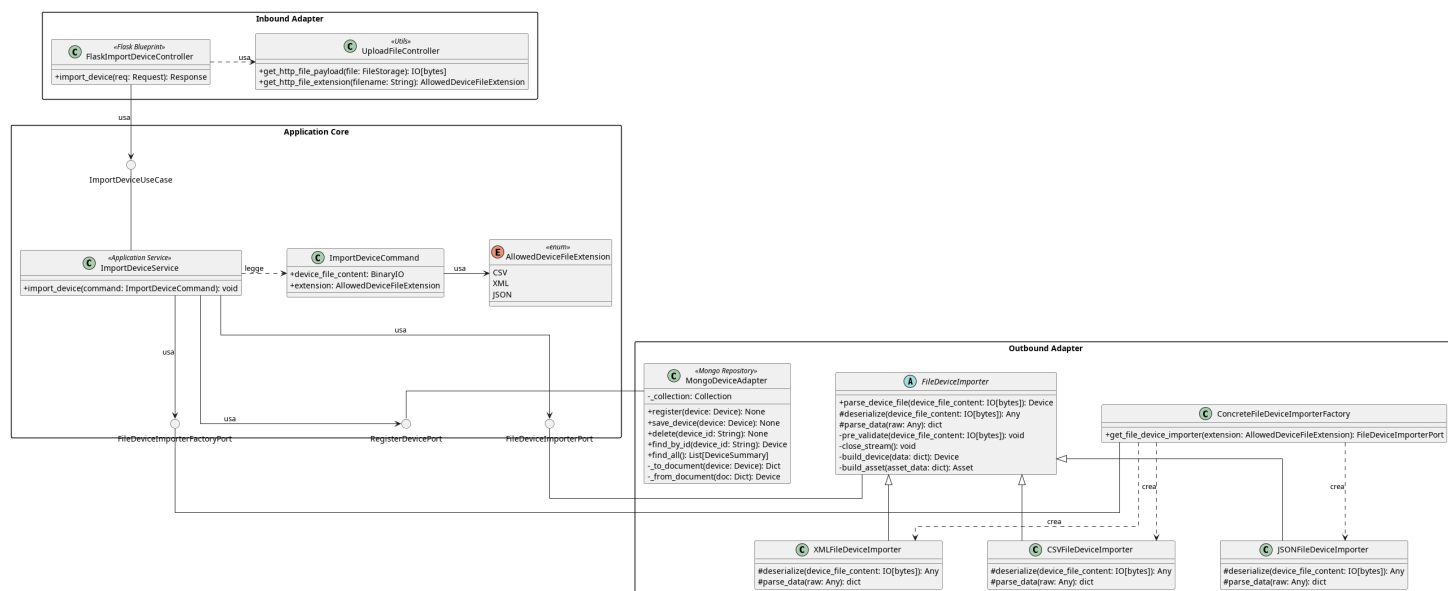


Figura 104: Caso d'uso_G ImportDevice

Il diagramma illustra l'architettura del modulo dedicato all'importazione_G di Dispositivi tramite file. Il modulo supporta tre formati — CSV_G, XML_G e JSON_G — gestiti tramite il pattern *Template Method* e una factory dedicata. Il componente *MongoDeviceAdapter* è già descritto nella sezione *CreateDevice*.

- Per la definizione di *MongoDeviceAdapter*, vedere la **Sezione 5.2.1.6**.
- Per la definizione di *RegisterDevicePort*, vedere la **Sezione 5.2.1.5**

Di seguito vengono documentati i componenti introdotti specificamente per questo caso d'uso_G.

5.4.1.1) UploadFileController

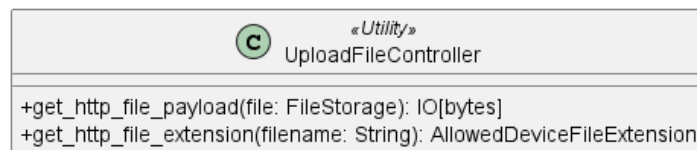


Figura 105: UploadFileController

Descrizione

UploadFileController è una classe di utilità appartenente all’Inbound Adapter che fornisce i metodi comuni per l’estrazione del contenuto e dell’estensione_G di un file dalla richiesta HTTP. Viene utilizzata da *FlaskImportDeviceController*.

Attributi

UploadFileController non definisce attributi propri.

Metodi

- + `get_http_file_payload(file: FileStorage): IO[bytes]` — estrae il contenuto binario del file dalla richiesta HTTP.
- + `get_http_file_extension(filename: String): AllowedDeviceFileExtension` — estrae l’estensione_G del file dalla richiesta HTTP.

5.4.1.2) FlaskImportDeviceController



Figura 106: FlaskImportDeviceController

Descrizione

FlaskImportDeviceController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP di importazione_G di Dispositivi tramite file. Utilizza *UploadFileController* per estrarre il contenuto binario e l’estensione_G del file dalla request.

Attributi

- - `_service: ImportDeviceUseCase` — riferimento alla porta di Inbound (caso d’uso_G) responsabile della logica applicativa di importazione_G.

Metodi

- + `import_device(req: Request): Response` — riceve la richiesta HTTP di importazione_G, estrae il file e la sua estensione_G tramite *UploadFileController* e inoltra il Command al livello applicativo; restituisce una risposta HTTP con l’esito dell’operazione.

5.4.1.3) ImportDeviceUseCase

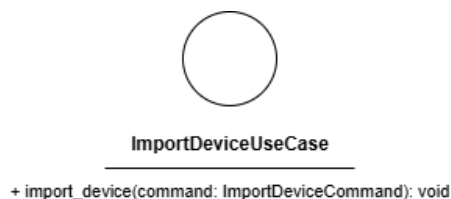


Figura 107: ImportDeviceUseCase

Descrizione

ImportDeviceUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per l'importazione_G di Dispositivi da file. Viene implementata da *ImportDeviceService* e utilizzata da *FlaskImportDeviceController*.

Attributi

ImportDeviceUseCase non definisce attributi.

Metodi

- + `import_device(command: ImportDeviceCommand): void` — firma del metodo delegato all'esecuzione della logica di importazione_G a partire dai dati contenuti nel Command.

5.4.1.4) ImportDeviceCommand

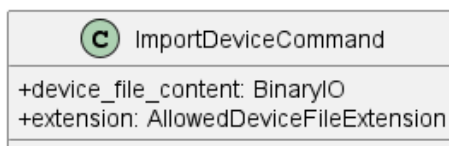


Figura 108: ImportDeviceCommand

Descrizione

ImportDeviceCommand è il Command Object che veicola i dati necessari all'importazione_G di Dispositivi dal controller al service.

Attributi

- + `device_file_content: BinaryIO` — contenuto binario del file da importare.
- + `extension: AllowedDeviceFileExtension` — estensione_G del file, vincolata ai valori dell'enumerazione *AllowedDeviceFileExtension*.

Metodi

ImportDeviceCommand non definisce metodi propri.

5.4.1.5) AllowedDeviceFileExtension

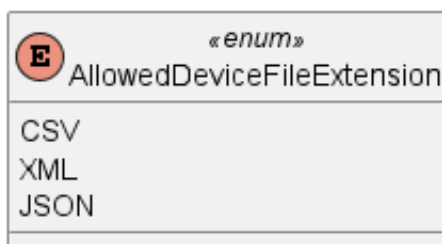


Figura 109: AllowedDeviceFileExtension

Descrizione

AllowedDeviceFileExtension è un'enumerazione che definisce i formati di file supportati per l'importazione_G di Dispositivi.

Valori

- `CSVG` — formato `CSVG`.
- `XMLG` — formato `XMLG`.
- `JSONG` — formato `JSONG`.

5.4.1.6) ImportDeviceService



Figura 110: ImportDeviceService

Descrizione

ImportDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di importazione_G di Dispositivi da file. Implementa l'interfaccia_G *ImportDeviceUseCase* e coordina il parsing del file tramite *FileDeviceImporterFactoryPort*, la lettura del contenuto tramite *FileDeviceImporterPort* e la persistenza tramite *RegisterDevicePort*.

Attributi

- `device_importer_factory`: *FileDeviceImporterFactoryPort*: outbound port usata per ottenere l'importer corretto in base al formato del file (`CSVG`, `JSONG` o `XMLG`).
- `register_device_port`: *RegisterDevicePort*: outbound port usata per registrare il dispositivo_G nel sistema di persistenza.

Metodi

- + `import_device(command: ImportDeviceCommand): void` — concretizza il contratto definito da *ImportDeviceUseCase*. Ottiene l'importer appropriato tramite la factory, effettua il parsing del file e persiste i Dispositivi estratti tramite *DeviceRepositoryPort*.

5.4.1.7) FileDeviceImporterPort

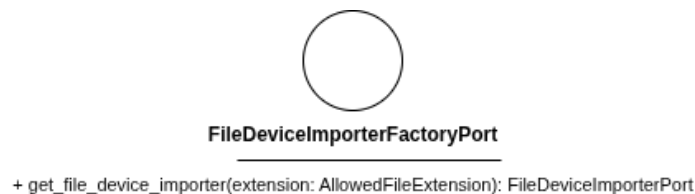


Figura 111: FileDeviceImporterPort

Descrizione

FileDeviceImporterPort è l'interfaccia_G (Outbound Port) che definisce il contratto per il parsing di un file contenente dati di Dispositivi. Viene implementata dalle classi concrete *XMLFileDeviceImporter*, *JSONFileDeviceImporter* e *CSVFileDeviceImporter* tramite *FileDeviceImporter*.

Attributi

FileDeviceImporterPort non definisce attributi.

Metodi

- + `parse_device_file(device_file_content: BinaryIO): Device` — firma del metodo che effettua il parsing del contenuto binario del file e restituisce l'entità *Device* estratta.

5.4.1.8) FileDeviceImporter

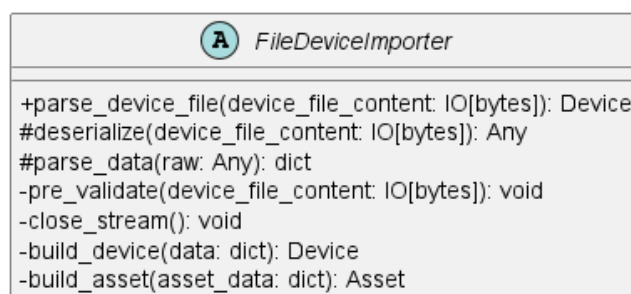


Figura 112: FileDeviceImporter

Descrizione

FileDeviceImporter è la classe astratta appartenente all'Outbound Adapter che implementa il pattern *Template Method* per il parsing dei file di Dispositivi. Definisce lo scheletro_G dell'algoritmo di importazione_G, delegando alle sottoclassi concrete l'implementazione dei passi specifici per ciascun formato.

Attributi

FileDeviceImporter non definisce attributi propri.

Metodi

- + `parse_device_file(device_file_content: IO[bytes]): Device` — metodo pubblico che orchestra l'algoritmo di parsing invocando in sequenza i metodi del template; restituisce l'entità *Device* estratta dal file.
- # `deserialize(device_file_content: IO[bytes]): Any` — metodo protetto astratto che deserializza il contenuto binario del file nella struttura dati grezza specifica del formato; deve essere implementato dalle sottoclassi.
- # `parse_data(raw: Any): dict` — metodo protetto astratto che estrae e normalizza i campi necessari dalla struttura dati grezza; deve essere implementato dalle sottoclassi.
- - `pre_validate(device_file_content: IO[bytes]): void` — metodo privato che verifica che il file non superi la dimensione massima consentita di 10 MB prima di procedere al parsing.
- - `close_stream(): void` — metodo privato che chiude lo stream di lettura del file.
- - `build_device(data: dict): Device` — metodo privato che costruisce l'entità *Device* a partire dal dizionario normalizzato, verificando la presenza dei campi obbligatori.
- - `build_asset(asset_data: dict): AssetG` — metodo privato che costruisce un'entità *Asset_G* a partire dai dati del singolo asset_G estratti dal file.

5.4.1.9) XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter

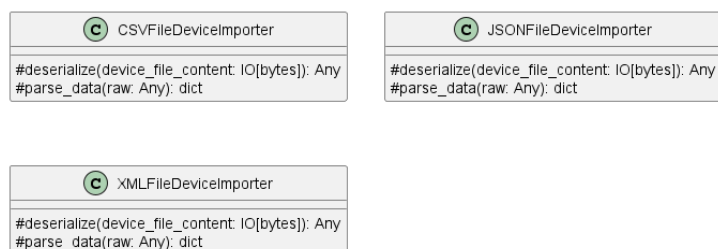


Figura 113: XMLFileDeviceImporter, JSONFileDeviceImporter, CSVFileDeviceImporter

Descrizione

XMLFileDeviceImporter, *JSONFileDeviceImporter* e *CSVFileDeviceImporter* sono le implementazioni concrete di *FileDeviceImporter*, ciascuna specializzata nel parsing del rispettivo formato di file. Implementano i metodi protetti del template per la gestione dello stream e del parsing specifico del formato.

Attributi

Le tre classi non definiscono attributi propri.

Metodi

Ciascuna classe implementa i metodi ereditati da *FileDeviceImporter*:

- # `deserialize(device_file_content: IO[bytes]): Any` — metodo protetto che deserializza il contenuto binario del file nel formato specifico della sottoclasse.
- # `parse_data(raw: Any): dict` — metodo protetto che estrae e normalizza i campi necessari dalla struttura dati grezza prodotta da `deserialize`.

5.4.1.10) FileDeviceImporterFactoryPort

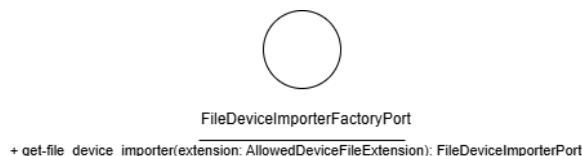


Figura 114: FileDeviceImporterFactoryPort

Descrizione

FileDeviceImporterFactoryPort è l'interfaccia_G (Outbound Port) che definisce il contratto per l'ottenimento dell'importer appropriato in base all'estensione_G del file. Viene implementata da *ConcreteFileDeviceImporterFactory*.

Attributi

FileDeviceImporterFactoryPort non definisce attributi.

Metodi

- + `get_file_device_importer(extension: AllowedDeviceFileExtension): FileDeviceImporterPort` — restituisce l'istanza dell'importer appropriato per il formato specificato.

5.4.1.11) ConcreteFileDeviceImporterFactory



Figura 115: ConcreteFileDeviceImporterFactory

Descrizione

ConcreteFileDeviceImporterFactory è la classe dell'Outbound Adapter che implementa *FileDeviceImporterFactoryPort*. Istanza e restituisce l'importer appropriato in base all'estensione_G del file fornita, selezionando tra *XMLFileDeviceImporter*, *JSONFileDeviceImporter* e *CSVFileDeviceImporter*.

Attributi

ConcreteFileDeviceImporterFactory non definisce attributi propri.

Metodi

- + `get_file_device_importer(extension: AllowedDeviceFileExtension): FileDeviceImporterPort` — restituisce l'istanza dell'importer corrispondente al formato specificato.

5.5) Export

5.5.1) ExportDevice

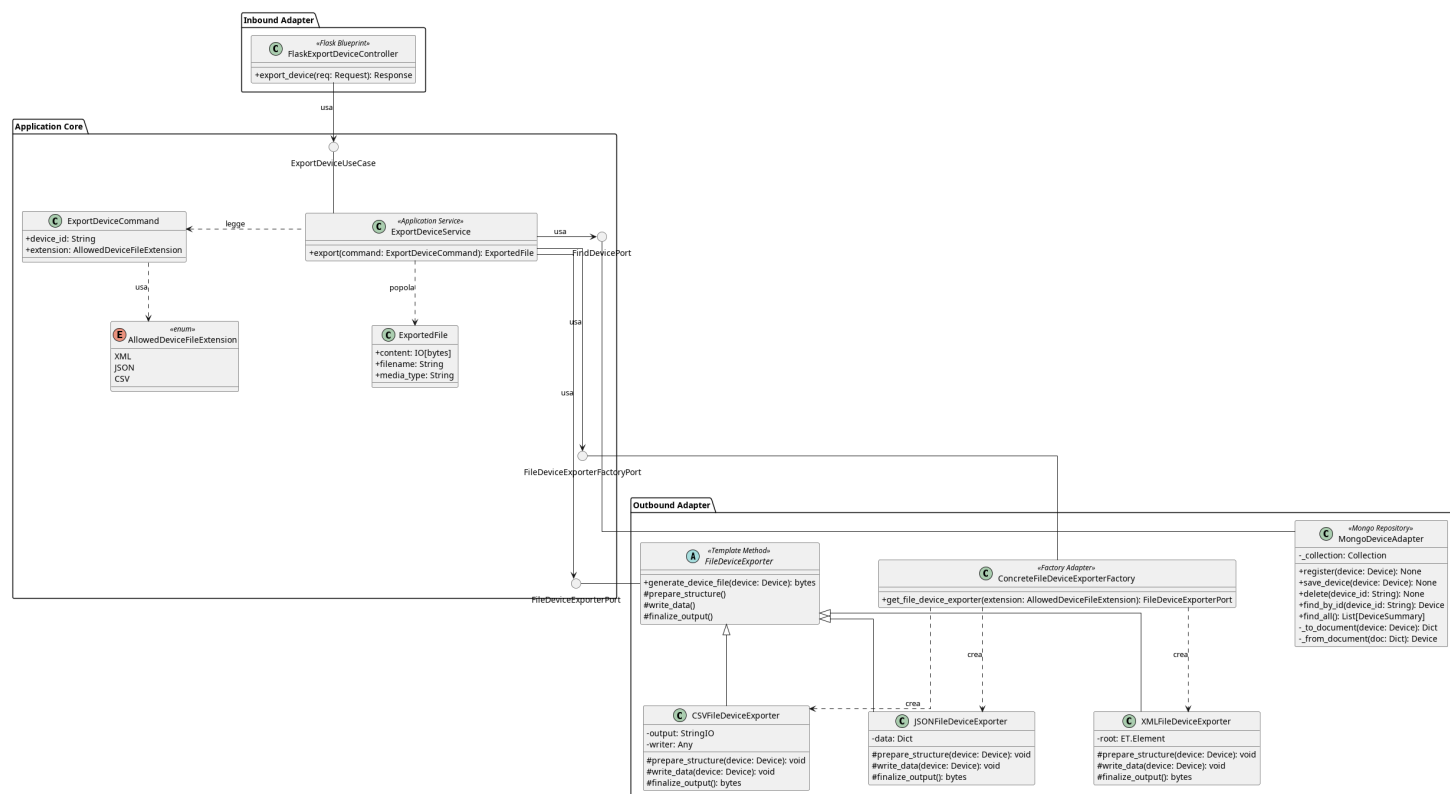


Figura 116: Caso d'uso_G ExportDevice

Il diagramma illustra l'architettura del modulo dedicato all'esportazione_G dei dati di un dispositivo_G (inclusi i suoi asset_G e le relative valutazioni di conformità_G) in un file scaricabile. Il sistema supporta tre formati — CSV_G, XML_G e JSON_G — gestiti tramite il pattern *Factory* per la selezione dell'esportatore corretto e il pattern *Template Method* per standardizzare l'algoritmo di generazione del file.

- Per la definizione di *MongoDeviceAdapter*, vedere la [Sezione 5.2.1.6](#).
- Per la definizione di *FindDevicePort*, vedere la [Sezione 5.2.3.5](#).
- Per la definizione di *AllowedDeviceFileExtension*, vedere la [Sezione 5.4.1.5](#).
- Per la definizione di *ExportedFile*, vedere la [Sezione 5.1.25](#)

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso_G.

5.5.1.1) FlaskExportDeviceController



Figura 117: FlaskExportDeviceController

Descrizione

FlaskExportDeviceController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP di esportazione_G di un dispositivo_G, estrae l’identificativo del dispositivo_G e il formato desiderato dalla request e inoltra il Command al livello applicativo.

Attributi

- - `export_device_use_case`: `ExportDeviceUseCase` — inbound port usata per esportare un dispositivo_G.

Metodi

- + `export_device(req: Request): Response` — riceve la richiesta HTTP, estrae l’identificativo del dispositivo_G e il formato desiderato come query parameter, invoca il caso d’uso_G di esportazione_G e restituisce il file generato come risposta HTTP scaricabile.

5.5.1.2) ExportDeviceUseCase



Figura 118: ExportDeviceUseCase

Descrizione

ExportDeviceUseCase è l’interfaccia_G (Inbound Port) che definisce il contratto per l’esportazione_G dei dati di un dispositivo_G in un file. Viene implementata da *ExportDeviceService* e utilizzata da *FlaskExportDeviceController*.

Attributi

ExportDeviceUseCase non definisce attributi.

Metodi

- + `export_device(command: ExportDeviceCommand): ExportedFile` — firma del metodo delegato all’esecuzione della logica di esportazione_G a partire dai dati contenuti nel Command.

5.5.1.3) ExportDeviceCommand

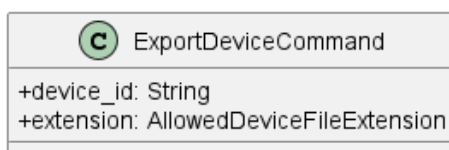


Figura 119: ExportDeviceCommand

Descrizione

ExportDeviceCommand è il Command Object che veicola i parametri della richiesta di esportazione_G dal controller al service.

Attributi

- + `device_id: String` — identificativo univoco del dispositivo_G da esportare.
- + `extension: AllowedDeviceFileExtension` — formato del file richiesto per l'esportazione_G, vincolato ai valori dell'enumerazione *AllowedDeviceFileExtension*.

Metodi

ExportDeviceCommand non definisce metodi propri.

5.5.1.4) ExportDeviceService



Figura 120: ExportDeviceService

Descrizione

ExportDeviceService è il service applicativo appartenente all'Application Core responsabile di orchestrare il processo di esportazione_G. Implementa l'interfaccia_G *ExportDeviceUseCase*, recupera il dispositivo_G tramite *FindDevicePort*, ottiene l'esportatore appropriato tramite *FileDeviceExporterFactoryPort* e avvia la generazione del file tramite *FileDeviceExporterPort*; restituisce il risultato incapsulato in un oggetto *ExportedFile*.

Attributi

- - `find_device: FindDevicePort` — outbound port usata per prelevare il dispositivo_G.
- - `exporter_factory: FileDeviceExporterFactoryPort` — outbound port usata per ottenere l'exporter desiderato.

Metodi

- + `export_device(command: ExportDeviceCommand): ExportedFile` — concretizza il contratto definito da *ExportDeviceUseCase*. Recupera il dispositivo_G, ottiene

l'esportatore corretto tramite la factory e restituisce il file generato incapsulato in *ExportedFile*.

5.5.1.5) FileDeviceExporterFactoryPort



Figura 121: FileDeviceExporterFactoryPort

Descrizione

FileDeviceExporterFactoryPort è l'interfaccia_G (Outbound Port) che definisce il contratto per l'ottenimento dell'esportatore appropriato in base al formato richiesto. Viene implementata da *ConcreteFileDeviceExporterFactory* e utilizzata da *ExportDeviceService*.

Attributi

FileDeviceExporterFactoryPort non definisce attributi.

Metodi

- + `get_file_device_exporter(extension: AllowedDeviceFileExtension): FileDeviceExporterPort` — restituisce l'istanza dell'esportatore corrispondente al formato specificato.

5.5.1.6) FileDeviceExporterPort

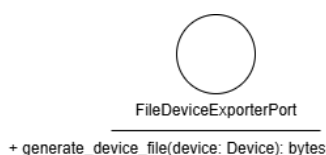


Figura 122: FileDeviceExporterPort

Descrizione

FileDeviceExporterPort è l'interfaccia_G (Outbound Port) che definisce il contratto per la generazione del file di esportazione_G. Viene implementata da *FileDeviceExporter* e utilizzata da *ExportDeviceService*.

Attributi

FileDeviceExporterPort non definisce attributi.

Metodi

- + `generate_device_file(device: Device): bytes` — firma del metodo delegato alla generazione del file; riceve l'entità *Device* e restituisce il file generato come bytes.

5.5.1.7) FileDeviceExporter

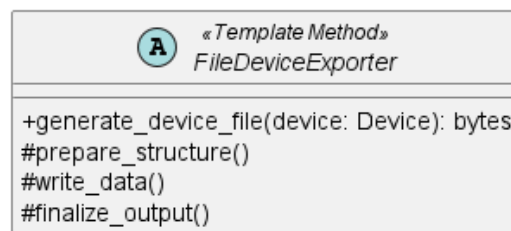


Figura 123: FileDeviceExporter

Descrizione

FileDeviceExporter è la classe astratta appartenente all'Outbound Adapter che implementa *FileDeviceExporterPort* definendo lo scheletro_G dell'algoritmo di esportazione_G tramite il pattern *Template Method*. Le sottoclassi concrete implementano i passi specifici per ciascun formato.

Attributi

FileDeviceExporter non definisce attributi propri.

Metodi

- + `generate_device_file(device: Device): bytes` — metodo pubblico che orchestra l'algoritmo di esportazione_G invocando in sequenza i metodi protetti del template.
- # `prepare_structure(device: Device): void` — metodo protetto astratto che inizializza la struttura del documento nel formato specifico.
- # `write_data(device: Device): void` — metodo protetto astratto che scrive i dati del dispositivo_G nella struttura inizializzata.
- # `finalize_output(): bytes` — metodo protetto astratto che chiude il documento e restituisce il contenuto serializzato come array di byte.

5.5.1.8) ConcreteFileDeviceExporterFactory



Figura 124: ConcreteFileDeviceExporterFactory

Descrizione

ConcreteFileDeviceExporterFactory è la classe dell'Outbound Adapter che implementa *FileDeviceExporterFactoryPort*. Istanza e restituisce l'esportatore appropriato in base al formato richiesto, selezionando tra *CSVFileDeviceExporter*, *JSONFileDeviceExporter* e *XMLFileDeviceExporter*.

Attributi

- - exporters: Dict[AllowedDeviceFileExtension, FileDeviceExporterPort]
— dizionario che mappa ogni estensione_G supportata alla corrispondente istanza dell'esportatore.

Metodi

- + get_file_device_exporter(extension: AllowedDeviceFileExtension): FileDeviceExporterPort — restituisce l'istanza dell'esportatore corrispondente al formato specificato.

5.5.1.9) CSVFileDeviceExporter, JSONFileDeviceExporter, XMLFileDeviceExporter

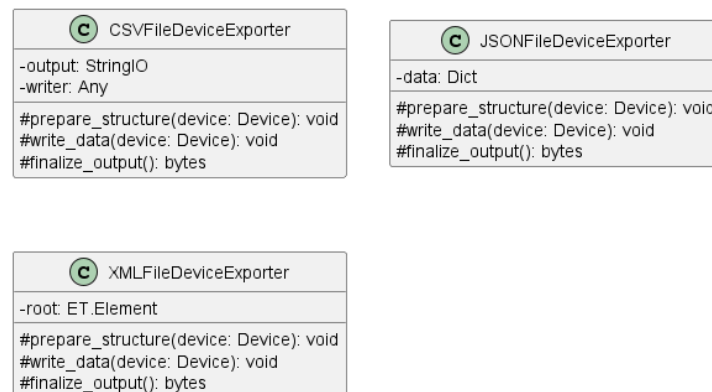


Figura 125: CSVFileDeviceExporter, JSONFileDeviceExporter, XMLFileDeviceExporter

Descrizione

CSVFileDeviceExporter, *JSONFileDeviceExporter* e *XMLFileDeviceExporter* sono le implementazioni concrete di *FileDeviceExporter*, ciascuna specializzata nella generazione del file nel rispettivo formato. Implementano i metodi protetti del template per la gestione della struttura e della serializzazione specifica del formato.

Attributi

- CSVFileDeviceExporter: - output: StringIO, - writer: Any — output è lo stream in memoria su cui viene scritto il contenuto CSV_G; writer è il csv_G.writer che scrive le righe su output.
- JSONFileDeviceExporter: - data: Dict — dizionario che accumula incrementalmente la struttura JSON_G del dispositivo_G durante write_data e viene serializzato in finalize_output.
- XMLFileDeviceExporter: - root: ET.Element — nodo_G radice dell'albero XML_G costruito durante write_data e serializzato in finalize_output.

Metodi

- # prepare_structure(device: Device): void — inizializza la struttura del documento nel formato specifico.

- `# write_data(device: Device): void` — scrive i dati del dispositivo_G nella struttura inizializzata.
- `# finalize_output(): bytes` — serializza il documento e restituisce il contenuto come array di byte.

5.5.2) ExportReport

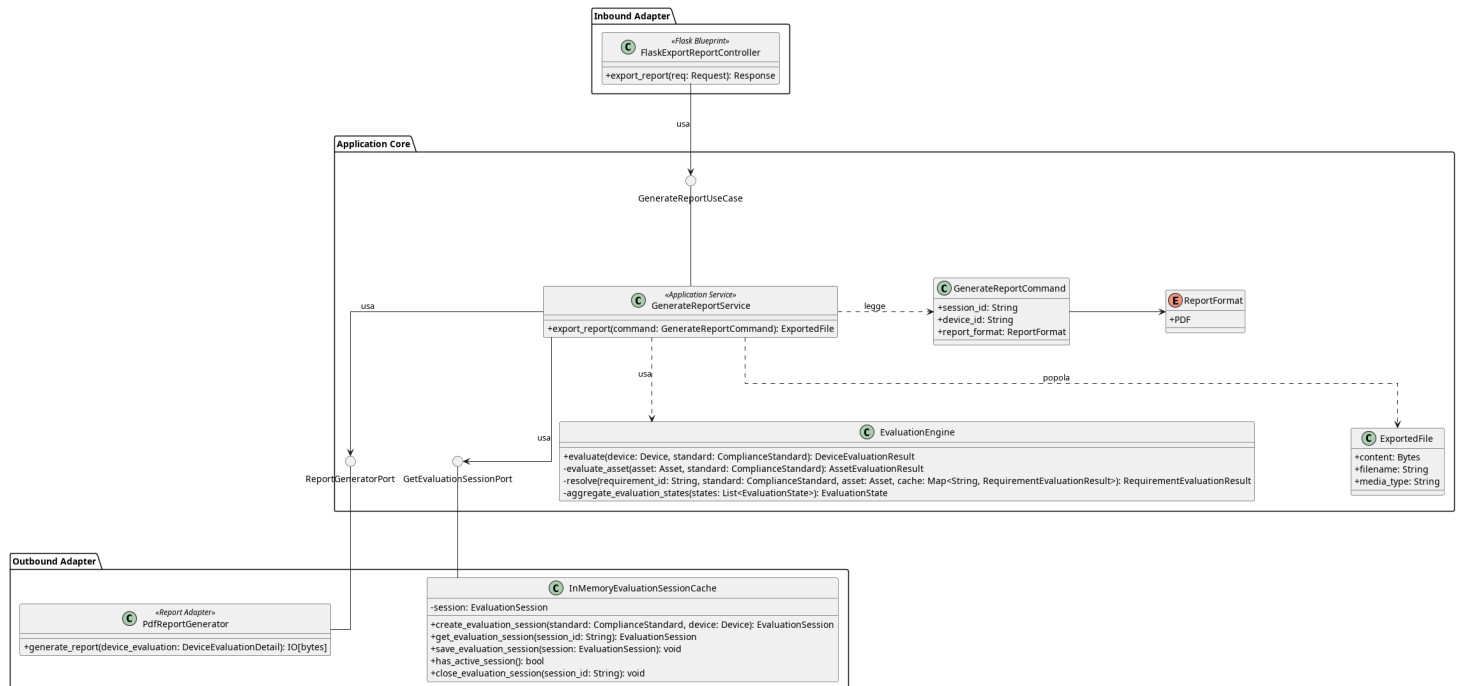


Figura 126: ExportReport

Il diagramma illustra l'architettura del modulo dedicato alla generazione e all'esportazione_G dei report riassuntivi di una valutazione_G. Il flusso permette di recuperare i dati di una sessione, valutare il dispositivo_G tramite *EvaluationEngine* e compilare dinamicamente un documento PDF_G da restituire all'utente tramite download.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la [Sezione 5.3.1.5](#).
- Per la definizione di *GetEvaluationSessionPort*, vedere la [Sezione 5.3.1.7](#).
- Per la definizione di *ExportedFile*, vedere la [Sezione 5.1.25](#).
- Per la definizione di *EvaluationEngine*, vedere la [Sezione 5.1.14](#)

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso_G.

5.5.2.1) FlaskExportReportController



Figura 127: ExportReportController

Descrizione

ExportReportController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP per la generazione e l'esportazione_G del report di una sessione di valutazione_G. Estrae dalla request i parametri necessari, costruisce il Command e inoltra la richiesta al livello applicativo.

Attributi

- `generate_report_use_case`: `GenerateReportUseCase` — porta di inbound usata per generare un Report.

Metodi

- `+ export_report(session_id: String, device_id: String, fmt: String): Response` — riceve la richiesta HTTP, valida il formato richiesto, costruisce il *GenerateReportCommand* e invoca il caso d'uso_G; restituisce il file generato come risposta HTTP scaricabile.

5.5.2.2) GenerateReportUseCase



Figura 128: GenerateReportUseCase

Descrizione

GenerateReportUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per la generazione e l'esportazione_G di un report. Viene implementata da *GenerateReportService* e utilizzata da *ExportReportController*.

Attributi

GenerateReportUseCase non definisce attributi.

Metodi

- `+ export_report(command: GenerateReportCommand): ExportedFile` — firma del metodo delegato all'esecuzione della logica di generazione del report a partire dai dati contenuti nel Command; restituisce un oggetto *ExportedFile* contenente il file generato, il nome del file e il tipo MIME necessari alla costruzione della risposta HTTP.

5.5.2.3) GenerateReportCommand

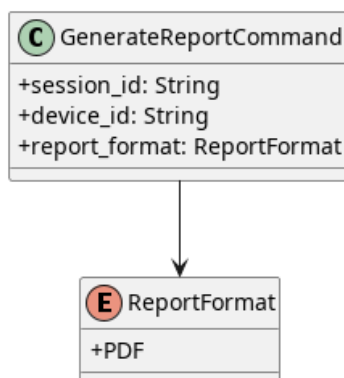


Figura 129: GenerateReportCommand

Descrizione

GenerateReportCommand è il Command Object che veicola i parametri necessari alla generazione del report dal controller al service.

Attributi

- `+ session_id: String` — identificativo univoco della sessione di valutazione_G da cui generare il report.
- `+ device_id: String` — identificativo univoco del dispositivo_G oggetto del report.
- `+ report_format: ReportFormat` — formato desiderato per il report in uscita, vincolato ai valori dell'enumerazione *ReportFormat*.

Metodi

GenerateReportCommand non definisce metodi propri.

5.5.2.4) ReportFormat

Descrizione

ReportFormat è un'enumerazione che definisce i formati di report supportati dal sistema.

Valori

- `PDFG` — formato PDF_G.

5.5.2.5) GenerateReportService



Figura 130: GenerateReportService

Descrizione

GenerateReportService è il service applicativo appartenente all'Application Core responsabile dell'orchestrazione del processo di generazione del report. Implementa l'interfacciag *GenerateReportUseCase*, recupera la sessione attiva tramite *GetEvaluationSessionPort*, valuta il dispositivo_G tramite *EvaluationEngine*, costruisce il dettaglio della valutazione_G e delega la generazione materiale del documento a *ReportGeneratorPort*; restituisce il risultato incapsulato in un oggetto *ExportedFile*. Internamente utilizza i metodi privati *_build_device_detail*, *_build_asset_detail* e *_build_requirement_detail* per costruire la struttura gerarchica dei dettagli di valutazione_G.

Attributi

- *get_evaluation_session_port*: *GetEvaluationSessionPort* — outbound port usata per prelevare la sessione di valutazione_G.
- *report_generator_port*: *ReportGeneratorPort* — outbound port usata per generare il Report.
- *evaluation_engine*: *EvaluationEngine* — oggetto di dominio usato per calcolare il *DeviceResult* i cui dati saranno immessi nel Report.

Metodi

- + *export_report*(command: *GenerateReportCommand*): *ExportedFile* — concretizza il contratto definito da *GenerateReportUseCase*. Recupera la sessione, valuta il dispositivo_G tramite *EvaluationEngine*, costruisce il dettaglio della valutazione_G e restituisce il file generato incapsulato in *ExportedFile*.

5.5.2.6) ReportGeneratorPort

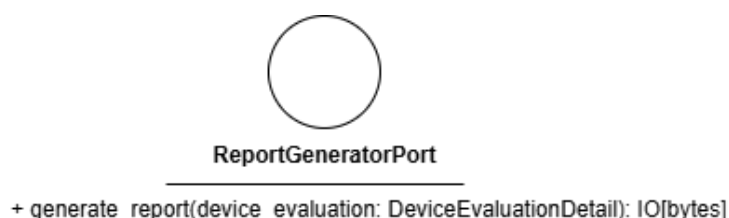


Figura 131: ReportGeneratorPort

Descrizione

ReportGeneratorPort è l'interfacciag (Outbound Port) che definisce il contratto per la generazione materiale del report a partire dal dettaglio della valutazione_G. Viene implementata da *PdfReportGenerator* e utilizzata da *GenerateReportService*.

Attributi

ReportGeneratorPort non definisce attributi.

Metodi

- + generate_report(device_evaluation: DeviceEvaluationDetail): IO[bytes]
— firma del metodo delegato alla generazione del file del report a partire dal dettaglio della valutazione_G del dispositivo_G.

5.5.2.7) PdfReportGenerator



Figura 132: PdfReportGenerator

Descrizione

PdfReportGenerator è l'Outbound Adapter che implementa *ReportGeneratorPort*. Si occupa della formattazione e della generazione materiale del file del report in formato PDF_G a partire dal dettaglio della valutazione_G. Internamente utilizza metodi privati per la formattazione dell'intestazione, del corpo e del piè di pagina del documento.

Attributi

PdfReportGenerator non definisce attributi propri.

Metodi

- + generate_report(device_evaluation: DeviceEvaluationDetail): IO[bytes]
— implementa il metodo dell'interfaccia_G, avviando il processo di creazione del PDF_G e restituendo lo stream di byte generato

5.6) Session

5.6.1) OpenEvaluationSession

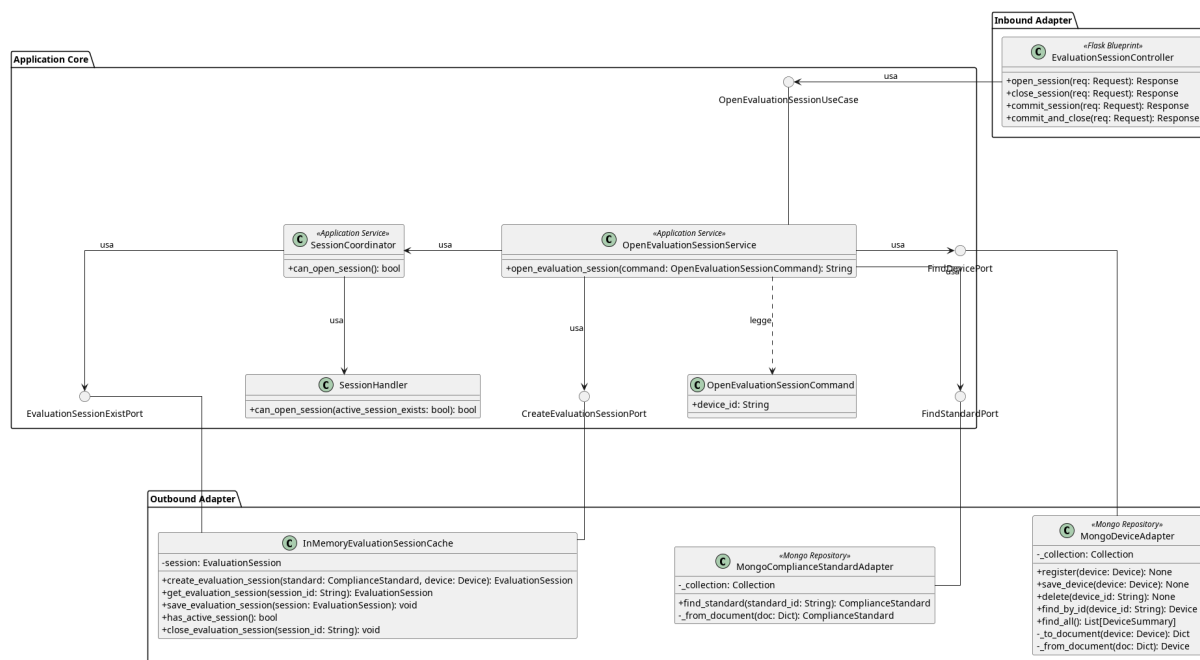


Figura 133: Caso d'uso_G OpenEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato all'apertura di una sessione di valutazione_G.

- Per la definizione di *Device*, vedere la **Sezione 5.1.1**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la **Sezione 5.3.1.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la **Sezione 5.2.1.6**.
- Per la definizione di *FindDevicePort*, vedere la **Sezione 5.2.3.5**.
- Per la definizione di *MongoStandardAdapter*, vedere la **Sezione 5.2.4.9**.
- Per la definizione di *FindStandardPort*, vedere la **Sezione 5.2.4.8**.
- Per la definizione di *SessionHandler*, vedere la **Sezione 5.1.24**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso_G.

5.6.1.1) EvaluationSessionController

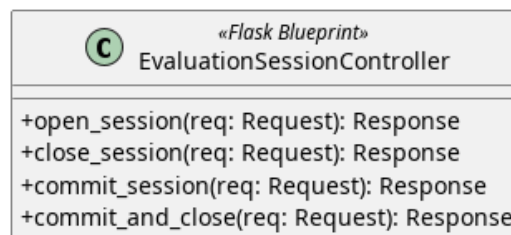


Figura 134: EvaluationSessionController

Descrizione

EvaluationSessionController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione del ciclo di vita della sessione di valutazione_G e le inoltra al livello applicativo.

Attributi

- - `open_use_case`: `OpenEvaluationSessionUseCase` — inbound port usata per aprire la sessione.
- - `close_use_case`: `CloseEvaluationSessionUseCase` — inbound port usata per chiudere la sessione.
- - `commit_use_case`: `CommitEvaluationSessionUseCase` — inbound port usata per salvare la sessione in memoria.

Metodi

- + `open_session(req: Request): Response` — riceve la richiesta HTTP di apertura di una nuova sessione di valutazione_G e restituisce una risposta HTTP con l'identificativo della sessione creata.
- + `close_session(req: Request): Response` — riceve la richiesta HTTP di chiusura della sessione corrente e restituisce una risposta HTTP con l'esito dell'operazione.
- + `commit_session(req: Request): Response` — riceve la richiesta HTTP di salvataggio definitivo della sessione e restituisce una risposta HTTP con l'esito dell'operazione.
- + `commit_and_close(req: Request): Response` — riceve la richiesta HTTP di salvataggio definitivo e chiusura contestuale della sessione e restituisce una risposta HTTP con l'esito dell'operazione.

5.6.1.2) OpenEvaluationSessionCommand

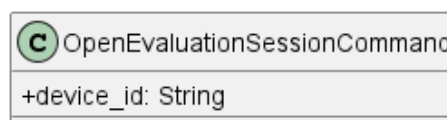


Figura 135: OpenEvaluationSessionCommand

Descrizione

OpenEvaluationSessionCommand è il Command Object utilizzato per trasportare i dati necessari all'apertura di una nuova sessione di valutazione_G. Incapsula i parametri di input del metodo esposto da *OpenEvaluationSessionUseCase*.

Attributi

- + device_id: String — identificativo univoco del Dispositivo_G per cui aprire la sessione.

Metodi

OpenEvaluationSessionCommand non definisce metodi propri.

5.6.1.3) OpenEvaluationSessionUseCase



Figura 136: OpenEvaluationSessionUseCase

Descrizione

OpenEvaluationSessionUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per l'apertura di una nuova sessione di valutazione_G. Viene implementata da *OpenEvaluationSessionService* e utilizzata da *EvaluationSessionController*.

Attributi

OpenEvaluationSessionUseCase non definisce attributi.

Metodi

- + open_evaluation_session(command: OpenEvaluationSessionCommand): String — firma del metodo delegato all'esecuzione della logica di apertura della sessione a partire dai dati contenuti nel Command.

5.6.1.4) OpenEvaluationSessionService

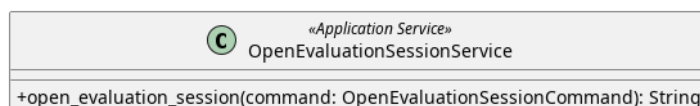


Figura 137: OpenEvaluationSessionService

Descrizione

OpenEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile della logica di apertura di una sessione di valutazione_G. Implementa l'interfaccia_G *OpenEvaluationSessionUseCase* e coordina il recupero del

Dispositivo_G tramite *FindDevicePort*, il recupero dello standard di conformità_G tramite *FindStandardPort* e la creazione della sessione tramite *CreateEvaluationSessionPort*.

Attributi

- - session_coordinator: SessionCoordinator — classe allo stato applicativo che centralizza le regole di business per autorizzare l'apertura di una nuova sessione.
- - create_session_port: CreateEvaluationSessionPort — outbound port usata per creare una sessione.
- - find_device_port: FindDevicePort — outbound port usata per trovare il dispositivo_G.
- - find_standard_port: FindStandardPort — outbound port usata per trovare lo Standard.

Metodi

- + open_evaluation_session(command: OpenEvaluationSessionCommand): String — concretizza il contratto definito da *OpenEvaluationSessionUseCase*. Recupera il Dispositivo_G e lo standard associato, inizializza una nuova *EvaluationSession* e ne richiede la creazione tramite *CreateEvaluationSessionPort*; restituisce l'identificativo della sessione creata.

5.6.1.5) SessionCoordinator

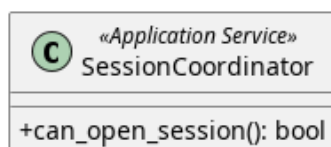


Figura 138: SessionCoordinator

Descrizione

SessionCoordinator è il Service che coordina la logica di dominio relativa alla gestione della sessione di valutazione_G. Verifica_G le precondizioni necessarie all'apertura di una sessione, come l'assenza di sessioni attive per il Dispositivo_G.

Attributi

- - exist_port: EvaluationSessionExistPort — Outbound port per verificare l'esistenza di sessioni attive nel sistema.
- - session_handler: SessionHandler — Componente di dominio che incapsula le regole di business per l'apertura delle sessioni.

Metodi

- + can_open_session(): bool — verifica_G se è possibile aprire una nuova sessione, restituendo true se le precondizioni sono soddisfatte.

5.6.1.6) EvaluationSessionExistPort

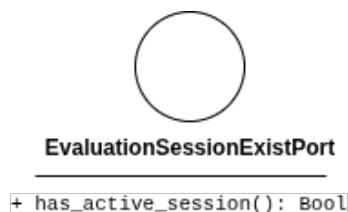


Figura 139: EvaluationSessionExistPort

Descrizione

EvaluationSessionExistPort è l'interfaccia_G (Outbound Port) che definisce il contratto per verificare la presenza di eventuali sessioni di valutazione_G attualmente attive all'interno del sistema.

Attributi

La classe *EvaluationSessionExistPort* non definisce attributi.

Metodi

- + `has_active_session(): bool` — interroga il sistema per determinare se esiste già una sessione attiva, restituendo il risultato come valore booleano.

5.6.1.7) CreateEvaluationSessionPort

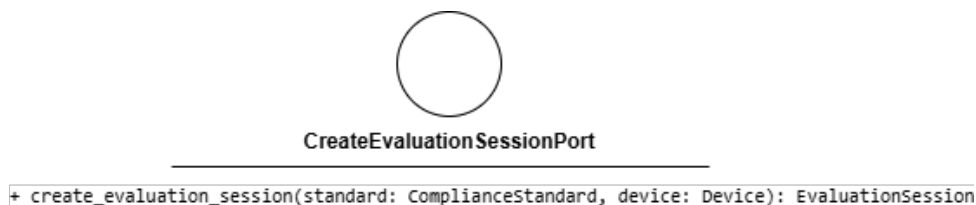


Figura 140: CreateEvaluationSessionPort

Descrizione

CreateEvaluationSessionPort è l'interfaccia_G (Outbound Port) che definisce il contratto per la creazione di una nuova sessione di valutazione_G nel sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da *OpenEvaluationSessionService*.

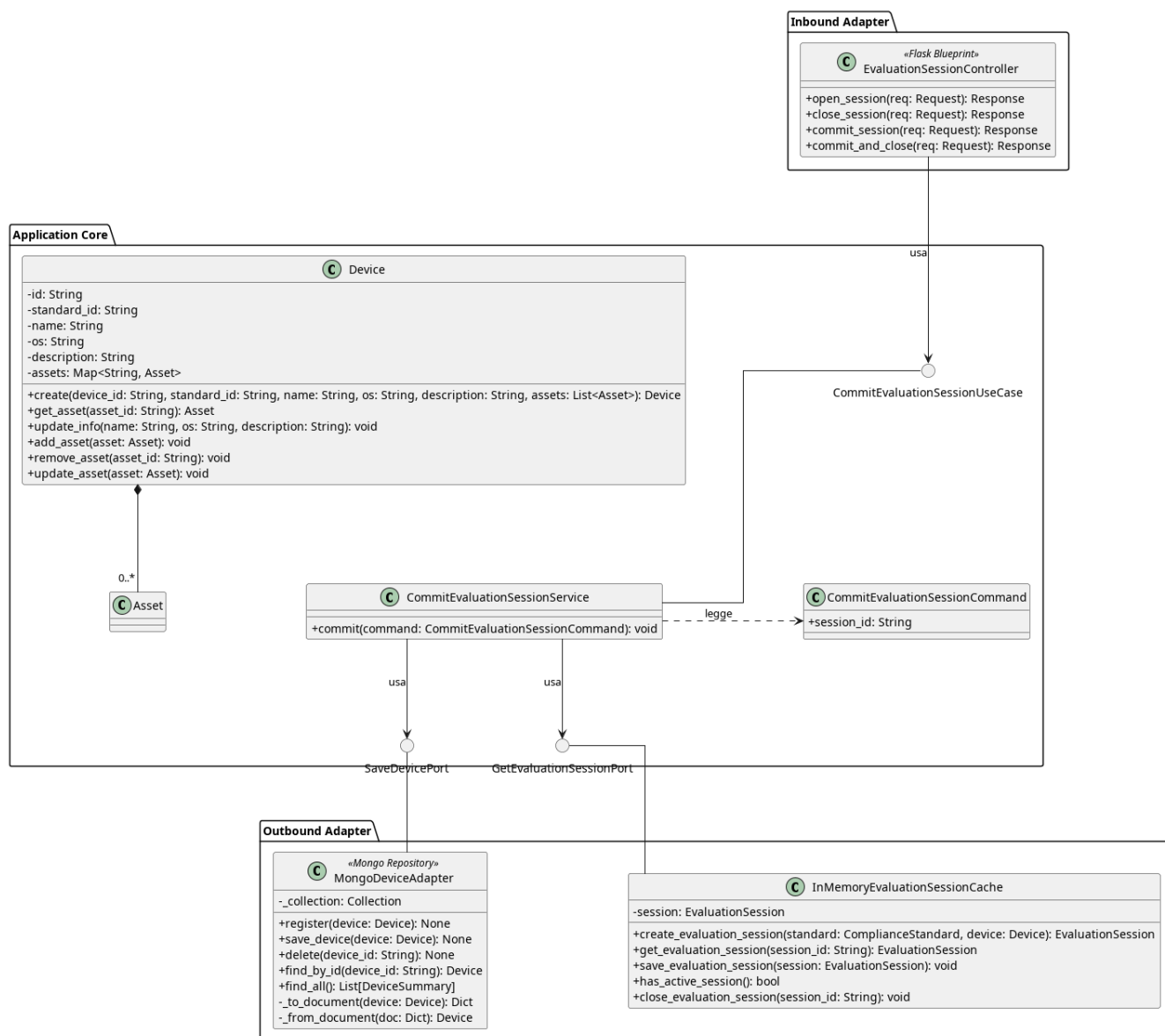
Attributi

CreateEvaluationSessionPort non definisce attributi.

Metodi

- + `create_session(): EvaluationSession` — firma del metodo che inizializza e registra una nuova sessione di valutazione_G nel sistema in memoria.

5.6.2) CommitEvaluationSession

Figura 141: Caso d'uso_G CommitEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato esclusivamente al consolidamento (commit) dei dati di una sessione di valutazione_G verso il dispositivo_G, senza richiederne la chiusura o l'eliminazione dalla memoria temporanea.

- Per la definizione di *EvaluationSessionController*, vedere la **Sezione 5.6.1.1**.
- Per la definizione di *Device*, vedere la **Sezione 5.1.1**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la **Sezione 5.3.1.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la **Sezione 5.2.1.6**.
- Per la definizione di *SaveDevicePort*, vedere la **Sezione 5.2.3.4**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti specifici introdotti per questo flusso operativo.

5.6.2.1) CommitEvaluationSessionUseCase



Figura 142: CommitEvaluationSessionUseCase

Descrizione

CommitEvaluationSessionUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per eseguire il consolidamento (commit) dei dati di una sessione di valutazione_G attiva, applicando definitivamente le modifiche all'entità dispositivo_G associata. Viene implementata a livello applicativo dal *CommitEvaluationSessionService* e utilizzata dal controller *EvaluationSessionController*.

Attributi

CommitEvaluationSessionUseCase non definisce attributi.

Metodi

- + commit(command: CommitEvaluationSessionCommand): void — firma del metodo delegato all'esecuzione della logica di consolidamento dei dati della sessione di valutazione_G a partire dal comando ricevuto in input.

5.6.2.2) CommitEvaluationSessionService

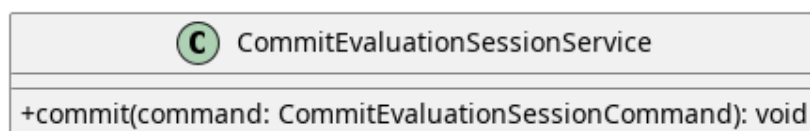


Figura 143: CommitEvaluationSessionService

CommitEvaluationSessionService è il service applicativo dell'Application Core responsabile di orchestrare l'operazione di commit. Implementa l'interfaccia_G *CommitEvaluationSessionUseCase*. Coordina il recupero della sessione attualmente in corso_G (tramite la porta *GetEvaluationSessionPort*), legge i dati necessari dal *CommitEvaluationSessionCommand* e applica le modifiche definitive delegando il salvataggio all'entità dispositivo_G (tramite *SaveDevicePort*).

Attributi

- save_device_port: SaveDevicePort — outbound port usata per salvare il dispositivo_G in memoria
- get_evaluation_session_port: GetEvaluationSessionPort — outbound port usata per prelevare la *EvaluationSession*

Metodi

- + `commit(command: CommitEvaluationSessionCommand): void` — concretizza la logica di business relativa al consolidamento dei dati. Utilizza i parametri incapsulati nel comando per applicare le modifiche allo stato persistente del dispositivo_G.

5.6.2.3) CommitEvaluationSessionCommand

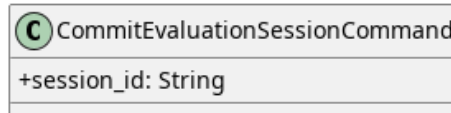


Figura 144: CommitEvaluationSessionCommand

Descrizione

CommitEvaluationSessionCommand incapsula i parametri necessari per richiedere il consolidamento (`commit`) di una sessione di valutazione_G. Serve a disaccoppiare i dati di input dalle firme dei metodi dei service applicativi.

Attributi

- + `session_id: String` — l'identificativo univoco della sessione di valutazione_G di cui si richiede il consolidamento dei dati.

Metodi

CommitEvaluationSessionCommand non definisce metodi.

5.6.3) CloseEvaluationSession

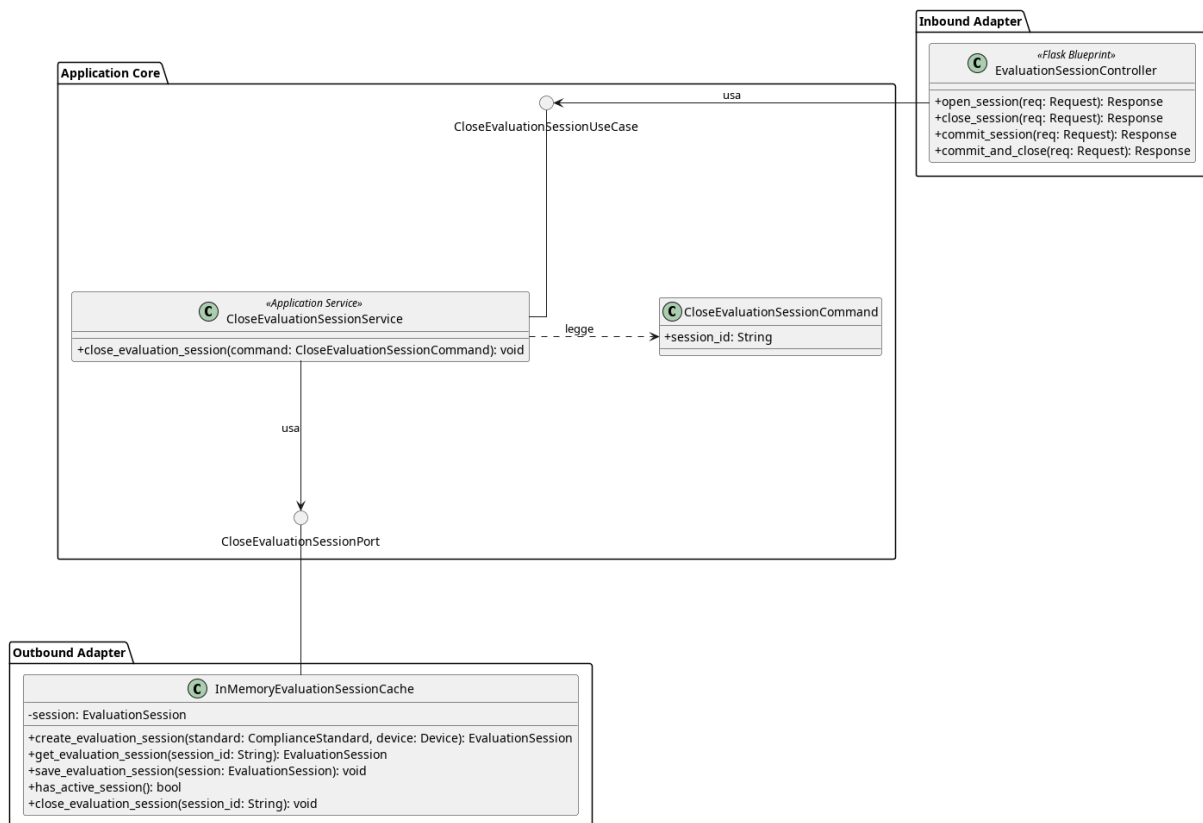


Figura 145: Caso d'uso_G CloseEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato alla chiusura e all'eliminazione di una sessione di valutazione_G.

- Per la definizione di *EvaluationSessionController*, vedere la **Sezione 5.6.1.1**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la **Sezione 5.3.1.5**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso_G.

5.6.3.1) CloseEvaluationSessionUseCase

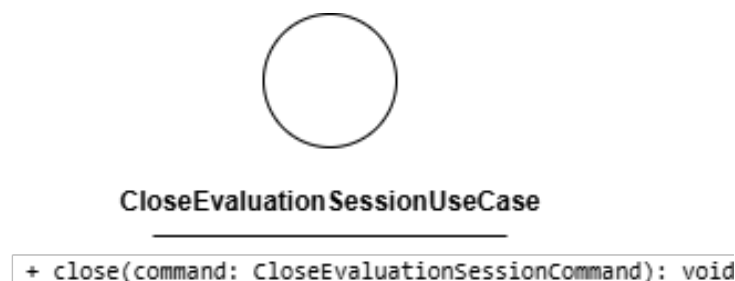


Figura 146: CloseEvaluationSessionUseCase

Descrizione

CloseEvaluationSessionUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per la chiusura definitiva di una sessione di valutazione_G attiva. Rappresenta l'operazione conclusiva del ciclo di vita della sessione e viene implementata da *CloseEvaluationSessionService* e utilizzata dal controller *EvaluationSessionController*.

Attributi

CloseEvaluationSessionUseCase non definisce attributi.

Metodi

- + close(command: CloseEvaluationSessionCommand): void — firma del metodo delegato all'esecuzione della logica di chiusura della sessione a partire dal comando ricevuto in input.

5.6.3.2) CloseEvaluationSessionService



Figura 147: CloseEvaluationSessionService

Descrizione

CloseEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile della logica di chiusura della sessione. Implementa l'interfaccia_G *CloseEvaluationSessionUseCase*. Una volta eseguite le operazioni di dominio necessarie, richiede la rimozione della sessione dal sistema di persistenza richiamando la porta in uscita *DeleteSessionPort*.

Attributi

- - delete_session_port: DeleteSessionPort — outbound port usata per eliminare la sessione.

Metodi

- + close(command: CloseEvaluationSessionCommand): void — concretizza il contratto definito da *CloseEvaluationSessionUseCase*. Coordina le operazioni di chiusura e inoltra la richiesta di eliminazione della sessione utilizzando i parametri incapsulati nel comando.

5.6.3.3) CloseEvaluationSessionCommand

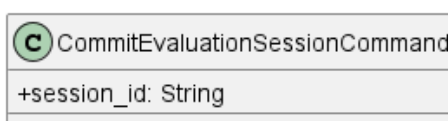


Figura 148: CloseEvaluationSessionCommand

Descrizione

CloseEvaluationSessionCommand incapsula i parametri necessari per richiedere la chiusura di una sessione di valutazione_G. Serve a disaccoppiare i dati di input dalle firme dei metodi dei service applicativi.

Attributi

- + session_id: String — l'identificativo univoco della sessione di valutazione_G di cui si richiede la chiusura e l'eliminazione.

Metodi

CloseEvaluationSessionCommand non definisce metodi.

5.6.3.4) DeleteSessionPort

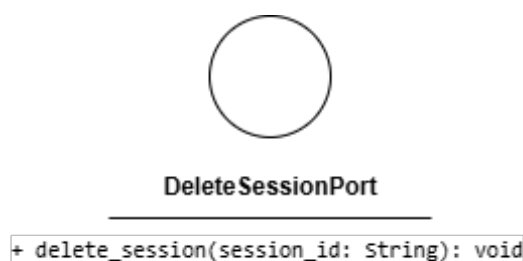


Figura 149: DeleteSessionPort

Descrizione

DeleteSessionPort è l'interfaccia_G (Outbound Port) che definisce il contratto per l'eliminazione di una sessione di valutazione_G dal sistema di archiviazione (es. la cache in memoria). Viene utilizzata da *CloseEvaluationSessionService* e implementata nel livello di adapter da *InMemoryEvaluationSessionCache*.

Attributi

DeleteSessionPort non definisce attributi.

Metodi

- + delete_session(session_id: String): void — firma del metodo che si occupa di rimuovere o invalidare lo stato di una specifica sessione di valutazione_G dal sistema di persistenza.

5.7) Area navigazione

5.7.1) EvaluateDecisionNode

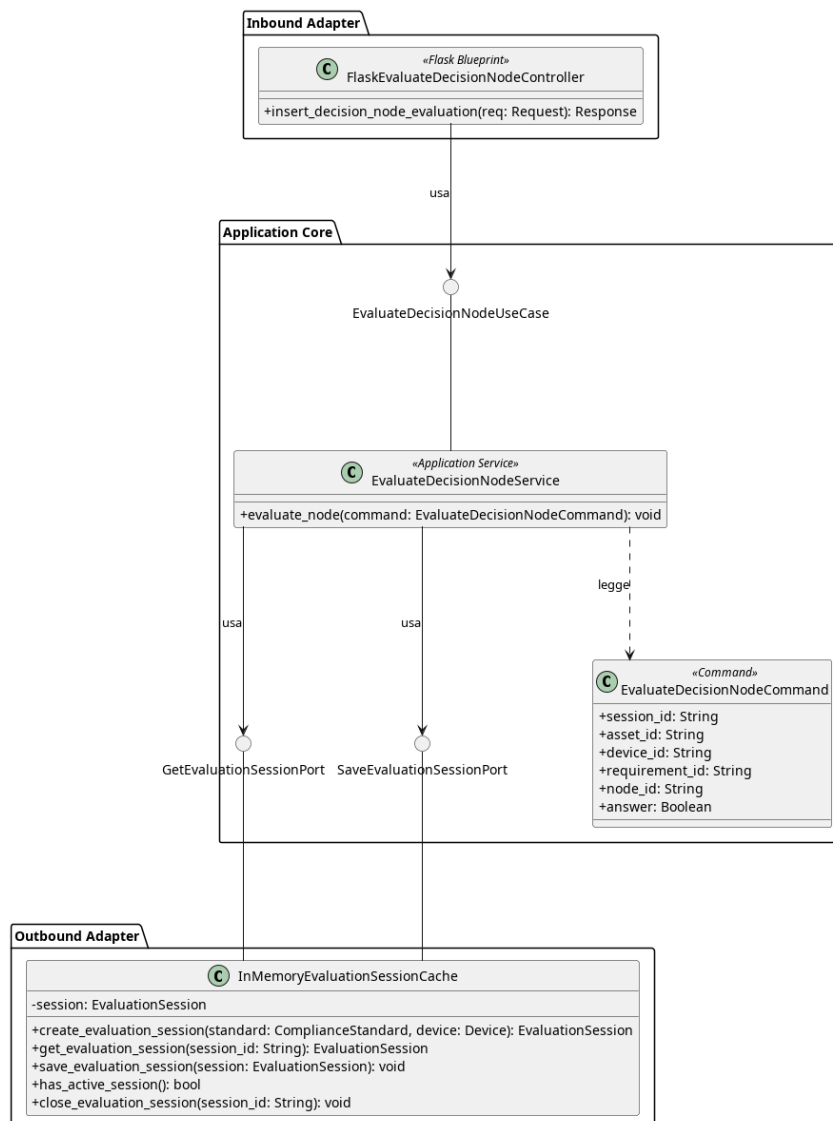


Figura 150: Caso d'uso_G EvaluateDecisionNode

Il diagramma illustra l'architettura del modulo dedicato alla valutazione_G di un nodo_G decisionale durante la verifica_G di conformità.

- Per la definizione di `InMemoryEvaluationSessionCache`, vedere la **Sezione 5.3.1.5**.
- Per la definizione di `GetEvaluationSessionPort`, vedere la **Sezione 5.3.1.7**.
- Per la definizione di `SaveEvaluationSessionPort`, vedere la **Sezione 5.3.1.6**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso_G.

5.7.1.1) EvaluateDecisionNodeController

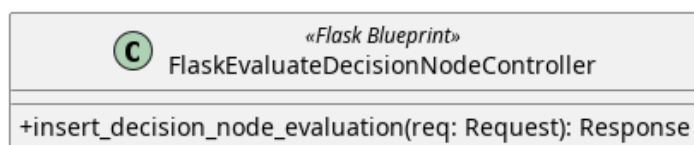


Figura 151: EvaluateDecisionNodeController

Descrizione

EvaluateDecisionNodeController è il controller Flask appartenente all’Inbound Adapter che riceve le richieste HTTP per la valutazione_G di un nodo_G decisionale e le inoltra al livello applicativo.

Attributi

- - `evaluate_decision_node_use_case`: *EvaluateDecisionNodeUseCase* — out-bound port usata per valutare un nodo_G

Metodi

- + `insert_decision_node_evaluation(req: Request): Response` — riceve la richiesta HTTP, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l’esito dell’operazione.

5.7.1.2) EvaluateDecisionNodeUseCase



Figura 152: EvaluateDecisionNodeUseCase

Descrizione

EvaluateDecisionNodeUseCase è l’interfaccia_G (Inbound Port) che definisce il contratto per la valutazione_G di un nodo_G decisionale. Viene implementata da *EvaluateDecisionNodeService* e utilizzata da *FlaskEvaluateDecisionNodeController*.

Attributi

EvaluateDecisionNodeUseCase non definisce attributi.

Metodi

- + `evaluate_node(command: EvaluateDecisionNodeCommand): void` — firma del metodo delegato all’esecuzione della logica di valutazione_G a partire dai dati contenuti nel comando.

5.7.1.3) EvaluateDecisionNodeCommand

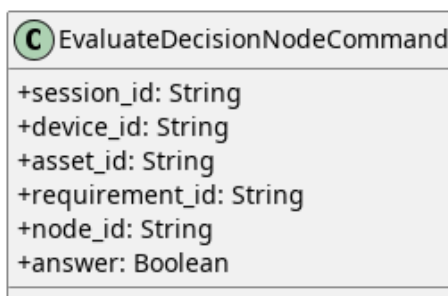


Figura 153: EvaluateDecisionNodeCommand

Descrizione

EvaluateDecisionNodeCommand è il Command Object che veicola i dati necessari alla valutazione_G di un nodo_G decisionale dal controller al service.

Attributi

- + session_id: String — identificativo della sessione di valutazione_G attiva.
- + device_id: String — identificativo del dispositivo_G oggetto di valutazione_G.
- + asset_id: String — identificativo dell'Asset_G oggetto di valutazione_G.
- + requirement_id: String — identificativo del requisito_G in fase di valutazione_G.
- + node_id: String — identificativo del nodo_G decisionale.
- + answer: Boolean — valore della risposta fornita per il nodo_G decisionale.

Metodi

EvaluateDecisionNodeCommand non definisce metodi propri.

5.7.1.4) EvaluateDecisionNodeService

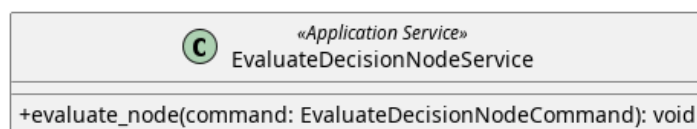


Figura 154: EvaluateDecisionNodeService

Descrizione

EvaluateDecisionNodeService è il service applicativo appartenente all'Application Core responsabile della logica di valutazione_G di un nodo_G decisionale. Implementa l'interfaccia_G *EvaluateDecisionNodeUseCase*. Recupera la sessione attiva tramite *GetEvaluationSessionPort*, individua l'Asset_G corrispondente all'interno del dispositivo_G, registra la risposta sul nodo_G decisionale e persiste la sessione aggiornata tramite *SaveEvaluationSessionPort*. In caso di sessione non trovata, asset_G non trovato o errore di salvataggio, propaga un'eccezione di tipo *EvaluateNodeFailure*.

Attributi

- `get_evaluation_session_port`: `GetEvaluationSessionPort` — outbound port usata per recuperare la sessione di valutazione_G.
- `save_evaluation_session_port`: `SaveEvaluationSessionPort` — outbound port usata per salvare le modifiche applicate alla sessione.

Metodi

- `+ evaluate_node(command: EvaluateDecisionNodeCommand): void` — concretizza il contratto definito da *EvaluateDecisionNodeUseCase*. Recupera la sessione attiva, individua l'Asset_G nel dispositivo_G, registra la risposta al nodo_G decisionale e persiste la sessione aggiornata.

5.7.2) InsertJustification

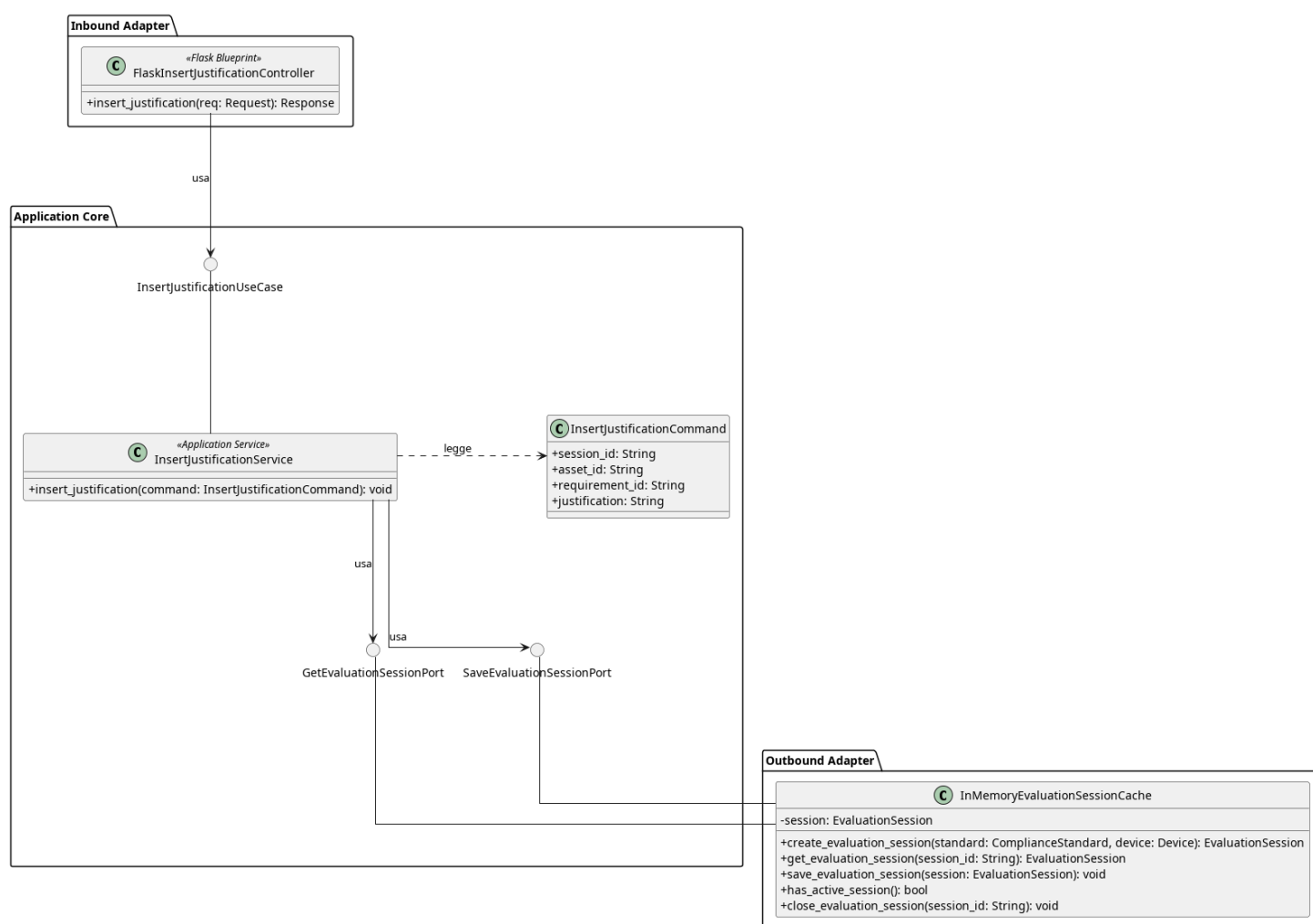


Figura 155: Caso d'uso_G InsertJustification

Il diagramma illustra l'architettura del modulo dedicato all'inserimento di una giustificazione testuale per un requisito_G di conformità_G durante la sessione di valutazione_G.

- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la **Sezione 5.3.1.5**.
- Per la definizione di *GetEvaluationSessionPort*, vedere la **Sezione 5.3.1.7**.
- Per la definizione di *SaveEvaluationSessionPort*, vedere la **Sezione 5.3.1.6**.

Di seguito vengono documentati i componenti introdotti specificamente per questo caso d'uso_G.

5.7.2.1) FlaskInsertJustificationController

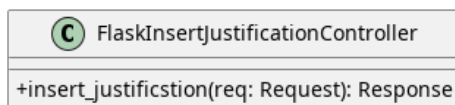


Figura 156: FlaskInsertJustificationController

Descrizione

FlaskInsertJustificationController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di inserimento di una giustificazione per un requisito_G e le inoltra al livello applicativo.

Attributi

- - `insert_justification_use_case`: `InsertJustificationUseCase` — inbound port usata per inserire la giustificazione.

Metodi

- + `insert_justification(req: Request): Response` — riceve la richiesta HTTP di inserimento della giustificazione, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.7.2.2) InsertJustificationUseCase

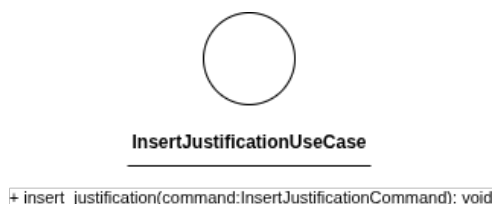


Figura 157: InsertJustificationUseCase

Descrizione

InsertJustificationUseCase è l'interfaccia_G (Inbound Port) che definisce il contratto per l'inserimento di una giustificazione testuale associata a un requisito_G di conformità_G. Viene implementata da *InsertJustificationService* e utilizzata da *FlaskInsertJustificationController*.

Attributi

InsertJustificationUseCase non definisce attributi.

Metodi

- + insert_justification(command: InsertJustificationCommand): void — firma del metodo delegato all'esecuzione della logica di inserimento della giustificazione a partire dai dati contenuti nel comando.

5.7.2.3) InsertJustificationCommand

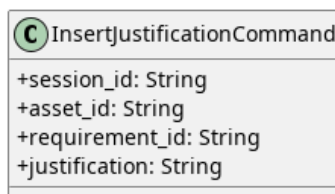


Figura 158: InsertJustificationCommand

Descrizione

InsertJustificationCommand è l'oggetto che veicola i dati necessari all'inserimento di una giustificazione dal controller al service.

Attributi

- + session_id: String — identificativo della sessione di valutazione_G attiva.
- + asset_id: String — identificativo dell'Asset_G oggetto di valutazione_G.
- + requirement_id: String — identificativo del requisito_G a cui si associa la giustificazione.
- + justification: String — testo della giustificazione da inserire.

Metodi

InsertJustificationCommand non definisce metodi.

5.7.2.4) InsertJustificationService



Figura 159: InsertJustificationService

Descrizione

InsertJustificationService è il service applicativo appartenente all'Application Core responsabile della logica di inserimento di una giustificazione per un requisito_G di conformità_G. Implementa l'interfaccia_G *InsertJustificationUseCase*, recupera la sessione attiva tramite *GetEvaluationSessionPort*, associa la giustificazione al requisito_G specificato e persiste la sessione aggiornata tramite *SaveEvaluationSessionPort*.

Attributi

- get_evaluation_session_port: GetEvaluationSessionPort — outbound port usata per recuperare la sessione di valutazione_G.

- `save_evaluation_session_port`: `SaveEvaluationSessionPort` — outbound port usata per salvare le modifiche applicate alla sessione.

Metodi

- `+ insert_justification(command: InsertJustificationCommand): void` — concretizza il contratto definito da *InsertJustificationUseCase*. Recupera la sessione attiva, individua il requisito_G corrispondente ai parametri incapsulati nel comando, registra la giustificazione fornita e persiste la sessione aggiornata.

5.8) Architettura Frontend: Widget Semplici

L'approccio server-side rendering adottato nel progetto è integrato da componenti e widget reattivi realizzati con Vue 3 (Composition API). Per ogni pagina, Flask rende-
rizza il template Jinja con i dati statici, mentre le parti interattive vengono delegate a
«isole» Vue indipendenti, montate su specifici punti del DOM.

Il sistema frontend gestisce solo l'aspetto grafico e la reattività, altre operazioni
vengono realizzate tramite chiamate API.

Per mantenere il codice aderente ai principi SOLID — in particolare Single Responsi-
bility e Dependency Inversion — l'architettura frontend si sviluppa su tre livelli:

3. Componenti Dumb

Componenti UI generici e riutilizzabili, privi di qualsiasi conoscenza del dominio
applicativo. Ricevono dati e callback esclusivamente tramite props ed emettono
eventi verso il padre. Non accedono a store, non eseguono chiamate di rete, non
conoscono la struttura degli URL.

2. Widget Smart

Isole applicative che orchestrano uno o più componenti dumb per realizzare una
funzionalità di dominio specifica. Contengono la logica di business (chiamate HTTP,
validazione_G, gestione del redirect post-azione) e compongono i componenti dumb
passando loro i dati necessari. Ogni widget è autocontenuto nella propria cartella.

1. Entry Point (Mount Point)

Livello di integrazione tra Flask/Jinja e Vue. Ogni entry point legge i dati iniettati
dal server tramite attributi `data-*` del DOM, li converte nel formato atteso dal widget
e monta l'applicazione Vue sul nodo_G HTML corrispondente. Non contiene logica
di business né di presentazione.

Logica Condivisa (Shared) Trasversalmente ai tre livelli, vi è logica shared riutiliz-
zabile che non è legata a un singolo widget né costituisce un componente visuale:

- **Composable** — Funzioni che incapsulano stato reattivo e logica riutilizzabile tra
widget diversi. Seguono la convenzione Vue `use*`. Esempio: `useFormModel`, che

gestisce lo stato dei campi, la validazione_G client-side e l'integrazione con errori server-side.

- **Definizioni di dominio** — Oggetti che descrivono la struttura dei dati condivisi tra widget. Esempio: `deviceFormFields` e `assetFormFields`, che definiscono campi e regole di validazione_G dei rispettivi form, condivisi tra i widget di creazione e modifica.
- **Utilità** — Funzioni pure senza dipendenze da Vue, utilizzabili sia dai widget che dagli entry point. Esempio: `navigationGuard`, che gestisce il blocco di navigazione per le pagine con dati non salvati.

Questa separazione garantisce che i componenti semplici siano riutilizzabili tra widget diversi, che i widget siano testabili in isolamento, e che la dipendenza_G dagli URL e dai dati del server sia confinata esclusivamente nel livello di integrazione.

5.8.1) Shared

5.8.1.1) Validation Rule

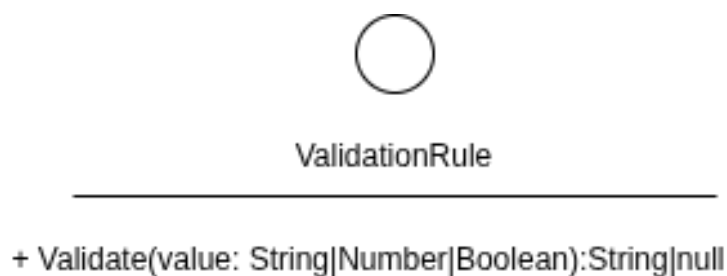


Figura 160: Shared - ValidationRule

Descrizione:

Viene utilizzato per la validazione_G dei dati inseriti dall'utente. Rappresenta un contratto, un oggetto deve esporre un metodo `validate` che accetta un valore primitivo, effettua un controllo, ritorna un messaggio di errore che indica perché non ha superato il controllo, se lo ha superato non ritorna nulla.

Attributi:

Le interfacce non hanno attributi

Metodi:

- `+ validate(value:String|Number|Boolean):String|null`: Accetta un valore primitivo, effettua un controllo, ritorna un messaggio di errore che indica perché non ha superato il controllo, se lo ha superato non ritorna nulla.

5.8.1.2) Field Definition

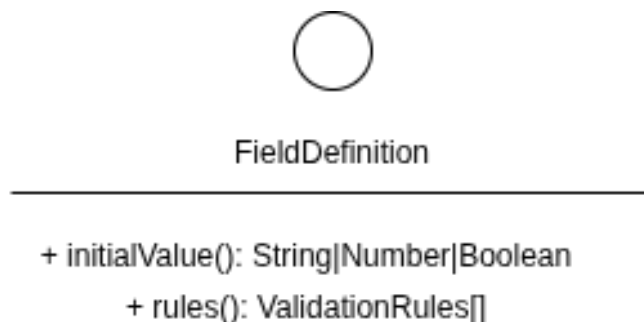


Figura 161: Shared - FieldDefinition

Descrizione:

Rappresenta un contratto strutturale che definisce i campi che un oggetto deve esporre per poter essere usato nel model di un form.

Attributi: L'interfaccia_G non ha attributi.

Metodi:

- `+ initialValue():String|Number|Boolean`:Il valore primitivo di default con cui il campo deve essere inizializzato nel DOM e a cui deve essere ripristinato in fase di reset.
- `+ rules(): ValidationRule[]`:Un array di elementi che implementano l'interfaccia_G `ValidationRule`, contenente l'elenco delle funzioni di validazione_G associate al campo.

5.8.1.3) Use Form Model

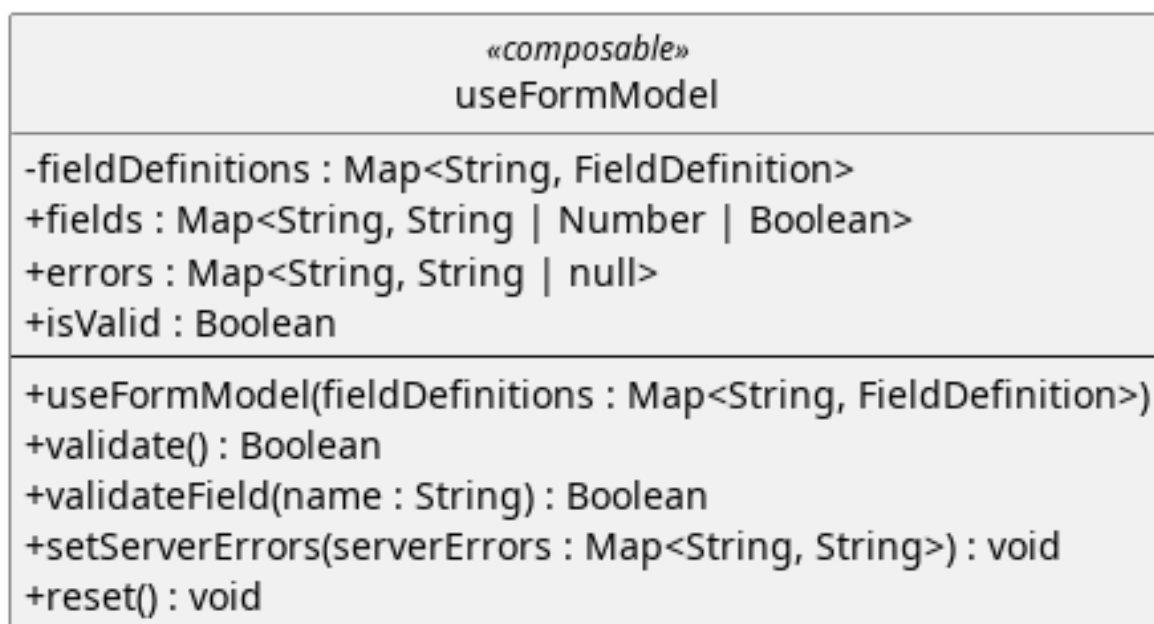


Figura 162: Shared - UseFormModel

Descrizione:

Inizializza e gestisce lo stato reattivo di un form di dominio a partire dalle sue definizioni. Si occupa della validazione client-side e dell'integrazione degli errori inviati dal server, isolando completamente la logica di business dal template visivo.

Attributi:

- - fieldDefinitions : Map<String, FieldDefinition>: Conserva internamente le field definition ricevute alla costruzione e le usa per usarlo nelle funzioni.
- + fields: Map<String, String | Number | Boolean> : Mappa reattiva che memorizza i valori correnti inseriti dall'utente per ciascun campo del form.
- + errors: Map<String, String | null>: Mappa reattiva che associa a ogni campo il relativo messaggio di errore (impostato a null se il campo è valido).
- + isValid: Boolean : Proprietà calcolata (computed). Restituisce true solo se tutti gli elementi all'interno di errors sono pari a null.

Metodi:

- + useFormModel(fieldDefinitions: Map<String, FieldDefinition>) : Costruttore/Funzione di inizializzazione. Riceve la configurazione dei campi e genera le strutture reattive per fields ed errors impostando i valori iniziali.
- + validate(): Boolean : Esegue la validazione su tutti i campi del form. Restituisce true se l'intero modulo è valido, altrimenti aggiorna la mappa degli errori e restituisce false.
- + validateField(name: String): Boolean : Invocato per convalidare un singolo campo. Applica le regole in ordine sequenziale e si interrompe al primo fallimento.
- + setServerErrors(serverErrors: Map<String, String>): void : Riceve una mappa di errori generati dalle API di Flask (backend) e li inietta direttamente nello stato errors per mostrarli all'utente.
- + reset(): void : Svuota tutti i messaggi di errore e ripristina i valori di fields allo stato iniziale definito in FieldDefinition.

5.8.1.4) Definizioni di Dominio (Constants)

Figura 163: Shared - Form Fields Configurations

Descrizione:

Oggetti JavaScript esportati come costanti, che fungono da singolo punto di configurazione per le regole di validazione_G degli input utente nei form.

Rappresentano una `Map<String, FieldDefinition>`, questi oggetti isolano le regole di validazione_G) e i valori iniziali dal resto del codice. Essendo condivisi, garantiscono una rigorosa simmetria di validazione_G tra le interfacce di creazione e di modifica.

Attributi:

Ogni attributo di questi oggetti rappresenta uno specifico campo del dominio e rispetta rigorosamente la forma definita da `FieldDefinition`:

Per `deviceFormFields`:

- + `deviceName`: `FieldDefinition`
- + `deviceOs`: `FieldDefinition`
- + `deviceDescription`: `FieldDefinition`

Per `assetFormFields`:

- + `name`: `FieldDefinition`
- + `assetType`: `FieldDefinition`
- + `description`: `FieldDefinition`

Metodi:

Trattandosi di oggetti di pura configurazione statica dei dati, non espongono alcun metodo operativo.

5.8.2) Component

5.8.2.1) AsyncButton

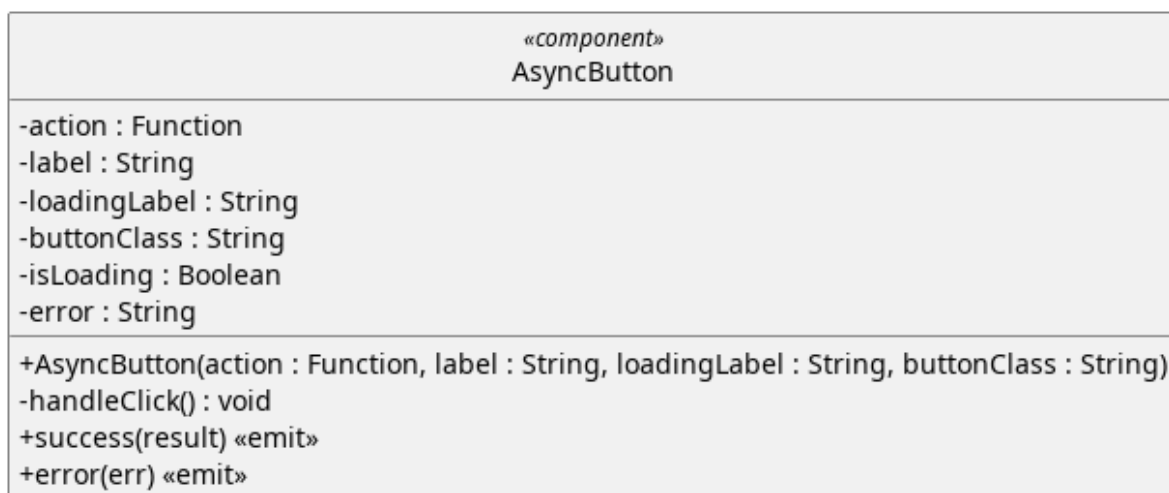


Figura 164: Componente - AsyncButton

Descrizione

Componente generico che esegue una funzione asincrona al click, gestendo lo stato di caricamento e la cattura degli errori.

Non possiede alcuna conoscenza del dominio applicativo: riceve la funzione da eseguire come parametro e comunica l'esito tramite eventi.

Attributi

- - `action: Function` : Funzione da eseguire al click del pulsante
- - `Label: String` : Testo del bottone.
- - `loadingLabel: String` : Testo del bottone durante l'esecuzione della funzione.
- - `buttonClass: String` : Stringa contenente l'elenco delle classi da applicare al pulsante
- - `isLoading : Boolean` — indica se la funzione asincrona è in esecuzione. Durante il caricamento il bottone è disabilitato per prevenire click multipli.
- - `error : String` — contiene il messaggio dell'ultimo errore verificatosi, null se l'ultima esecuzione ha avuto successo.

Metodi

- + `AsyncButton(action:Function, label:string, loadingLabel:String, buttonClass:String)` — costruttore. Riceve la funzione asincrona da eseguire, il testo del bottone nei due stati (default e caricamento), e una classe CSS opzionale per lo stile.
- `handleClick()` — metodo privato invocato al click. Verifica che non sia già in corso un'esecuzione, imposta lo stato di caricamento, esegue la funzione asincrona e gestisce il risultato.
- + `success(result)` — evento emesso al completamento con successo della funzione asincrona. Trasporta il valore restituito dalla funzione.
- + `error(err)` — evento emesso in caso di fallimento. Trasporta l'eccezione catturata.

5.8.2.2) BaseModal

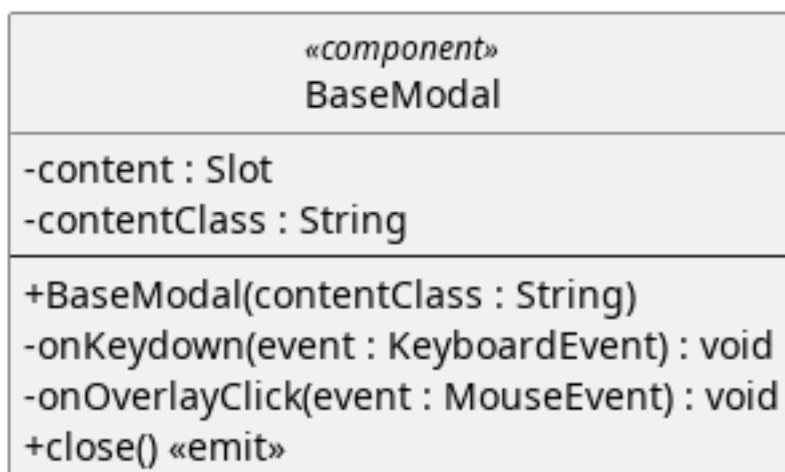


Figura 165: Componente - BaseModal

Descrizione Componente generico che fornisce l'infrastruttura per qualsiasi finestra di dialogo. Non possiede attributi interni né conosce il contenuto che ospita, interamente delegato a uno slot.

La decisione di quando il modale debba apparire o scomparire è responsabilità del componente padre, che lo monta e smonta tramite direttiva condizionale.

Attributi:

- - ContentClass: String : Stringa contenente l'elenco delle classi da applicare al contenuto del modal.
- - Content: Slot : slot di default che accetta il contenuto del modale. Il componente padre inietta tramite questo slot titolo, messaggio, bottoni e qualsiasi altro elemento necessario. Il BaseModal non ha alcuna conoscenza di ciò che lo slot contiene.

Metodi:

- + BaseModal(contentClass:String) : Stringa contenente l'elenco delle classi da applicare al contenuto del modal.
- - onKeyDown(Event:KeyboardEvent): metodo privato registrato come listener globale al mount. Intercetta la pressione del tasto Escape e emette l'evento di chiusura.
- - onOverlayClick(event) — metodo privato invocato al click sull'overlay. Emette l'evento di chiusura solo se il click avviene sull'overlay stesso e non su un elemento figlio.
- close() — evento emesso quando l'utente richiede la chiusura del modale (tramite Escape o click sull'overlay). Il padre decide come reagire.

5.8.2.3) FormField

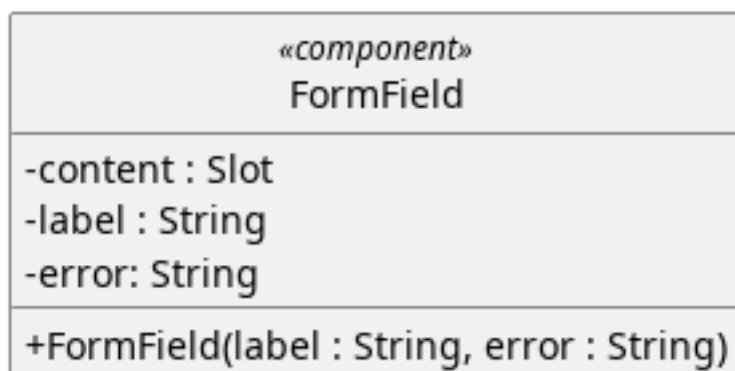


Figura 166: Componente - FormField

Descrizione:

Componente generico che funge da wrapper per un singolo campo di un form. Si occupa di mostrare la label associata al campo e l'eventuale messaggio di errore di validazione_G. Non conosce il tipo di input che ospita — il campo vero e proprio (text, select, textarea, radio) è delegato allo slot.

Attributi:

- - label: String : Testo da mostrare nella label del campo di input.
- - error: String : Testo da mostrare in caso di errore.
- - Content: Slot : slot di default che accetta l'input vero e proprio. Il widget padre inietta tramite questo slot il campo appropriato con il relativo binding ai dati.

Metodi:

- + FormField(label:String, error: String):

Costruttore. Riceve il testo della label e l'eventuale messaggio di errore. Se error è null il campo è considerato valido e il messaggio non viene mostrato.

5.8.2.4) Toast

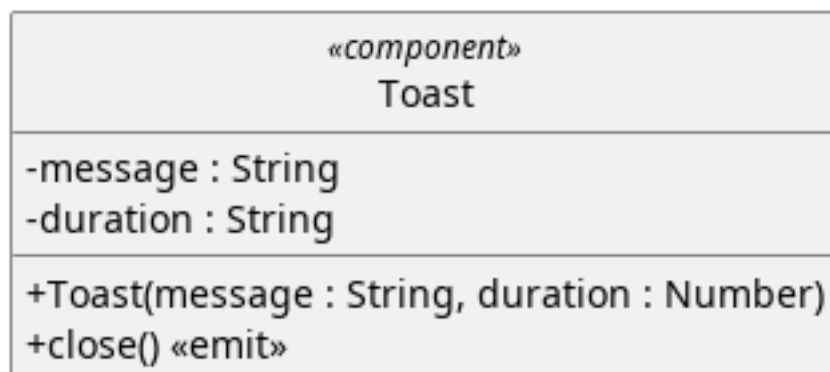


Figura 167: Componente - Toast

Descrizione:

Componente generico che mostra un messaggio temporaneo a schermo sotto forma di notifica. Scompare automaticamente dopo un tempo definito senza richiedere interazione da parte dell'utente. Non mantiene stato interno — il padre lo monta quando serve mostrare un messaggio e lo smonta quando riceve l'evento di chiusura.

Attributi:

- - message:String: messaggio da mostrare.
- - duration:Number: durata a schermo del componente.

Metodi:

- + Toast(message:String, duration:Number): costruttore. Riceve il testo del messaggio e la durata in millisecondi prima della chiusura automatica. Al mount del componente viene avviato un timer che, allo scadere, emette l'evento di chiusura.
- + close(): evento emesso alla scadenza del timer. Il padre reagisce smontando il componente.

5.8.2.5) FileDropZone

Figura 168: Componente - FileDropZone

Descrizione:

Componente generico che realizza un'area di caricamento file con supporto per drag & drop e selezione tramite click. Gestisce il feedback visivo durante il trascinamento, valida il tipo di file in base alle estensioni accettate e comunica il file selezionato al padre tramite evento. Non conosce cosa verrà fatto del file — l'upload o l'elaborazione sono responsabilità del widget che lo utilizza.

Attributi:

- - accept:String[]: Elenco di estensioni supportate.
- - placeholder: String: Testo del placeholder.
- - hint: String: Suggerimento testuale sui formati accettati.
- - disabled: Boolean: Flag per disabilitare l'interazione.
- - isDragging: Boolean: Indica se un file è attualmente trascinato sopra l'area di drop. Utilizzato per il feedback visivo.
- - selectedFile: File: Riferimento al file selezionato dall'utente, null se nessun file è stato scelto.

Metodi:

- + FileDropZone(accept: String[], placeholder: String, hint: String, disabled: Boolean): Costruttore. Riceve le estensioni accettate, il testo placeholder, un suggerimento sui formati supportati e un flag per disabilitare l'interazione.
- - handleFileChange(event:Event): Metodo privato invocato alla selezione di un file tramite il file picker nativo del browser.
- - handleDropEvent(event:DragEvent): Metodo privato invocato al rilascio di un file trascinato sull'area di drop. Valida il file e lo accetta o emette un errore.
- - isFileAccepted(file:File): Metodo privato che verifica se l'estensione del file è tra quelle accettate.
- - formatSize(bytes:Number): Metodo privato che converte una dimensione in byte in formato leggibile (Bytes, KB, MB).
- + reset(): Metodo pubblico esposto al padre tramite template ref. Resetta lo stato del componente rimuovendo il file selezionato.
- + select(file:File): Evento emesso quando un file valido viene selezionato o trascinato.
- + error(message:String): Evento emesso quando il file trascinato non supera la validazione del formato.

5.8.2.6) Device Form

«Component» DeviceForm
-fields: Map<String, String Number Boolean> -errors: Map<String, String null> -fieldComponents: FormField[]
+DeviceForm(fields: Map<String, String Number Boolean>, errors: Map<String, String null>)

Figura 169: Component - DeviceForm

Descrizione:

Componente di presentazione specifico per il dominio dei dispositivi. Si occupa esclusivamente di renderizzare l'interfaccia utente del modulo e di stabilire un binding

bidirezionale con i dati forniti dal componente genitore. È agnostico rispetto al contesto: non sa se sta creando un nuovo dispositivo_G o modificandone uno esistente, e non esegue alcuna logica di validazione_G o salvataggio.

Attributi:

- + fields: JSON_G: Oggetto reattivo iniettato dal padre contenente i valori dei campi del dispositivo_G.
- + errors: JSON_G: Oggetto iniettato dal padre contenente gli eventuali messaggi di errore da visualizzare per ciascun campo.

Metodi:

Il componente è puramente dichiarativo e non espone metodi operativi. Espone unicamente il costruttore:

- + DeviceForm(fields: Map<String, String | Number | Boolean>, errors: Map<String, String | null>)

5.8.2.7) Asset_G Form

«Component» AssetForm
-fields: Map<String, String Number Boolean> -errors: Map<String, String null> -fieldComponents: FormField[]
+AssetForm(fields: Map<String, String Number Boolean>, errors: Map<String, String null>)

Figura 170: Component - AssetForm

Descrizione:

Componente di presentazione delegato al rendering del modulo per la gestione degli Asset_G. Si occupa esclusivamente di renderizzare l'interfaccia_G utente del modulo e di stabilire un binding bidirezionale con i dati forniti dal componente genitore. È agnostico rispetto al contesto: non sa se sta creando un nuovo asset_G o modificandone uno esistente, e non esegue alcuna logica di validazione_G o salvataggio.

Attributi:

- + fields: Map<String, String | Number | Boolean>: Oggetto reattivo iniettato dal padre contenente i valori dei campi del dispositivo_G.
- + errors: Map<String, String | null>: Oggetto iniettato dal padre contenente gli eventuali messaggi di errore da visualizzare per ciascun campo.

Metodi:

Il componente è puramente dichiarativo e non espone metodi operativi. Espone unicamente il costruttore:

- + AssetForm(fields: Map<String, String | Number | Boolean>, errors: Map<String, String | null>)

5.8.3) Widget

5.8.3.1) AssetCreate

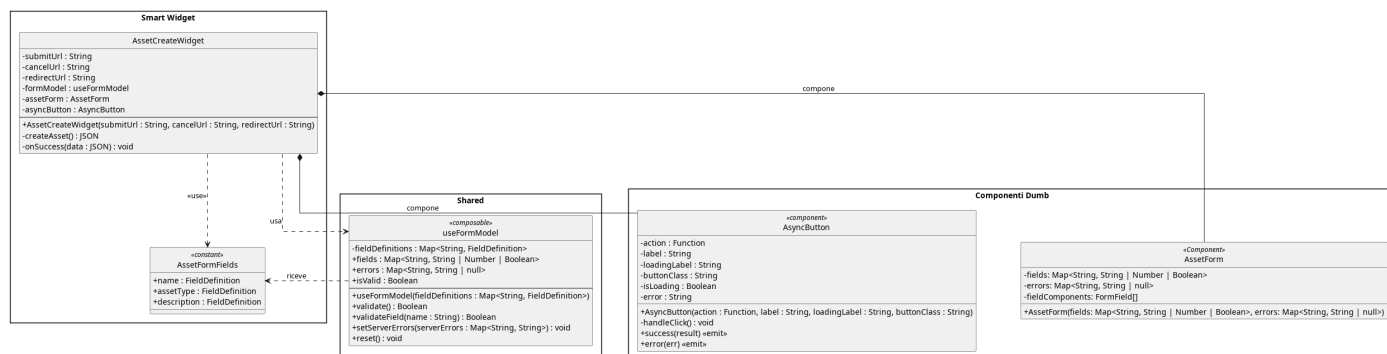


Figura 171: AssetCreate

Il diagramma illustra l'architettura del widget dedicato alla creazione di un nuovo asset_G, che segue il medesimo pattern del widget dispositivo_G adattando le definizioni di campo e il componente di presentazione al dominio degli asset_G.

5.8.3.1.1) AssetCreateWidget

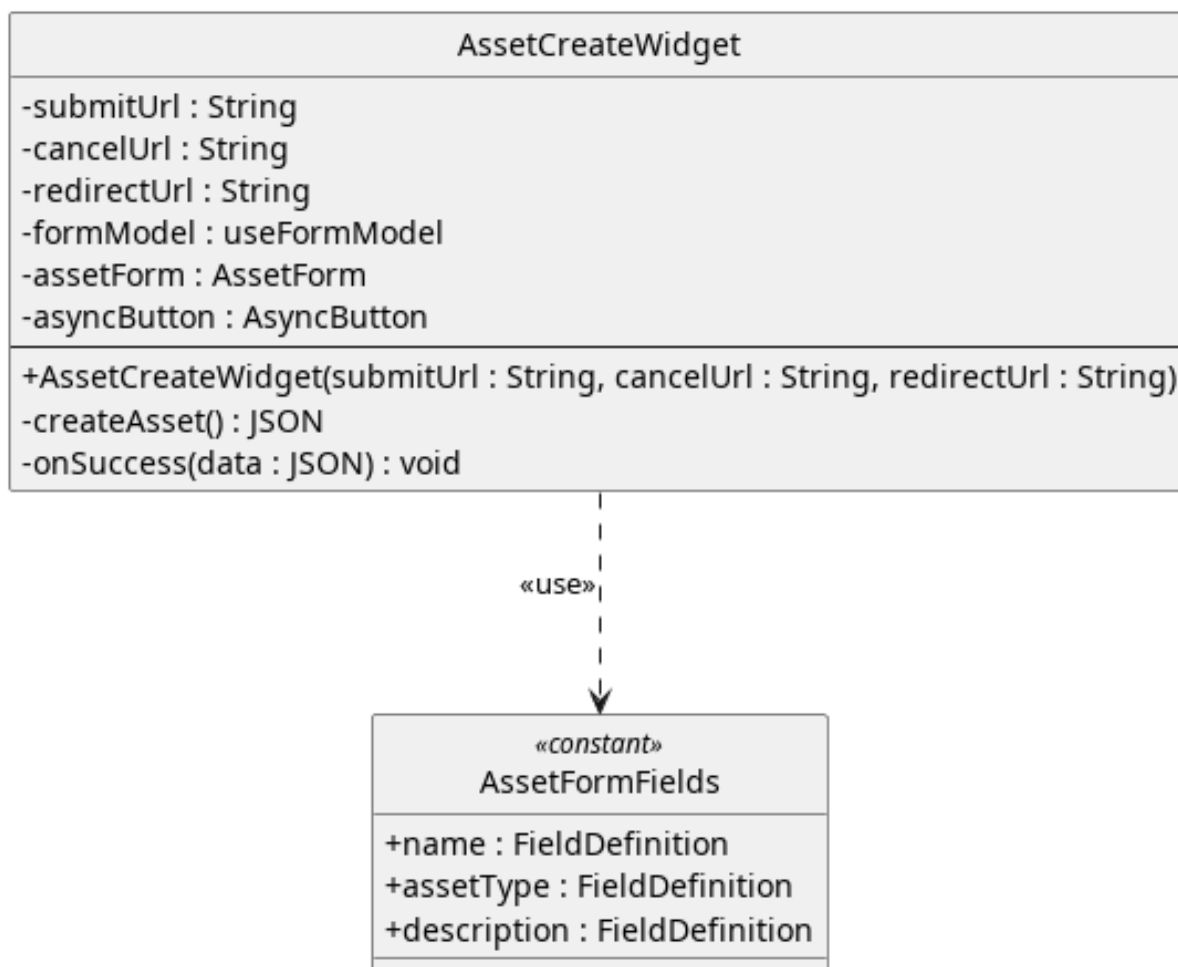


Figura 172: Widget - AssetCreateWidget

Descrizione:

Rappresenta un'isola applicativa responsabile di orchestrare la creazione di un nuovo AssetG. Agisce come controller di facciata: funge da ponte tra il livello di presentazione e il livello di comunicazione col backend. Detiene la logica di business specifica per questa casistica, ma delega il rendering grafico e la gestione reattiva ai componenti e ai composabile sottostanti.

Attributi:

- + submitUrl: String : L'endpoint Flask a cui inviare il payload POST.
- + cancelUrl: String : L'URL di fallback in caso di annullamento dell'operazione.
- + redirectUrl: String : L'URL a cui reindirizzare l'utente in caso di successo.
- - formModel: UseFormModel : Model di riferimento per il componente.
- - assetForm: AssetForm : Componente che mostra graficamente il form degli assetG.
- - confirmButton: AsyncButton : Componente che mostra graficamente il pulsante di conferma.

Metodi:

- + AssetCreateWidget(submitUrl : String, cancelUrl : String, redirectUrl : String) : Costruttore che riceve esternamente gli url a cui corrispondono le azioni.
- - createAsset(): Promise<JSONG> : Metodo asincrono invocato dal bottone di salvataggio. Esegue la validazioneG invocando il composabile, compone il payload JSONG e gestisce la chiamata di rete, catturando e smistando eventuali errori server-side, ritorna un oggetto jsonG contenente la risposta.
- - onSuccess(data: JSONG): void : Callback eseguita al completamento positivo della chiamata.

5.8.3.2) AssetDelete

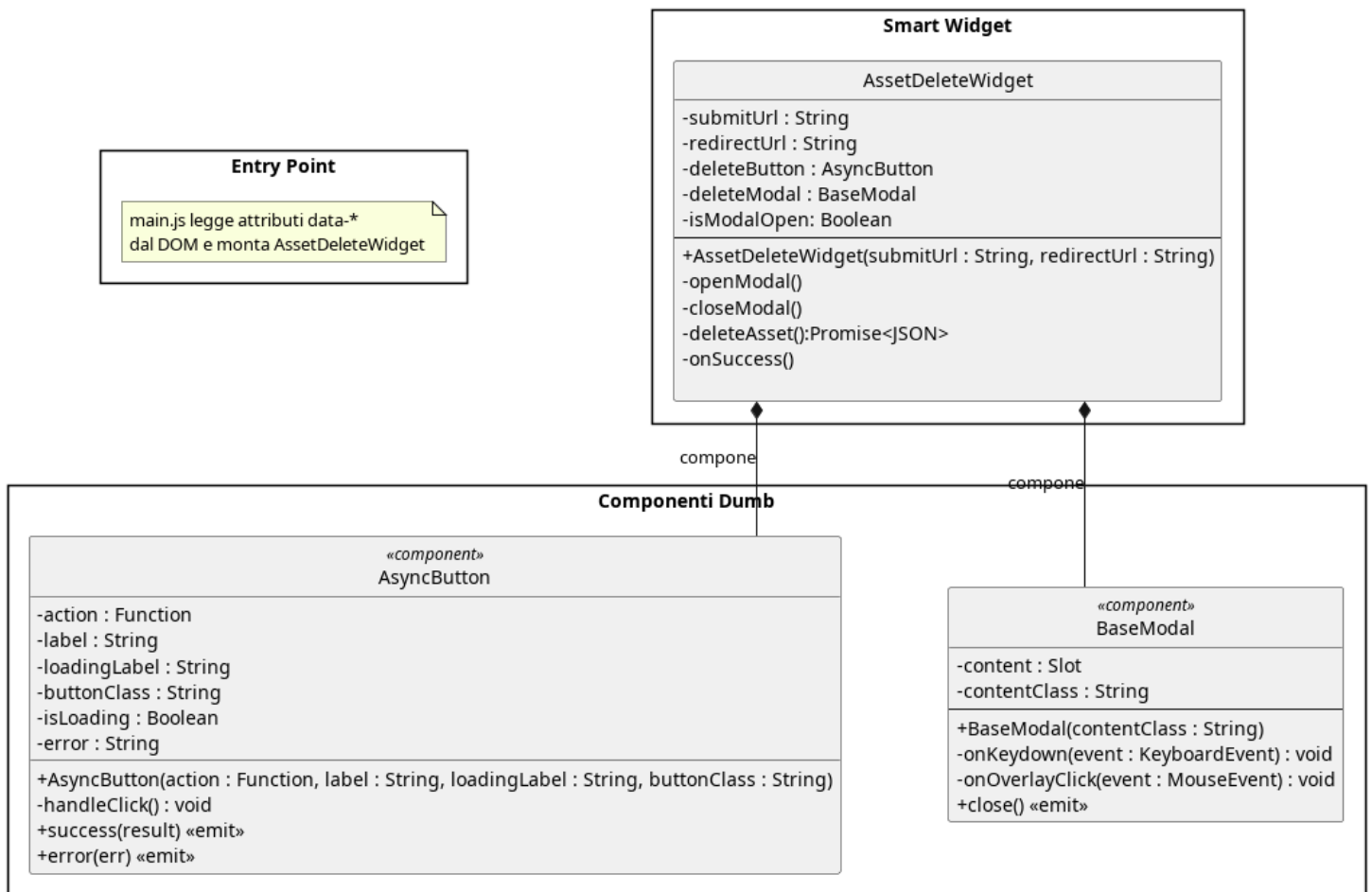


Figura 173: AssetDelete

Il diagramma illustra l'architettura del widget dedicato all'eliminazione di un asset, che interpone un modale di conferma prima di eseguire la chiamata di cancellazione e gestisce il reindirizzamento al completamento.

5.8.3.2.1) AssetDeleteWidget

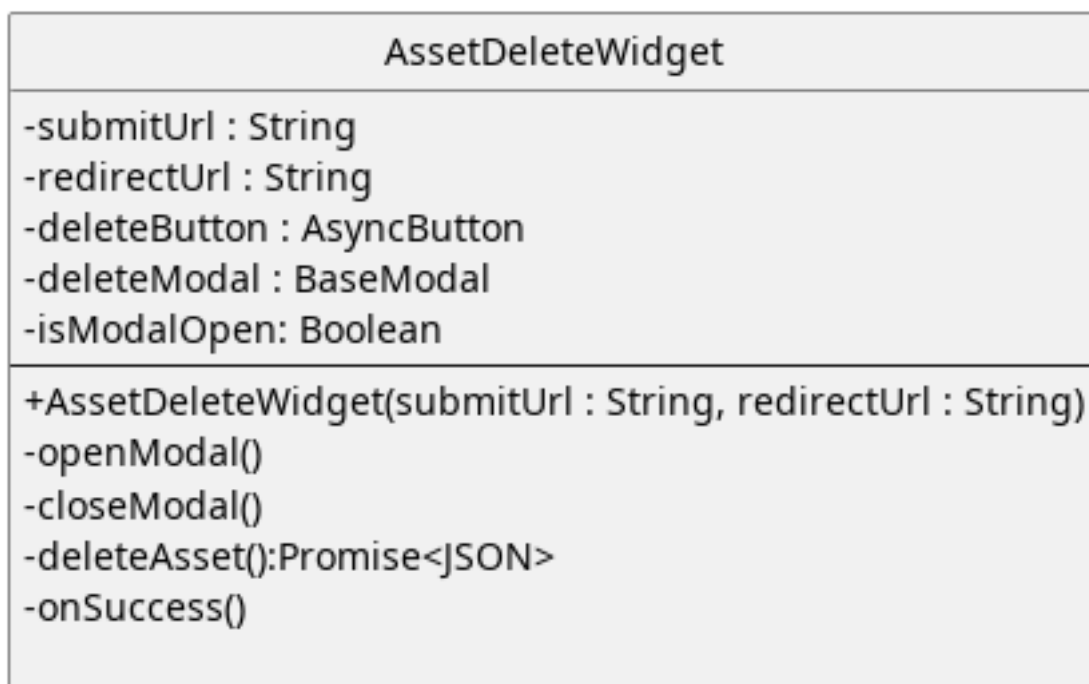


Figura 174: Widget - AssetDeleteWidget

Descrizione:

Rappresenta un'isola applicativa responsabile di chiedere conferma dell'eliminazione ed effettuare la chiamata API di eliminazione dell'asset_G

Attributi:

- + submitUrl: String: Url a cui fare la chiamata API per eliminare un asset_G, passato tramite props
- + redirectUrl: String: Url della pagina a cui reindirizzare dopo l'esecuzione dell'operazione.
- - deleteButton: AsyncButton: Componente reattivo che rappresenta un pulsante di eliminazione
- - deleteModal: BaseModal: Componente reattivo che rappresenta un modale per richiedere la conferma di eliminazione.
- - isModalOpen: Boolean: Indica se il modale deve essere mostrato a schermo o meno

Metodi:

- + AssetDeleteWidget(submitUrl:string,redirectUrl:String):Costruttore che riceve dall'esterno gli url da utilizzare.
- - openModal(): Mostra a schermo il modale di conferma.
- - closeModal(): Nasconde il modale di conferma.

- + DeleteAsset(): Promise<JSON>: Esegue la chiamata API di eliminazione dell'asset_G, ritorna un oggetto che rappresenta l'esito dell'operazione
- - onSuccess(): Callback eseguita al completamento positivo della chiamata.

5.8.3.3) AssetEditWidget

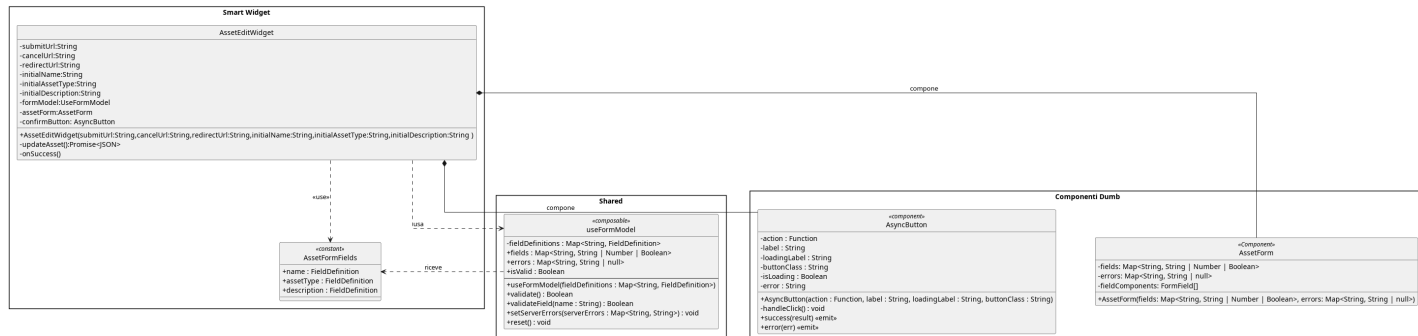


Figura 175: AssetEdit

Il diagramma illustra l'architettura del widget dedicato alla modifica di un asset_G esistente, che precarica i valori iniziali nel form condiviso e orchestra la chiamata di aggiornamento verso il backend.

5.8.3.3.1) AssetEditWidget

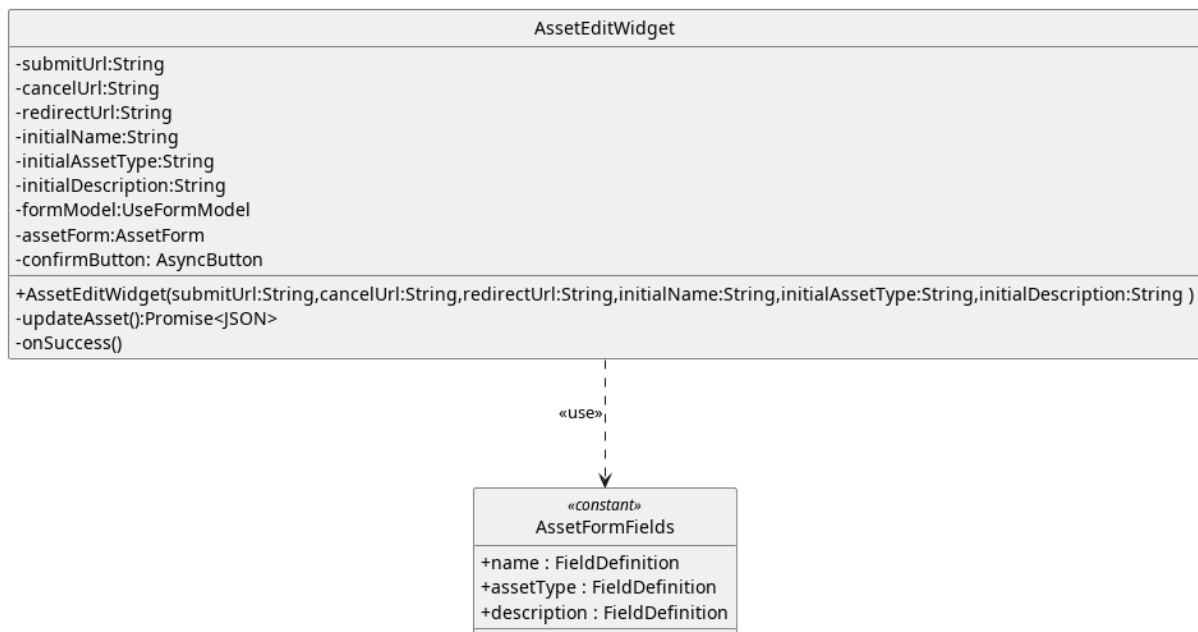


Figura 176: Widget - AssetEditWidget

Descrizione:

Rappresenta l'isola applicativa che l'utente usa per modificare i dati di un asset_G, rappresenta il viewModel del MVVM

Attributi:

- - submitUrl: String: Url a cui effettuare la chiamata API di modifica.

- - `cancelUrl:String`: Url a cui reindirizzare l'utente in caso di annullamento.
- - `redirectUrl:String`: Url a cui reindirizzare l'utente dopo la modifica.
- - `initialName`: Valore a cui inizializzare i campo del form relativo al nome dell'asset_G.
- - `initialAssetType`: Valore a cui inizializzare i campo del form relativo al tipo dell'asset_G.
- - `initialDescription`: Valore a cui inizializzare i campo del form relativo alla descrizione dell'asset_G.
- - `formModel`: Model di riferimento che tiene traccia dello stato interno.
- - `assetForm`: `AssetForm`: Componente che visualizza il form per l'inserimento dei dati dell'asset_G
- - `confirmButton`: `AsyncButton`: Pulsante per confermare il salvataggio delle modifiche.

Metodi:

- + `AssetEditWidget(- submitUrl:String, cancelUrl:String, redirectUrl:String, initialName:String, initialAssetType:String, initialDescription:String)`: Costruttore che riceve dall'esterno i valori a cui inizializzare il form e gli endpoint a cui effettuare le chiamate e reindirizzare l'utente.
- - `updateAsset()`: `Promise<JSONG>`: Effettua la chiamata API per la modifica dell'asset_G
- - `onSuccess()`: Callback eseguita al completamento positivo della chiamata.

5.8.3.4) DeviceCreate

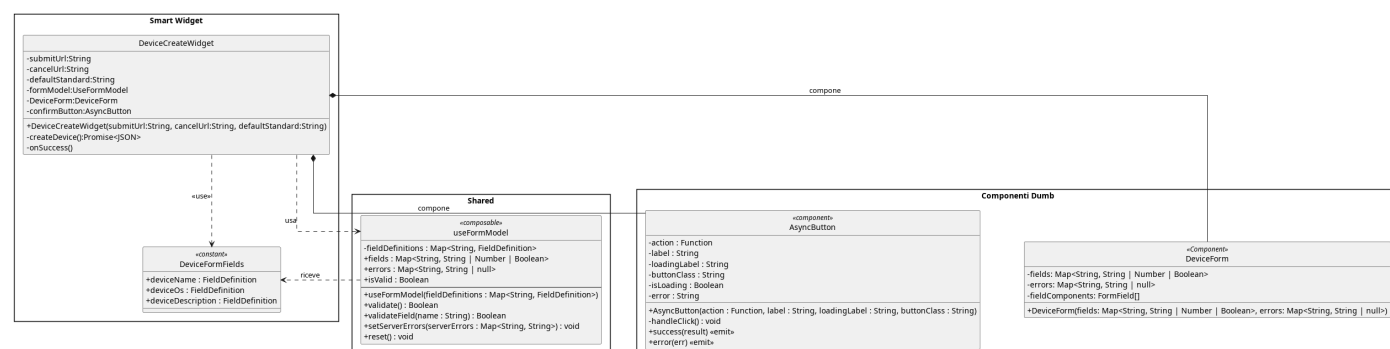


Figura 177: DeviceCreate

Il diagramma illustra l'architettura del widget dedicato alla creazione di un nuovo dispositivo_G, mostrando come il componente smart orchestri il composabile di

gestione form, le definizioni di campo e i componenti dumb di presentazione e conferma.

5.8.3.4.1) DeviceCreateWidget

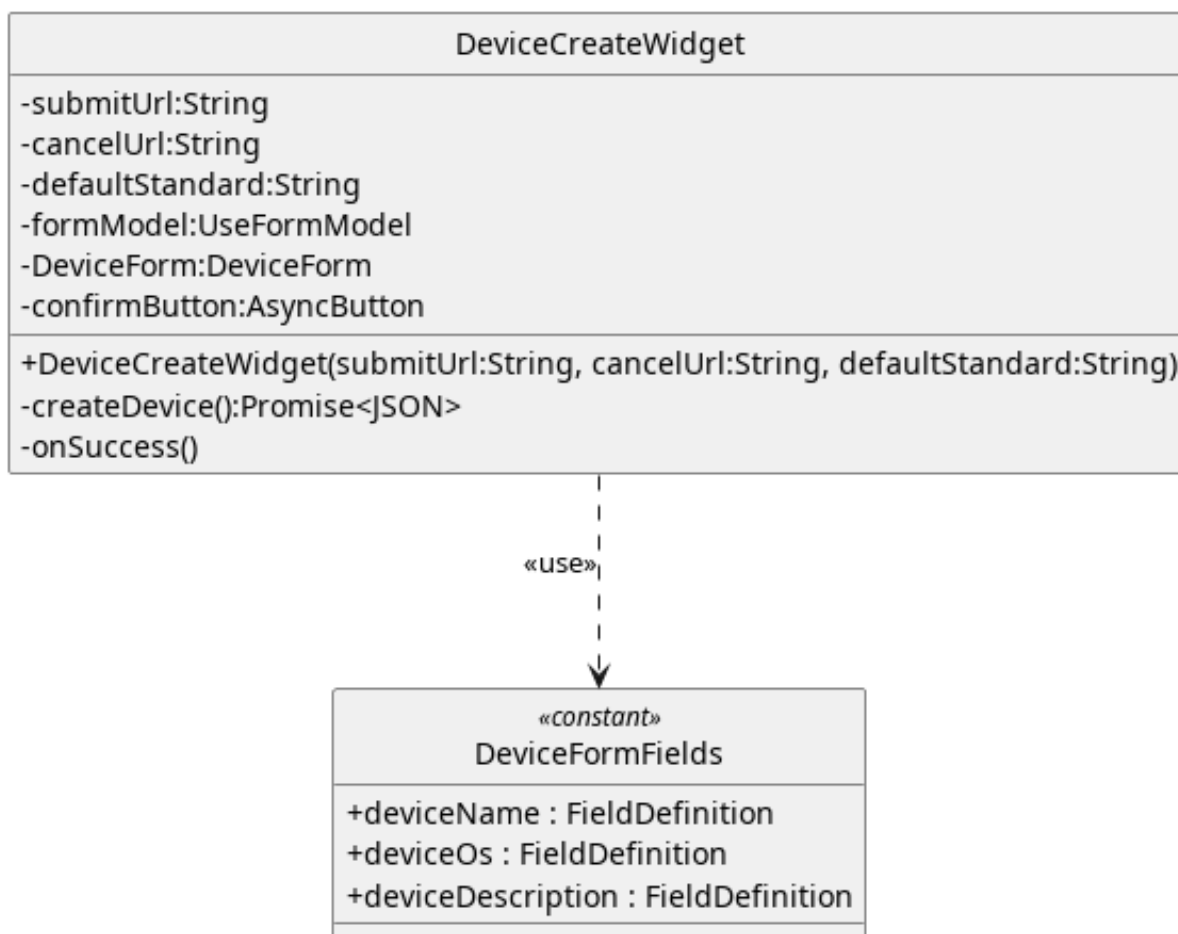


Figura 178: Widget - DeviceCreateWidget

Descrizione:

Rappresenta il widget per la creazione di un nuovo dispositivo_G.

Attributi:

- - submitUrl: String: Url a cui effettuare la chiamata API.
- - cancelUrl: String: Url a cui reindirizzare l'utente in caso di annullamento dell'operazione.
- - defaultStandard: String: Id dello standard di da usare di default durante la valutazione_G.
- - formModel: useFormModel: Model di riferimento che tiene traccia dello stato interno.
- - deviceForm: DeviceForm: Componente che permette all'utente di visualizzare il form per la creazione del dispositivo_G.

- `confirmButton`: `AsyncButton`: Bottone per la conferma della creazione del dispositivo_G.

Metodi:

- + `DeviceCreateWidget( submitUrl: String, cancelUrl: String, defaultStandard: String)`: Costruttore che riceve gli url per gli endpoint e a cui fare i redirect dall'esterno.
- + `createDevice()`: `Promise<JSONG>`: Effettua la chiamata API per la creazione di un nuovo dispositivo_G.
- `onSuccess()`: Callback eseguita al completamento positivo della chiamata.

5.8.3.5) DeviceEdit

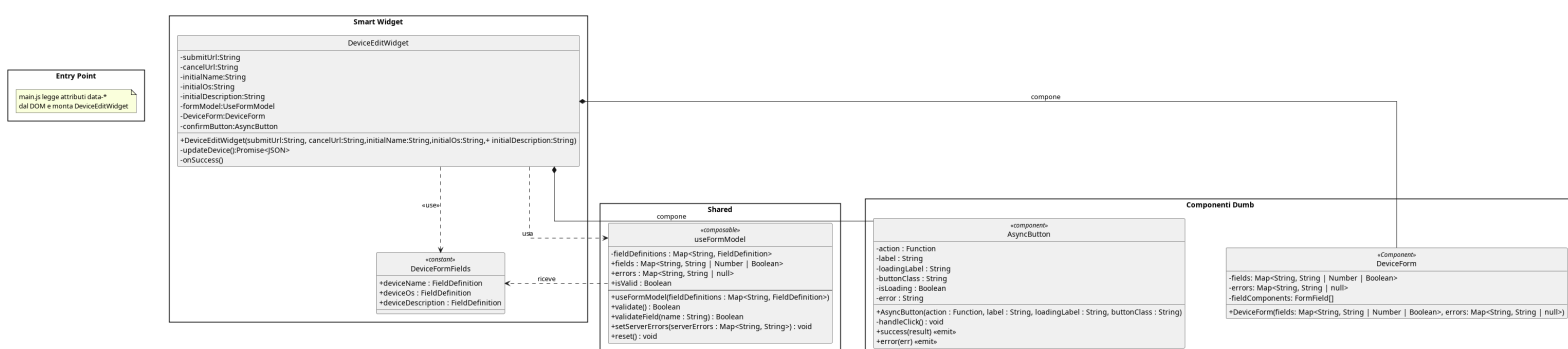


Figura 179: DeviceEdit

Il diagramma illustra l'architettura del widget dedicato alla modifica di un dispositivo_G esistente, che inizializza il form con i valori correnti ricevuti dal server e delega la validazione_G e il rendering agli stessi layer condivisi del widget di creazione.

5.8.3.5.1) DeviceEditWidget

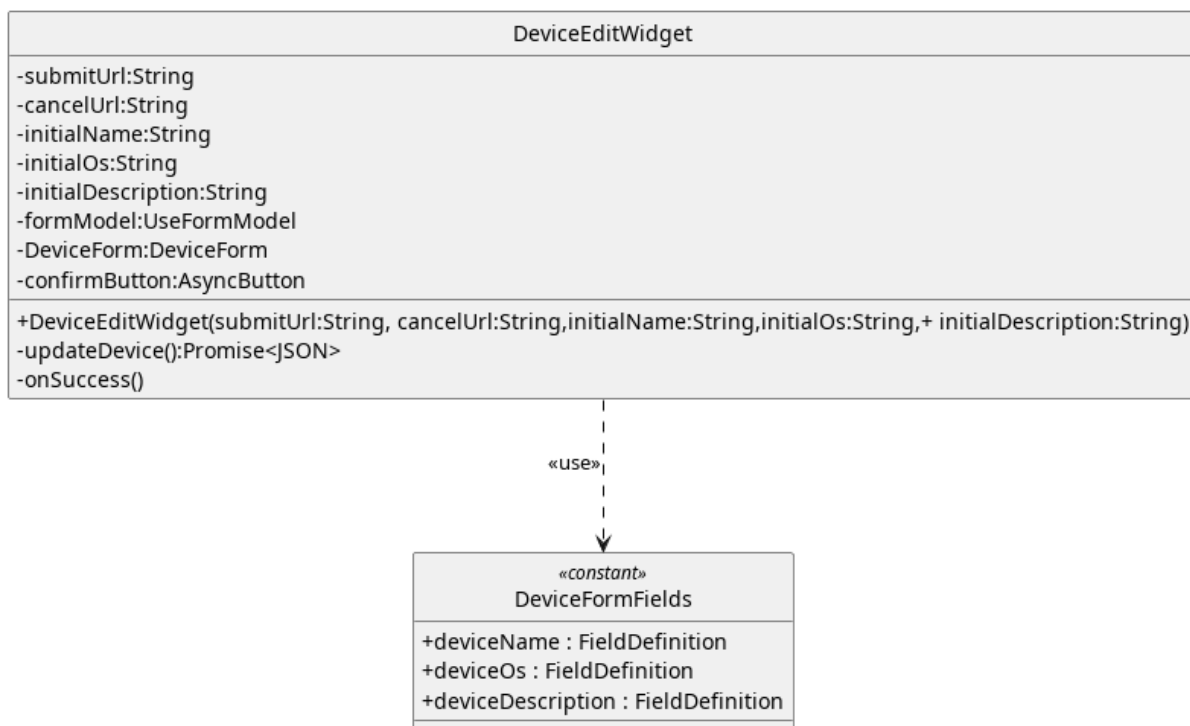


Figura 180: Widget - DeviceEditWidget

Descrizione:

Rappresenta Il widget per la modifica dei dati del dispositivo_G.

Attributi:

- - submitUrl: String: Url a cui effettuare la chiamata API.
- - cancelUrl: Url a cui reindirizzare l'utente in caso di annullamento dell'operazione.
- - initialName: String: Valore a cui inizializzare il nome del dispositivo_G.
- - initialOs: String: Valore a cui inizializzare il nome del sistema operativo del dispositivo_G.
- - initialDescription: String: Valore a cui inizializzare la descrizione del dispositivo_G.
- - formModel: useFormModel: Model di riferimento che tiene traccia dello stato interno.
- - deviceForm: DeviceForm: Componente che permette all'utente di visualizzare il form per la creazione del dispositivo_G.
- - confirmButton: AsyncButton: Bottone per la conferma della modifica del dispositivo_G.

Metodi:

- +

DeviceEditWidget(submitUrl:String, cancelUrl:String, initialName:String, initialOs:St

Costruttore che riceve gli url come dipendenze dall'esterno.

- - updateDevice(): Promise<JSONG>: Effettua la chiamata API.
- - onSuccess(): Callback eseguita al completamento positivo della chiamata.

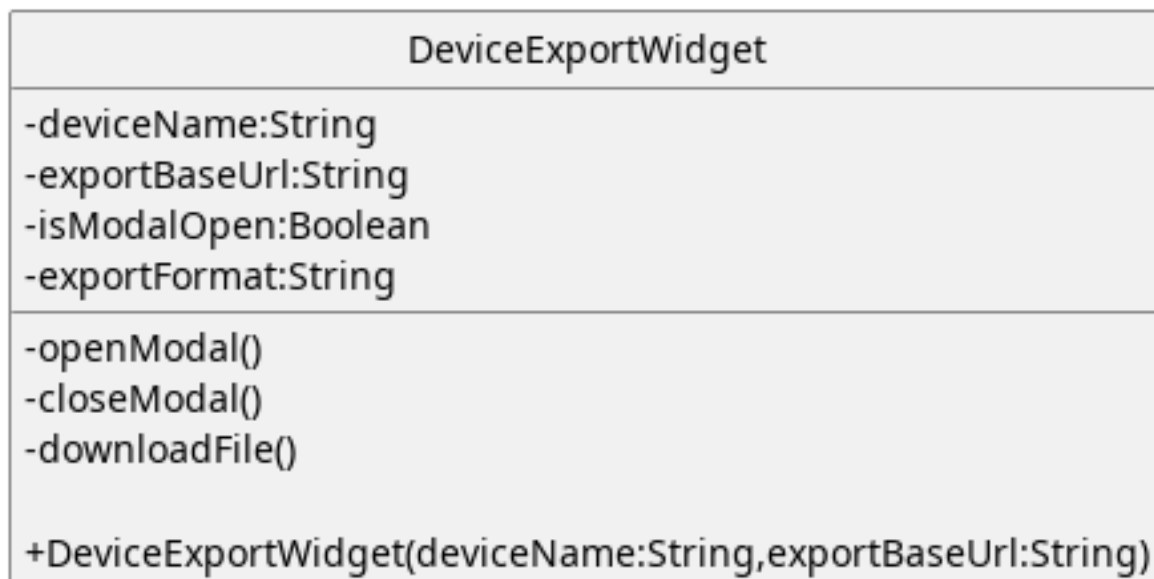
5.8.3.6) DeviceExportWidget

Figura 181: Widget - DeviceExportWidget

Descrizione:

Rappresenta il widget che permette di esportare il dispositivo_G come file e di selezionare l'estensione_G desiderata.

Attributi:

- - deviceName: String: Nome del dispositivo_G da esportare.
- - exportBaseUrl: String: Url per la chiamata API per ricevere il file rappresentante le informazioni del dispositivo_G.
- - isModalOpen: Boolean: Serve a gestire la visibilità del modale a schermo.
- - exportFormat: String: Tiene traccia del formato per l'esportazione_G selezionato dall'utente.

Metodi:

- - openModal(): Apre il modale per l'esportazione_G del dispositivo_G.
- - closeModal(): Chiude il modale per l'esportazione_G.

- `downloadFile(): Promise<JSON_G>`: Esegue la chiamata API per scaricare il file.

5.8.3.7) DeviceImport

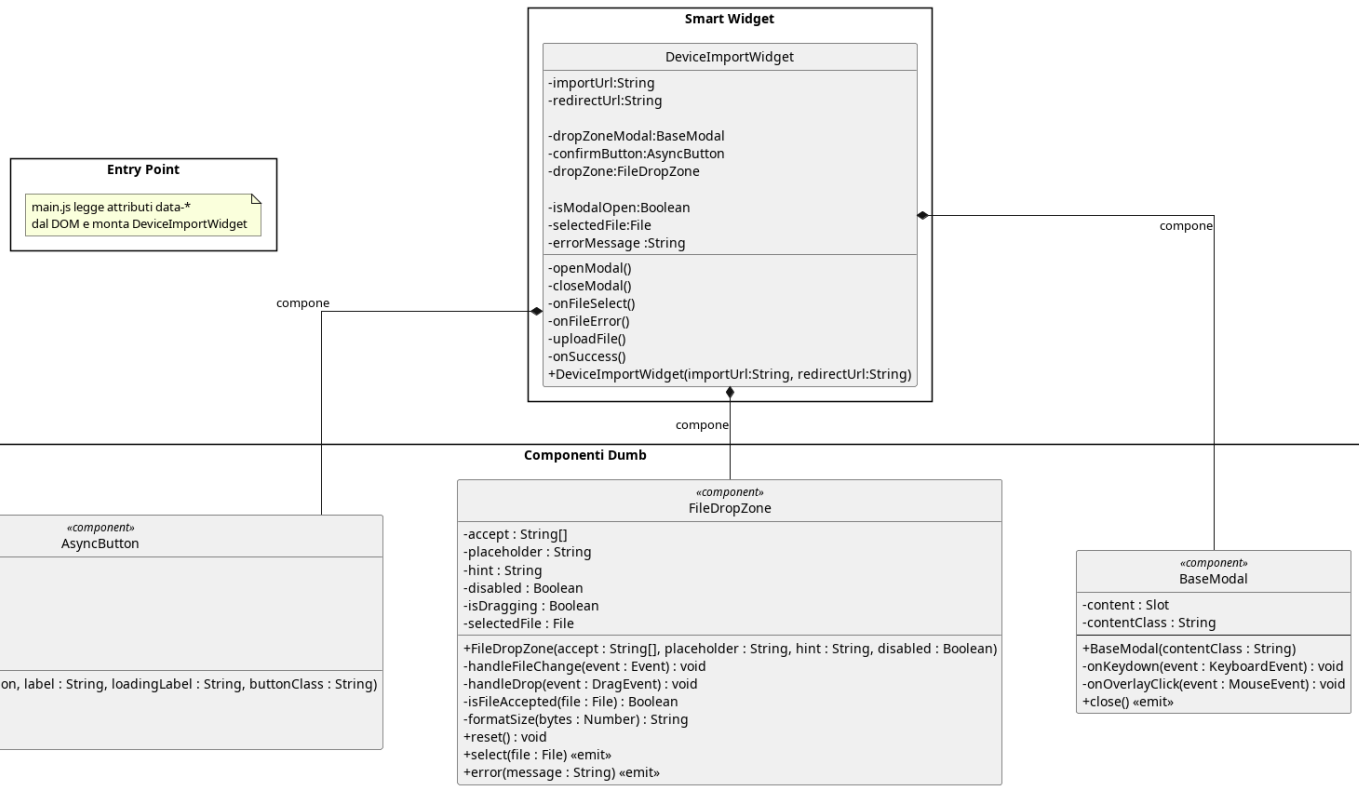


Figura 182: DeviceImport

Il diagramma illustra l'architettura del widget dedicato all'importazione di un dispositivo da file, che compone un modale contenente un'area di drag-and-drop per la selezione del file e un pulsante asincrono per l'invio al backend.

5.8.3.7.1) DeviceImportWidget

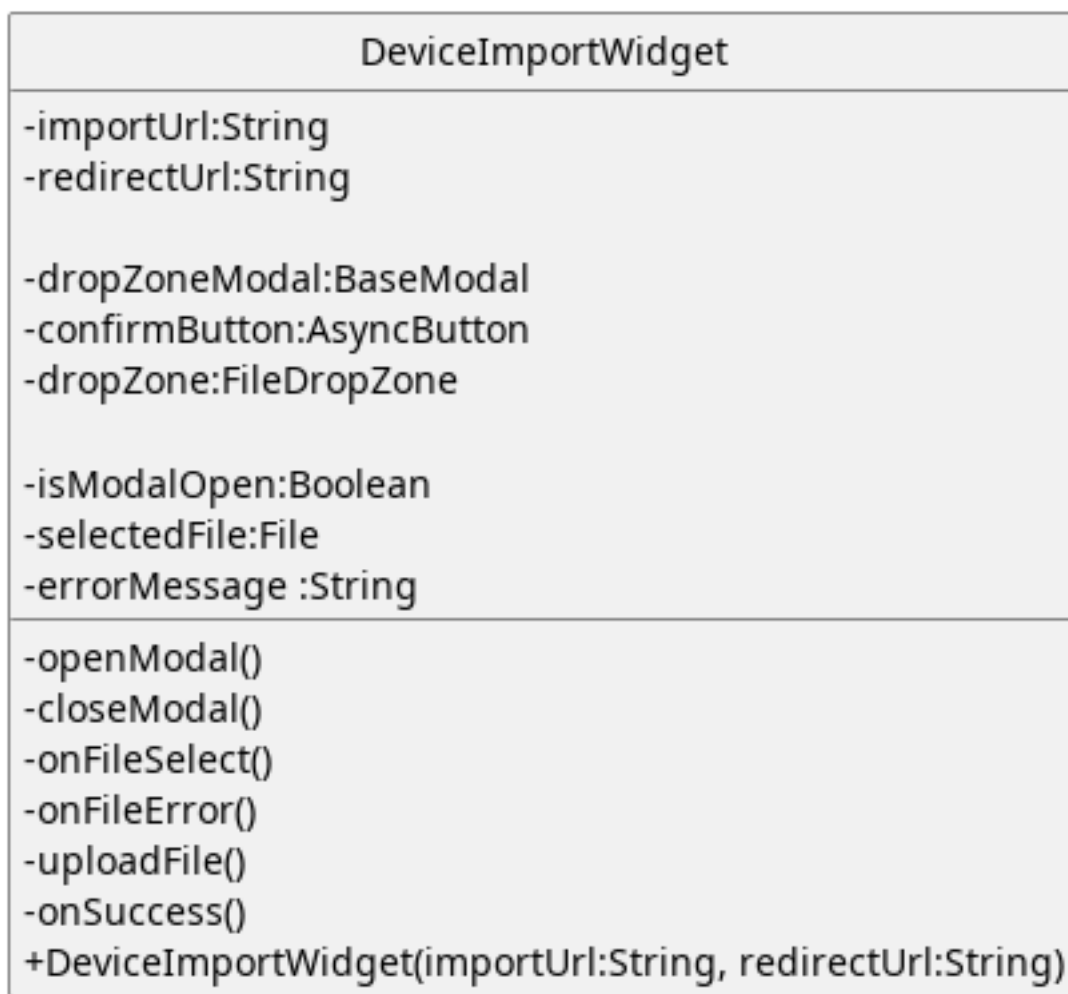


Figura 183: Widget - DeviceImportWidget

Descrizione:

Rappresenta un widget usato durante l'importazioni delle informazioni legate ad un dispositivo_G tramite file.

Attributi:

- - importUrl: String: Url da usare per la chiamata API relativa all'importazione_G del file.
- - redirectUrl: String: Url della pagina verso cui reindirizzare l'utente dopo l'importazione_G.
- - dropZoneModal: BaseModal: Modal per l'import del file
- - dropZone: FileDropZone: File drop zone per il caricamento di file.
- - confirmButton: AsyncButton: Pulsante per la conferma
- - isModalOpen: Boolean: Viene usato per gestire la visibilità del modale

- - `selectedFile: File`: Tiene traccia del file caricato dall'utente.
- - `errorMessage: String`: Messaggio di errore da visualizzare a schermo.

Metodi:

- + `DeviceImportWidget(importUrl: String, redirectUrl: String)`: Costruttore che riceve gli url per chiamate e redirect dall'esterno.
- - `openModal()`: Mostra a schermo il modale per l'importazione_G.
- - `closeModal()`: Nasconde il modale per l'importazione_G.
- - `onFileSelect()`: Funzione eseguita al caricamento del file per aggiornare lo stato del widget.
- - `onFileError()`: Funzione eseguita in caso di errori nel caricamento del file.
- - `uploadFile()`: Funzione che esegue la chiamata API di importazione_G.
- - `onSuccess()`: Callback eseguita al completamento positivo della chiamata.

5.8.3.8) SessionClose

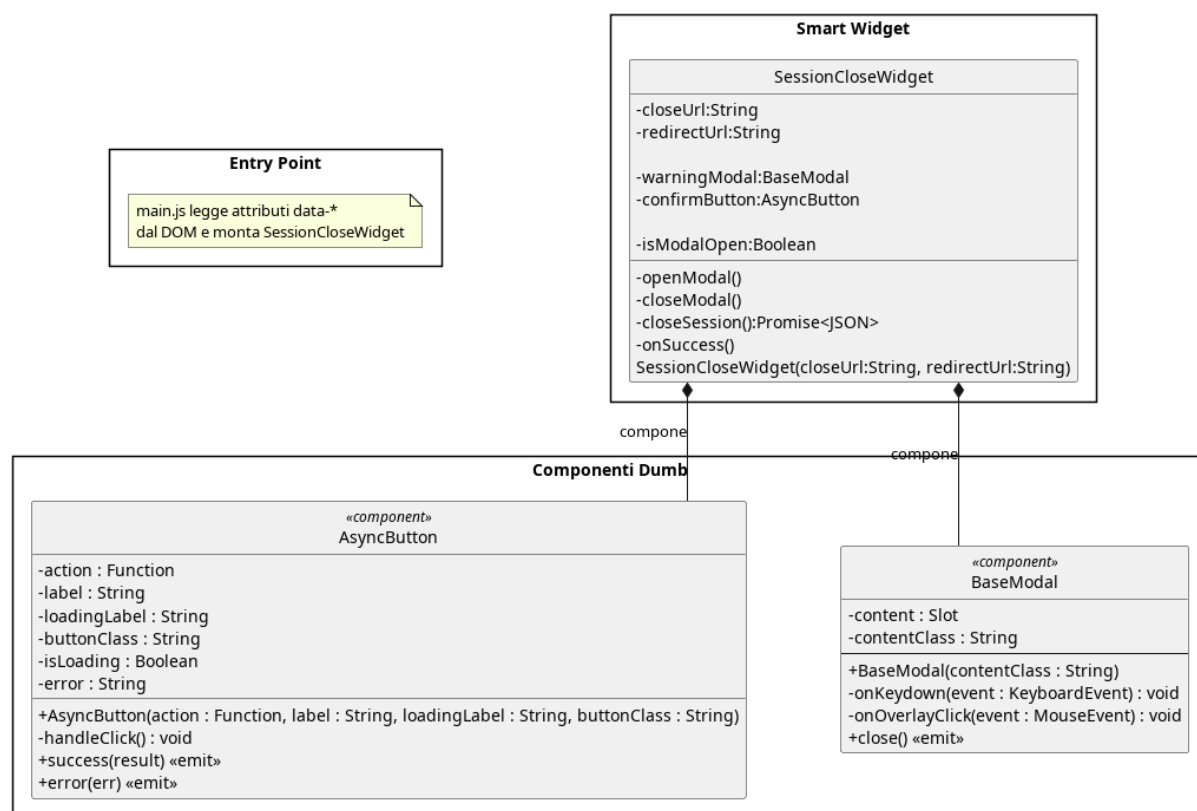


Figura 184: SessionClose

Il diagramma illustra l'architettura del widget dedicato alla chiusura di una sessione di valutazione_G, che richiede conferma esplicita tramite modale prima di eseguire la chiamata di terminazione e reindirizzare l'utente.

5.8.3.8.1) SessionCloseWidget

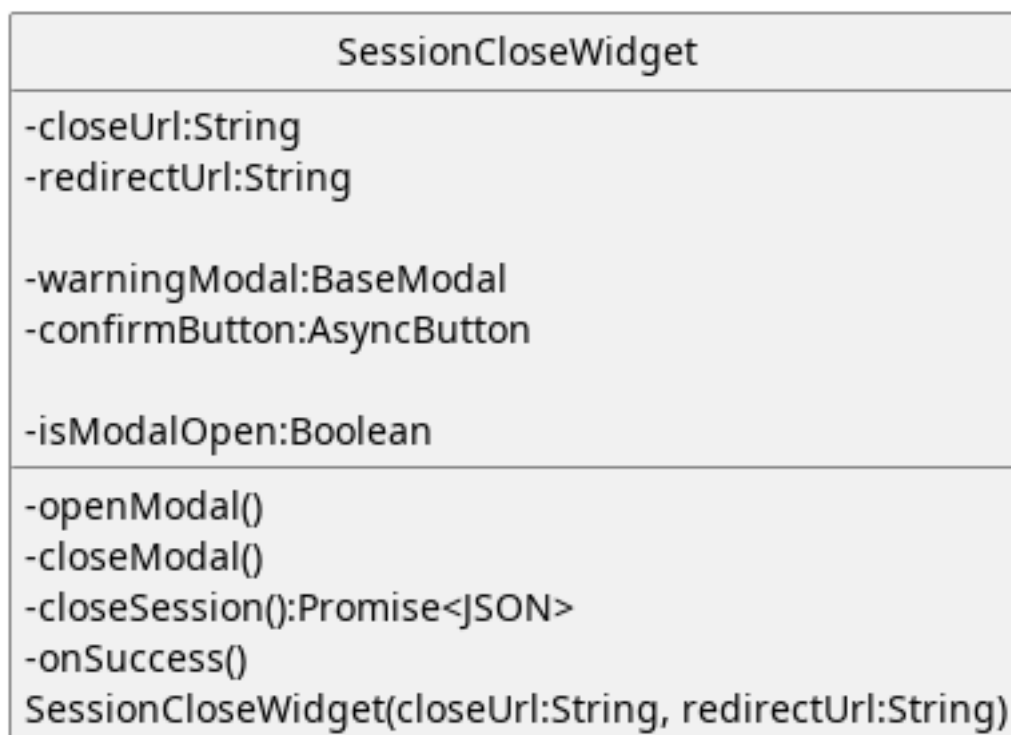


Figura 185: Widget - SessionCloseWidget

Descrizione:

Rappresenta il widget che permette all'utente di chiudere una sessione di valutazione_G di un dispositivo_G.

Attributi:

- - closeUrl: String: Url a cui effettuare la chiamata API per la chiusura della sessione di valutazione_G.
- - redirectUrl: String: Url a cui reindirizzare l'utente dopo la chiusura della sessione.
- - warningModal: BaseModal: Modale mostrato all'utente per chiedere conferma della chiusura della sessione.
- - confirmButton: AsyncButton: Pulsante per chiudere la sessione.
- - isModalOpen: Boolean: Gestisce la visibilità del modale.

Metodi:

- + SessionCloseWidget(closeUrl: String, redirectUrl: String): Costruttore che riceve gli url per le chiamate API e per reindirizzare l'utente.
- - openModal(): Apre il modale di conferma.

- - `closeModal()`: Chiude il modale di conferma.
- - `closeSession()`: `Promise<JSON>`: Effettua la chiamata per chiudere la sessione.
- - `onSuccess()`: Callback eseguita al completamento positivo della chiamata.

5.8.3.9) SessionCommitAndClose

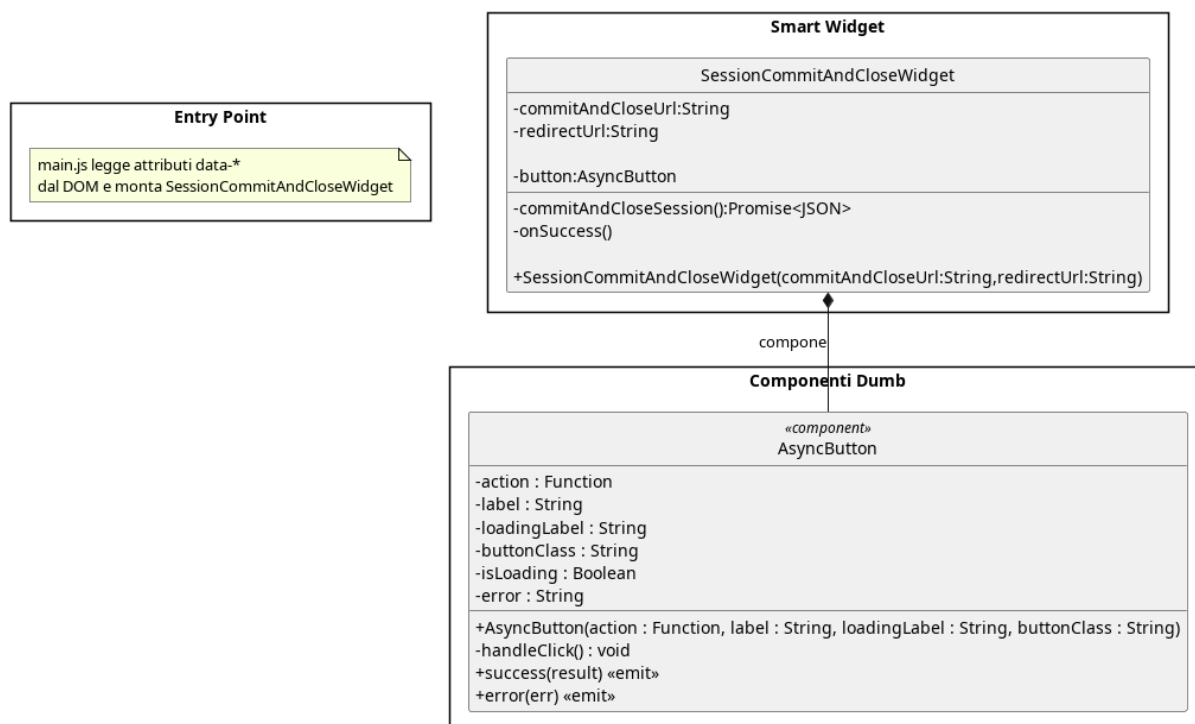


Figura 186: SessionCommitAndClose

Il diagramma illustra l'architettura del widget dedicato al salvataggio e alla chiusura contestuale della sessione, che consolida le due operazioni in un'unica azione delegata al pulsante asincrono.

5.8.3.9.1) SessionCommitAndCloseWidget

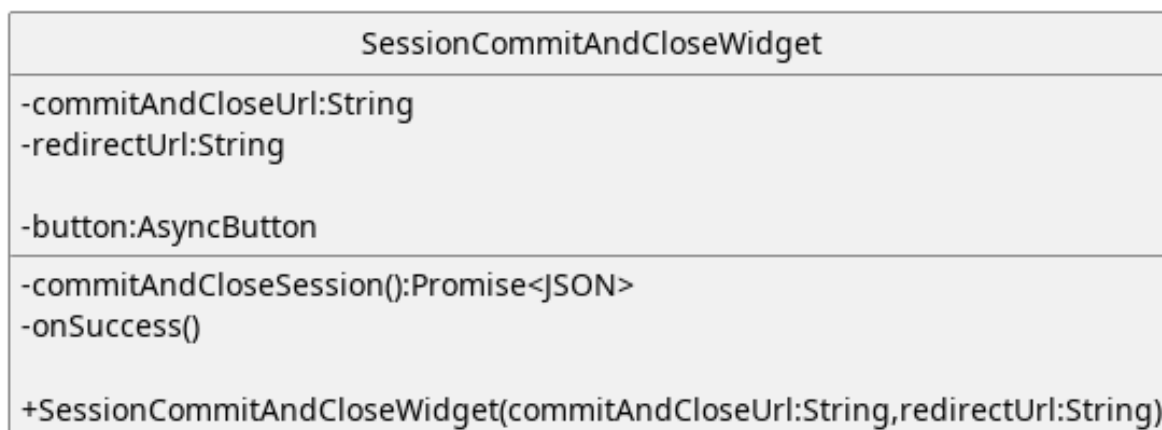


Figura 187: Widget - SessionCommitAndCloseWidget

Descrizione:

Rappresenta il widget che permette di salvare e chiudere la sessione.

Attributi:

- - commitAndCloseUrl: String: Url a cui fare la chiamata API per salvare e chiudere la sessione.
- - redirectUrl: String: Url a cui reindirizzare l'utente dopo il salvataggio e la chiusura della sessione.
- - confirmButton: AsyncButton: Pulsante con cui l'utente salva e chiude la sessione.

Metodi:

- + SessionCommitAndCloseWidget(commitAndCloseUrl: String, redirectUrl: String): Costruttore che riceve gli url per le chiamate API e i redirect dall'esterno.
- - commitAndCloseSession(): Promise<JSON>: Effettua la chiamata API per il salvataggio e chiusura della sessione.
- - onSuccess(): Callback eseguita al completamento positivo della chiamata.

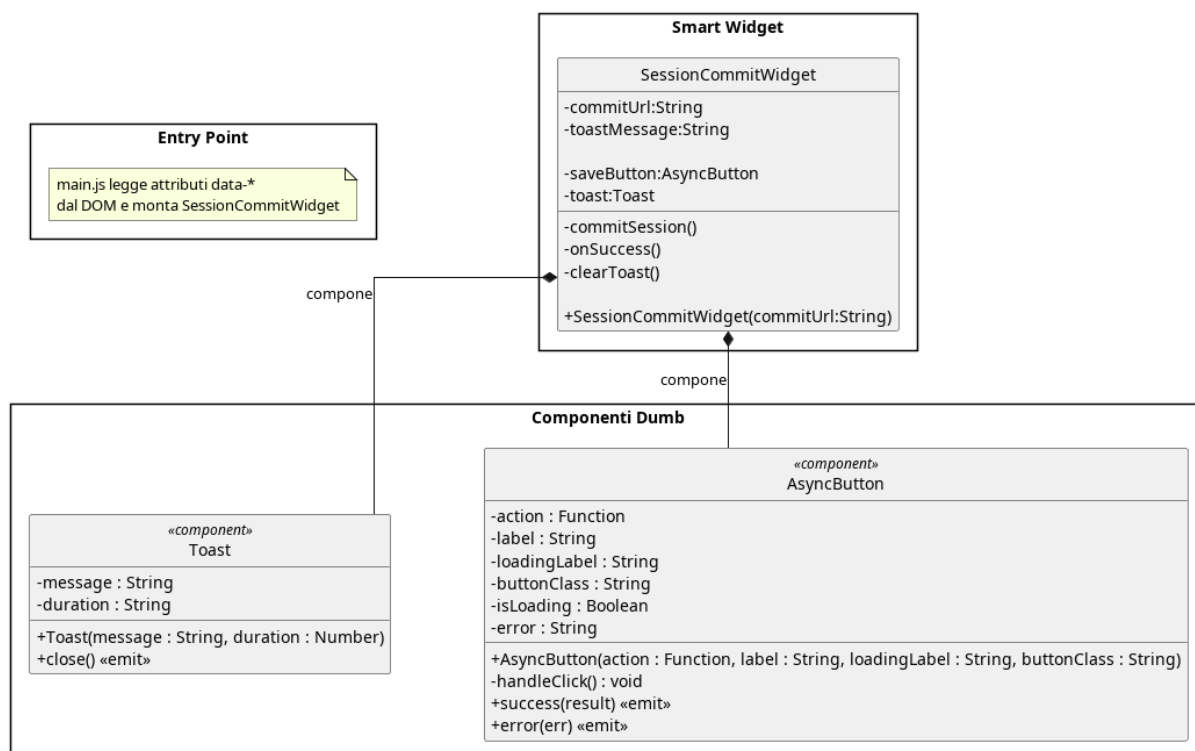
5.8.3.10) SessionCommit

Figura 188: SessionCommit

Il diagramma illustra l'architettura del widget dedicato al salvataggio della sessione attiva, che al completamento positivo della chiamata mostra un messaggio temporaneo tramite il componente Toast.

5.8.3.10.1) SessionCommitWidget

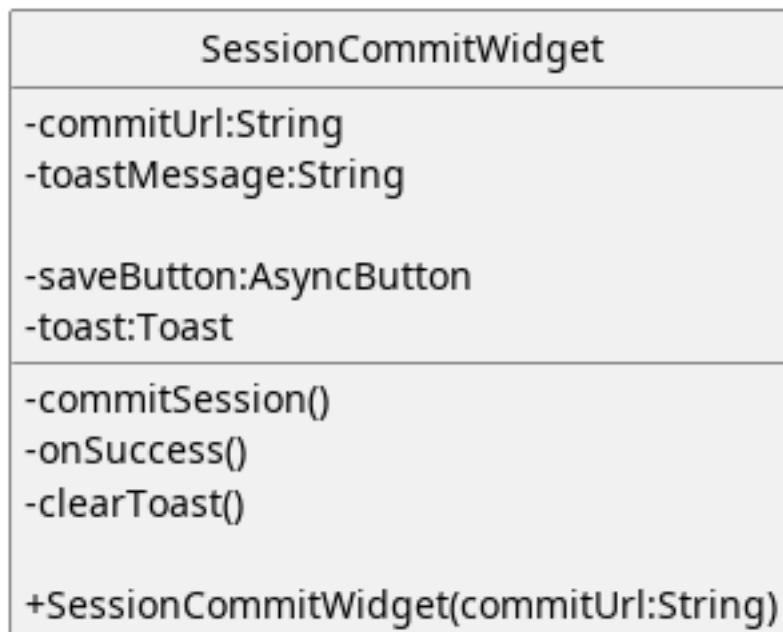


Figura 189: Widget - SessionCommitWidget

Descrizione:

Rappresenta il widget che permette all'utente di salvare la sessione

Attributi:

- - `commitUrl:String`:Url a cui fare la chiamata API per il salvataggio della sessione di modifica.
- - `toastMessage:String`:Messaggio da visualizzare tramite toast.
- - `saveButton:AsyncButton`:Pulsante per salvare la sessione
- - `toast:Toast`:Componente visivo che mostra temporaneamente un messaggio

Metodi:

- + `SessionCommitWidget(commitUrl:String)`:Costruttore che riceve gli url a cui effettuare le chiamate API dall'esterno
- - `onSuccess()`:Callback eseguita al completamento positivo della chiamata.
- - `clearToast()`:Funzione che nasconde il messaggio temporaneo.
- - `commitSession():Promise<JSON>`:Funzione che esegue la chiamata API per il salvataggio della sessione.

5.8.3.11) SessionOpen

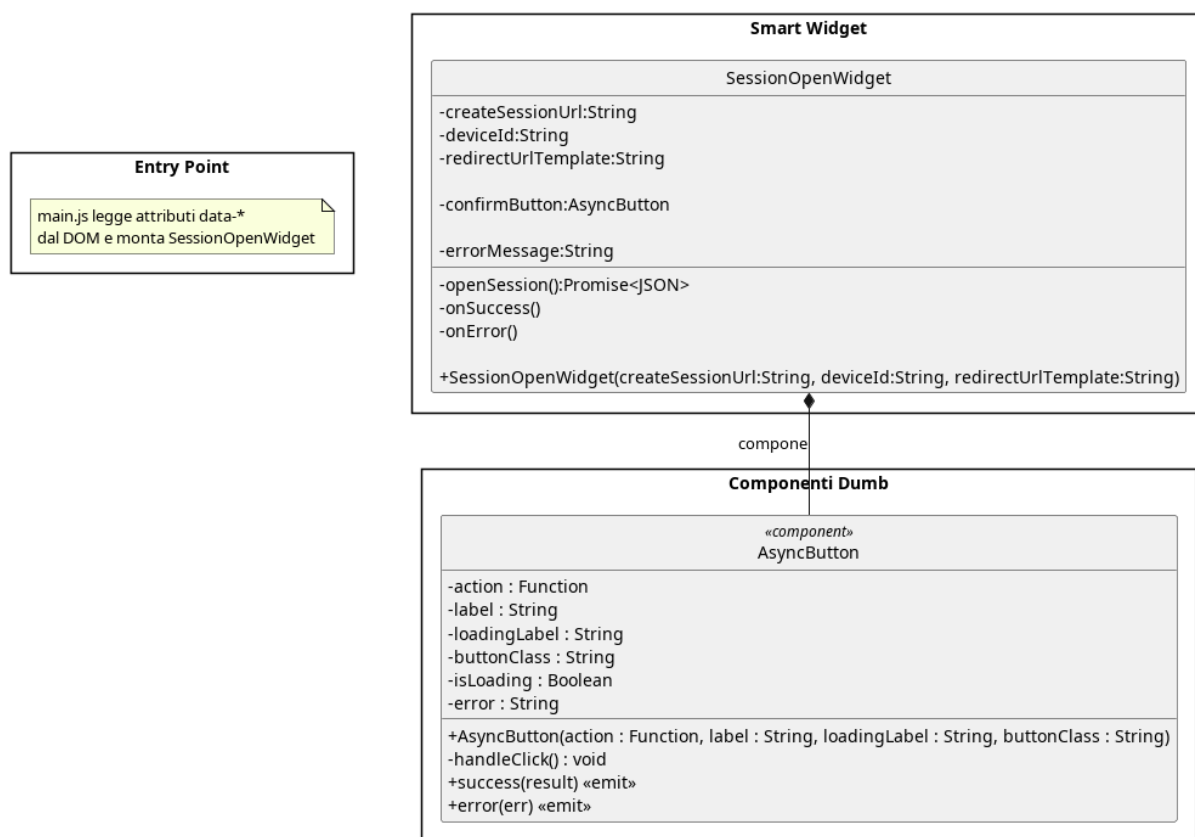


Figura 190: SessionOpen

Il diagramma illustra l'architettura del widget dedicato all'apertura di una nuova sessione di valutazione_G, che delega l'esecuzione della chiamata al pulsante asincrono e gestisce la visualizzazione di eventuali errori di apertura.

5.8.3.11.1) SessionOpenWidget



Figura 191: Widget - SessionOpenWidget

Descrizione:

Rappresenta il widget per l'apertura di una nuova sessione di valutazione_G.

Attributi:

- - `createSessionUrl`: String: Url a cui effettuare la chiamata API per aprire una nuova sessione.
- - `redirectUrlTemplate`: String: Url a cui reindirizzare l'utente dopo l'apertura della sessione.
- - `deviceId`: String: Id del dispositivo_G a cui associare la sessione di valutazione_G.
- - `confirmButton`: AsyncButton: Pulsante per aprire una nuova sessione.
- - `errorMessage`: String: Messaggio di errore da far visualizzare all'utente.

Metodi:

- + `SessionOpenWidget(createSessionUrl:String, deviceId:String, redirectUrl:String)`: Costruttore che riceve gli url per chiamate API e redirect dall'esterno.
- - `openSession()`: Promise<JSON_G>: Funzione per eseguire la chiamata API per aprire una nuova sessione.
- - `onError()`: Funzione eseguita in caso di errori durante l'apertura della sessione.
- - `onSuccess()`: Callback eseguita al completamento positivo della chiamata.

5.9) Architettura Frontend: Modulo Decision Tree_G

Per via della differenza di complessità tra i widget semplici e l'albero di decisione interattivo, abbiamo ritenuto opportuno dedicare a quest'ultimo una sezione apposita.

Il frontend per la valutazione_G dei requisiti è progettato per essere reattivo e disaccoppiato. Per evitare di sovraccaricare il server a ogni interazione e permettere un'esperienza più fluida, una porzione della logica di dominio viene eseguita direttamente lato client, tale logica si limita unicamente alla navigazione e alla presentazione grafica dell'albero di decisione. Lo riteniamo accettabile perché il sistema backend fornisce dati grezzi e il sistema frontend li organizza in un'interfaccia_G comprensibile all'utente.

L'architettura si articola su quattro pilastri:

1. **Core di Valutazione_G (EvaluationEngine)**: Il widget del sistema frontend calcola il percorso attivo dell'albero in base alle risposte dell'utente, offrendo un feedback visivo.
2. **Gestione dello Stato (DecisionTreeStore)**: Basato su Pinia, funge da «Single Source of Truth» o Model del MVVM.

Contiene le logiche di business, salva i dati ed effettua le chiamate API, tramite l'API client.

3. **Interfaccia_G Utente**: Suddivisa in due macro-aree cooperanti. Il **Tree Canvas** disegna la topologia dell'albero delegando i calcoli geometrici al D3LayoutEngine, mentre la **Tree Sidebar** funge da pannello interattivo per leggere le domande e inserire le risposte, realizza la parte di View del MVVM.
4. **Integrazione Backend (EvaluationApiClient)**: Incapsula la comunicazione con il server.

Nei paragrafi successivi verranno analizzati nel dettaglio i diagrammi delle classi dei singoli sottosistemi.

5.9.1) Logica di valutazione_G

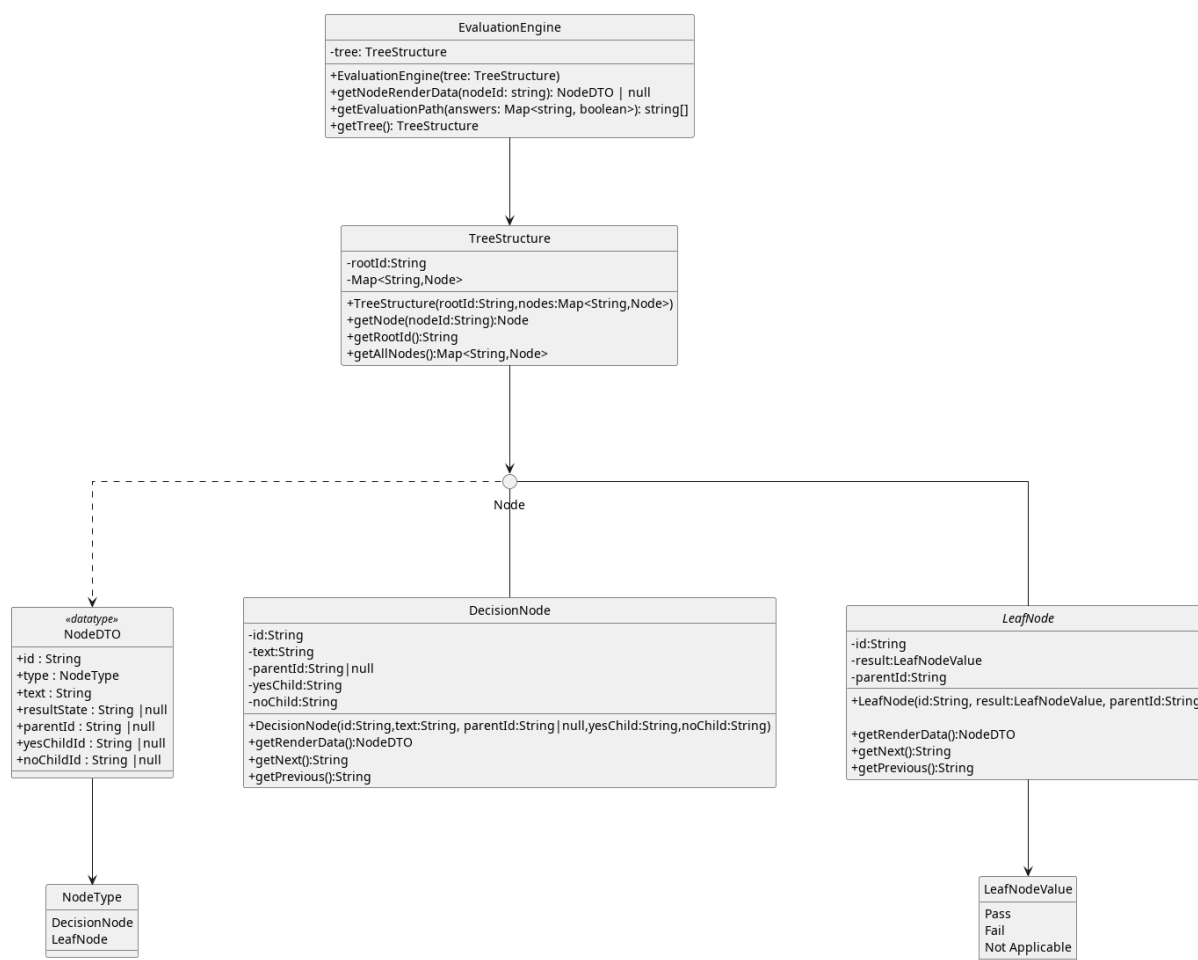


Figura 192: Logica di valutazione_G

Non esegue logica di business di competenza del backend, effettua i calcoli necessari a mostrare l'albero e il percorso attivo da mostrare.

5.9.1.1) TreeStructure

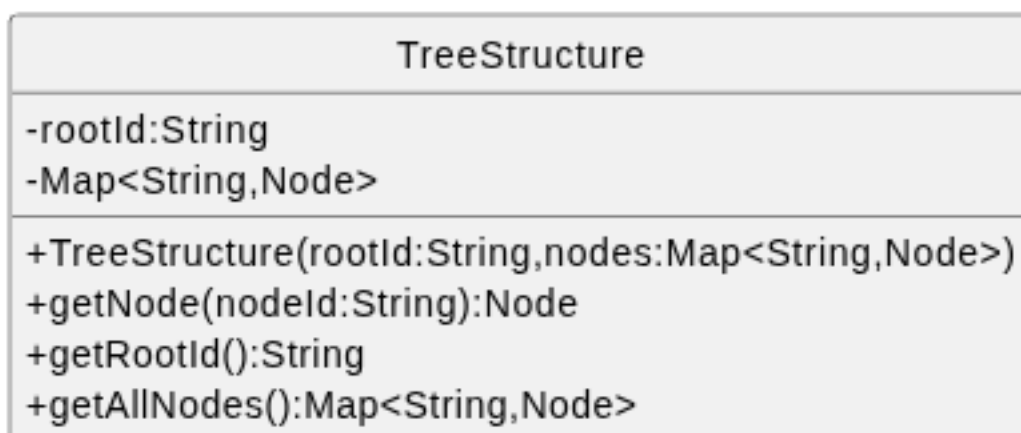


Figura 193: TreeStructure

Descrizione

TreeStructure funge da contenitore organizzato per l'intero albero decisionale. Al fine di massimizzare le prestazioni di ricerca lato client, utilizza una rappresentazione «piatta» basata su una mappa anziché una struttura nidificata.

Attributi

- - nodes: Map<String, Node> — Mappa che indicizza tutti i nodi tramite il loro ID univoco per un accesso immediato ($O(1)$).
- - rootId:String — Id del nodo_G root.

Metodi

- + getNode(NodeId: String): Node — Recupera l'oggetto nodo_G corrispondente all'ID fornito.
- + getRootId(): String — Restituisce l'ID del punto di partenza della valutazione_G.
- + getAllNodes(): Map<String, Node> — Restituisce tutti i nodi del decision tree_G.
- + TreeStructure(rootId:String,nodes:Map<String, Node>) — Costruttore

5.9.1.2) Node

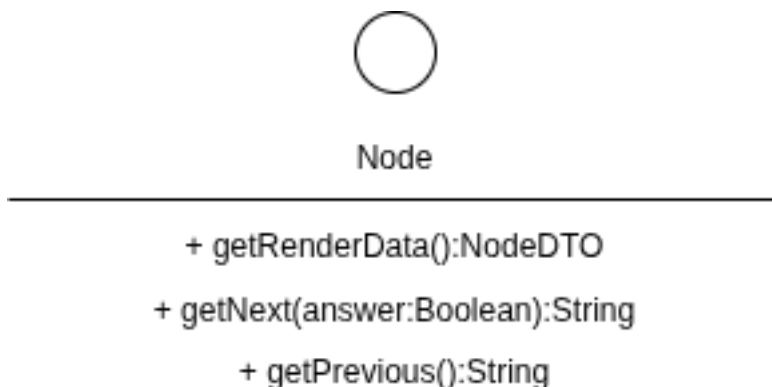


Figura 194: Node

Descrizione:

Rappresenta il contratto che deve implementare un nodo_G dell'albero di decisione.

Attributi:

È un'interfaccia_G, non ha attributi.

Metodi:

- + getRenderData():NodeDTO — Recupera i dati del nodo_G per poterli mostrare a schermo.
- + getNext(answer:Boolean):String — Recupera l'id del nodo_G successore.
- + getPrevious():String — Recupera l'id del nodo_G padre.

5.9.1.3) DecisionNode

DecisionNode
-id:String -text:String -parentId:String null -yesChild:String -noChild:String
+DecisionNode(id:String, text:String, parentId:String null, yesChild:String, noChild:String) +getRenderData():NodeDTO +getNext():String +getPrevious():String

Figura 195: DecisionNode

Descrizione

DecisionNode è la classe che implementa un nodo_G intermedio (domanda). Contiene la logica decisionale binaria (Sì/No) per determinare quale ramo dell'albero debba essere percorso.

Attributi

- - id, - question: String — Identificativo e testo della domanda.
- - text: String — Testo della domanda associata al nodo_G.
- - parentId: String | null — Riferimento al nodo_G precedente.
- - yesChildId: String|null — Riferimento al nodo_G figlio associato alla risposta «Sì»
- - noChildId: String|null — Riferimento al nodo_G figlio associato alla risposta «No»

Metodi

- + DecisionNode(id:String, text:String ,parentId:String| null,yesChild:String,noChild:String) — Costruttore.
- + getRenderData():NodeDTO — Recupera i dati del nodo_G per poterli mostrare a schermo.
- + getNext(answer:Boolean):String — Recupera l'id del nodo_G successore.
- + getPrevious():String — Recupera l'id del nodo_G padre.

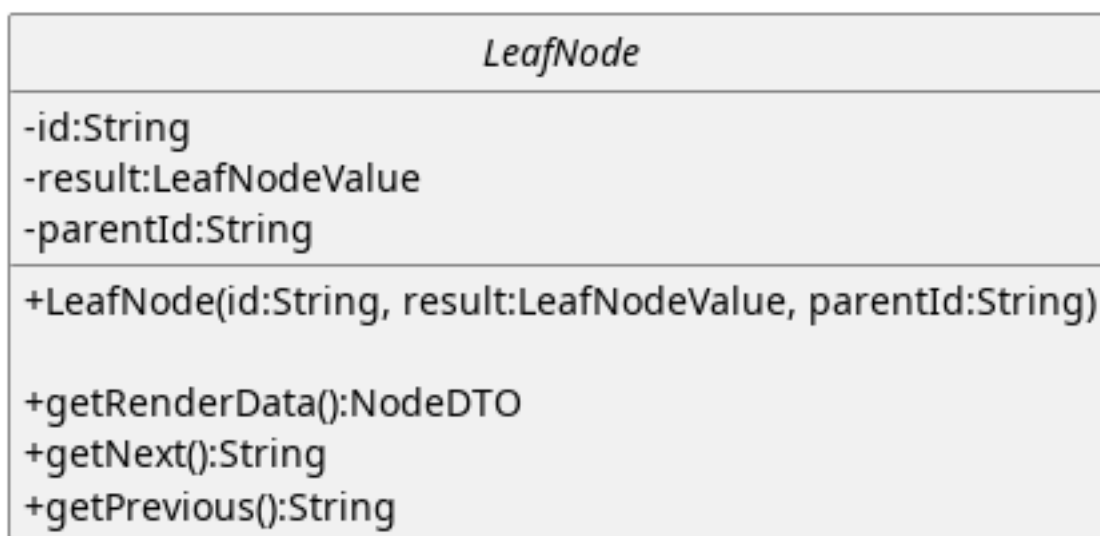
5.9.1.4) LeafNode

Figura 196: LeafNode

Descrizione

LeafNode rappresenta il punto terminale di un ramo dell'albero (foglia). Non permette ulteriori avanzamenti e contiene lo stato finale della valutazione_G per quel determinato percorso.

Attributi

- - parentId — Id del nodo_G padre.

- - result: evaluationStateType — definisce l'esito finale (es. «Conforme», «Non Conforme»).

Metodi

- +LeafNode(id:String, result:LeafNodeValue, parentId:String) — Costruttore.
- + getRenderData():NodeDTO — Recupera i dati del nodo_G per poterli mostrare a schermo.
- + getNext(answer:Boolean):String — Recupera l'id del nodo_G successore.
- + getPrevious():String — Recupera l'id del nodo_G padre.

5.9.1.5) NodeDTO

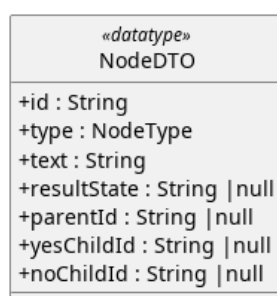


Figura 197: NodeDTO

Descrizione

NodeDTO è una struttura dati passiva utilizzata esclusivamente per il trasferimento e la formattazione dei dati. Incapsula le informazioni «grezze» del nodo_G in modo che il componente visivo possa stamparle a schermo senza dover accedere alla complessa logica di dominio.

Attributi

- + id: String — Identificativo univoco del nodo_G.
- + type: NodeType — Tipo del nodo_G (Decisione o Foglia).
- + text: String — il contenuto testuale della domanda o dell'esito da mostrare.
- + parentId: String | null — riferimento all'ID del nodo_G padre, necessario per calcolare la navigazione a ritroso.
- + yesChildId: String | null — riferimento all'ID del nodo_G figlio in caso di risposta affermativa.
- + noChildId: String | null — riferimento all'ID del nodo_G figlio in caso di risposta negativa.

Metodi

NodeDTO non espone metodi, trattandosi di un oggetto dedicato al solo trasporto dati.

5.9.1.6) NodeType

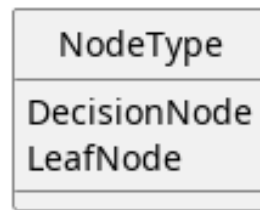


Figura 198: NodeType

Descrizione

NodeType è un enum che serve a distinguere i nodi di decisione e i nodi foglia quando questi vengono trasformati in strutture dati pure.

Attributi

- `DecisionNode` — Elemento dell'enum che identifica un nodo_G di decisione.
- `LeafNode` — Elemento dell'enum che identifica un nodo_G foglia.

Metodi

Questo enum non espone metodi

5.9.1.7) LeafNodeValue

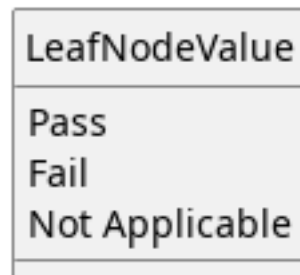


Figura 199: Leaf Node Value

Descrizione

`LeafNodeValue` è un enum che raccoglie i possibili valori di un nodo_G foglia.

Attributi

- `PassG` — Elemento dell'enum che identifica un nodo_G foglia `passG`.
- `FailG` — Elemento dell'enum che identifica un nodo_G foglia `failG`.
- `NotApplicable` — Elemento dell'enum che identifica un nodo_G foglia `not applicableG`.

Metodi

Questo enum non espone metodi

5.9.1.8) EvaluationEngine

EvaluationEngine
-tree: TreeStructure
+EvaluationEngine(tree: TreeStructure) +getNodeRenderData(nodeId: string): NodeDTO null +getEvaluationPath(answers: Map<string, boolean>): string[] +getTree(): TreeStructure

Figura 200: EvaluationEngine

Descrizione

EvaluationEngine è il motore logico principale che calcola il percorso attivo. Traduce le risposte dell'utente in un percorso visivo coerente e navigabile.

Attributi

- - tree: TreeStructure — la struttura dell'albero su cui operare.

Metodi

- + EvaluationEngine(tree: TreeStructure) — Costruttore.
- + getEvaluationPath(answers: Map<String, Boolean>): List<String> — incrocia la mappa delle risposte fornite dall'utente con la struttura dell'albero per generare la lista ordinata di ID che compongono l'attuale «Percorso Attivo» (Active Path).
- + getNodeRenderData(nodeId: String): NodeDTO — metodo di utilità per ottenere i dati grafici di un nodo_G specifico tramite il motore.
- + getTree() — metodo di utilità per recuperare la struttura del decision tree_G.

5.9.2) APIClient

5.9.3) EvaluationApiClient

EvaluationApiClient
-answerUrl:String -stateUrl:String -justificationUrl:String -detailUrl:String
+EvaluationApiClient(answerUrl:String, stateUrl:String, justificationUrl:String, detailUrl:String) +saveAnswer(nodeId:String, answer:Boolean):Promise<JSON> +saveJustification(justification:String):Promise<JSON> +fetchState():Promise<JSON> +fetchDetail():Promise<JSON>

Figura 201: Dettaglio dell'EvaluationApiClient

Descrizione

EvaluationApiClient è l'Outbound Adapter del frontend. Questa classe incapsula tutta la logica di comunicazione HTTP, isolando il resto dell'applicazione (Store e Compo-

nenti) dai dettagli implementativi delle chiamate API. Utilizza il pattern delle *Promise* per gestire l'asincronia tipica delle richieste di rete.

Attributi

- - `answerUrl: String` — Url presso cui effettuare la chiamata API per salvare una risposta.
- - `stateUrl: String` — Url presso cui effettuare la chiamata API per recuperare solo lo stato di valutazione_G del requisito_G.
- - `justificationUrl: String` — Url presso cui effettuare la chiamata API per inserire una giustificazione da associare al requisito_G.
- - `detailUrl: String` — Url presso cui effettuare la chiamata API per recuperare tutti i dati relativi alla valutazione_G del requisito_G.

Metodi

- `+EvaluationAPIClient(answerUrl: String, stateUrl: String, justificationUrl: String, detailUrl: String)` — Costruttore che riceve dall'esterno gli url a cui effettuare le chiamate API, gli url contengono anche il contesto della valutazione_G (dispositivo_G, asset_G, requisito_G).
- `+ saveAnswer( node_id:String, answer:Boolean): Promise<JSONG>` — invia una richiesta asincrona al server per salvare la risposta fornita a un determinato nodo_G dell'albero. Riceve tutti i parametri di contesto necessari (dispositivo_G, asset_G e requisito_G) per garantire la corretta associazione dei dati nel database.
- `+ saveJustification(justification:String)` — invia una richiesta asincrona al server per salvare la giustificazione fornita al requisito_G.
- `+ fetchRequirementEvaluationState(): Promise<JSONG>` — recupera dal backend lo stato attuale della valutazione_G per un determinato requisito_G. Recupera solo lo stato della valutazione_G.
- `+ fetchRequirementEvaluationDetail(): Promise<JSONG>` — recupera dal backend lo stato attuale della valutazione_G per un determinato requisito_G, utile in caso serva effettuare un reset.

5.9.4) Layout

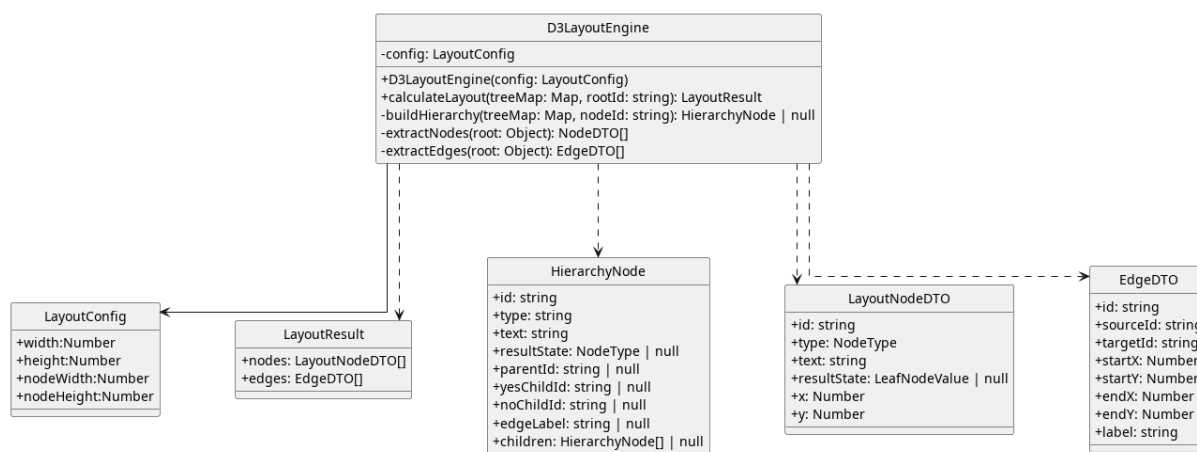


Figura 202: LayoutEngineCompressivo

Incapsula i calcoli necessari a mostrare correttamente a schermo il decision tree_G.

5.9.4.1) LayoutNodeDTO

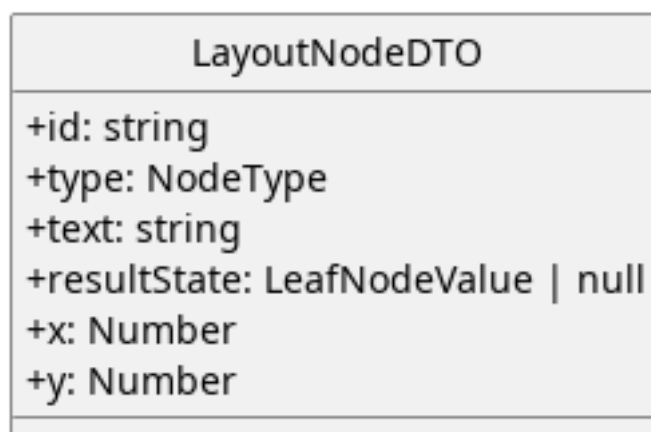


Figura 203: LayoutNodeDTO

Descrizione: Incapsula tutti i dati necessari al rendering grafico.

Attributi:

- + id:String — Id del nodo_G.
- + type:NodeType — Tipo di nodo_G.
- + text:String — Testo descrittivo del nodo_G.
- + resultState:LeafNodeValue|null — Se è un nodo_G foglia, è il valore di quel nodo_G.
- + x:Number — Coordinata x del nodo_G.
- + y:Number — Coordinata y del nodo_G.

Metodi:

Non espone metodi

5.9.4.2) EdgeDTO

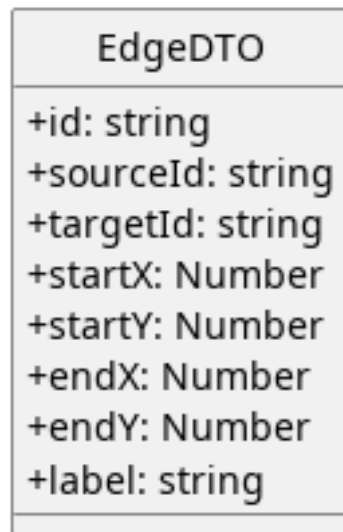


Figura 204: EdgeDTO

Descrizione:

Incapsula i dati necessari al rendering degli archi che connettono i nodi

Attributi:

- `+id:String` — Id dell'arco .
- `+sourceId:String` — Id del nodo_G da cui origina l'arco.
- `+targetId:String` — Id del nodo_G in cui termina l'arco.
- `+startX:Number` — Coordinata X da cui far partire gli arco.
- `+startY:Number` — Coordinata Y da cui far partire gli arco.
- `+endX:Number` — Coordinata X in cui far terminare l'arco.
- `+endY:Number` — Coordinata Y in cui far terminare l'arco.
- `+label:String` — Testo da mostrare sull'arco .

Metodi:

Non espone metodi.

5.9.4.3) HierarchyNode

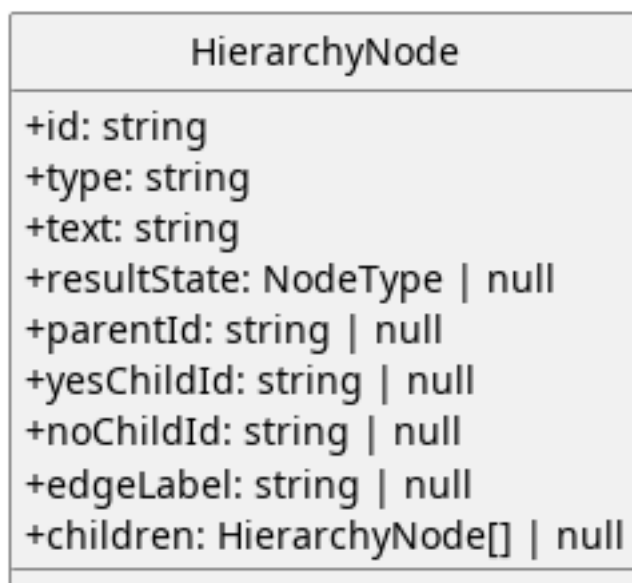


Figura 205: HierarchyNode

Descrizione:

Struttura dati ricorsiva utilizzata internamente dal `D3LayoutEngine` come formato intermedio per alimentare `d3.hierarchy()`. Estende i dati del `NodeDTO` con i campi `children` (riferimenti ricorsivi ai nodi figli) e `edgeLabel` (etichetta dell'arco, «Yes» o «No»). Questa struttura non esce dal layout engine — viene costruita da e consumata da `d3` per calcolare le posizioni, dopodiché viene scartata.

Attributi:

- `+ id : String` — identificativo univoco del nodo_G, corrisponde all'id del nodo_G di dominio.
- `+ type : NodeType` — tipo del nodo_G.
- `+ text : String` — testo da visualizzare, la domanda per i nodi decisione o il verdetto per i nodi foglia.
- `+ resultState : String` — stato di valutazione_G per i nodi foglia (`passG`, `failG`, `not_applicable`), null per i nodi decisione.
- `+ parentId : String` — identificativo del nodo_G genitore, null per la radice.
- `+ yesChildId : String` — identificativo del figlio raggiunto con risposta affermativa, null per i nodi foglia.
- `+ noChildId : String` — identificativo del figlio raggiunto con risposta negativa, null per i nodi foglia.
- `+ _edgeLabel : String` — etichetta dell'arco che collega questo nodo_G al suo genitore («Yes» o «No»). Il prefisso underscore indica una proprietà ausiliaria aggiunta durante la costruzione della gerarchia, non presente nei dati di dominio originali.

- + children : HierarchyNode[] — array ricorsivo dei nodi figli nel formato atteso da d3.hierarchy(). Null o assente per i nodi foglia.

Metodi:

Non espone metodi

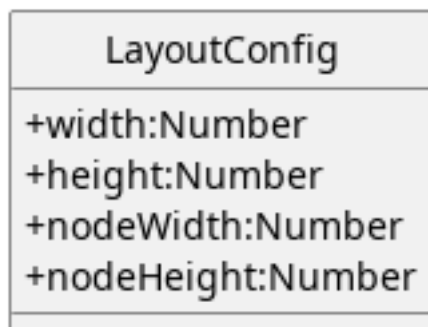
5.9.4.4) LayoutConfig

Figura 206: LayoutConfig

Descrizione:

Incapsula le configurazioni base per il layout

Attributi:

- +width: Number — larghezza complessiva a cui il layout si deve adattare.
- +height: Number — Altezza complessiva a cui il layout si deve adattare.
- +nodeWidth: Number — Larghezza dei nodi.
- +nodeHeight: Number — Altezza massima dei nodi.

Metodi:

Non espone metodi

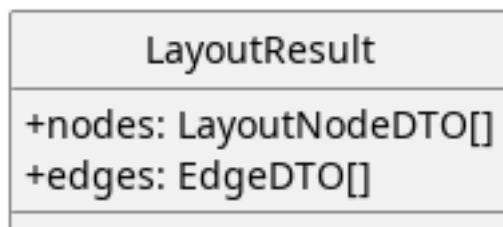
5.9.4.5) LayoutResult

Figura 207: LayoutResult

Descrizione:

Racchiude tutte le informazioni necessarie a mostrare i nodi.

Attributi:

- + nodes:LayoutNodeDT0[] — Elenco dei nodi da mostrare a schermo.
- + edges:EdgeDT0[] — Elenco degli archi da mostrare a schermo.

Metodi:

Non espone metodi.

5.9.4.6) D3LayoutEngine

D3LayoutEngine
-config: LayoutConfig
+D3LayoutEngine(config: LayoutConfig)
+calculateLayout(treeMap: Map, rootId: string): LayoutResult
-buildHierarchy(treeMap: Map, nodeId: string): HierarchyNode null
-extractNodes(root: Object): NodeDTO[]
-extractEdges(root: Object): EdgeDTO[]

Figura 208: D3LayoutEngine

Descrizione:

Incapsula i calcoli e le altre operazioni relative alla costruzione del layout.

Attributi:

- - config: LayoutConfig — Impostazioni base per personalizzare il layout.

Metodi:

- + D3LayoutEngine(config: LayoutConfig) — Costruttore
- + calculateLayout(treeMap: Map, rootId: string): LayoutResult — Calcola il layout partendo dalla struttura del decision tree_G.
- - buildHierarchy(treeMap: Map, nodeId: string): HierarchyNode | null — Funzione interna per il calcolo del hierarchy.
- - extractNodes(root: HierarchyNode): LayoutNodeDT0[] — Funzione di utilità che trasforma il formato di D3 in un formato di dominio.
- - extractEdges(root: HierarchyNode): EdgeDT0[] — Funzione di utilità che trasforma il formato di D3 in un formato di dominio.

5.9.5) Store

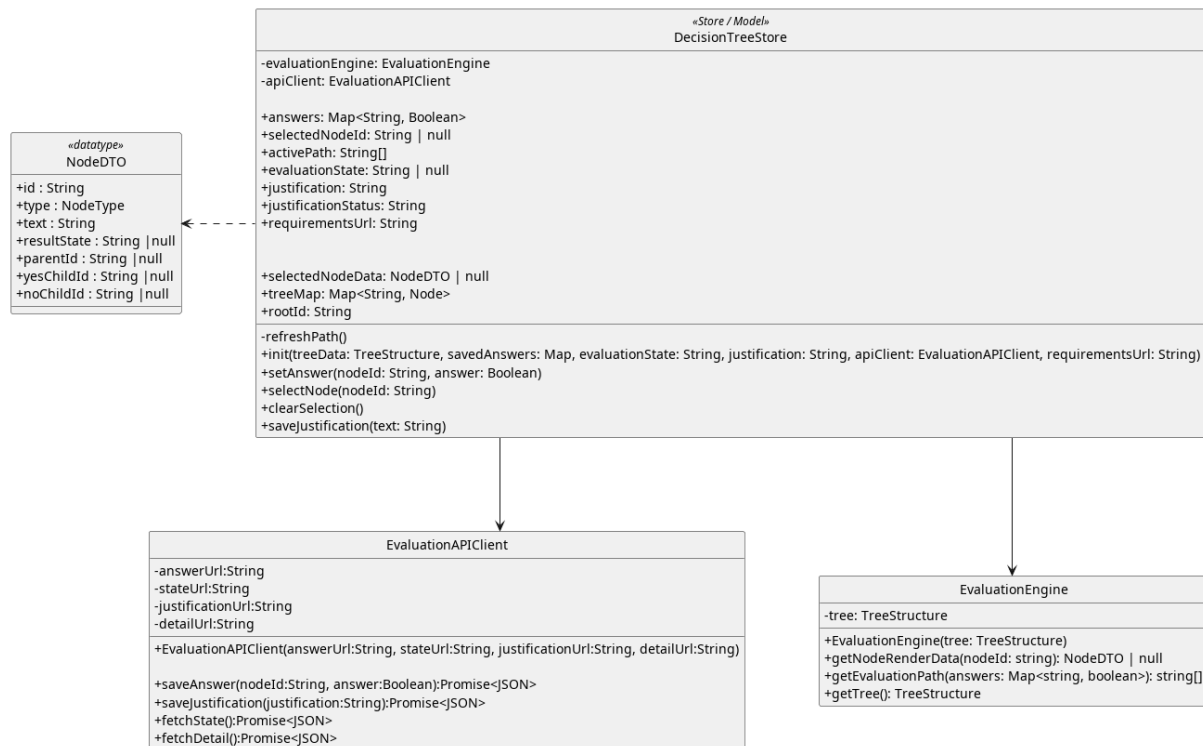


Figura 209: DecisionTreeComplessivo

5.9.5.1) DecisionTreeStore

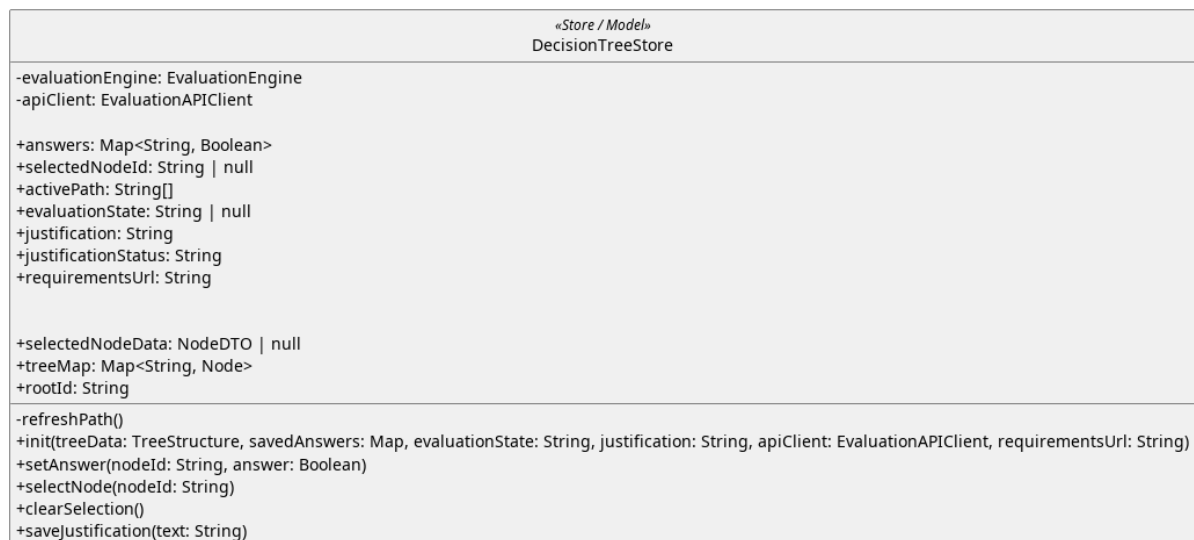


Figura 210: DecisionTreeStore

Descrizione

DecisionTreeStore rappresenta il cuore reattivo dell'applicazione. È lo Store che permette la comunicazione disaccoppiata tra i vari componenti Vue (Canvas e Sidebar). Invece di passare dati tramite «props» e «events» complessi, i componenti leggono e scrivono direttamente in questo store, che si occupa di mantenere la coerenza dei dati.

Diversi requisiti non condividono lo stesso store.

Funge da model del MVVM.

Attributi (State)

- - `evaluationEngine: EvaluationEngine` — Istanza del motore logico utilizzata per i calcoli di navigazione.
- - `apiClient: EvaluationAPIClient` — Istanza dell'API client per effettuare le chiamate col backend.
- + `answers: Map<String, Boolean>` — Stato reattivo delle risposte fornite dall'utente durante la sessione corrente.
- + `selectedNodeId: String|null` — Id del nodo_G selezionato al momento.
- + `activePath: String[]` — Lista reattiva degli ID dei nodi che compongono il percorso corrente, aggiornata automaticamente al variare delle risposte.
- + `evaluationState: LeafNodeValue` — oggetto reattivo che rappresenta lo stato della valutazione_G.
- + `justification: String` — Giustificazione per il requisito_G sottoposto a valutazione_G.
- + `justificationStatus: String` — Stato del salvataggio della giustificazione da mostrare all'utente
- + `requirementsUrl: String` — Url della lista requisiti.
- + `selectedNodeData: NodeDTO` — Oggetto reattivo che tiene traccia dei dati del nodo_G selezionato.
- + `treeMap: Map<String, Node>` — Mantiene l'elenco dei nodi in memoria e facilmente accessibile.
- + `rootId: String` — Root dell'albero decisionale.

Metodi (Actions)

- - `refreshPath()` — metodo privato invocato internamente per aggiornare l'attributo `activePath` interpellando l'*EvaluationEngine*.
- + `init(treeData: TreeStructure, savedAnswers: Map<String, Boolean>, justification: String)` — inzializza lo store caricando la struttura dell'albero e le eventuali risposte già salvate, la giustificazione, lo stato della valutazione_G, l'api client e l'url della lista dei requisiti per il inserirlo in un link.
- + `setAnswer(nodeId, answer)` — aggiorna una risposta nello stato locale, invoca il ricalcolo del percorso e avvia la persistenza asincrona tramite l'API.
- + `selectNode(nodeId)` — aggiorna il nodo_G selezionato, permettendo alla Sidebar di mostrare le informazioni contestuali corrette.
- + `clearSelection()` — aggiorna lo stato interno per togliere il focus dal nodo_G corrente.
- + `saveJustification(text: String)` — Salva la giustificazione inserita tramite il backend.

5.9.6) Componenti

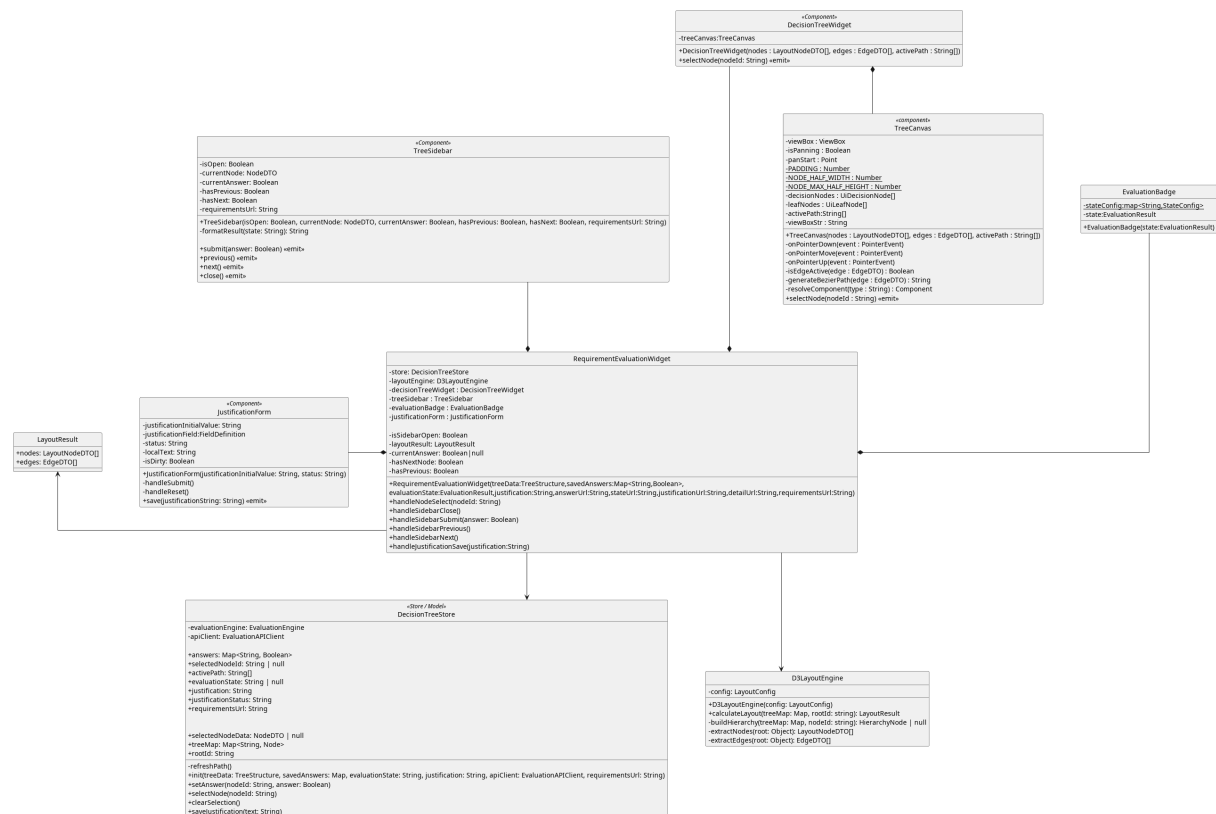


Figura 211: RequirementEvaluationWidget

Questo widget realizza il MVVM, lo store rappresenta il model, il RequirementEvaluationWidget mantiene un riferimento allo store.

Come convenzione di Vue il componente è diviso in 2 parti script(ViewModel) e template (View). La reattività è realizzata usando le funzionalità di vue.

5.9.6.1) RequirementEvaluationWidget

RequirementEvaluationWidget
<pre> - store: DecisionTreeStore - layoutEngine: D3LayoutEngine - decisionTreeWidget : DecisionTreeWidget - treeSidebar : TreeSidebar - evaluationBadge : EvaluationBadge - justificationForm : JustificationForm - isSidebarOpen: Boolean - layoutResult: LayoutResult - currentAnswer: Boolean null - hasNextNode: Boolean - hasPreviousNode: Boolean + RequirementEvaluationWidget(treeData: TreeStructure, savedAnswers: Map<String, Boolean>, evaluationState: EvaluationResult, justification: String, answerUrl: String, stateUrl: String, justificationUrl: String, detailUrl: String, requirementsUrl: String) + handleNodeSelect(nodeId: String) + handleSidebarClose() + handleSidebarSubmit(answer: Boolean) + handleSidebarPrevious() + handleSidebarNext() + handleJustificationSave(justification: String) </pre>

Figura 212: RequirementEvaluationWidget Dettaglio

Descrizione

Rappresenta il widget per la valutazione_G el requisito_G, funge da ViewModel e monta la View realizza il binding tra il model e i componenti visivi(view). Le responsabilità all'interno del componente sono separate come da convenzione di Vue (tag <script> per la parte di ViewModel e tag <template> per la parte di view).

Attributi:

- - store : DecisionTreeStore — Riferimento allo store Pinia, unico punto di accesso allo stato.
- - layoutEngine: D3LayoutEngine — Layout engine usato per calcolare la disposizione di archi e nodi sullo schermo.
- - decisionTreeWidget : DecisionTreeWidget — Componente che contiene il canvas dell'albero.
- - treeSidebar : TreeSidebar — Pannello laterale per l'interazione con i nodi.
- - evaluationBadge : EvaluationBadge — Badge che mostra lo stato corrente della valutazione_G.
- - justificationForm : JustificationForm — Form per l'inserimento della giustificazione.
- - isSidebarOpen:Boolean — Flag che mantiene lo stato della iu e visualizzazione della sidebar per la visualizzazione del nodo_G.
- - layoutResult:LayoutResult — Mantiene il layout che il decision tree_G deve rispettare.
- - currentAnswer:Boolean|null — Mantiene la risposta associata al nodo_G corrente, elabora i dati del modello e li espone in un formato facilmente leggibile dalla View.
- - hasNextNode:Boolean — Indica se un nodo_G ha un successore nel path attivo, espone questa informazione in un formato facilmente leggibile dalla View.
- - hasPrevious:Boolean — Indica se un nodo_G ha un predecessore, espone questa informazione in un formato facilmente leggibile dalla View.

Metodi

- + RequirementEvaluationWidget(treeData:TreeStructure, savedAnswers:Map<String,Boolean>) — costruttore. Riceve i dati dal livello di integrazione, crea l'API client e inizializza lo store.
- - handleNodeSelect(nodeId : String) — Handler che riceve l'emit dal canvas e chiama store.
- - handleSidebarClose() — Handler che riceve l'emit dalla sidebar e chiama store.
- - handleSidebarSubmit(answer : Boolean) — Handler che riceve l'emit dalla sidebar e chiama store.
- - handleSidebarPrevious() — Handler che recupera il nodo_G precedente nel path attivo chiamando lo store.
- - handleSidebarNext() — Handler che recupera il nodo_G successivo nel path attivo chiamando lo store.

- - `handleJustificationSave(text : String)` — Handler che chiama lo store per salvare la nuova giustificazione.

5.9.6.2) TreeSidebar

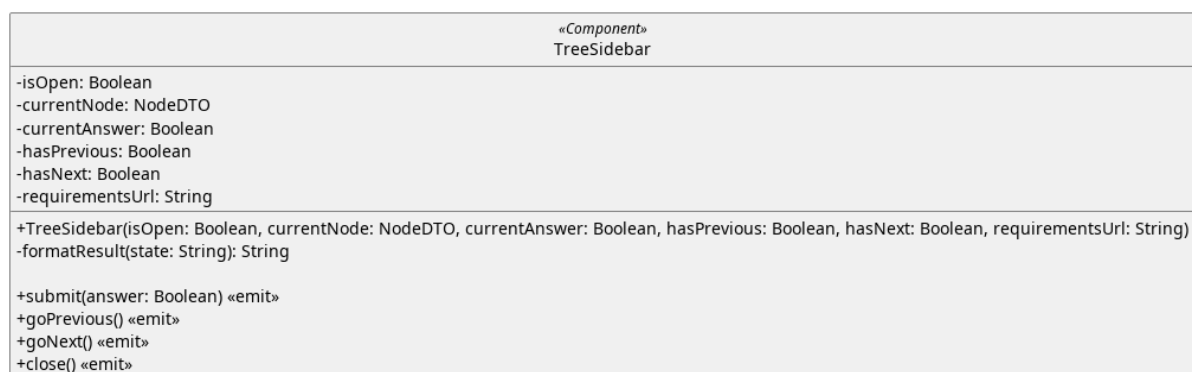


Figura 213: TreeSideBar

Descrizione

TreeSidebar è il componente Vue che implementa il pannello laterale interattivo. Il suo compito è intercettare l'input dell'utente e presentare le informazioni del nodo_G selezionato. Comunica con l'esterno tramite gli emit.

Attributi:

- - `isOpen: Boolean` — Flag che gestisce la visualizzazione della sidebar, il riferimento viene passato alla costruzione.
- - `currentNode: NodeDTO | null` — Oggetto reattivo che racchiude le informazioni da mostrare, viene aggiornato al cambiamento del nodo_G selezionato, la side bar rileva questo cambiamento e si aggiorna di conseguenza.
- - `currentAnswer: Boolean` — La risposta per il nodo_G corrente, se presente.
- - `hasPrevious: Boolean` — Indica se il nodo_G corrente ha un predecessore.
- - `hasNext: Boolean` — Indica se il nodo_G corrente ha un successore.
- - `requirementsUrl` — Url della lista requisiti.

Metodi

- + `TreeSidebar(currentNode : NodeDTO, currentAnswer : Boolean, hasPrevious : Boolean, hasNext : Boolean, requirementsUrl : String)` — Costruttore che riceve gli oggetti reattivi per costruire il widget.
- + `submit(answer: Boolean)<<emit>>` — gestisce l'invio della risposta.
- + `goPrevious()<<emit>>` — innesca la navigazione per tornare al nodo_G precedente lungo il percorso di valutazione_G.
- + `goNext()<<emit>>` — innesca la navigazione per avanzare al nodo_G logico successivo all'interno del flusso.

- + close()«emit» — Chiude la sidebar.
- - formatResult(state:String) — Incapsula la conversione del risultato della valutazione_G in testo ed eventuali label comprensibili all'utente.

5.9.6.3) DecisionTreeWidget

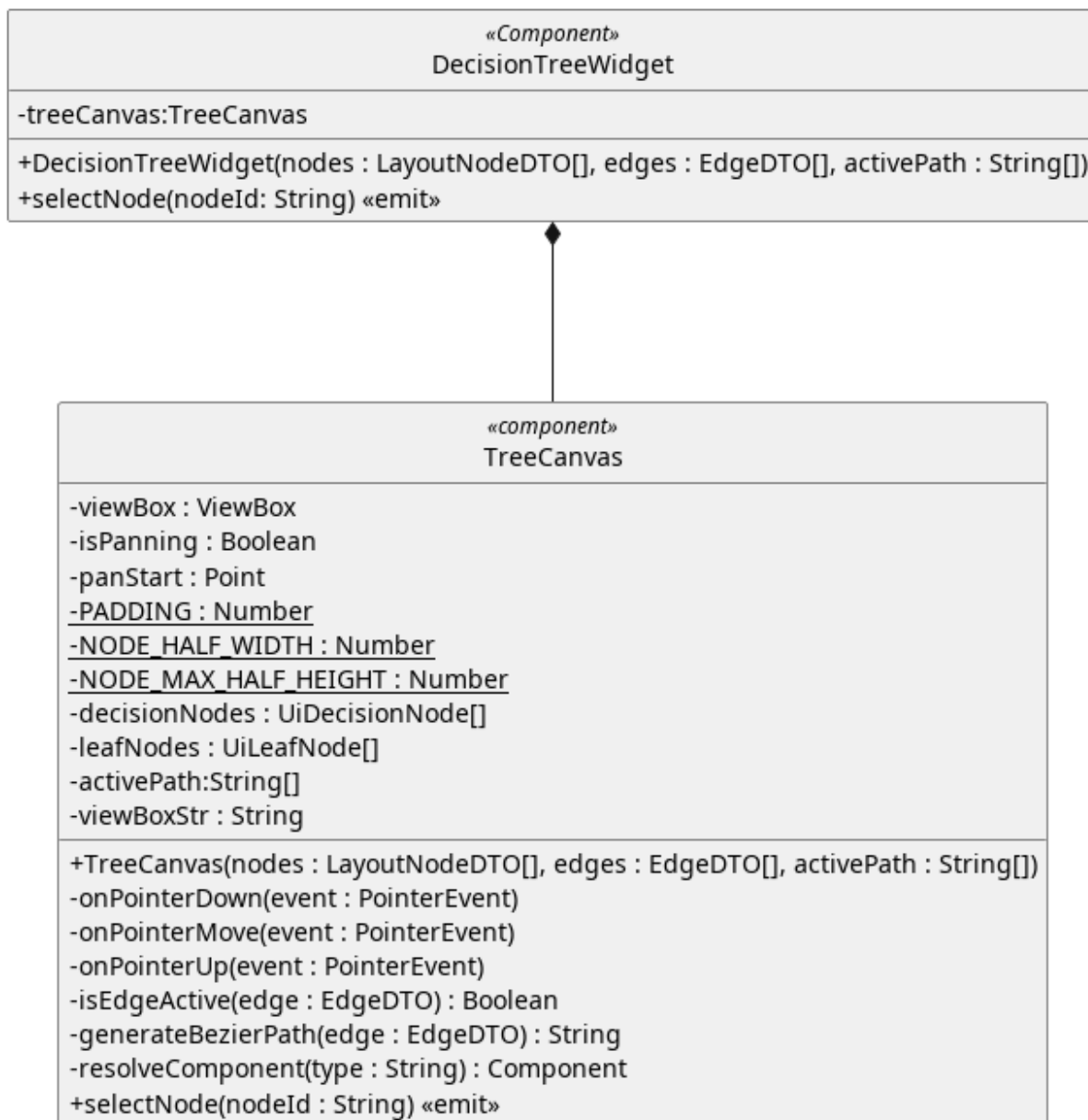


Figura 214: DecisionTreeWidget

Descrizione:

Wrapper del decision tree_G canvas.

Attributi:

- - treeCanvas:TreeCanvas — Canvas su cui disegnare il decision tree_G.

Metodi:

- + DecisionTreeWidget(nodes : LayoutNodeDT0[], edges : EdgeDT0[], activePath : String[]) — Costruttore che riceve i dati reattivi da osservare dall'esterno e li passa alla canvas.
- + selectNode(nodeId:String)<<emit>> — Ritrasmette l'evento emesso dalla canvas.

5.9.6.4) TreeCanvas

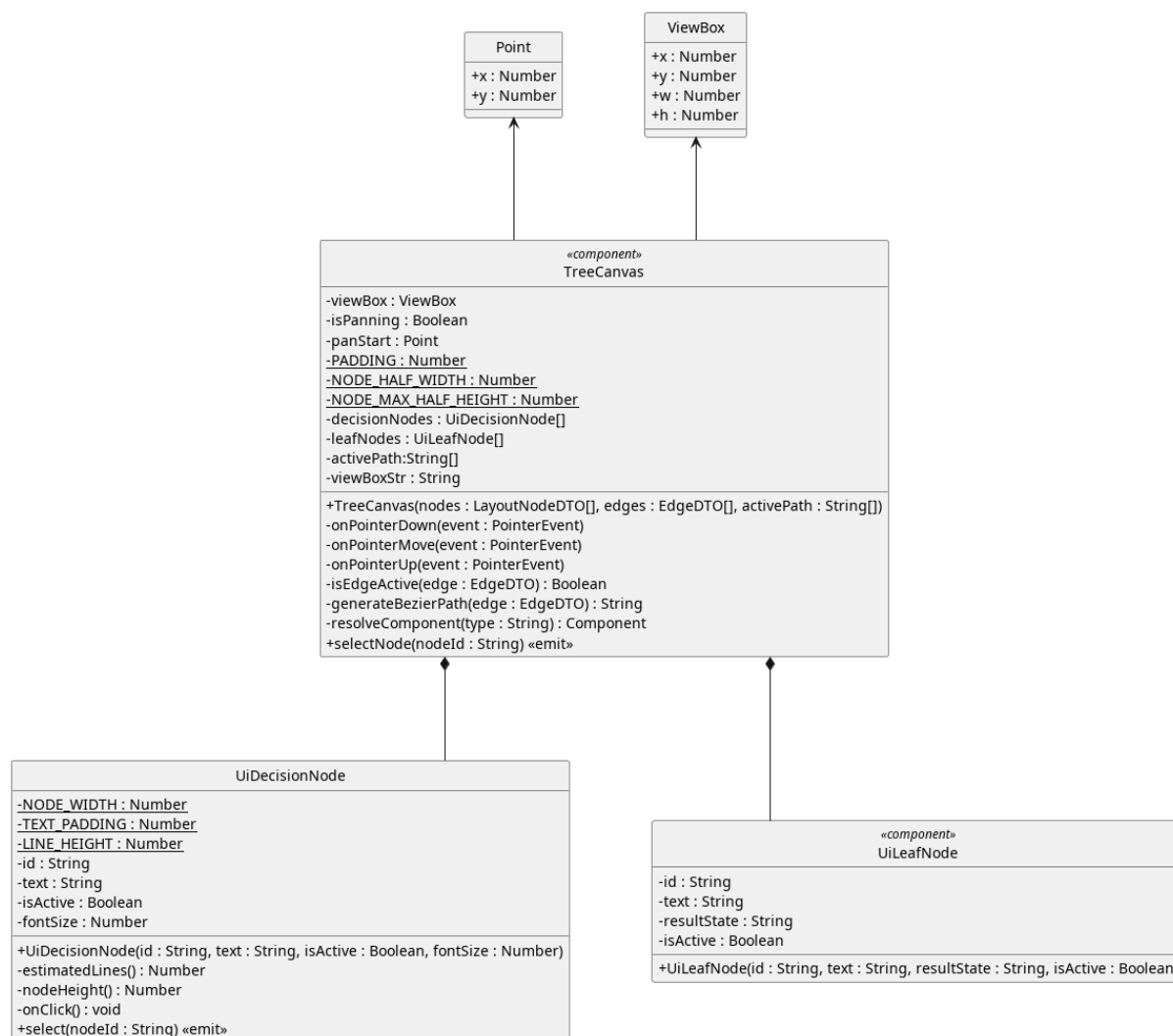


Figura 215: TreeCanvas

Descrizione:

Rappresenta la canvas su cui viene disegnato l'albero di decisione. Riceve gli oggetti reattivi da osservare alla costruzione, comunica con l'esterno tramite emit.

Attributi:

- - viewBox:VBox — VBox in cui disegnare l'albero.
- - isPanning:Boolean — Flag che aiuta a distinguere un click su un nodo_C da un click sullo sfondo.
- - panStart:Point — Punto di origine per disegnare il decision tree_C

- - `PADDING:Number` — Costante che incapsula il padding dell'svg, semplifica le configurazioni.
- - `NODE_HALF_WIDTH:Number` — Costante che incapsula la larghezza dei nodi, semplifica le configurazioni.
- - `NODE_MAX_HALF_HEIGHT:Number` — Costante che incapsula l'altezza massima di un `nodeG` del decision treeG.
- - `decisionNodes:UiDecisionNode[]` — Lista dei dati relativi ai componenti che rappresentano i nodi di decisione.
- - `leafNodes:UiDecisionNode[]` — Lista dei dati relativi ai componenti che rappresentano i nodi foglia.
- - `activePath:String[]` — Lista dei nodi attivi.
- - `viewBox:String` — Stringa da passare con attributo del tag html.

Metodi:

- + `TreeCanvas(nodes:LayoutNodeDTO[],edges:EdgeDTO[],activePath:String[])` — Costruttore che riceve i dati reattivi dall'esterno.
- - `onPointerDown(event:PointerEvent)` — Callback da eseguire in caso di evento `pointerDown`.
- - `onPointerMove(event:PointerEvent)` — Callback da eseguire in caso di evento `pointerMove`.
- - `onPointerUp(event:PointerEvent)` — Callback da eseguire in caso di evento `pointerUp`.
- - `isEdgeActive(EdgeDTO):Boolean` — Incapsula il controllo dello stato di attività di un `nodeG`.
- - `generateBezierPath(edge:EdgeDTO):String` — Genera la stringa per renderizzare l'arco nell'svg.
- - `resolveComponent(type:String):Component` — Mappa il `NodeType` del `LayoutNodeDTO` al component corretto, è il modo inteso per realizzare il polimorfismo in Vue.
- + `selectNode(nodeId:String)<<emit>>` — Evento emesso verso l'esterno per notificare la selezione di un `nodeG`.

5.9.6.5) EvaluationBadge

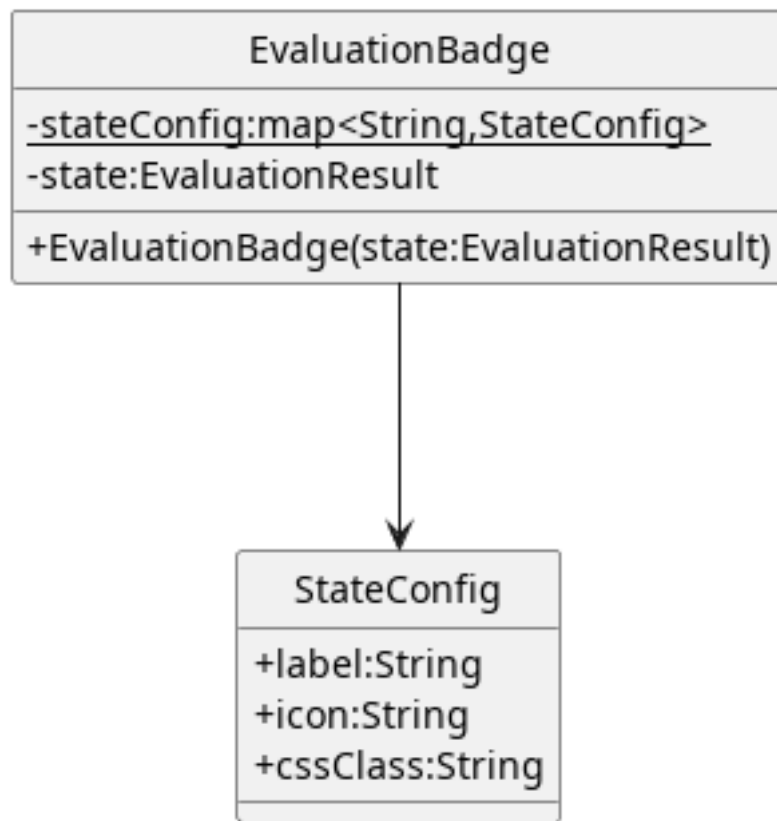


Figura 216: EvaluationBadge

Descrizione: Rappresenta il badge che notifica lo stato della valutazione_G.

Attributi:

- - `stateConfig:map<String,StateConfig>` — Mappa il valore del risultato della valutazione_G alle informazioni relative alla presentazione.
- - `state:EvaluationState` — Oggetto reattivo che mappa il valore dello stato di valutazione_G.

Metodi:

- + `EvaluationBadge(state:EvaluationResult)` — Costruttore che riceve l'oggetto reattivo da osservare.

5.9.6.6) JustificationForm

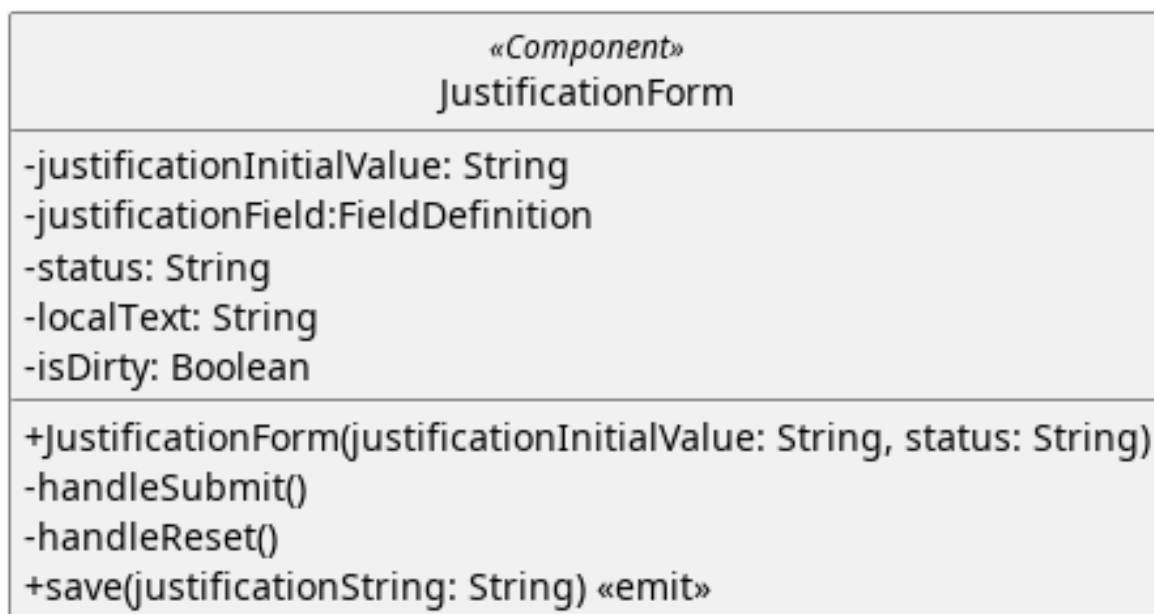


Figura 217: JustificationForm

Descrizione: Componente che mostra a schermo un form per modificare la descrizione.

Attributi:

- - justificationInitialValue:String — Testo della giustificazione salvato.
- - justificationField:FieldDefinition — Oggetto che gestisce il singolo campo del form, con le relative regole di validazione.
- - status:String — Stato della modifica della giustificazione.
- - localText:String — Testo attualmente inserito dall'utente.
- - isDirty:Boolean — Flag che traccia lo stato di aggiornamento della giustificazione.

Metodi:

- + JustificationForm(justificationInitialValue:String, status:String) — Costruttore che riceve dall'esterno il valore della giustificazione salvato sullo store.
- - handleSubmit() — Gestisce il salvataggio della nuova giustificazione.
- - handleReset() — Ripristina il testo al valore dell'initial value.
- + save(justification:String) <<emit>> — Evento emesso quando l'utente cerca di salvare una giustificazione.

5.9.6.7) Point

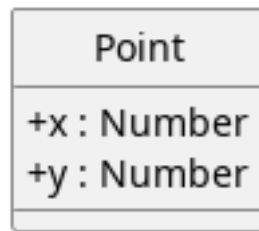


Figura 218: Point

Descrizione: Incapsula le coordinate di un singolo punto.

Attributi:

- + x:Number — Coordinata x.
- + y:Number — Coordinata y.

Metodi:

Non espone metodi.

5.9.6.8) StateConfig

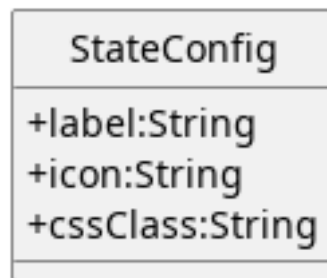


Figura 219: StateConfig

Descrizione:

Incapsula delle informazioni per la presentazione.

Attributi:

- + label:String — Etichetta da far visualizzare all'utente.
- + icon:String — Icona da mostrare.
- + cssClass:String — Classe o classi css da applicare.

Metodi:

Non espone metodi.

5.9.6.9) UiDecisionNode

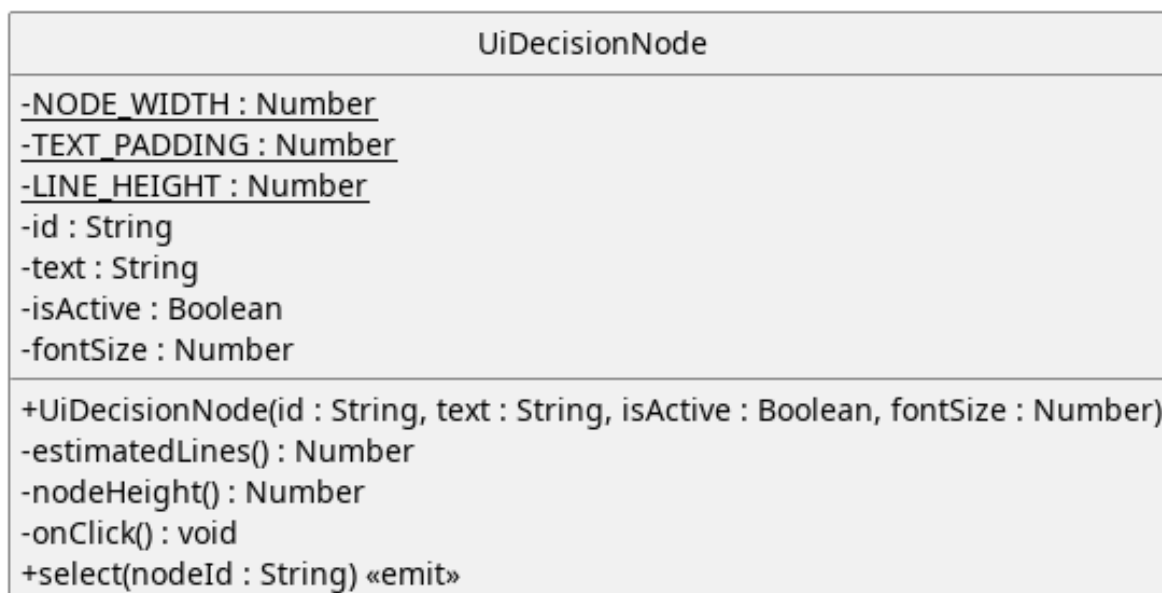


Figura 220: UiDecisionNode

Descrizione:

Componente che mostra graficamente il nodo_G di decisione.

Attributi:

- -NODE_WIDTH:Number — Valore statico che semplifica la configurazione della larghezza dei nodi.
- - TEXT_PADDING — Valore statico che semplifica del padding.
- - LINE_HEIGHT — Valore statico che semplifica la configurazione della distanza tra le linee.
- -id:String — Id del nodo_G di decisione.
- - text:String — Testo da mostrare a schermo.
- - isActive:Boolean — Flag usata per mostrare lo stato di attività del nodo_G.
- - fontSize:Number — Dimensione del testo.

Metodi:

- +
UiDecisionNode(id:String, text:String, isActive:Boolean, fontSize:Number)
—
- - estimatedLines():Number — Funzione di utilità per calcolare il numero di linee stimate del nodo_G.
- - nodeHeight:Number — Altezza del nodo_G.
- - onClick() — Callback eseguito al click di un nodo_G.
- - select(nodeId:String) — Evento emesso alla selezione di un nodo_G.

5.9.6.10) UiLeafNode

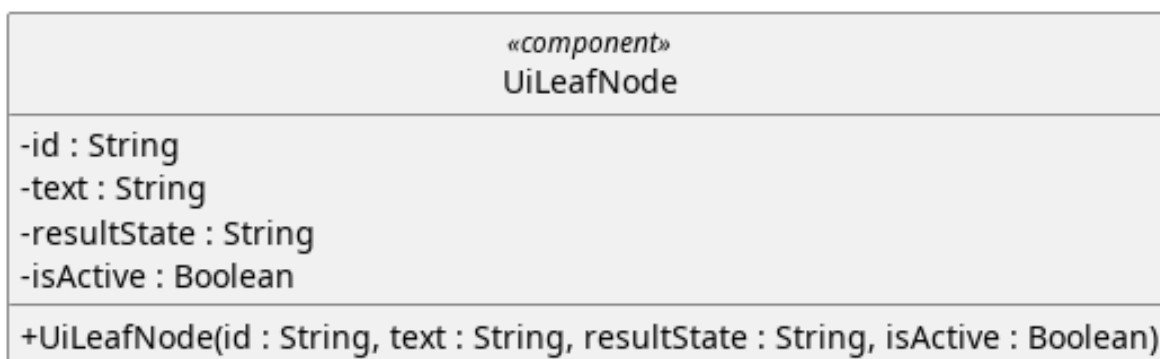


Figura 221: UiLeafNode

Descrizione:

Componente che disegna il singolo nodo_G foglia.

Attributi:

- - id:String — Id del nodo_G foglia.
- - text:String — Testo da mostrare.
- - resultState:String — Valore del nodo_G foglia.
- - isActive:Boolean — Flag utilizzato per mostrare lo stato di attività del nodo_G.

Metodi:

- + UiLeafNode(id:String, text:String, resultState:String, isActive:Boolean) — Costruttore che riceve i dati reattivi.

5.9.6.11) ViewBox

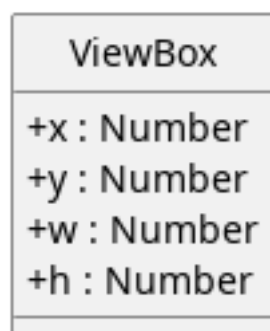


Figura 222: ViewBox

Descrizione:

Oggetto che incapsula i dati relative alla view box della canvas.

Attributi:

- - x:Number — Coordinata x da cui far partire la ViewBox.
- - y:Number — Coordinata y da cui far partire la ViewBox.

- - w:Number — Larghezza della ViewBox.
- - h:Number — Altezza della ViewBox.

Metodi:

Non espone metodi.

6) Tracciamento

6.1) Tracciamento dei Requisiti

6.1.1) Requisiti Obbligatori

Codice	Descrizione	Stato
RObb001	L'Utente deve poter visualizzare la lista di tutti i dispositivi registrati.	Implementato
RObb002	L'Utente deve poter visualizzare le informazioni generali dei dispositivi dalla lista di tutti i dispositivi registrati.	Implementato
RObb003	L'Utente deve poter visualizzare il nome di ogni dispositivo _G nella lista dei dispositivi.	Implementato
RObb004	L'Utente deve poter inserire e registrare nuovi dispositivi nel sistema.	Implementato
RObb005	L'Utente deve poter creare un nuovo dispositivo _G inserendo manualmente i dati richiesti.	Implementato
RObb006	L'Utente deve poter inserire il nome del dispositivo _G durante la creazione manuale. La lunghezza del nome deve essere compresa tra 1 e 64 caratteri.	Implementato
RObb007	L'utente deve poter visualizzare un messaggio di errore se in fase di creazione di un dispositivo _G inserisce un nome di lunghezza non compresa tra 1 e 64 caratteri.	Implementato
RObb008	L'Utente deve poter inserire il sistema operativo del dispositivo _G durante la creazione manuale.	Implementato
RObb009	L'Utente deve poter inserire la descrizione del dispositivo _G durante la creazione manuale.	Implementato
RObb010	L'Utente deve poter annullare la procedura di inserimento di un nuovo dispositivo _G in qualsiasi momento.	Implementato
RObb011	L'Utente deve poter inserire un nuovo dispositivo _G importando i dati tramite l'importazione _G di un file esterno.	Implementato
RObb012	L'Utente deve poter selezionare il file da usare in fase di importazione _G di un nuovo dispositivo _G .	Implementato
RObb013	L'Utente deve poter selezionare un file in formato JSON _G come sorgente per l'importazione _G del dispositivo _G .	Implementato
RObb014	L'Utente deve poter selezionare un file in formato XML _G come sorgente per l'importazione _G del dispositivo _G .	Implementato
RObb015	L'Utente deve poter selezionare un file in formato CSV _G come sorgente per l'importazione _G del dispositivo _G .	Implementato

RObb016	L'Utente deve poter visualizzare un messaggio di errore esplicativo se il file selezionato non rientra tra i formati supportati.	Implementato
RObb017	L'Utente deve poter visualizzare un messaggio di errore esplicativo se il file selezionato non ha una struttura interna interpretabile dal sistema.	Implementato
RObb018	L'Utente deve poter visualizzare un messaggio di errore esplicativo se il file selezionato ha una dimensione pari a 0 byte o superiore a 10 MB.	Implementato
RObb019	L'Utente deve poter visualizzare i dati del dispositivo _G nella vista dati e nella dashboard _G .	Implementato
RObb020	L'Utente deve poter visualizzare il nome del dispositivo _G nella vista dati e nella dashboard _G .	Implementato
RObb021	L'Utente deve poter visualizzare il sistema operativo del dispositivo _G nella vista dati e nella dashboard _G .	Implementato
RObb022	L'Utente deve poter visualizzare la descrizione del dispositivo _G nella vista dati e nella dashboard _G .	Implementato
RObb023	L'Utente deve poter visualizzare il modello di standard associato al dispositivo _G .	Implementato
RObb024	L'Utente deve poter visualizzare il nome del modello di standard associato al dispositivo _G .	Implementato
RObb025	L'Utente deve poter visualizzare la versione del modello di standard associato al dispositivo _G .	Implementato
RObb026	L'Utente deve poter eliminare un dispositivo _G dal sistema.	Implementato
RObb027	L'Utente deve poter eliminare un dispositivo _G senza effettuare un backup preventivo, previa conferma esplicita.	Implementato
RObb028	L'Utente deve poter avviare la sessione di valutazione _G di un dispositivo _G registrato nel sistema.	Implementato
RObb029	L'Utente deve poter scartare tutte le modifiche apportate dall'ultimo salvataggio effettuato e chiudere la sessione senza salvare.	Implementato
RObb030	L'Utente deve poter salvare le modifiche apportate durante la sessione di valutazione _G sul sistema di permanenza.	Implementato
RObb031	L'Utente deve poter salvare le modifiche apportate durante la valutazione _G e chiudere la sessione.	Implementato
RObb032	L'Utente deve poter salvare le modifiche apportate durante la valutazione _G mantenendo la sessione aperta per continuare.	Implementato

RObb033	L'Utente deve poter visualizzare un messaggio di errore esplicativo se il salvataggio della valutazione _G non va a buon fine a causa di un errore imprevisto, mantenendo attiva la sessione.	Implementato
RObb034	L'Utente deve poter visualizzare la dashboard _G riepilogativa della valutazione _G del dispositivo _G durante la sessione di valutazione _G .	Implementato
RObb035	L'Utente deve poter visualizzare lo stato aggregato _G della valutazione _G del dispositivo _G nella dashboard _G .	Implementato
RObb036	L'Utente deve poter esportare le informazioni del dispositivo _G su file.	Implementato
RObb037	L'Utente deve poter esportare le informazioni del dispositivo _G in formato XML _G .	Implementato
RObb038	L'Utente deve poter esportare le informazioni del dispositivo _G in formato JSON _G .	Implementato
RObb039	L'Utente deve poter esportare le informazioni del dispositivo _G in formato CSV _G .	Implementato
RObb040	L'Utente deve poter aggiungere un nuovo asset _G al dispositivo _G durante la sessione di valutazione _G .	Implementato
RObb041	L'Utente deve poter inserire il nome dell'asset _G durante l'aggiunta. Il nome dell'asset _G deve essere compreso tra 1 e 32 caratteri.	Implementato
RObb042	L'Utente deve poter selezionare il tipo dell'asset _G durante l'aggiunta.	Implementato
RObb043	L'Utente deve poter selezionare il tipo security asset _G per l'asset _G .	Implementato
RObb044	L'Utente deve poter selezionare il tipo network asset _G per l'asset _G .	Implementato
RObb045	L'Utente deve poter inserire la descrizione dell'asset _G durante l'aggiunta.	Implementato
RObb046	L'Utente deve poter visualizzare un messaggio di errore se inserisce un nome per l'asset _G di lunghezza non compresa tra 1 e 32 caratteri .	Implementato
RObb047	L'Utente deve poter annullare la procedura di aggiunta di un nuovo asset _G in qualsiasi momento.	Implementato
RObb048	L'Utente deve poter visualizzare la lista degli asset _G associati al dispositivo _G nella dashboard _G .	Implementato
RObb049	L'Utente deve poter visualizzare le informazioni generali di ogni asset _G nella lista degli asset _G del dispositivo _G .	Implementato

RObb050	L'Utente deve poter visualizzare il nome dell'asset _G sia nella lista che nel vista in dettaglio dell'asset _G .	Implementato
RObb051	L'Utente deve poter visualizzare il tipo dell'asset _G sia nella lista che nel vista in dettaglio dell'asset _G .	Implementato
RObb052	L'Utente deve poter visualizzare lo stato aggregato _G dell'asset _G sia nella lista che nel vista in dettaglio dell'asset _G .	Implementato
RObb053	L'Utente deve poter visualizzare il dettaglio completo di uno specifico asset _G selezionandolo dalla lista.	Implementato
RObb054	L'utente deve poter valutare i singoli asset _G di cui si compone il dispositivo _G .	Implementato
RObb055	L'Utente deve poter visualizzare la descrizione dell'asset _G nel dettaglio.	Implementato
RObb056	L'Utente deve poter eliminare un asset _G dal dispositivo _G durante la sessione di valutazione _G , previa conferma espressa.	Implementato
RObb057	L'Utente deve poter visualizzare la lista dei requisiti applicabili all'asset _G nel contesto della sessione di valutazione _G .	Implementato
RObb058	L'Utente deve poter visualizzare le informazioni generali di ogni requisito _G nella lista dei requisiti dell'asset _G .	Implementato
RObb059	L'Utente deve poter visualizzare il codice del requisito _G sia nella lista che nel dettaglio.	Implementato
RObb060	L'Utente deve poter visualizzare lo stato di valutazione _G del requisito _G sia nella lista che nel dettaglio.	Implementato
RObb061	L'Utente deve poter visualizzare il dettaglio completo di un requisito _G selezionandolo dalla lista dei requisiti dell'asset _G .	Implementato
RObb062	L'Utente deve poter visualizzare il nome del requisito _G nel dettaglio.	Implementato
RObb063	L'Utente deve poter visualizzare la descrizione normativa del requisito _G nel dettaglio.	Implementato
RObb064	L'Utente deve poter visualizzare lo stato PASS _G per la valutazione _G del requisito _G quando questa è completa e il risultato è PASS _G .	Implementato
RObb065	L'Utente deve poter visualizzare lo stato FAIL _G per la valutazione _G del requisito _G quando questa è completa e il risultato è FAIL _G .	Implementato

RObb066	L'Utente deve poter visualizzare lo stato NA per la valutazione _G del requisito _G quando questa è completa e il risultato è NA.	Implementato
RObb067	L'Utente deve poter visualizzare lo stato In corso _G per la valutazione _G del requisito _G quando la compilazione del decision tree _G non è ancora completata.	Implementato
RObb068	L'utente deve poter visualizzare lo stato in corso _G per la valutazione _G del requisito _G Quando la valutazione _G di una dipendenza _G è fallita o non è stata completata risulta.	Implementato
RObb069	L'Utente deve poter visualizzare la lista delle dipendenze del requisito _G nel dettaglio.	Implementato
RObb070	L'Utente deve poter visualizzare le informazioni sintetiche di ogni dipendenza _G nella lista delle dipendenze del requisito _G .	Implementato
RObb071	L'Utente deve poter visualizzare il codice di ogni dipendenza _G nella lista delle dipendenze del requisito _G .	Implementato
RObb072	L'Utente deve poter visualizzare lo stato di valutazione _G di ogni dipendenza _G nella lista delle dipendenze del requisito _G .	Implementato
RObb073	L'Utente deve poter visualizzare il decision tree _G associato al requisito _G nel contesto dell'asset _G .	Implementato
RObb074	L'Utente deve poter visualizzare le informazioni associate a ogni nodo _G del decision tree _G .	Implementato
RObb075	L'Utente deve poter visualizzare lo stato di attività di ogni nodo _G del decision tree _G . - Attivo: Il nodo_G fa parte del cammino principale del decision tree_G - Non attivo: Il nodo_G non fa parte del cammino principale del decision tree_G	Implementato
RObb076	L'Utente deve poter visualizzare le informazioni di un nodo _G di decisione all'interno del decision tree _G .	Implementato
RObb077	L'Utente deve poter visualizzare il codice del requisito _G a cui è associato il decision tree _G di cui fa parte il nodo _G di decisione.	Implementato
RObb078	L'Utente deve poter visualizzare il codice del nodo _G nella visualizzazione del decision tree _G .	Implementato

RObb079	L'Utente deve poter visualizzare la domanda associata al nodo _G nella visualizzazione del decision tree _G .	Implementato
RObb080	L'Utente deve poter visualizzare la risposta associata al nodo _G nella visualizzazione del decision tree _G .	Implementato
RObb081	L'Utente deve poter visualizzare un'indicazione visiva che un nodo _G non ha ancora una risposta associata nella visualizzazione del decision tree _G .	Implementato
RObb082	L'Utente deve poter visualizzare le informazioni di un nodo _G foglia all'interno del decision tree _G .	Implementato
RObb083	L'Utente deve poter visualizzare il valore associato al nodo _G foglia (Pass _G , Fail _G o Not Applicable _G).	Implementato
RObb084	L'Utente deve poter visualizzare la giustificazione associata alle risposte inserite nel decision tree _G nel dettaglio del requisito _G .	Implementato
RObb085	L'Utente deve poter visualizzare il dettaglio completo di un nodo _G decisionale attivo selezionandolo dal decision tree _G .	Implementato
RObb086	L'Utente deve poter visualizzare il codice del requisito _G a cui è associato il decision tree _G nel dettaglio del nodo _G .	Implementato
RObb087	L'Utente deve poter visualizzare il codice del nodo _G nel dettaglio del nodo _G decisionale.	Implementato
RObb088	L'Utente deve poter visualizzare la domanda associata al nodo _G nel dettaglio del nodo _G decisionale.	Implementato
RObb089	L'Utente deve poter visualizzare la risposta associata al nodo _G nel dettaglio del nodo _G decisionale.	Implementato
RObb090	L'Utente deve poter visualizzare un'indicazione visiva che il nodo _G non ha ancora una risposta associata nel dettaglio del nodo _G decisionale.	Implementato
RObb091	L'Utente deve poter inserire una risposta per il nodo _G di decisione corrente durante la compilazione del decision tree _G .	Implementato
RObb092	L'Utente deve poter selezionare una risposta per il nodo _G di decisione corrente.	Implementato
RObb093	L'Utente deve poter selezionare la risposta Yes per il nodo _G di decisione corrente.	Implementato
RObb094	L'Utente deve poter selezionare la risposta No per il nodo _G di decisione corrente.	Implementato
RObb095	L'Utente deve poter navigare al nodo _G successivo del decision tree _G dopo aver selezionato una risposta.	Implementato

RObb096	L'Utente deve poter visualizzare un avviso e non poter procedere al nodo _G successivo se non ha selezionato alcuna risposta per il nodo _G corrente.	Implementato
RObb097	L'Utente deve poter visualizzare una notifica di completamento del percorso decisionale ed essere reindirizzato al dettaglio del requisito _G quando il nodo _G successore è un nodo _G foglia.	Implementato
RObb098	L'Utente deve poter tornare al nodo _G precedente del decision tree _G per modificare una risposta già inserita.	Implementato
RObb099	L'Utente deve poter visualizzare un avviso ed essere reindirizzato al dettaglio del requisito _G se tenta di tornare indietro dal nodo _G root del decision tree _G .	Implementato
RObb100	L'Utente deve poter inserire o modificare il testo della giustificazione associata alla valutazione _G del requisito _G durante la sessione di valutazione _G .	Implementato

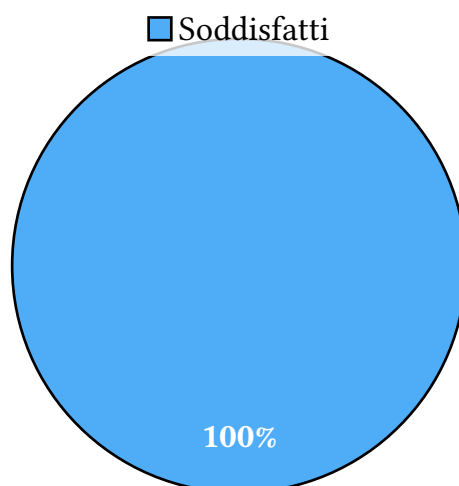


Figura 223: Diagramma dei requisiti obbligatori soddisfatti

6.1.2) Requisiti Desiderabili

Codice	Descrizione	Stato
RDes001	L'Utente deve poter eliminare un dispositivo _G scaricando un file di backup con i relativi dati.	Implementato
RDes002	L'Utente deve poter eliminare un dispositivo _G scaricando un file di backup in formato JSON _G .	Implementato
RDes003	L'Utente deve poter eliminare un dispositivo _G scaricando un file di backup in formato XML _G .	Implementato

RDes004	L'Utente deve poter eliminare un dispositivo _G scaricando un file di backup in formato CSV _G .	Implementato
---------	--	--------------

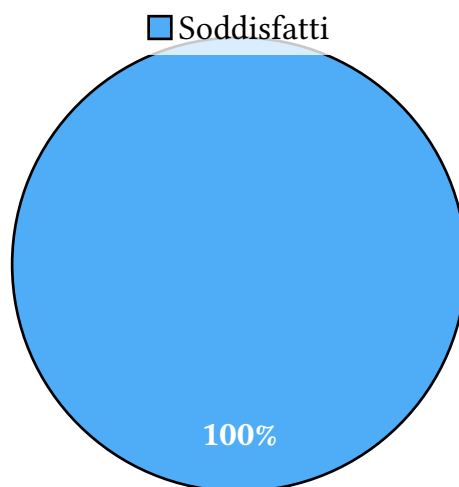


Figura 224: Diagramma dei requisiti desiderabili soddisfatti

6.1.3) Requisiti Opzionali

Codice	Descrizione	Stato
ROpzG-001	L'Utente deve poter modificare le informazioni di un dispositivo _G esistente.	Implementato
ROpzG-002	L'Utente deve poter modificare il nome associato al dispositivo _G con un nuovo nome di lunghezza compresa tra 1 e 64 caratteri	Implementato
ROpzG-003	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il nuovo nome inserito per il dispositivo _G non ha una lunghezza compresa tra 1 e 64 caratteri.	Implementato
ROpzG-004	L'Utente deve poter modificare il sistema operativo del dispositivo _G durante la fase di modifica.	Implementato
ROpzG-005	L'Utente deve poter modificare la descrizione del dispositivo _G durante la fase di modifica.	Implementato
ROpzG-006	L'Utente deve poter annullare le modifiche apportate ai dati del dispositivo _G durante la fase di modifica.	Implementato
ROpzG-007	L'Utente deve poter modificare le informazioni di un asset _G esistente durante una sessione di valutazione _G .	Implementato

ROpzG-008	L'Utente deve poter modificare il nome dell'asset _G durante la fase di modifica. Il nome dell'asset _G deve essere compreso tra 1 e 32 caratteri	Implementato
ROpzG-009	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il nuovo nome inserito per l'asset _G non è di lunghezza compresa tra 1 e 32 caratteri.	Implementato
ROpzG-010	L'Utente deve poter modificare il tipo dell'asset _G durante la fase di modifica.	Implementato
ROpzG-011	L'Utente deve poter selezionare il tipo security asset _G come nuovo tipo durante la modifica dell'asset _G .	Implementato
ROpzG-012	L'Utente deve poter selezionare il tipo network asset _G come nuovo tipo durante la modifica dell'asset _G .	Implementato
ROpzG-013	L'Utente deve poter modificare la descrizione dell'asset _G durante la fase di modifica.	Implementato
ROpzG-014	L'Utente deve poter annullare le modifiche apportate a un asset _G durante la fase di modifica.	Implementato
ROpzG-015	L'Utente deve poter esportare un report rappresentativo della valutazione _G del dispositivo _G .	Implementato
ROpzG-016	L'Utente deve poter esportare un report in formato pdf _G rappresentativo della valutazione _G del dispositivo _G .	Implementato
ROpzG-017	L'Utente deve poter visualizzare la lista dei modelli registrati nel sistema.	Non implementato
ROpzG-018	L'Utente deve poter visualizzare le informazioni generali di ogni singolo elemento nella lista dei modelli.	Non implementato
ROpzG-019	L'Utente deve poter visualizzare il nome del modello nella lista dei modelli.	Non implementato
ROpzG-020	L'Utente deve poter visualizzare la versione del modello nella lista dei modelli.	Non implementato
ROpzG-021	L'Utente deve poter visualizzare il dettaglio completo di uno specifico modello.	Non implementato
ROpzG-022	L'Utente deve poter visualizzare il codice identificativo del modello nel dettaglio del modello.	Non implementato

ROpzG-023	L'Utente deve poter visualizzare il nome del modello nel dettaglio del modello.	Non implementato
ROpzG-024	L'Utente deve poter visualizzare il numero di versione del modello nel dettaglio del modello.	Non implementato
ROpzG-025	L'Utente deve poter visualizzare la lista dei requisiti associati a un modello nel dettaglio del modello.	Non implementato
ROpzG-026	L'Utente deve poter visualizzare le informazioni generali di ogni singolo elemento nella lista dei requisiti del modello.	Non implementato
ROpzG-027	L'Utente deve poter visualizzare il codice del requisito _G nella lista dei requisiti del modello.	Non implementato
ROpzG-028	L'Utente deve poter visualizzare il nome del requisito _G nella lista dei requisiti del modello.	Non implementato
ROpzG-029	L'Utente deve poter inserire un nuovo modello nel sistema.	Non implementato
ROpzG-030	L'Utente deve poter creare manualmente un nuovo modello vuoto.	Non implementato
ROpzG-031	L'Utente deve poter inserire il nome del modello durante la creazione di un nuovo modello. Il nome del modello deve essere compreso tra 1 e 32 caratteri	Non implementato
ROpzG-032	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il nome inserito per il nuovo modello non è compreso tra 1 e 32 caratteri.	Non implementato
ROpzG-033	L'Utente deve poter importare un nuovo modello tramite file.	Non implementato
ROpzG-034	L'Utente deve poter importare un modello tramite un file in formato XML _G .	Non implementato
ROpzG-035	L'Utente deve poter importare un modello tramite un file in formato JSON _G .	Non implementato
ROpzG-036	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il modello che si tenta di importare esiste già nel sistema.	Non implementato
ROpzG-037	L'Utente deve poter annullare l'operazione di inserimento di un nuovo modello.	Non implementato
ROpzG-038	L'Utente deve poter modificare l'anagrafica _G di un modello esistente.	Non implementato
ROpzG-039	L'Utente deve poter modificare il nome del modello durante la modifica dell'anagrafica _G .	Non implementato

	Il nuovo nome deve avere una lunghezza compresa tra 1 e 32 caratteri	
ROpz _G -040	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il nuovo nome inserito per il modello ha una lunghezza non compresa tra 1 e 32 caratteri.	Non implementato
ROpz _G -041	L'Utente deve poter modificare la struttura di un modello.	Non implementato
ROpz _G -042	L'Utente deve poter salvare le modifiche apportate alla struttura del modello. La struttura non è valida al momento del salvataggio se vi è almeno uno scheletro _G di decision tree _G con almeno un percorso che non termina in un nodo _G foglia.	Non implementato
ROpz _G -043	L'utente deve poter salvare le modifiche significative apportate alla struttura dello standard senza invalidare le valutazioni del dispositivo _G già associate. Le modifiche significative comprendono operazioni di creazione di nuovi requisiti, eliminazione di requisiti e modifica dello scheletro _G di un qualsiasi decision tree _G , modifiche ai codici identificativi dei requisiti e dei nodi dei decision tree _G .	Non implementato
ROpz _G -044	L'utente deve poter salvare le modifiche non significative apportate alla struttura dello standard. Le modifiche non significative comprendono modifiche a campi puramente testuali: quali descrizione dei requisiti, nomi dei requisiti, testo delle domande dei nodi decisionali dello scheletro _G del decision tree _G .	Non implementato
ROpz _G -045	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando la struttura del modello non è valida al momento del salvataggio. Esiste almeno un decision tree _G il cui scheletro _G non è completo.	Non implementato
ROpz _G -046	L'Utente deve poter scartare le modifiche apportate alla struttura del modello durante una sessione di modifica.	Non implementato

ROpzG-047	L'Utente deve poter eliminare un modello esistente dal sistema.	Non implementato
ROpzG-048	L'Utente deve poter esportare le informazioni di un modello su file.	Non implementato
ROpzG-049	L'Utente deve poter esportare il modello in formato XML _G .	Non implementato
ROpzG-050	L'Utente deve poter esportare il modello in formato JSON _G .	Non implementato
ROpzG-051	L'Utente deve poter visualizzare il dettaglio completo di uno specifico requisito _G associato a un modello.	Non implementato
ROpzG-052	L'Utente deve poter visualizzare l'anagrafica _G del requisito _G nel dettaglio del requisito _G del modello.	Non implementato
ROpzG-053	L'Utente deve poter visualizzare il codice del requisito _G nell'anagrafica _G del requisito _G del modello.	Non implementato
ROpzG-054	L'Utente deve poter visualizzare il nome del requisito _G nell'anagrafica _G del requisito _G del modello.	Non implementato
ROpzG-055	L'Utente deve poter visualizzare la descrizione del requisito _G nel dettaglio del requisito _G del modello.	Non implementato
ROpzG-056	L'Utente deve poter visualizzare la lista dei requisiti da cui dipende il requisito _G nel contesto del modello.	Non implementato
ROpzG-057	L'Utente deve poter visualizzare il codice di ogni requisito _G dipendenza _G nella lista delle dipendenze del modello.	Non implementato
ROpzG-058	L'Utente deve poter visualizzare la lista dei requisiti da cui non dipende il requisito _G nel contesto del modello.	Non implementato
ROpzG-059	L'Utente deve poter visualizzare il codice di ogni requisito _G nella lista delle non dipendenze del modello.	Non implementato
ROpzG-060	L'Utente deve poter visualizzare lo scheletro _G del decision tree _G associato a un requisito _G del modello.	Non implementato
ROpzG-061	L'Utente deve poter visualizzare le informazioni associate a ogni nodo _G del decision tree _G di un requisito _G del modello.	Non implementato

ROpzG-062	L'Utente deve poter visualizzare le informazioni di un nodo _G decisionale nel decision tree _G del modello.	Non implementato
ROpzG-063	L'Utente deve poter visualizzare il codice del requisito _G a cui è associato il decision tree _G nel nodo _G decisionale del modello.	Non implementato
ROpzG-064	L'Utente deve poter visualizzare il codice del nodo _G decisionale nel decision tree _G del modello.	Non implementato
ROpzG-065	L'Utente deve poter visualizzare la domanda associata al nodo _G decisionale nel decision tree _G del modello.	Non implementato
ROpzG-066	L'Utente deve poter visualizzare le informazioni associate a un nodo _G foglia nel decision tree _G del modello.	Non implementato
ROpzG-067	L'Utente deve poter visualizzare il dettaglio delle informazioni legate a uno specifico nodo _G del decision tree _G del modello.	Non implementato
ROpzG-068	L'Utente deve poter visualizzare il dettaglio di uno specifico nodo _G decisionale nel decision tree _G del modello.	Non implementato
ROpzG-069	L'Utente deve poter visualizzare il codice del requisito _G padre nel dettaglio del nodo _G decisionale del modello.	Non implementato
ROpzG-070	L'Utente deve poter visualizzare il codice del nodo _G nel dettaglio del nodo _G decisionale del modello.	Non implementato
ROpzG-071	L'Utente deve poter visualizzare la domanda associata al nodo _G nel dettaglio del nodo _G decisionale del modello.	Non implementato
ROpzG-072	L'Utente deve poter visualizzare il dettaglio di uno specifico nodo _G foglia nel decision tree _G del modello.	Non implementato
ROpzG-073	L'Utente deve poter aggiungere un nuovo requisito _G al modello durante una sessione di modifica.	Non implementato
ROpzG-074	L'Utente deve poter eliminare un requisito _G dal modello durante una sessione di modifica.	Non implementato
ROpzG-075	L'Utente deve poter modificare l'anagrafica _G di un requisito _G esistente nel modello.	Non implementato
ROpzG-076	L'Utente deve poter inserire un codice univoco e di lunghezza compresa tra 4 e 10 caratteri da asso-	Non implementato

	ciare al requisito _G durante la creazione di un nuovo requisito _G .	
ROpz _G -077	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il codice inserito per il requisito _G ha una lunghezza non compresa tra 4 e 10 caratteri.	Non implementato
ROpz _G -078	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il codice inserito per il requisito _G è già associato a un altro requisito _G .	Non implementato
ROpz _G -079	L'Utente deve poter inserire un nome di lunghezza compresa tra 1 e 64 caratteri da associare al requisito _G durante la creazione di un nuovo requisito _G .	Non implementato
ROpz _G -080	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il nome inserito per il requisito _G non ha una lunghezza compresa tra 1 e 64 caratteri.	Non implementato
ROpz _G -081	L'Utente deve poter inserire la descrizione del requisito _G durante la creazione di un nuovo requisito _G .	Non implementato
ROpz _G -082	L'Utente deve poter modificare il codice di un requisito _G esistente nel modello. Il nuovo nome del requisito _G deve avere una lunghezza compresa tra 4 e 10 caratteri e non deve già appartenere a un altro requisito _G .	Non implementato
ROpz _G -083	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il codice modificato per il requisito _G ha una lunghezza non compresa tra 4 e 10 caratteri.	Non implementato
ROpz _G -084	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il codice modificato per il requisito _G è già associato a un altro requisito _G .	Non implementato
ROpz _G -085	L'Utente deve poter modificare il nome di un requisito _G esistente nel modello, inserendo un nuovo nome di lunghezza compresa tra 1 e 64 caratteri.	Non implementato
ROpz _G -086	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il nuovo nome inserito per il requisito _G non ha una lunghezza compresa tra 1 e 64 caratteri.	Non implementato
ROpz _G -087	L'Utente deve poter modificare la descrizione di un requisito _G esistente nel modello.	Non implementato

ROpz _G -088	L'Utente deve poter aggiungere una dipendenza _G tra requisiti del modello durante una sessione di modifica.	Non implementato
ROpz _G -089	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando l'aggiunta di una dipendenza _G causa una dipendenza circolare _G .	Non implementato
ROpz _G -090	L'Utente deve poter rimuovere una dipendenza _G tra requisiti del modello durante una sessione di modifica.	Non implementato
ROpz _G -091	L'Utente deve poter aggiungere un nodo _G figlio a un nodo _G dello scheletro _G del decision tree _G durante una sessione di modifica del modello.	Non implementato
ROpz _G -092	L'Utente deve poter aggiungere un nodo _G figlio con una relazione di bivio logico YES a un nodo _G dello scheletro _G del decision tree _G durante una sessione di modifica del modello.	Non implementato
ROpz _G -093	L'Utente deve poter aggiungere un nodo _G figlio con relazione di bivio logico NO a un nodo _G dello scheletro _G del decision tree _G durante una sessione di modifica del modello.	Non implementato
ROpz _G -094	L'Utente deve poter aggiungere un nuovo nodo _G allo scheletro _G del decision tree _G durante una sessione di modifica del modello.	Non implementato
ROpz _G -095	L'Utente deve poter aggiungere un nodo _G foglia allo scheletro _G del decision tree _G durante una sessione di modifica del modello.	Non implementato
ROpz _G -096	L'Utente deve poter aggiungere un nodo _G foglia con valore PASS _G allo scheletro _G del decision tree _G durante una sessione di modifica del modello.	Non implementato
ROpz _G -097	L'Utente deve poter aggiungere un nodo _G foglia con valore FAIL _G allo scheletro _G del decision tree _G durante una sessione di modifica del modello.	Non implementato
ROpz _G -098	L'Utente deve poter aggiungere un nodo _G foglia con valore NOT APPLICABLE _G allo scheletro _G del decision tree _G durante una sessione di modifica del modello.	Non implementato
ROpz _G -099	L'Utente deve poter aggiungere un nodo _G di decisione allo scheletro _G del decision tree _G durante una sessione di modifica del modello.	Non implementato

ROpz _G -100	L'Utente deve poter inserire il codice del nodo _G durante l'aggiunta di un nodo _G di decisione allo scheletro _G del decision tree _G . Il codice deve essere univoco e di lunghezza compresa tra 4 e 10 caratteri.	Non implementato
ROpz _G -101	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il codice inserito per il nodo _G non ha una lunghezza compresa tra 4 e 10 caratteri.	Non implementato
ROpz _G -102	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il codice inserito per il nodo _G è già associato a un altro nodo _G del decision tree _G .	Non implementato
ROpz _G -103	L'Utente deve poter inserire la domanda associata al nodo _G durante l'aggiunta di un nodo _G di decisione al decision tree _G .	Non implementato
ROpz _G -104		Non implementato
ROpz _G -105	L'Utente deve poter modificare le informazioni di un nodo _G di decisione esistente nel decision tree _G .	Non implementato
ROpz _G -106	L'Utente deve poter modificare il codice di un nodo _G di decisione esistente nel decision tree _G . Il codice deve essere univoco e di lunghezza compresa tra 4 e 10 caratteri.	Non implementato
ROpz _G -107	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il codice inserito per il nodo _G non ha una lunghezza compresa tra 4 e 10 caratteri.	Non implementato
ROpz _G -108	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando il codice modificato per il nodo _G è già associato a un altro nodo _G del decision tree _G .	Non implementato
ROpz _G -109	L'Utente deve poter modificare la domanda associata a un nodo _G di decisione esistente nel decision tree _G .	Non implementato
ROpz _G -110	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando la domanda modificata per il nodo _G è vuota.	Non implementato

ROpz _G -111	L'Utente deve poter rimuovere un nodo _G dal decision tree _G durante una sessione di modifica del modello.	Non implementato
ROpz _G -112	L'Utente deve poter visualizzare un messaggio di errore esplicativo quando tenta di eliminare il nodo _G root del decision tree _G .	Non implementato

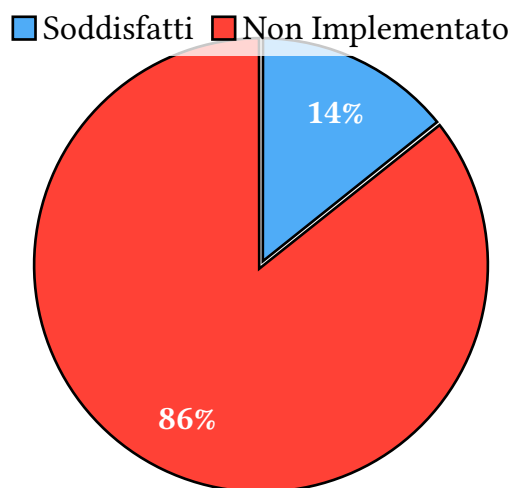


Figura 225: Diagramma dei requisiti opzionali soddisfatti

7) Gestione degli Errori

L'applicazione adotta una strategia di gestione degli errori stratificata. Le eccezioni non attraversano liberamente i confini architetturali, vengono invece intercettate, contestualizzate e tradotte man mano che si propagano dai livelli più profondi verso l'esterno. Per garantire chiarezza d'intenti e manutenibilità del codice, l'implementazione si basa su una netta distinzione semantica definita dai suffissi delle classi:

- **Suffisso **Error* (Livello di Dominio e Infrastruttura):** Viene utilizzato all'interno del Domain Layer (ereditando da una classe base `DomainError`) per segnalare la violazione di regole di business, oppure negli Outbound Adapter per segnalare problemi di interazione con risorse esterne o fallimenti infrastrutturali.
- **Suffisso **Failure* (Livello Applicativo / Casi d'Uso):** Identifica il fallimento di un intero flusso di lavoro orchestrato da un **Application Service**. Queste eccezioni descrivono l'impossibilità di portare a termine un'operazione dal punto di vista dell'utente o del sistema chiamante.

7.0.1) Il Flusso di Traduzione degli Errori (Exception Mapping)

L'intento di questa gestione degli errori è di evitare che dettagli tecnici (come l'uso specifico di MongoDB) inquinino i livelli superiori. Il meccanismo di propagazione si articola in tre fasi:

1. **Adattatori di Output (Infrastruttura):** Sollevano eccezioni custom di tipo `Error` per segnalare l'impossibilità di completare un'operazione di propria competenza. Questo avviene sia intercettando e traducendo errori generati da librerie di terze parti (es. eccezioni dei driver del database), sia generandole direttamente in base alla logica interna dell'adapter (es. risorse non trovate in un sistema di cache in memoria).
2. **Application Services (Casi d'Uso):** Durante l'orchestrazione, intercettano le eccezioni `Error` provenienti dal dominio o dai repository e le sollevano nuovamente (wrapping) traducendole in eccezioni di tipo `Failure`, aggiungendo il contesto specifico dell'operazione in corso.
3. **Adattatori di Input (Controller):** Comunicano esclusivamente con i Service, ricevendo solo eccezioni `Failure`. Il loro unico compito è mappare questi fallimenti nei corretti codici di stato del protocollo di comunicazione (es. HTTP 400 Bad Request, 404 Not Found, 409 Conflict), restituendo al client messaggi chiari e sicuri.